



软件源代码缺陷检测 TDengine 检测报告

苏州棱镜七彩信息科技有限公司

2026-04-17 21:26:36



2004

2010

2012

2006

CHART SEGMENT TITLE

CHART SEGMENT TITLE

报告声明

棱镜七彩是苏州棱镜七彩信息科技有限公司的简称，PrismSAST 是棱镜七彩具有自主知识产权的软件源代码安全缺陷检测平台的简称。

本检测报告由棱镜七彩软件源代码安全缺陷检测平台（PrismSAST）采用智能检测引擎进行自动化检测，并对检测结果经过人工复核后产生的检测报告。棱镜七彩保留对检测报告中各款项的解读权利，但并不意味着对检测结果中的缺陷或安全漏洞可能引起的任何直接经济损失或任何间接损失承担责任。

本检测报告的最终解释权归属棱镜七彩。

1 项目信息

项目名称	TDengine
提交时间	2026-04-20 20:48:24
完成时间	2026-04-20 21:05:34
用时	00:17:10
代码总行数	1826546
检测文件数	3860
缺陷总数	5648
致命缺陷数	2
缺陷密度（缺陷/KLOC）	3.09
检测类别	常规检测
报告创建日期时间	2026-04-17 21:26:36
检测团队	安全部
版本号	1.3.0

参考标准

*CMMI（软件能力成熟度模型集成）中缺陷密度参考值：		
CMM5 级	< 0.32	国际一流软件研发企业
CMM4 级	< 0.92	
CMM3 级	< 2.39	国内一流软件研发企业
CMM2 级	< 5.52	国内其他软件研发企业
CMM1 级	< 11.95	

2 风险等级

缺陷/安全漏洞严重等级	等级描述
致命	会造成程序崩溃或系统性故障的缺陷；或可能被容易利用且造成系统重大风险的安全漏洞
严重	会影响程序部分功能和性能，或者造成未定义行为的缺陷；或可能被利用且造成较为严重的风险的安全漏洞
一般	会造成对程序功能或性能的轻度影响；或存在可能被利用的低危安全漏洞。
强制	强制性标准是要求必须严格执行的标准条款
建议	建议性标准是企业可参考，自愿采用的标准条款
提示	不会直接影响程序的功能和性能，但这不是良好的代码风格，可能会有潜在的影响。
此等级划分是按照相关行业标准、国家标准，并参考《GB/T30279-2020 信息安全技术网络安全漏洞分类分级指南》进行定义	

3 评分评级

1. 用户可以在项目级别自定义下列三个影响因子的权重及情况：

序号	影响因子	描述
A	是否外部可达	可以分为物理隔绝、内部局域网连接、连接外网三个级别
B	影响程度	可以分为对别的组件或是部门影响较大、有一般的影响、不影响其它组件或是部门
C	缺陷/安全漏洞严重等级	致命、严重、一般、强制、建议、提示

2. 评分计算公式：A*A 权重 + B*B 权重 + C*C 权重

A、B、C 取值在【0，9】，三个权重相加等于 1，因此结果取值范围也在【0，9】；

最后根据公式的结果分级：低危【0，4），中危【4，6），高危【7，9）。

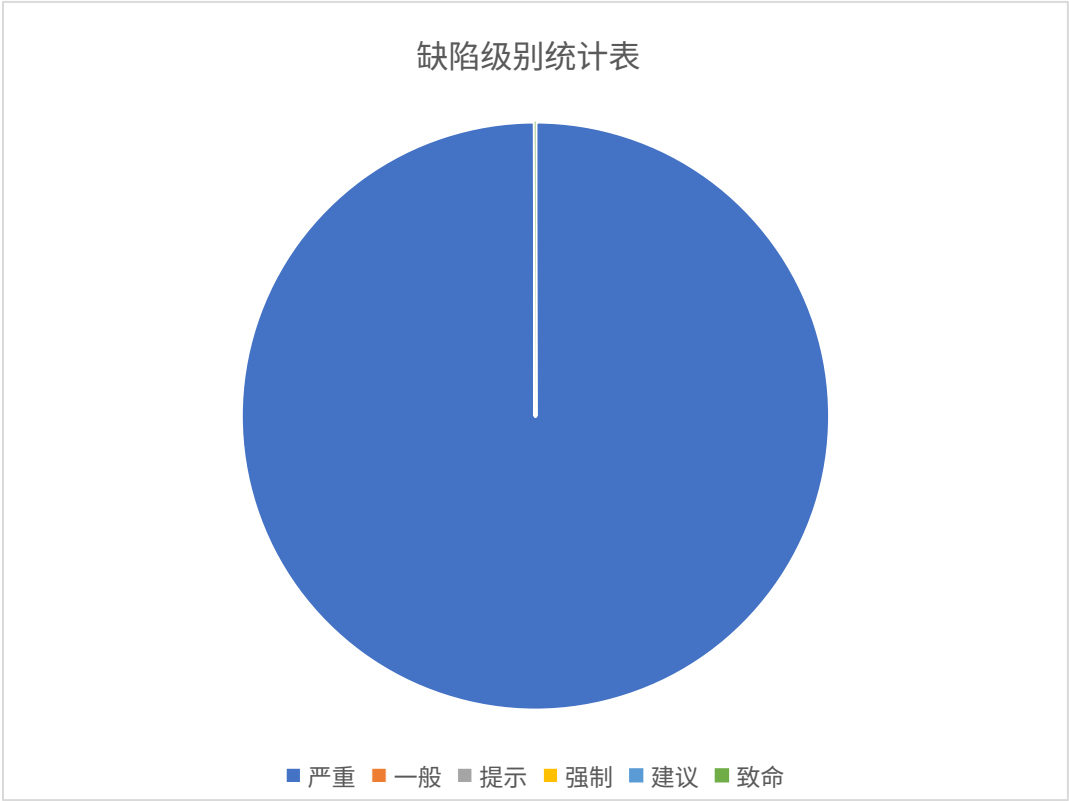
3. 缺省情况下：A 权重 20%，B 权重 20%，C 权重 60%

A = 5，B=5，C：致命=9，严重=7，一般=3，提示=1。

4 汇总统计

4.1 图表统计

4.1.1 按缺陷级别统计



4.1.2 按 CWE 2023 (Top25)统计



4.1.3 按 OWASP 2021 (TOP10)统计



4.2 选择检测集及检测结果统计表

序号	规则集	检测集	选择数量	发现缺陷/安全漏洞数
1	C/C++	CWE C/C++	24	1
2	C/C++缺陷类型	数据加密	3	0
3	C/C++缺陷类型	初始化问题	14	591
4	C/C++缺陷类型	缓冲区溢出	5	0
5	C/C++缺陷类型	空指针解引用	9	332
6	C/C++缺陷类型	并发	2	0
7	C/C++缺陷类型	除零	2	0
8	C/C++缺陷类型	数组问题	6	338
9	C/C++缺陷类型	不匹配的类型转换	11	385
10	C/C++缺陷类型	比较的问题	11	183
11	C/C++缺陷类型	整数溢出	2	0
12	C/C++缺陷类型	字符串问题	6	31
13	C/C++缺陷类型	注入问题	3	0
14	C/C++缺陷类型	返回值问题	10	2494
15	C/C++缺陷类型	不可达代码	2	1
16	C/C++缺陷类型	代码质量-头文件问题	7	214
17	C/C++缺陷类型	代码质量-无用数据	8	0
18	C/C++缺陷类型	算术运算	12	549
19	C/C++缺陷类型	恒真恒假	2	0
20	C/C++缺陷类型	跨站攻击	1	0
21	C/C++缺陷类型	浮点数问题	2	9
22	C/C++缺陷类型	不安全密码	4	0
23	C/C++缺陷类型	日志管理	2	0
24	C/C++缺陷类型	资源泄露	3	3
25	C/C++缺陷类型	使用已释放的内存	4	0
26	C/C++缺陷类型	其他内存相关问题	10	109
27	CSHARP	CWE CSHARP	26	6
28	HTML	HTML 通用缺陷	64	0
29	Java	CWE JAVA	55	72
30	Java	OWASP	28	19
31	Java	SEI CERT JAVA	27	70
32	Java	SJ/T 11683-2017	58	40
33	Java	GB/T 34944-2017	44	0
34	Java	GB/T 39412-2020 JAVA	67	57

35	Java	GB/T 38674-2020 JAVA	78	70
36	Java	Java 安全编码规范	1	0
37	Java	其它	42	0
38	Java 缺陷类型	API 误用	4	0
39	Java 缺陷类型	敏感信息暴露	13	0
40	Java 缺陷类型	日志问题	5	0
41	Java 缺陷类型	使用已释放的资源	1	0
42	Java 缺陷类型	输入验证不当	8	0
43	Java 缺陷类型	数据加密	7	0
44	Java 缺陷类型	算术运算	4	0
45	Java 缺陷类型	异常情况检查或处理不当	9	38
46	Java 缺陷类型	注入	11	0
47	Java 缺陷类型	资源泄露	2	19
48	Java 缺陷类型	比较问题	7	0
49	Java 缺陷类型	并发	11	0
50	Java 缺陷类型	不安全密码	3	0
51	Java 缺陷类型	除零	1	0
52	Java 缺陷类型	代码质量	63	15
53	Java 缺陷类型	访问控制	13	0
54	Java 缺陷类型	空指针解引用	1	0
55	Java 缺陷类型	跨站攻击	4	0
56	Python	Python 通用缺陷	105	2

4.3 人工复核结果统计表

缺陷级别	真实缺陷	误报缺陷	待确认缺陷	忽略缺陷	总计
致命	0	0	2	0	2
严重	0	0	5646	0	5646
一般	0	0	0	0	0
提示	0	0	0	0	0
强制	0	0	0	0	0
建议	0	0	0	0	0

4.4 代码质量度量统计表

变量描述	数量	简述
总行数	1826546	源代码行数统计
总文件数	3860	文件数量统计
总类个数	818	编译后中间文件个数
总方法个数	26889	所有的方法的个数
总有效代码行数	662353	有效代码行数=总代码行数-注释行数-空行数 总行数 "c= a+b; //some comments" 如上面一行代码会同时归入有效代码行和注释行
总注释行数	145072	总注释行=文件头注释+方法头注释+方法内注释
文件头注释总行数	13242	文件头注释
函数声明注释总函数	55569	方法函数注释，域变量注释
方法内注释总行数	76261	方法内部的注释
总空行数	130626	空行数
注释比率	8.40	注释比率=注释行总数/代码总行数
声明注释比率	8.38	声明注释比率=头注释行数/有效代码
代码注释比例	11.51	代码注释比例=函数内注释行数/有效代码
总基本块数	0	基本块数
出度为 1 的基本块数	0	出度为 1 的基本块数
总操作数	21197	加，减，乘，除，取余，与，或，异或等操作
参与操作符运算的变量数	10397	参与操作符运算的变量数
程序出口个数	593	程序出口个数
扇入数比率	0.00	扇入数/函数数量
扇出数比率	0.00	扇入数/函数数量
扇入数	0	当前节点的所有被调用数量
扇出数	0	当前节点的所有调用数量
循环数	8016	程序中所有的 loop 操作数量
圈复杂度	4.54	$(E-N+2)/\text{总方法数量}$

5 CWE C/C++

5.1 分项汇总表

序号	CWE C/C++	缺陷/安全漏洞名称	数量	缺陷级别
1	CWE459	资源未关闭/释放	1	严重

5.2 缺陷/安全漏洞详解

5.2.1 严重类

5.2.1.1 CWE459 资源未关闭/释放

缺陷数量： 1 个		
缺陷描述： 在使用临时或配套资源后，软件没有正确“清理”和删除这些资源。		
修复意见： 临时文件和其他支持资源在不再需要后立即删除/释放。		
1	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c 步骤描述： 不完整的清理 代码片段： 115: strcpy (stpcpy (stpcpy (stpcpy (fname, test_dir), "/"), base), "XXXXXX"); 116: 117: fd = mkstemp (fname); 118: if (fd == -1) 119: {	行号： 117

	污染路径: 无

6 C/C++缺陷类型-初始化问题

6.1 分项汇总表

序号	C/C++缺陷类型-初始化问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4757	枚举元素定义中的初始化必须完整	68	严重
2	WK4787	派生类构造函数必须在初始化列表中说明直接基类构造函数	1	严重
3	WK5723	结构体变量初始化的类型必须一致	341	严重
4	WK4755	变量初始化禁止隐含依赖于系统的缺省值	59	严重
5	WK4786	类中所有成员变量必须在构造函数中初始化	103	严重
6	WK4113	类成员变量未初始化	19	严重

6.2 缺陷/安全漏洞详解

6.2.1 严重类

6.2.1.1 WK4757 枚举元素定义中的初始化必须完整

缺陷数量：68 个		
缺陷描述： 枚举元素定义中的初始化必须完整。 注:规范的枚举元素初始化方法是,或初始化第一个元素,或初始化所有元素。		
修复意见： 无		
1	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v03.c	行号： 570
	步骤描述： 枚举元素定义中的初始化必须完整	
	代码片段： 568: 569: #define ERROR_GENERATE_ENUM(ENUM) ENUM, 570: typedef enum { ERROR_LIST(ERROR_GENERATE_ENUM) } ERR_codes; /* enum is exposed, to detect & handle specific errors; compare function result to -enum value */ 571: 572: #define ERROR_CONVERTTOSTRING(STRING) #STRING,	
	污染路径： 无	
2	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v04.c	行号： 256
	步骤描述： 枚举元素定义中的初始化必须完整	
	代码片段： 254: 255: /** from faster to stronger */ 256: typedef enum { ZSTD_fast, ZSTD_greedy, ZSTD_lazy, ZSTD_lazy2, ZSTD_btlazy2 } ZSTD_strategy; 257: 258: typedef struct	

	污染路径: 无	
3	缺陷发生位置: /contrib/lemon/lemon.c	行号: 405
	步骤描述: e_action 枚举元素定义中的初始化必须完整	
	代码片段: 403: }; 404: 405: enum e_action { 406: SHIFT, 407: ACCEPT,	
	污染路径: 无	
4	缺陷发生位置: /source/client/wrapper/jni/windows/win32/bridge/ AccessBridgePackages.h	行号: 1775
	步骤描述: PackageType 枚举元素定义中的初始化必须完整	
	代码片段: 1773: * What is the type of this package 1774: */ 1775: typedef enum PackageType { 1776: 1777: cMemoryMappedFileCreatedPackage = 0x11000,	
	污染路径: 无	
5	缺陷发生位置: /contrib/lemon/lemon.c	行号: 389
	步骤描述: cfgstatus 枚举元素定义中的初始化必须完整	
	代码片段: 387: ** symbols which are allowed to immediately follow the end of the rule. 388: ** Every configuration is recorded as an instance of the following: */ 389: enum cfgstatus { 390: COMPLETE, 391: INCOMPLETE	

	污染路径: 无	
6	缺陷发生位置: /source/dnode/vnode/src/meta/metaOpen.c	行号: 397
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 395: } 396: 397: typedef enum { 398: E_META_REPAIR_FROM_UID, 399: E_META_REPAIR_FROM_REDO,	
	污染路径: 无	
7	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 2215
	步骤描述: SMEMNEXTROWSTATES 枚举元素定义中的初始化必须完整	
	代码片段: 2213: } 2214: 2215: typedef enum SMEMNEXTROWSTATES { 2216: SMEMNEXTROW_ENTER, 2217: SMEMNEXTROW_NEXT,	
	污染路径: 无	
8	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 306
	步骤描述: _show_db_index 枚举元素定义中的初始化必须完整	
	代码片段: 304: } QUERY_CLASS; 305: 306: enum _show_db_index { 307: TSDB_SHOW_DB_NAME_INDEX, 308: TSDB_SHOW_DB_CREATED_TIME_INDEX,	
	污染路径: 无	
9	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 3226

	步骤描述: SFSNEXTROWSTATES 枚举元素定义中的初始化必须完整	
	代码片段: 3224: } 3225: 3226: typedef enum SFSNEXTROWSTATES { 3227: SFSNEXTROW_FS, 3228: SFSNEXTROW_FILESET,	
	污染路径: 无	
10	缺陷发生位置: /source/common/src/ttime.c	行号: 1309
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 1307: } TSFormatKeyWord; 1308: 1309: typedef enum { 1310: // TSFKW_AD, // BC AD 1311: // TSFKW_A_D, // A.D. B.C.	
	污染路径: 无	
11	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 1017
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 1015: } 1016: 1017: typedef enum { ZSTDcrp_continue, ZSTDcrp_noMemset } ZSTD_compResetPolicy_e; 1018: 1019: static void*	
	污染路径: 无	
12	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 2715
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段:	

	2713: #define ZSTDv07_BLOCKHEADERSIZE 3 /* C standard doesn't allow `static const` variable to be init using another `static const` variable */ 2714: static const size_t ZSTDv07_blockHeaderSize = ZSTDv07_BLOCKHEADERSIZE; 2715: typedef enum { bt_compressed, bt_raw, bt_rle, bt_end } blockType_t; 2716: 2717: #define MIN_SEQUENCES_SIZE 1 /* nbSeq==0 */	
	污染路径: 无	
13	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 2801
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 2799: * Context management 2800: ***** / 2801: typedef enum { ZSTDds_getFrameHeaderSize, ZSTDds_decodeFrameHeader, 2802: ZSTDds_decodeBlockHeader, ZSTDds_decompressBlock } ZSTDv06_dStage; 2803:	
	污染路径: 无	
14	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1400
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 1398: typedef struct ZSTD_Cctx_s ZSTD_Cctx; 1399: 1400: typedef enum { bt_compressed, bt_raw, bt_rle, bt_end } blockType_t; 1401: 1402: typedef struct	
	污染路径: 无	

15	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 605
	步骤描述: EChildOperatorStatus 枚举元素定义中的初始化必须完整	
	代码片段: 603: <pre>//===== ===== ===== 604: // Group Sort Operator 605: typedef enum EChildOperatorStatus { CHILD_OP_NEW_GROUP, CHILD_OP_SAME_GROUP, CHILD_OP_FINISHED } EChildOperatorStatus; 606: 607: typedef struct SGroupSortOperatorInfo {</pre>	
	污染路径: 无	
16	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 2493
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 2491: * Context management 2492: <pre>***** / 2493: typedef enum { ZSTDds_getFrameHeaderSize, ZSTDds_decodeFrameHeader, 2494: ZSTDds_decodeBlockHeader, ZSTDds_decompressBlock } ZSTD_dStage; 2495:</pre>	
	污染路径: 无	
17	缺陷发生位置: /source/libs/decimal/src/decimal.c	行号: 26
	步骤描述: DecimalRoundType 枚举元素定义中的初始化必须完整	
	代码片段: 24: } DecimalInternalType; 25: 26: typedef enum DecimalRoundType {	

	27: ROUND_TYPE_CEIL, 28: ROUND_TYPE_FLOOR,	
	污染路径: 无	
18	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 3916
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 3914: * ***** *****/ 3915: 3916: typedef enum { ZBUFFds_init, ZBUFFds_loadHeader, 3917: ZBUFFds_read, ZBUFFds_load, ZBUFFds_flush } ZBUFFv06_dStage; 3918:	
	污染路径: 无	
19	缺陷发生位置: /tools/taos-tools/inc/dump.h	行号: 357
	步骤描述: AVRO_CODEC 枚举元素定义中的初始化必须完整	
	代码片段: 355: 356: 357: enum AVRO_CODEC { 358: AVRO_CODEC_START = 0, 359: AVRO_CODEC_NULL = AVRO_CODEC_START,	
	污染路径: 无	
20	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 300
	步骤描述: enumQUERY_CLASS 枚举元素定义中的初始化必须完整	
	代码片段: 298: }; 299: 300: typedef enum enumQUERY_CLASS { 301: SPECIFIED_CLASS,	

	302: STABLE_CLASS,	
	污染路径: 无	
21	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 290
	步骤描述: enum_TAOS_INTERFACE 枚举元素定义中的初始化必须完整	
	代码片段: 288: enum enumSYNC_MODE { SYNC_MODE, ASYNC_MODE, MODE_BUT }; 289: 290: enum enum_TAOS_INTERFACE { 291: TAOSC_IFACE, 292: REST_IFACE,	
	污染路径: 无	
22	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 73
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 71: }; 72: 73: enum { 74: DND_CREATE, 75: DND_ADD,	
	污染路径: 无	
23	缺陷发生位置: /source/client/inc/clientSml.h	行号: 115
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 113: return code; 114: 115: typedef enum { 116: SCHEMA_ACTION_NULL, 117: SCHEMA_ACTION_CREATE_STABLE,	
	污染路径:	

	无	
24	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v04.c	行号： 3320
	步骤描述： 枚举元素定义中的初始化必须完整	
	代码片段： 3318: * *****/ 3319: 3320: typedef enum { ZBUFFds_init, ZBUFFds_readHeader, ZBUFFds_loadHeader, ZBUFFds_decodeHeader, 3321: ZBUFFds_read, ZBUFFds_load, ZBUFFds_flush } ZBUFF_dStage; 3322:	
	污染路径： 无	
25	缺陷发生位置： /source/dnode/mnode/impl/src/mndDnode.c	行号： 68
	步骤描述： 枚举元素定义中的初始化必须完整	
	代码片段： 66: }; 67: 68: enum { 69: DND_ACTIVE_CODE, 70: DND_CONN_ACTIVE_CODE,	
	污染路径： 无	
26	缺陷发生位置： /source/libs/executor/test/queryPlanTests.cpp	行号： 67
	步骤描述： 枚举元素定义中的初始化必须完整	
	代码片段： 65: #define QPT_QUERY_NODE_COL 10000 66: 67: typedef enum { 68: QPT_NODE_COLUMN, 69: QPT_NODE_EXPR,	
	污染路径： 无	
27	缺陷发生位置：	行号：

	/tools/taos-tools/inc/bench.h	281
	步骤描述: TEST_MODE 枚举元素定义中的初始化必须完整	
	代码片段: 279: ((t) == TSDB_DATA_TYPE_MEDIUMBLOB)) 280: 281: enum TEST_MODE { 282: INSERT_TEST, // 0 283: QUERY_TEST, // 1	
	污染路径: 无	
28	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 3788
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 3786: * *****/ 3787: 3788: typedef enum { ZBUFFv05ds_init, ZBUFFv05ds_readHeader, ZBUFFv05ds_loadHeader, ZBUFFv05ds_decodeHeader, 3789: ZBUFFv05ds_read, ZBUFFv05ds_load, ZBUFFv05ds_flush } ZBUFFv05_dStage; 3790:	
	污染路径: 无	
29	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 2923
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 2921: * Context management 2922: *****/ / 2923: typedef enum { ZSTDds_getFrameHeaderSize, ZSTDds_decodeFrameHeader, 2924: ZSTDds_decodeBlockHeader, ZSTDds_decompressBlock, 2925: ZSTDds_decodeSkippableHeader, ZSTDds_skipFrame } ZSTDv07_dStage;	

	污染路径: 无	
30	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 386
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 384: #define MIN_CBLOCK_SIZE (3 /*litCSize*/ + MIN_SEQUENCES_SIZE) 385: 386: typedef enum { bt_compressed, bt_raw, bt_rle, bt_end } blockType_t; 387: 388:	
	污染路径: 无	
31	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 288
	步骤描述: enumSYNC_MODE 枚举元素定义中的初始化必须完整	
	代码片段: 286: }; 287: 288: enum enumSYNC_MODE { SYNC_MODE, ASYNC_MODE, MODE_BUT }; 289: 290: enum enum_TAOS_INTERFACE {	
	污染路径: 无	
32	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 494
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 492: #define WILDCOPY_OVERLENGTH 8 493: 494: typedef enum { bt_compressed, bt_raw, bt_rle, bt_end } blockType_t; 495: 496:	

	污染路径: 无	
33	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.h	行号: 28
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 26: typedef struct STFileObj STFileObj; 27: 28: typedef enum { 29: TSDB_FTYPE_HEAD = 0, // .head 30: TSDB_FTYPE_DATA, // .data	
	污染路径: 无	
34	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/getopt_int.h	行号: 86
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 84: `--' can cause `getopt' to return -1 with `optind' != ARGC. */ 85: 86: enum 87: { 88: REQUIRE_ORDER, PERMUTE, RETURN_IN_ORDER	
	污染路径: 无	
35	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.h	行号: 80
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 78: * Advanced Streaming API 79: *****/ 80: typedef enum { ZSTDv05_fast, ZSTDv05_greedy, ZSTDv05_lazy, ZSTDv05_lazy2, ZSTDv05_btlazy2, ZSTDv05_opt, ZSTDv05_btopt } ZSTDv05_strategy; 81: typedef struct { 82: U64 srcSize;	

	污染路径: 无	
36	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress_internal.h	行号: 260
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 258: }; 259: 260: typedef enum { ZSTD_dtlm_fast, ZSTD_dtlm_full } ZSTD_dictTableLoadMethod_e; 261: 262: typedef enum { ZSTD_noDict = 0, ZSTD_extDict = 1, ZSTD_dictMatchState = 2 } ZSTD_dictMode_e;	
	污染路径: 无	
37	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.h	行号: 86
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 84: /* ZSTDMT_parameter : 85: * List of parameters that can be set using ZSTDMT_setMTCtxParameter() */ 86: typedef enum { 87: ZSTDMT_p_jobSize, /* Each job is compressed in parallel. By default, this value is dynamically determined depending on compression parameters. Can be set explicitly here. */ 88: ZSTDMT_p_overlapSectionLog /* Each job may reload a part of previous job to enhance compression ratio; 0 == no overlap, 6(default) == use 1/8th of window, >=9 == use full window. This is a "sticky" parameter : its value will be re-used on next compression job */	
	污染路径: 无	
38	缺陷发生位置: /tools/taos-tools/inc/dump.h	行号: 146
	步骤描述: _show_db_index 枚举元素定义中的初始化必须完整	

	代码片段: 144: 145: // ----- SHOW DATABASE INTERFACE----- 146: enum _show_db_index { 147: TSDB_SHOW_DB_NAME_INDEX, 148: TSDB_SHOW_DB_CREATED_TIME_INDEX,	
	污染路径: 无	
39	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 568
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 566: 567: #define ERROR_GENERATE_ENUM(ENUM) ENUM, 568: typedef enum { ERROR_LIST(ERROR_GENERATE_ENUM) } ERR_codes; /* enum is exposed, to detect & handle specific errors; compare function result to -enum value */ 569: 570: #define ERROR_CONVERTTOSTRING(STRING) #STRING,	
	污染路径: 无	
40	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 42
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 40: #define MAX_SQL_LEN 1048576 41: 42: typedef enum { 43: NOTIFY_CMD_START_CONSUM, 44: NOTIFY_CMD_START_COMMIT,	
	污染路径: 无	
41	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 2721
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段:	

	2719: 2720: #define HufLog 12 2721: typedef enum { lbt_huffman, lbt_repeat, lbt_raw, lbt_rle } litBlockType_t; 2722: 2723: #define LONGNBSEQ 0x7F00	
	污染路径: 无	
42	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 50
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 48: } 49: 50: typedef enum { A, B, C, D, E } T; 51: 52: static void func(T t) {	
	污染路径: 无	
43	缺陷发生位置: /source/util/test/terrorTest.cpp	行号: 19
	步骤描述: ParseStatus 枚举元素定义中的初始化必须完整	
	代码片段: 17: using namespace std; 18: 19: enum class ParseStatus { 20: Success, 21: FileNotExist,	
	污染路径: 无	
44	缺陷发生位置: /contrib/TSZ/zstd/common/zstd_internal.h	行号: 93
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 91: 92: #define HufLog 12 93: typedef enum { set_basic, set_rle, set_compressed,	

	set_repeat } symbolEncodingType_e; 94: 95: #define LONGNBSEQ 0x7F00	
	污染路径: 无	
45	缺陷发生位置: /contrib/TSZ/zstd/common/zstd_internal.h	行号: 87
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 85: #define ZSTD_BLOCKHEADERSIZE 3 /* C standard doesn't allow `static const` variable to be init using another `static const` variable */ 86: static const size_t ZSTD_blockHeaderSize = ZSTD_BLOCKHEADERSIZE; 87: typedef enum { bt_raw, bt_rle, bt_compressed, bt_reserved } blockType_e; 88: 89: #define MIN_SEQUENCES_SIZE 1 /* nbSeq==0 */	
	污染路径: 无	
46	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 23
	步骤描述: _line_status_ 枚举元素定义中的初始化必须完整	
	代码片段: 21: * This enum stores the status for each parsed line (internal use only). 22: */ 23: typedef enum _line_status_ { 24: LINE_UNPROCESSED, 25: LINE_ERROR,	
	污染路径: 无	
47	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 48
	步骤描述: enumQUERY_TYPE 枚举元素定义中的初始化必须完整	
	代码片段: 46: } NOTIFY_CMD_ID;	

	47: 48: typedef enum enumQUERY_TYPE { NO_INSERT_TYPE, INSERT_TYPE, QUERY_TYPE_BUT } QUERY_TYPE; 49: 50: typedef struct {	
	污染路径: 无	
48	缺陷发生位置: /tools/taos-tools/inc/benchCsv.h	行号: 24
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 22: 23: 24: typedef enum { 25: CSV_NAMING_I_SINGLE, 26: CSV_NAMING_I_TIME_SLICE,	
	污染路径: 无	
49	缺陷发生位置: /source/dnode/mnode/impl/inc/mndDef.h	行号: 1366
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 1364: 1365: // SGrantLogObj 1366: typedef enum { 1367: GRANT_STATE_INIT = 0, 1368: GRANT_STATE_UNGRANTED = 1,	
	污染路径: 无	
50	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 2636
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 2634: * Context management 2635: ***** /	

	2636: typedef enum { ZSTDv05ds_getFrameHeaderSize, ZSTDv05ds_decodeFrameHeader, 2637: ZSTDv05ds_decodeBlockHeader, ZSTDv05ds_decompressBlock } ZSTDv05_dStage; 2638:	
	污染路径: 无	
51	缺陷发生位置: /tools/taos-tools/inc/dump.h	行号: 182
	步骤描述: _describe_table_index 枚举元素定义中的初始化必须完整	
	代码片段: 180: 181: // DESCRIBE STABLE CONFIGURE ----- 182: enum _describe_table_index { 183: TSDB_DESCRIBE_METRIC_FIELD_INDEX, 184: TSDB_DESCRIBE_METRIC_TYPE_INDEX,	
	污染路径: 无	
52	缺陷发生位置: /tools/taos-tools/inc/benchLog.h	行号: 30
	步骤描述: LOG_LOCK 枚举元素定义中的初始化必须完整	
	代码片段: 28: // 29: 30: enum LOG_LOCK { 31: LOG_STDOUT, // 0 32: LOG_STDERR, // 1	
	污染路径: 无	
53	缺陷发生位置: /source/common/test/tmsgTest.cpp	行号: 31
	步骤描述: ParseStatus 枚举元素定义中的初始化必须完整	
	代码片段: 29: using namespace std; 30: 31: enum class ParseStatus { 32: Success,	

	33: FileNotExist,	
	污染路径: 无	
54	缺陷发生位置: /contrib/lemon/lemon.c	行号: 319
	步骤描述: symbol_type 枚举元素定义中的初始化必须完整	
	代码片段: 317: /* Symbols (terminals and nonterminals) of the grammar are stored 318: ** in the following: */ 319: enum symbol_type { 320: TERMINAL, 321: NONTERMINAL,	
	污染路径: 无	
55	缺陷发生位置: /include/client/taos.h	行号: 58
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 56: #define TSDB_DATA_TYPE_MAX 22 57: 58: typedef enum { 59: TSDB_OPTION_LOCALE, 60: TSDB_OPTION_CHARSET,	
	污染路径: 无	
56	缺陷发生位置: /source/dnode/mnode/impl/inc/mndDef.h	行号: 1375
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 1373: } EGrantState; 1374: 1375: typedef enum { 1376: GRANT_STATE_REASON_INIT = 0, 1377: GRANT_STATE_REASON_ALTER = 1, // alter activeCode 'revoked' or 'xxx'	

	污染路径: 无	
57	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 2392
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 2390: * Local structures 2391: *****/ 2392: typedef enum { bt_compressed, bt_raw, bt_rle, bt_end } blockType_t; 2393: 2394: typedef struct	
	污染路径: 无	
58	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 379
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 377: const SMetaEntry *pEntry = pParam->pEntry; 378: const SMetaEntry *pOldEntry = pParam->pOldEntry; 379: enum { ADD_COLUMN, DROP_COLUMN } action; 380: 381: if (pOldColumn && pNewColumn) {	
	污染路径: 无	
59	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 476
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 474: #define ZSTDv06_BLOCKHEADERSIZE 3 /* because C standard does not allow a static const value to be defined using another static const value :(*/ 475: static const size_t ZSTDv06_blockHeaderSize = ZSTDv06_BLOCKHEADERSIZE; 476: typedef enum { bt_compressed, bt_raw, bt_rle, bt_end } blockType_t; 477: 478: #define MIN_SEQUENCES_SIZE 1 /* nbSeq==0 */	

	污染路径: 无	
60	缺陷发生位置: /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号: 89
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 87: * Context management 88: ***** / 89: typedef enum { ZSTDds_getFrameHeaderSize, ZSTDds_decodeFrameHeader, 90: ZSTDds_decodeBlockHeader, ZSTDds_decompressBlock, 91: ZSTDds_decompressLastBlock, ZSTDds_checkChecksum,	
	污染路径: 无	
61	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 279
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 277: const SMetaEntry *pEntry = pParam->pEntry; 278: const SMetaEntry *pOldEntry = pParam->pOldEntry; 279: enum { ADD_INDEX, DROP_INDEX } action; 280: 281: if (pOldColumn && pNewColumn) {	
	污染路径: 无	
62	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 36
	步骤描述: _RUN_MODE 枚举元素定义中的初始化必须完整	
	代码片段: 34: #define MAX_ROW_STR_LEN (16 * 1024) 35: 36: enum _RUN_MODE { 37: TMQ_RUN_INSERT_AND_CONSUME,	

	38: TMQ_RUN_ONLY_INSERT,	
	污染路径: 无	
63	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 43
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 41: 42: #define FSE_GENERATE_ENUM(ENUM) ENUM, 43: typedef enum { FSE_LIST_ERRORS(FSE_GENERATE_ENUM) } FSE_errorCodes; /* enum is exposed, to detect & handle specific errors; compare function result to -enum value */ 44: 45:	
	污染路径: 无	
64	缺陷发生位置: /contrib/lemon/lemon.c	行号: 2304
	步骤描述: e_state 枚举元素定义中的初始化必须完整	
	代码片段: 2302: 2303: /* The state of the parser */ 2304: enum e_state { 2305: INITIALIZE, 2306: WAITING_FOR_DECL_OR_RULE,	
	污染路径: 无	
65	缺陷发生位置: /tools/taos-tools/inc/dump.h	行号: 170
	步骤描述: _show_tables_index 枚举元素定义中的初始化必须完整	
	代码片段: 168: 169: // SHOW TABLES CONFIGURE ----- 170: enum _show_tables_index { 171: TSDB_SHOW_TABLES_NAME_INDEX, 172: TSDB_SHOW_TABLES_CREATED_TIME_INDEX,	

	污染路径: 无	
66	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 2751
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 2749: * Local structures 2750: *****/ 2751: typedef enum { bt_compressed, bt_raw, bt_rle, bt_end } blockType_t; 2752: 2753: typedef struct	
	污染路径: 无	
67	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 4272
	步骤描述: 枚举元素定义中的初始化必须完整	
	代码片段: 4270: * *****/ 4271: 4272: typedef enum { ZBUFFds_init, ZBUFFds_loadHeader, 4273: ZBUFFds_read, ZBUFFds_load, ZBUFFds_flush } ZBUFFv07_dStage; 4274:	
	污染路径: 无	
68	缺陷发生位置: /contrib/lemon/lemon.c	行号: 324
	步骤描述: e_assoc 枚举元素定义中的初始化必须完整	
	代码片段: 322: MULTITERMINAL 323: }; 324: enum e_assoc { 325: LEFT, 326: RIGHT,	

	污染路径: 无

6.2.1.2 WK4787 派生类构造函数必须在初始化列表中说明直接基类构造函数

缺陷数量： 1 个		
缺陷描述： 派生类构造函数必须在初始化列表中说明直接基类构造函数。		
修复意见： 无		
1	缺陷发生位置： /source/libs/parser/test/mockCatalogService.cpp	行号： 80
	步骤描述： 派生类构造函数必须在初始化列表中说明直接基类构造函数	
	代码片段： 78: } 79: 80: TableBuilder(STableMeta* schemaMeta) : colId_(1), rowsize_(0), meta_(new MockTableMeta()) { 81: meta_->schema = schemaMeta; 82: }	
	污染路径： 无	

6.2.1.3 WK5723 结构体变量初始化的类型必须一致

缺陷数量： 341 个		
缺陷描述： 结构体变量的初值类型必须与结构体变量的定义类型一致。		
修复意见： 无		
1	缺陷发生位置： /tools/rocks-reader/rreader.cpp	行号： 95

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 93: 94: pLastCol->rowKey.ts = pLastColV0->ts; 95: pLastCol->rowKey.numOfPKs = 0; 96: pLastCol->dirty = pLastColV0->dirty; 97: pLastCol->colVal.cid = pLastColV0->colVal.cid;	
	污染路径: 无	
2	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 251
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 249: char msg[256] = {0}; 250: SSmlMsgBuf msgBuf; 251: msgBuf.buf = msg; 252: msgBuf.len = 256; 253: int32_t len = strlen(data[i]);	
	污染路径: 无	
3	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1483
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1481: symbolNext[s] = 1; 1482: } else { 1483: if (normalizedCounter[s] >= largeLimit) DTableH.fastMode=0; 1484: symbolNext[s] = normalizedCounter[s]; 1485: } } }	
	污染路径: 无	
4	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 505
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	503: code = tsdbDataFileReadBrinBlk(reader, &brinBlkArray); 504: if (code != 0) { 505: result->action = TSDB_REPAIR_ACTION_DROP; 506: tsdbRepairSetReason(&result->reason, "read_brin_index_failed"); 507: tsdbWarn("vgId:%d fid:%d failed to read brin index, downgrade action to %s since %s", vgId, pFileSet->fid,	
	污染路径: 无	
5	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 166
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 164: DTableH->fastMode = 0; 165: 166: cell->newState = 0; 167: cell->symbol = symbolValue; 168: cell->nbBits = 0;	
	污染路径: 无	
6	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 64
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 62: static struct argp argp3 = 63: { 64: opt3, NULL, "args doc3", "doc3", NULL, NULL, NULL 65: }; 66:	
	污染路径: 无	
7	缺陷发生位置: /tools/taos-tools/src/benchUtilDs.c	行号: 39
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 37:	

	38: ds->magic = MAGIC_NUMBER; 39: ds->custom = 0; 40: ds->cap = cap; 41: ds->len = 0;	
	污染路径: 无	
8	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 607
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 605: DTableH->fastMode = 0; 606: 607: cell->newState = 0; 608: cell->symbol = symbolValue; 609: cell->nbBits = 0;	
	污染路径: 无	
9	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 870
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 868: case TSDB_DATA_TYPE_TINYINT: 869: // pBase[bindIdx].buffer_length = sizeof(int8_t); 870: pBase[bindIdx].buffer = data->tinyData + rowIdx; 871: pBase[bindIdx].length = NULL; 872: pBase[bindIdx].is_null = data->isNull ? (data->isNull + rowIdx) : NULL;	
	污染路径: 无	
10	缺陷发生位置: /tools/taos-tools/src/benchMain.c	行号: 163
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 161: // read json config 162: if (g_arguments->metaFile) { 163: g_arguments->totalChildTables = 0; 164: if (readJsonConfig(g_arguments->metaFile)) {	

	165: errorPrint("failed to readJsonConfig %s\n", g_arguments->metaFile);	
	污染路径: 无	
11	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 1180
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1178: conn->hostThrd = pThrd; 1179: conn->dstAddr = taosStrdup(addr); 1180: conn->ipStr = taosStrdup(ip); 1181: conn->port = port; 1182: if (conn->dstAddr == NULL conn->ipStr == NULL) {	
	污染路径: 无	
12	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 242
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 240: 241: t->type = AllocatedTupleType; 242: t->data = pTuple; 243: 244: *pDesc = t;	
	污染路径: 无	
13	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.c	行号: 525
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 523: 524: p->pTableMeta = pMgt->pMgt->pTableMetaMgt->pTableMeta; 525: p->pBuilderMgt = pMgt; 526: pMgt->p = p; 527:	

	污染路径: 无	
14	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1295
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1293: struct action *ap; 1294: for(ap=lemp->sorted[i]->ap; ap; ap=ap->next){ 1295: if(ap->type==REDUCE) ap->x.rp->canReduce = LEMON_TRUE; 1296: } 1297: }	
	污染路径: 无	
15	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 335
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 333: params[i+8].buffer_type = TSDB_DATA_TYPE_BINARY; 334: params[i+8].buffer_length = (uintptr_t)lenOfBinaryDef; 335: params[i+8].buffer = v->br; 336: params[i+8].length = lb; 337: params[i+8].is_null = is_null;	
	污染路径: 无	
16	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 3124
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 3122: param.joinType = JOIN_TYPE_LEFT; 3123: param.subType = JOIN_STYPE_OUTER; 3124: param.cond = TEST_NO_COND; 3125: param.asc = true; 3126:	
	污染路径: 无	

17	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 203
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 201: { 202: bitC->bitContainer = 0; 203: bitC->bitPos = 0; 204: bitC->startPtr = (char*)startPtr; 205: bitC->ptr = bitC->startPtr;	
	污染路径: 无	
18	缺陷发生位置: /source/os/src/osDir.c	行号: 244
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 242: code = chmod(temp, mode); 243: if (-1 == code) { 244: struct stat statbuf = {0}; 245: code = stat(temp, &statbuf); 246: if (code != 0 (statbuf.st_mode & mode) != mode) {	
	污染路径: 无	
19	缺陷发生位置: /source/libs/parser/test/parTestUtil.cpp	行号: 79
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 77: void useDb(const string& acctId, const string& db) { 78: caseEnv_.acctId_ = acctId; 79: caseEnv_.db_ = db; 80: } 81:	
	污染路径: 无	
20	缺陷发生位置: /source/libs/parser/test/parTestUtil.cpp	行号: 75
	步骤描述: 结构体变量的初始化要一致.	

	代码片段: 73: } 74: 75: void login(const std::string& user) { caseEnv_.user_ = user; } 76: 77: void useDb(const string& acctId, const string& db) {	
	污染路径: 无	
21	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 541
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 539: { 540: entry->opt = o; 541: entry->num = 0; 542: entry->short_options = so; 543: entry->group = cur_group =	
	污染路径: 无	
22	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 855
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 853: case TSDB_DATA_TYPE_TINYINT: 854: pBase[bindIdx].buffer_length = sizeof(int8_t); 855: pBase[bindIdx].buffer = data->tinyData + rowIdx; 856: pBase[bindIdx].length = NULL; 857: pBase[bindIdx].is_null = data->isNull ? (data->isNull + rowIdx) : NULL;	
	污染路径: 无	
23	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 29
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	27: typedef int64_t tb_uid_t; 28: 29: struct SValue { 30: int8_t type; 31: union {	
	污染路径: 无	
24	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 732
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 730: else 731: { 732: na.numerator_ex = 0; 733: na.numerator = numerator; 734: na.divisor = denominator;	
	污染路径: 无	
25	缺陷发生位置: /source/libs/index/test/fstUT.cc	行号: 81
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 79: return false; 80: } 81: _size = nRead; 82: _s = fstSliceCreate((uint8_t*)buf, _size); 83: _fst = fstCreate(&_s);	
	污染路径: 无	
26	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1495
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1493: { FSEv07_DTableHeader DTableH; 1494: DTableH.tableLog = (U16)tableLog; 1495: DTableH.fastMode = 1; 1496: { S16 const largeLimit= (S16)(1 << (tableLog-	

	1)); 1497: U32 s;	
	污染路径: 无	
27	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1660
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1658: (void)memset(&args, 0, sizeof(struct __sysctl_args)); 1659: args.name = name; 1660: args.nlen = sizeof(name) / sizeof(name[0]); 1661: args.oldval = &old_usespid; 1662: args.oldlenp = &old_len;	
	污染路径: 无	
28	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1679
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1677: old_len = 0; 1678: (void)memset(&args, 0, sizeof(struct __sysctl_args)); 1679: args.name = name; 1680: args.nlen = sizeof(name) / sizeof(name[0]); 1681: args.oldval = &old_usespid;	
	污染路径: 无	
29	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 880
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 878: prctl(PR_SET_NAME, "createTable"); 879: #endif 880: pThreadInfo->buffer = benchCalloc(1, TOOLS_MAX_ALLOWED_SQL_LEN, false); 881: infoPrint(882: "thread[%d] start creating table from %" PRIu64 " to %" PRIu64	

	污染路径: 无	
30	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 571
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 569: bitD->start = (const char*)srcBuffer; 570: bitD->ptr = bitD->start; 571: bitD->bitContainer = *(const BYTE*)(bitD->start); 572: switch(srcSize) 573: {	
	污染路径: 无	
31	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 2222
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 2220: } 2221: memcpy(p->name, path, sizeof(path)); 2222: p->pTableMeta = pMeta; 2223: 2224: p->pBlkHandle = taosArrayInit(128, sizeof(SBlkHandle));	
	污染路径: 无	
32	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 157
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 155: params[i+9].buffer_type = TSDB_DATA_TYPE_NCHAR; 156: params[i+9].buffer_length = (uintptr_t)lenOfBinaryDef; 157: params[i+9].buffer = v->nr; 158: params[i+9].length = lb; 159: params[i+9].is_null = is_null;	
	污染路径:	

	无	
33	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.c	行号: 482
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 480: } 481: 482: p->pMemTable->pTableBuilder = p; 483: } 484: code = tableBuilderPut(p, pBatch);	
	污染路径: 无	
34	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 882
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 880: case TSDB_DATA_TYPE_INT: 881: // pBase[bindIdx].buffer_length = sizeof(int32_t); 882: pBase[bindIdx].buffer = data->intData + rowIdx; 883: pBase[bindIdx].length = NULL; 884: pBase[bindIdx].is_null = data->isNull ? (data->isNull + rowIdx) : NULL;	
	污染路径: 无	
35	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 1005
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1003: 1004: // end 1005: if (0 == info->totalDelay) info->totalDelay = 1; 1006: 1007:	
	污染路径: 无	
36	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 160

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 158: ZSTD_pthread_mutex_lock(&bufPool->poolMutex); 159: DEBUGLOG(4, "ZSTDMT_setBufferSize: bSize = %u", (U32)bSize); 160: bufPool->bufferSize = bSize; 161: ZSTD_pthread_mutex_unlock(&bufPool->poolMutex); 162: }	
	污染路径: 无	
37	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 240
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 238: params[2].buffer_type = TSDB_DATA_TYPE_TINYINT; 239: params[2].buffer_length = sizeof(v.v1); 240: params[2].buffer = &v.v1; 241: params[2].length = &boolLen; 242: params[2].is_null = NULL;	
	污染路径: 无	
38	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 3281
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 3279: state->iFileSet = TARRAY2_SIZE(state->aDFileSet); 3280: 3281: state->state = SFSNEXTROW_FILESET; 3282: } 3283:	
	污染路径: 无	
39	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 3037
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	3035: param.joinType = JOIN_TYPE_INNER; 3036: param.subType = JOIN_STYPE_NONE; 3037: param.cond = TEST_EQ_COND; 3038: param.asc = true; 3039: param.grpJoin = false;	
	污染路径: 无	
40	缺陷发生位置: /tools/taos-tools/src/benchMain.c	行号: 67
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 65: infoPrint("prepared_rand(%"PRIu64") < num_of_records_per_req(%d), so set num_of_records_per_req = prepared_rand\n", 66: g_arguments->prepared_rand, g_arguments- >reqPerReq); 67: g_arguments->reqPerReq = g_arguments- >prepared_rand; 68: } 69:	
	污染路径: 无	
41	缺陷发生位置: /tools/taos-tools/src/benchUtilDs.c	行号: 38
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 36: } 37: 38: ds->magic = MAGIC_NUMBER; 39: ds->custom = 0; 40: ds->cap = cap;	
	污染路径: 无	
42	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 3009
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	3007: param.joinType = JOIN_TYPE_INNER; 3008: param.subType = JOIN_STYPE_NONE; 3009: param.cond = TEST_NO_COND; 3010: param.asc = true; 3011: param.grpJoin = false;	
	污染路径: 无	
43	缺陷发生位置: /examples/c/prepare.c	行号: 72
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 70: char blob[80]; 71: int8_t varbin[16]; 72: } v = {0}; 73: 74: int32_t boolLen = sizeof(int8_t);	
	污染路径: 无	
44	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 302
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 300: } else { 301: bitD->ptr = bitD->start; 302: bitD->bitContainer = *(const BYTE*)(bitD->start); 303: switch(srcSize) 304: {	
	污染路径: 无	
45	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 163
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 161: FSE_decode_t* const cell = (FSE_decode_t*)dPtr; 162: 163: DTableH->tableLog = 0; 164: DTableH->fastMode = 0;	

	165:	
	污染路径: 无	
46	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 368
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 366: strcpy(v.c6, "abcdefgh"); 367: v.c7 = 1; 368: v.c8 = 1; 369: v.c9 = 1; 370: strcpy(v.c10, "一二三四五六七八");	
	污染路径: 无	
47	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 369
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 367: v.c7 = 1; 368: v.c8 = 1; 369: v.c9 = 1; 370: strcpy(v.c10, "一二三四五六七八"); 371: uintptr_t c10len=strlen(v.c10);	
	污染路径: 无	
48	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 157
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 155: struct timespec ts; 156: ts.tv_sec = 0; 157: ts.tv_nsec = 100000000; 158: nanosleep (&ts, NULL); 159: }	
	污染路径:	

	无	
49	缺陷发生位置： /tools/taos-tools/src/benchQuery.c	行号： 1070
	步骤描述： 结构体变量的初始化要一致。	
	代码片段： 1068: pThreadInfo->end_sql = i < b ? start_sql + a : start_sql + a - 1; 1069: start_sql = pThreadInfo->end_sql + 1; 1070: pThreadInfo->total_delay = 0; 1071: // total zero 1072: pThreadInfo->nSucc = 0;	
	污染路径： 无	
50	缺陷发生位置： /source/libs/nodes/src/nodesMsgFuncs.c	行号： 101
	步骤描述： 结构体变量的初始化要一致。	
	代码片段： 99: return termo; 100: } 101: pEncoder->pBuf = pNewBuf; 102: } 103: STlv* pTlv = (STlv*)(pEncoder->pBuf + pEncoder->offset);	
	污染路径： 无	
51	缺陷发生位置： /utils/test/c/tmqSim.c	行号： 275
	步骤描述： 结构体变量的初始化要一致。	
	代码片段： 273: g_stConflInfo.saveRowFlag = atol(argv[++i]); 274: } else if (strcmp(argv[i], "-y") == 0) { 275: g_stConflInfo.consumeDelay = atol(argv[++i]); 276: } else if (strcmp(argv[i], "-e") == 0) { 277: g_stConflInfo.useSnapshot = atol(argv[++i]);	
	污染路径： 无	
52	缺陷发生位置：	行号：

	/source/client/src/clientTmq.c	530
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 528: } 529: taosMemoryFree(conf->pass); 530: conf->pass = tmp; 531: return TMQ_CONF_OK; 532: }	
	污染路径: 无	
53	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 495
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 493: code = tsdbDataFileReaderOpen(NULL, &readerConfig, &reader); 494: if (code != 0) { 495: result->action = TSDB_REPAIR_ACTION_DROP; 496: tsdbRepairSetReason(&result->reason, "open_core_reader_failed"); 497: tsdbWarn("vgId:%d fid:%d failed to open core reader, downgrade action to %s since %s", vgId, pFileSet->fid,	
	污染路径: 无	
54	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdblter.c	行号: 517
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 515: } 516: 517: iter[0]->type = config->type; 518: iter[0]->noMoreData = false; 519: iter[0]->filterByVersion = config->filterByVersion;	
	污染路径: 无	
55	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 117

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 115: return NULL; 116: } 117: bufPool->bufferSize = 64 KB; 118: bufPool->totalBuffers = maxNbBuffers; 119: bufPool->nbBuffers = 0;	
	污染路径: 无	
56	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 1072
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1070: pThreadInfo->total_delay = 0; 1071: // total zero 1072: pThreadInfo->nSucc = 0; 1073: pThreadInfo->nFail = 0; 1074:	
	污染路径: 无	
57	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 288
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 286: /* Try to avoid dumping core. */ 287: struct rlimit core_limit; 288: core_limit.rlim_cur = 0; 289: core_limit.rlim_max = 0; 290: setrlimit (RLIMIT_CORE, &core_limit);	
	污染路径: 无	
58	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 849
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	847: case TSDB_DATA_TYPE_BOOL: 848: pBase[bindIdx].buffer_length = sizeof(bool); 849: pBase[bindIdx].buffer = data->boolData + rowIdx; 850: pBase[bindIdx].length = NULL; 851: pBase[bindIdx].is_null = data->isNull ? (data->isNull + rowIdx) : NULL;	
	污染路径: 无	
59	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 231
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 229: errorPrintReqArg2(CUS_PROMPT"Benchmark", "P"); 230: } 231: g_arguments->port = atoi(arg); 232: if (g_arguments->port <= 0) { 233: errorPrint(	
	污染路径: 无	
60	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 6030
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 6028: pReader->status.memTableMaxKey = INT64_MIN; 6029: } 6030: pReader->flag = READER_STATUS_SUSPEND; 6031: 6032: if (pReader->type == TIMEWINDOW_RANGE_EXTERNAL) {	
	污染路径: 无	
61	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 76
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 74: }	

	75: 76: pMem->size = sizeof(*pMem) + size; 77: pMem->next = pPool->next; 78: pMem->prev = pPool;	
	污染路径: 无	
62	缺陷发生位置: /examples/c/prepare.c	行号: 179
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 177: params[j].is_null = ((i == j) ? &is_null : 0); 178: } 179: v.b = (int8_t)i % 2; 180: v.v1 = (int8_t)i; 181: v.v2 = (int16_t)(i * 2);	
	污染路径: 无	
63	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 27
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 25: SSdbRaw* pRaw = (SSdbRaw*)malloc(sizeof(SSdbRaw)); 26: memset(pRaw, 0, sizeof(SSdbRaw)); 27: pRaw->sver = version; 28: return pRaw; 29: }	
	污染路径: 无	
64	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 878
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 876: DStatePtr->state = BITv07_readBits(bitD, DTableH->tableLog); 877: BITv07_reloadDStream(bitD); 878: DStatePtr->table = dt + 1; 879: }	

	880:	
	污染路径: 无	
65	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1281
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1279: FSE_decode_t* const cell = (FSE_decode_t*)dPtr; 1280: 1281: DTableH->tableLog = 0; 1282: DTableH->fastMode = 0; 1283:	
	污染路径: 无	
66	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 918
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 916: } 917: 918: http->quit = 0; 919: 920: err = taosThreadCreate(&http->thread, NULL, httpThread, (void*)http);	
	污染路径: 无	
67	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 3391
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 3389: 3390: if (SFSNEXTROW_INDEXLIST != state->state) { 3391: state->state = SFSNEXTROW_NEXTSTTROW; 3392: 3393: *ppRow = state->pLastRow;	
	污染路径:	

	无	
68	缺陷发生位置： /tools/taos-tools/src/dumpUtil.c	行号： 829
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 827: 828: // set idx 829: col->idx = i; 830: } 831:	
	污染路径： 无	
69	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号： 257
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 255: static void tsdbRepairCoreResultInit(STsdbRepairCoreResult *result) { 256: memset(result, 0, sizeof(*result)); 257: result->action = TSDB_REPAIR_ACTION_KEEP; 258: } 259:	
	污染路径： 无	
70	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v06.c	行号： 1112
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 1110: DStatePtr->state = BITv06_readBits(bitD, DTableH->tableLog); 1111: BITv06_reloadDStream(bitD); 1112: DStatePtr->table = dt + 1; 1113: } 1114:	
	污染路径： 无	
71	缺陷发生位置：	行号：

	/contrib/TSZ/zstd/legacy/zstd_v05.c		1387
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: 1385: DTableH->fastMode = 0; 1386: 1387: cell->newState = 0; 1388: cell->symbol = symbolValue; 1389: cell->nbBits = 0;		
	污染路径: 无		
72	缺陷发生位置: /tools/taos-tools/src/benchInsert.c		行号: 1571
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: 1569: } 1570: if (0 == pThreadInfo->totalDelay) { 1571: pThreadInfo->totalDelay = 1; 1572: } 1573: succPrint(		
	污染路径: 无		
73	缺陷发生位置: /tools/taos-tools/src/benchSys.c		行号: 326
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: 324: } 325: 326: stbInfo->insert_interval = atoi(arg); 327: if (stbInfo->insert_interval <= 0) { 328: errorPrint(		
	污染路径: 无		
74	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c		行号: 132
	步骤描述: 结构体变量的初始化要一致.		
	代码片段:		

	130: global_hooks.allocate = malloc; 131: global_hooks.deallocate = free; 132: global_hooks.reallocate = realloc; 133: return; 134: }	
	污染路径: 无	
75	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 238
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 236: vnodeAsyncCancelAllTasks(async, cancelArray); 237: } 238: worker->state = EVA_WORKER_STATE_STOP; 239: async->numLaunchWorkers--; 240: (void)taosThreadMutexUnlock(&async->mutex);	
	污染路径: 无	
76	缺陷发生位置: /examples/c/prepare.c	行号: 202
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 200: stmt = taos_stmt_init(taos); 201: taos_stmt_prepare(stmt, "SELECT * FROM m1 WHERE v1 > ? AND v2 < ?", 0); 202: v.v1 = 5; 203: v.v2 = 15; 204: taos_stmt_bind_param(stmt, params + 2);	
	污染路径: 无	
77	缺陷发生位置: /test/new_test_framework/script/api/stmt-crash.c	行号: 209
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 207: code = taos_stmt_set_tbname_tags(stmt, "m2", tags); 208: v.ts = 1591060628000 + 1; 209: v.b = (int8_t)1 % 2;	

	210: v.v1 = (int8_t)1; 211: v.v2 = (int16_t)(1 * 2);	
	污染路径: 无	
78	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 192
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 190: 191: *pBuilder = p; 192: p->pMemTable->pTableBuilder = p; 193: 194: _error:	
	污染路径: 无	
79	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 372
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 370: pCol->dtype = TSDB_DATA_TYPE_TIMESTAMP; 371: pCol->len = qptGetColumnRandLen(pCol->dtype); 372: pCol->inUse = 0; 373: pCol->hasIndex = false; 374: pCol->isPrimTs = true;	
	污染路径: 无	
80	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 367
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 365: v.c5 = (double)1; 366: strcpy(v.c6, "abcdefgh"); 367: v.c7 = 1; 368: v.c8 = 1; 369: v.c9 = 1;	
	污染路径:	

	无	
81	缺陷发生位置: /source/client/src/clientTmq.c	行号: 541
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 539: } 540: taosMemoryFree(conf->token); 541: conf->token = tmp; 542: return TMQ_CONF_OK; 543: }	
	污染路径: 无	
82	缺陷发生位置: /source/util/test/memPoolTest.cpp	行号: 569
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 567: 568: atomic_store_ptr(&pJobCtx->pJob, pJob); 569: pJob->pCtx = pJobCtx; 570: 571: char id[sizeof(tld) + sizeof(eld) + 1] = {0};	
	污染路径: 无	
83	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 800
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 798: bitD->start = (const char*)srcBuffer; 799: bitD->ptr = bitD->start; 800: bitD->bitContainer = *(const BYTE*)(bitD->start); 801: switch(srcSize) 802: {	
	污染路径: 无	
84	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 3365
	步骤描述:	

	结构体变量的初始化要一致.	
	代码片段: 3363: } 3364: 3365: state->state = SFSNEXTROW_INDEXLIST; 3366: state->iBrinIndex = 1; 3367: }	
	污染路径: 无	
85	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 653
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 651: } else { 652: if (field->min < (-1 * (RAND_MAX >> 1))) { 653: field->min = -1 * (RAND_MAX >> 1); 654: } 655: if (field->max > (RAND_MAX >> 1)) {	
	污染路径: 无	
86	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1530
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1528: FSEv06_decode_t* const cell = (FSEv06_decode_t*)dPtr; 1529: 1530: DTableH->tableLog = 0; 1531: DTableH->fastMode = 0; 1532:	
	污染路径: 无	
87	缺陷发生位置: /tools/taos-tools/src/benchUtilDs.c	行号: 41
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 39: ds->custom = 0;	

	40: ds->cap = cap; 41: ds->len = 0; 42: ds->data[0] = '\0'; 43:	
	污染路径: 无	
88	缺陷发生位置: /source/dnode/vnode/src/bse/bseMgt.c	行号: 738
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 736: 737: p->startSeq = sseq; 738: p->pBse = pBse; 739: *pBatch = p; 740: _error:	
	污染路径: 无	
89	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 483
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 481: if (pScanInfo->pTagCond != NULL) { 482: // tb_uid_t id = mr.me.uid; 483: item.check = 1; 484: } 485: if (taosArrayPush(tUid, &item) == NULL) {	
	污染路径: 无	
90	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 118
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 116: } 117: bufPool->bufferSize = 64 KB; 118: bufPool->totalBuffers = maxNbBuffers; 119: bufPool->nbBuffers = 0; 120: bufPool->cMem = cMem;	

	污染路径: 无	
91	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 731
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 729: } 730: } else { 731: grpSortOpInfo->childOpStatus = CHILD_OP_FINISHED; 732: } 733: }	
	污染路径: 无	
92	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 6006
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 6004: bins[k].percentage = bins[k].count / (double)totalCount; 6005: } else { 6006: bins[k].percentage = 0; 6007: } 6008: }	
	污染路径: 无	
93	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1661
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1659: args.name = name; 1660: args.nlen = sizeof(name) / sizeof(name[0]); 1661: args.oldval = &old_usespid; 1662: args.oldlenp = &old_len; 1663: args.newval = &new_usespid;	
	污染路径:	

	无	
94	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 107
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 105: 106: // other 107: field->type = convertStringToDatatype(token, 0, NULL); 108: field->length = convertTypeToLength(field->type); 109:	
	污染路径: 无	
95	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 269
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 267: 268: if (worker->runningTask == NULL) { 269: worker->state = EVA_WORKER_STATE_IDLE; 270: async->numIdleWorkers++; 271: (void)taosThreadCondWait(&async->hasTask, &async->mutex);	
	污染路径: 无	
96	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 262
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 260: static void tsdbRepairSttResultInit(STsdbRepairSttResult *result) { 261: memset(result, 0, sizeof(*result)); 262: result->action = TSDB_REPAIR_ACTION_KEEP; 263: } 264:	
	污染路径: 无	

97	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 741
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 739: best->liveJobs = 0; 740: best->dict = NULL; 741: best->dictSize = 0; 742: best->compressedSize = (size_t)-1; 743: memset(&best->parameters, 0, sizeof(best->parameters));	
	污染路径: 无	
98	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1290
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1288: FSE_decode_t* const cell = (FSE_decode_t*)(ptr) + 1; 1289: 1290: DTableH->tableLog = 0; 1291: DTableH->fastMode = 0; 1292:	
	污染路径: 无	
99	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/bug-argp1.c	行号: 17
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 15: static struct argp test_argp = 16: { 17: test_options 18: }; 19:	
	污染路径: 无	
100	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/	行号: 246

	test/argp-test.c	
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 244: struct params params; 245: params.foonly = 0; 246: params.foonly_default = random (); 247: argp_parse (&argp, argc, argv, 0, 0, ¶ms); 248: printf ("After parsing: foonly = %x\n", params.foonly);	
	污染路径: 无	
101	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1026
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1024: g_args.start_time = strtoll((const char *)timeStr, NULL, 10); 1025: } else { 1026: g_args.end_time = strtoll((const char *)timeStr, NULL, 10); 1027: } 1028: }	
	污染路径: 无	
102	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 459
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 457: if (pInfo->scanOrder == SCAN_ORDER_BOTH) { 458: // for window, only keep asc order when both first and last exist 459: pInfo->scanOrder = SCAN_ORDER_ASC; 460: } 461: pScan->interval = ((SWindowLogicNode*)pParent)- >interval;	
	污染路径: 无	
103	缺陷发生位置:	行号:

	/source/dnode/vnode/src/vnd/vnodeAsync.c	273
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 271: (void)taosThreadCondWait(&async->hasTask, &async->mutex); 272: async->numIdleWorkers--; 273: worker->state = EVA_WORKER_STATE_ACTIVE; 274: } else { 275: worker->runningTask->state = EVA_TASK_STATE_RUNNING;	
	污染路径: 无	
104	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 1725
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1723: SQPTCol* pNode = (SQPTCol*)nodesListGetNode(qptCtx.makeCtx.pInputList, i); 1724: TD_ALWAYS_ASSERT(pNode->type == QPT_QUERY_NODE_COL); 1725: pNode->inUse = 1; 1726: } 1727: return;	
	污染路径: 无	
105	缺陷发生位置: /utils/test/c/varbinary_test.c	行号: 464
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 462: params[j].is_null = ((i == j) ? &is_null : 0); 463: } 464: v.b = (int8_t)i % 2; 465: 466: for (int j = 0; j < vbinLen; ++j) {	
	污染路径: 无	
106	缺陷发生位置:	行号:

	/contrib/TSZ/zstd/legacy/zstd_v03.c		708
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: <pre> 706: DStatePtr->state = BIT_readBits(bitD, DTableH.tableLog); 707: BIT_reloadDStream(bitD); 708: DStatePtr->table = dt + 1; 709: } 710: </pre>		
	污染路径: 无		
107	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c		行号: 88
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: <pre> 86: static struct argp argp1 = 87: { 88: opt1, NULL, "args doc1", "doc1", children1, NULL, NULL 89: }; 90: </pre>		
	污染路径: 无		
108	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c		行号: 728
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: <pre> 726: tag = benchArrayGet(stbInfo->tags, stbInfo- >tags->size - 1); 727: tag->type = type; 728: tag->length = length; 729: if (length == 0) { 730: tag->null = true; </pre>		
	污染路径: 无		
109	缺陷发生位置: /test/new_test_framework/script/api/apitest.c		行号: 247

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 245: params[3].buffer_type = TSDB_DATA_TYPE_SMALLINT; 246: params[3].buffer_length = sizeof(v.v2); 247: params[3].buffer = &v.v2; 248: params[3].length = &sintLen; 249: params[3].is_null = NULL;	
	污染路径: 无	
110	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 900
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 898: int tableSum = pThreadInfo->end_table_to - pThreadInfo->start_table_from + 1; 899: if (stbInfo->useTagTableName) { 900: pThreadInfo->childNames = benchCalloc(tableSum, sizeof(char *), false); 901: pThreadInfo->childTblCount = tableSum; 902: }	
	污染路径: 无	
111	缺陷发生位置: /source/libs/parser/src/parInsertSql.c	行号: 4844
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 4842: 4843: // Set default CSV format 4844: parser->delimiter = CSV_DEFAULT_DELIMITER; 4845: parser->quote = CSV_QUOTE_SINGLE; // Default to single quote for TDengine compatibility 4846: parser->escape = CSV_ESCAPE_CHAR;	
	污染路径: expanded from macro 'CSV_DEFAULT_DELIMITER'	
112	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 670
	步骤描述:	

	结构体变量的初始化要一致.	
	代码片段: 668: 669: if (field->min < (-1 * (RAND_MAX >> 1))) { 670: field->min = -1 * (RAND_MAX >> 1); 671: } 672: if (field->max > (RAND_MAX >> 1)) {	
	污染路径: 无	
113	缺陷发生位置:	行号:
	/source/os/test/osTimeTests.cpp	224
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 222: char fmt[1024] = {0}; 223: struct tm tm = {0}; 224: struct timeval tv = {0}; 225: 226: retp = taosStrpTime(buf, fmt, NULL);	
	污染路径: 无	
114	缺陷发生位置:	行号:
	/source/libs/tdb/test/tdbPageFlushTest.cpp	102
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 100: } 101: 102: pMem->size = sizeof(*pMem) + size; 103: pMem->next = pPool->next; 104: pMem->prev = pPool;	
	污染路径: 无	
115	缺陷发生位置:	行号:
	/source/libs/transport/src/transCli.c	1178
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1176: char addr[TSDB_FQDN_LEN + 64] = {0}; 1177: CONN_CONSTRUCT_HASH_KEY(addr, ip, port);	

	1178: conn->hostThrd = pThrd; 1179: conn->dstAddr = taosStrdup(addr); 1180: conn->ipStr = taosStrdup(ip);	
	污染路径: 无	
116	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 433
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 431: } 432: 433: group->parser = argp->parser; 434: group->argp = argp; 435: group->short_end = cvt->short_end;	
	污染路径: 无	
117	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 436
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 434: group->argp = argp; 435: group->short_end = cvt->short_end; 436: group->args_processed = 0; 437: group->parent = parent; 438: group->parent_index = parent_index;	
	污染路径: 无	
118	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 61
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 59: } 60: 61: pMem->size = sizeof(*pMem) + size; 62: pMem->next = pPool->next;	

	63: pMem->prev = pPool;	
	污染路径: 无	
119	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 451
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 449: col = benchArrayGet(stbInfo->cols, stbInfo->cols->size - 1); 450: col->type = type; 451: col->length = length; 452: if (length == 0) { 453: col->null = true;	
	污染路径: 无	
120	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 865
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 863: void benchArrayClear(BArray* pArray) { 864: if (pArray == NULL) return; 865: pArray->size = 0; 866: } 867:	
	污染路径: 无	
121	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1873
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1871: 1872: TOOLS_STRNCPY(tableDes->name, table, TSDB_TABLE_NAME_LEN); 1873: tableDes->isVirtual = stbTableDes->isVirtual; 1874: tableDes->columns = stbTableDes->columns; 1875: tableDes->tags = stbTableDes->tags;	

	污染路径: 无	
122	缺陷发生位置: /test/new_test_framework/script/api/stmt-crash.c	行号: 258
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 256: taos_stmt_prepare(stmt, "SELECT * FROM m1 WHERE v1 > ? AND v2 < ?", 0); 257: v.v1 = 5; 258: v.v2 = 15; 259: taos_stmt_bind_param(stmt, params + 2); 260: if (taos_stmt_execute(stmt) != 0) {	
	污染路径: 无	
123	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 54
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 52: static struct argp argp5 = 53: { 54: opt5, NULL, "args doc5", "doc5", NULL, NULL, NULL 55: }; 56:	
	污染路径: 无	
124	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 114
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 112: __argp_fmtstream_update (__fs); 113: __old = __fs->wmargin; 114: __fs->wmargin = __wmargin; 115: return __old; 116: }	
	污染路径:	

	无	
125	缺陷发生位置： /source/client/test/smlTest.cpp	行号： 270
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 268: char msg[256] = {0}; 269: SSmlMsgBuf msgBuf; 270: msgBuf.buf = msg; 271: msgBuf.len = 256; 272: SSmlHandle *info = nullptr;	
	污染路径： 无	
126	缺陷发生位置： /tools/taos-tools/src/taosdump.c	行号： 1874
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 1872: TOOLS_STRNCPY(tableDes->name, table, TSDB_TABLE_NAME_LEN); 1873: tableDes->isVirtual = stbTableDes->isVirtual; 1874: tableDes->columns = stbTableDes->columns; 1875: tableDes->tags = stbTableDes->tags; 1876: memcpy(tableDes->cols, stbTableDes->cols,	
	污染路径： 无	
127	缺陷发生位置： /contrib/lemon/lemon.c	行号： 925
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 923: 924: for(i=0; i<lemp->nsymbol; i++){ 925: lemp->symbols[i]->lambda = LEMON_FALSE; 926: } 927: for(i=lemp->nterminal; i<lemp->nsymbol; i++){	
	污染路径： 无	
128	缺陷发生位置： /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号： 276

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 274: 275: #ifdef SIGHUP 276: sa.sa_handler = ttq_handle_sighup; 277: (void)sigaction(SIGHUP, &sa, NULL); 278: #endif	
	污染路径: 无	
129	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 528
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 526: } 527: 528: hol->entries = malloc (sizeof (struct hol_entry) * hol->num_entries); 529: hol->short_options = malloc (num_short_options + 1); 530:	
	污染路径: 无	
130	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 272
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 270: (void)sigemptyset(&sa.sa_mask); 271: (void)sigaction(SIGINT, &sa, NULL); 272: sa.sa_handler = ttq_handle_sigint; 273: (void)sigaction(SIGTERM, &sa, NULL); 274:	
	污染路径: 无	
131	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 343
	步骤描述:	

	结构体变量的初始化要一致.	
	代码片段: 341: int ret = 0; 342: 343: struct timespec ts = {0}; 344: 345: if (clock_gettime(CLOCK_REALTIME, &ts) == -1) {	
	污染路径: 无	
132	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbCache.c	3417
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 3415: 3416: clearLastFileSet(state); 3417: state->state = SFSNEXTROW_FILESET; 3418: goto _next_fileset; 3419: } else {	
133	污染路径: 无	
	缺陷发生位置:	行号:
	/docs/examples/c/prepare.c	203
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 201: taos_stmt_prepare(stmt, "SELECT * FROM m1 WHERE v1 > ? AND v2 < ?", 0); 202: v.v1 = 5; 203: v.v2 = 15; 204: taos_stmt_bind_param(stmt, params + 2); 205: if (taos_stmt_execute(stmt) != 0) {	
134	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/tdb/test/tdbExOVFLTest.cpp	61
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 59: }	

	60: 61: pMem->size = sizeof(*pMem) + size; 62: pMem->next = pPool->next; 63: pMem->prev = pPool;	
	污染路径: 无	
135	缺陷发生位置: /test/new_test_framework/script/api/stmt-crash.c	行号: 194
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 192: code = taos_stmt_set_tbname_tags(stmt, "m1", tags); 193: v.ts = 1591060628000; 194: v.b = (int8_t)0 % 2; 195: v.v1 = (int8_t)0; 196: v.v2 = (int16_t)(0 * 2);	
	污染路径: 无	
136	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 811
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 809: 810: FORCE_INLINE int64_t toolsGetTimestampNs() { 811: struct timespec systemTime = {0}; 812: (void)toolsClockGetTime(CLOCK_REALTIME, &systemTime); 813: return (int64_t)systemTime.tv_sec * 1000000000L +	
	污染路径: 无	
137	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 691
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 689: for (int j = 0; j < nConcurrent; j++) { 690: qThreadInfo *pThreadInfo = infos + j; 691: pThreadInfo->threadID = i * nConcurrent + j; 692: pThreadInfo->querySeq = i;	

	693: pThreadInfo->dbName = dbName;	
	污染路径: 无	
138	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 150
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 148: fs->rmargin = rmargin; 149: fs->wmargin = wmargin; 150: fs->point_col = 0; 151: fs->point_offs = 0; 152:	
	污染路径: 无	
139	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 3218
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 3216: pCol->dtype = TSDB_DATA_TYPE_VARCHAR; 3217: pCol->len = qptGetColumnRandLen(pCol->dtype); 3218: pCol->inUse = 0; 3219: pCol->hasIndex = QPT_RAND_BOOL_V; 3220: pCol->isPrimTs = QPT_RAND_BOOL_V;	
	污染路径: 无	
140	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 641
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 639: write_meta->end_ts = stb->startTimestamp + stb->timestamp_step * stb->insertRows; 640: write_meta->ts_step = stb->timestamp_step * stb->insertRows; 641: write_meta->interlace_step = stb-> timestamp_step * stb->interlaceRows; 642:	

	643: int ret = csvGenCsvHeader(write_meta);	
	污染路径: 无	
141	缺陷发生位置: /contrib/lemon/lemon.c	行号: 942
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 940: } 941: if(i==rp->nrhs){ 942: rp->lhs->lambda = LEMON_TRUE; 943: progress = 1; 944: }	
	污染路径: 无	
142	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 727
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 725: *ppBlock = block; 726: } else { 727: grpSortOpInfo->childOpStatus = CHILD_OP_NEW_GROUP; 728: grpSortOpInfo->prefetchedSortInput = block; 729: }	
	污染路径: 无	
143	缺陷发生位置: /source/util/test/trefTest.c	行号: 99
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 97: 98: pSpace->p = (void **)taosMemoryCalloc(sizeof(void *), pSpace->refNum); 99: pSpace->rid = taosMemoryCalloc(pSpace->refNum, sizeof(int64_t)); 100: 101: TdThreadAttr thattr;	

	污染路径: 无	
144	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 1740
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1738: } 1739: 1740: pNode->inUse = 1; 1741: colInUse++; 1742: } while (colInUse < colNum);	
	污染路径: 无	
145	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 269
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 267: struct sigaction sa = {0}; 268: 269: sa.sa_handler = ttq_handle_sigint; 270: (void)sigemptyset(&sa.sa_mask); 271: (void)sigaction(SIGINT, &sa, NULL);	
	污染路径: 无	
146	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 679
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 677: bitD->start = (const char*)srcBuffer; 678: bitD->ptr = bitD->start; 679: bitD->bitContainer = *(const BYTE*)(bitD->start); 680: switch(srcSize) 681: {	
	污染路径: 无	
147	缺陷发生位置:	行号:

	/source/libs/scalar/src/scifunc.c	5945
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 5943: (*bins)[i].lower = intervals[i] < intervals[i + 1] ? intervals[i] : intervals[i + 1]; 5944: (*bins)[i].upper = intervals[i + 1] > intervals[i] ? intervals[i + 1] : intervals[i]; 5945: (*bins)[i].count = 0; 5946: } 5947:	
	污染路径: 无	
148	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 289
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 287: struct rlimit core_limit; 288: core_limit.rlim_cur = 0; 289: core_limit.rlim_max = 0; 290: setrlimit (RLIMIT_CORE, &core_limit); 291: #endif	
	污染路径: 无	
149	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 87
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 85: exit(EXIT_FAILURE); 86: } 87: field->precision = p; 88: field->scale = s; 89: field->type = convertStringToDatatype("DECIMAL", 0, &field->precision);	
	污染路径: 无	
150	缺陷发生位置:	行号:

	/contrib/TSZ/zstd/legacy/zstd_v05.c	1006
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1004: DStatePtr->state = BITv05_readBits(bitD, DTableH->tableLog); 1005: BITv05_reloadDStream(bitD); 1006: DStatePtr->table = dt + 1; 1007: } 1008:	
	污染路径: 无	
151	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 673
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 671: } 672: if (field->max > (RAND_MAX >> 1)) { 673: field->max = RAND_MAX >> 1; 674: } 675: intTmp = field->min;	
	污染路径: expanded from macro 'RAND_MAX'	
152	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 861
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 859: case TSDB_DATA_TYPE_SMALLINT: 860: pBase[bindIdx].buffer_length = sizeof(int16_t); 861: pBase[bindIdx].buffer = data->smallData + rowIdx; 862: pBase[bindIdx].length = NULL; 863: pBase[bindIdx].is_null = data->isNull ? (data->isNull + rowIdx) : NULL;	
	污染路径: 无	
153	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 187
	步骤描述:	

	结构体变量的初始化要一致.	
	代码片段: 185: 186: gzipStream.next_in = (Bytef*)pSrc; 187: gzipStream.avail_in = (uLong)srcLen; 188: gzipStream.next_out = (Bytef*)pDest; 189: gzipStream.avail_out = (uLong)(destLen);	
	污染路径: 无	
154	缺陷发生位置:	行号:
	/source/libs/tdb/test/tdbTest.cpp	23
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 21: 22: pPool->prev = pPool->next = pPool; 23: pPool->size = 0; 24: 25: return pPool;	
	污染路径: 无	
155	缺陷发生位置:	行号:
	/tools/shell/src/shellEngine.c	878
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 876: dump_info->vertical = vertical; 877: tsem_init(&dump_info->sem, 0, 0); 878: dump_info->numOfAllRows = 0; 879: 880: dump_info->numFields = taos_num_fields(tres);	
	污染路径: 无	
156	缺陷发生位置:	行号:
	/tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/gnu-test-skeleton.c	156
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 154: finally died. */	

	155: struct timespec ts; 156: ts.tv_sec = 0; 157: ts.tv_nsec = 100000000; 158: nanosleep (&ts, NULL);	
	污染路径: 无	
157	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 209
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 207: } 208: 209: g_arguments->prepared_rand = atol(arg); 210: if (g_arguments->prepared_rand <= 0) { 211: errorPrint(	
	污染路径: 无	
158	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 150
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 148: params[i+8].buffer_type = TSDB_DATA_TYPE_BINARY; 149: params[i+8].buffer_length = (uintptr_t)lenOfBinaryDef; 150: params[i+8].buffer = v->br; 151: params[i+8].length = lb; 152: params[i+8].is_null = is_null;	
	污染路径: 无	
159	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 254
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 252: params[4].buffer_type = TSDB_DATA_TYPE_INT; 253: params[4].buffer_length = sizeof(v.v4); 254: params[4].buffer = &v.v4; 255: params[4].length = &intLen;	

	256: params[4].is_null = NULL;	
	污染路径: 无	
160	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 273
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 271: g_stConflInfo.showRowFlag = atol(argv[++i]); 272: } else if (strcmp(argv[i], "-s") == 0) { 273: g_stConflInfo.saveRowFlag = atol(argv[++i]); 274: } else if (strcmp(argv[i], "-y") == 0) { 275: g_stConflInfo.consumeDelay = atol(argv[++i]);	
	污染路径: 无	
161	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 233
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 231: params[1].buffer_type = TSDB_DATA_TYPE_BOOL; 232: params[1].buffer_length = sizeof(v.b); 233: params[1].buffer = &v.b; 234: params[1].length = &boolLen; 235: params[1].is_null = NULL;	
	污染路径: 无	
162	缺陷发生位置: /contrib/lemon/lemon.c	行号: 2390
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 2388: psp->errorcnt++; 2389: }else if(strcmp(x, "{NEVER-REDUCE")==0){ 2390: psp->prevrule->neverReduce = 1; 2391: }else{ 2392: psp->prevrule->line = psp->tokenlineno;	
	污染路径: 无	

163	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 164
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 162: 163: DTableH->tableLog = 0; 164: DTableH->fastMode = 0; 165: 166: cell->newState = 0;	
	污染路径: 无	
164	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 215
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 213: } 214: v.c7 = 32767; 215: v.c8 = 127; 216: v.c9 = 1; 217: strcpy(v.c10, "一二三四五六七八");	
	污染路径: 无	
165	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 409
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 407: /* OPT can be used as a long option. */ 408: { 409: cvt->long_end->name = opt->name; 410: cvt->long_end->has_arg = 411: (real->arg	
	污染路径: 无	
166	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 360
	步骤描述:	

	结构体变量的初始化要一致.	
	代码片段: <pre> 358: int8_t c9; 359: char c10[32]; 360: } v = {0}; 361: v.c1 = (int64_t)1591060628000; 362: v.c2 = (int32_t)1; </pre>	
	污染路径: 无	
167	缺陷发生位置:	行号:
	/source/libs/tdb/test/tdbPageFlushTest.cpp	38
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 36: 37: pPool->prev = pPool->next = pPool; 38: pPool->size = 0; 39: 40: return pPool; </pre>	
	污染路径: 无	
168	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbRead2.c	6096
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 6094: // open reader failure may cause the flag still to be READER_STATUS_SUSPEND, which may cause suspend reader failure. 6095: // So we need to set it A.S.A.P 6096: pReader->flag = READER_STATUS_NORMAL; 6097: 6098: if (pReader->type == TIMEWINDOW_RANGE_CONTAINED) { </pre>	
	污染路径: 无	
169	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	264
	步骤描述: 结构体变量的初始化要一致.	

	代码片段: 262: snprintf(obj.url, obj.urlLen, "node%d:6050", i); 263: obj.createTime = 1000000 + i; 264: obj.updateTime = 2000000 + i; 265: 266: SSdbRaw* pRaw = sdbAllocRaw(1, 1, 256);	
	污染路径: 无	
170	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 893
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 891: pThreadInfo->end_sql = i < b ? start_sql + a : start_sql + a - 1; 892: start_sql = pThreadInfo->end_sql + 1; 893: pThreadInfo->total_delay = 0; 894: pThreadInfo->dbName = dbName; 895:	
	污染路径: 无	
171	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttDaemon.c	行号: 221
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 219: return termo; 220: } 221: msgInfo->rpcType = MQTTD_RPC_RETRIVE_FUNC; 222: msgInfo->param = mqtt; 223: if (uv_sem_init(&msgInfo->resultSem, 0) != 0) {	
	污染路径: 无	
172	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1106
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1104: } 1105: pThreadInfo->start_table_from = tableFrom;	

	1106: pThreadInfo->ntables = i < mod ? div + 1 : div; 1107: pThreadInfo->end_table_to = i < mod ? tableFrom + div : tableFrom + div - 1; 1108: tableFrom = pThreadInfo->end_table_to + 1;	
	污染路径: 无	
173	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 136
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 134: } 135: 136: global_hooks.allocate = malloc; 137: if (hooks->malloc_fn != NULL) 138: {	
	污染路径: 无	
174	缺陷发生位置: /source/os/test/osThreadTests.cpp	行号: 266
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 264: EXPECT_NE(reti, 0); 265: 266: struct timespec abstime = {0}; 267: reti = taosThreadCondTimedWait(&cond, NULL, &abstime); 268: EXPECT_NE(reti, 0);	
	污染路径: 无	
175	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 309
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 307: } 308: 309: g_arguments->nthreads = atoi(arg);	

	310: if (g_arguments->nthreads <= 0) { 311: errorPrint(	
	污染路径: 无	
176	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 345
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 343: void destroyFillOperatorInfo(void* param) { 344: SFillOperatorInfo* pInfo = (SFillOperatorInfo*)param; 345: pInfo->pFillInfo = taosDestroyFillInfo(pInfo->pFillInfo); 346: blockDataDestroy(pInfo->pRes); 347: pInfo->pRes = NULL;	
	污染路径: 无	
177	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 824
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 822: strncpy(col->field, json_string_value(value), TSDB_COL_NAME_LEN - 1); 823: } else if (0 == strcmp(key, "type")) { 824: col->type = json_integer_value(value); 825: } 826: }	
	污染路径: 无	
178	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 599
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 597: } 598: tag->type = type; 599: tag->length = JSON_FIXED_LENGTH; // json datatype is fixed length: 4096 600: return 0; 601: }	

	污染路径: expanded from macro 'JSON_FIXED_LENGTH'	
179	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 131
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 129: /* Reset hooks */ 130: global_hooks.allocate = malloc; 131: global_hooks.deallocate = free; 132: global_hooks.reallocate = realloc; 133: return;	
	污染路径: 无	
180	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 129
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 127: strcpy(obj.status, "online"); 128: obj.createTime = 1234567890; 129: obj.updateTime = 1234567900; 130: 131: // Encode	
	污染路径: 无	
181	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 245
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 243: { 244: struct params params; 245: params.foonly = 0; 246: params.foonly_default = random (); 247: argp_parse (&argp, argc, argv, 0, 0, ¶ms);	
	污染路径: 无	

182	缺陷发生位置: /source/libs/tdb/test/tdbBtreeToStackTest.cpp	行号: 63
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 61: } 62: 63: pMem->size = sizeof(*pMem) + size; 64: pMem->next = pPool->next; 65: pMem->prev = pPool;	
	污染路径: 无	
183	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 797
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 795: BArray* pArray = (BArray *)benchCalloc(1, sizeof(BArray), true); 796: 797: pArray->size = 0; 798: pArray->pData = benchCalloc(size, elemSize, true); 799:	
	污染路径: 无	
184	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 604
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 602: FSE_decode_t* const cell = (FSE_decode_t*)(ptr) + 1; /* because dt is unsigned */ 603: 604: DTableH->tableLog = 0; 605: DTableH->fastMode = 0; 606:	
	污染路径: 无	
185	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 1089

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1087: memset(¶ms, 0, sizeof(params)); 1088: params.d = 8; 1089: params.steps = 4; 1090: /* Default to level 6 since no compression level information is available */ 1091: params.zParams.compressionLevel = 6;	
	污染路径: 无	
186	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1293
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1291: DTableH->fastMode = 0; 1292: 1293: cell->newState = 0; 1294: cell->symbol = symbolValue; 1295: cell->nbBits = 0;	
	污染路径: 无	
187	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 202
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 200: void* startPtr, size_t dstCapacity) 201: { 202: bitC->bitContainer = 0; 203: bitC->bitPos = 0; 204: bitC->startPtr = (char*)startPtr;	
	污染路径: 无	
188	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 401
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	631: for (s=0; s<=maxSymbolValue; s++) 632: {	
	污染路径: 无	
192	缺陷发生位置: /source/util/src/tlru_cache.c	行号: 700
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 698: 699: int numShards = 1 << numShardBits; 700: cache->shards = taosMemoryCalloc(numShards, sizeof(SLRUCacheShard)); 701: if (!cache->shards) { 702: taosMemoryFree(cache);	
	污染路径: 无	
193	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1289
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1287: FSE_decode_t* const cell = (FSE_decode_t*)(ptr) + 1; /* because dt is unsigned */ 1288: 1289: DTableH->tableLog = 0; 1290: DTableH->fastMode = 0; 1291:	
	污染路径: 无	
194	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1659
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1657: 1658: (void)memset(&args, 0, sizeof(struct __sysctl_args)); 1659: args.name = name; 1660: args.nlen = sizeof(name) / sizeof(name[0]); 1661: args.oldval = &old_usespid;	

	污染路径: 无	
195	缺陷发生位置: /test/new_test_framework/script/api/stmt-crash.c	行号: 257
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 255: stmt = taos_stmt_init(taos); 256: taos_stmt_prepare(stmt, "SELECT * FROM m1 WHERE v1 > ? AND v2 < ?", 0); 257: v.v1 = 5; 258: v.v2 = 15; 259: taos_stmt_bind_param(stmt, params + 2);	
	污染路径: 无	
196	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 305
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 303: g_arguments->reqPerReq, g_arguments->prepared_rand, 304: g_arguments->prepared_rand); 305: g_arguments->reqPerReq = g_arguments->prepared_rand; 306: } 307:	
	污染路径: 无	
197	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 640
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 638: write_meta->start_ts = stb->startTimestamp; 639: write_meta->end_ts = stb->startTimestamp + stb->timestamp_step * stb->insertRows; 640: write_meta->ts_step = stb->timestamp_step * stb->insertRows; 641: write_meta->interlace_step = stb-	

	>timestamp_step * stb->interlaceRows; 642:	
	污染路径: 无	
198	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 876
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 874: case TSDB_DATA_TYPE_SMALLINT: 875: // pBase[bindIdx].buffer_length = sizeof(int16_t); 876: pBase[bindIdx].buffer = data->smallData + rowIdx; 877: pBase[bindIdx].length = NULL; 878: pBase[bindIdx].is_null = data->isNull ? (data->isNull + rowIdx) : NULL;	
	污染路径: 无	
199	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 212
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 210: { buffer_t buffer; 211: void* const start = ZSTD_malloc(bSize, bufPool->cMem); 212: buffer.start = start; /* note : start can be NULL if malloc fails ! */ 213: buffer.capacity = (start==NULL) ? 0 : bSize; 214: if (start==NULL) {	
	污染路径: 无	
200	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	行号: 151
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 149: fs->wmargin = wmargin; 150: fs->point_col = 0; 151: fs->point_offs = 0;	

	152: 153: fs->buf = (char *) malloc (INIT_BUF_SIZE);	
	污染路径: 无	
201	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 2248
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 2246: if (pMemRow) { 2247: *ppRow = pMemRow; 2248: state->state = SMEMNEXTROW_NEXT; 2249: 2250: TAOS_RETURN(code);	
	污染路径: 无	
202	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1024
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1022: static void copyTimestampToArg(char *timeStr, bool isStartTime) { 1023: if (isStartTime) { 1024: g_args.start_time = strtoll((const char *)timeStr, NULL, 10); 1025: } else { 1026: g_args.end_time = strtoll((const char *)timeStr, NULL, 10);	
	污染路径: 无	
203	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 275
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 273: worker->state = EVA_WORKER_STATE_ACTIVE; 274: } else { 275: worker->runningTask->state = EVA_TASK_STATE_RUNNING;	

	<pre> 276: break; 277: } </pre>	
	污染路径: 无	
204	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 186
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 184: } 185: memset(serv_addr, 0, sizeof(struct sockaddr_in)); 186: serv_addr->sin_family = AF_INET; 187: serv_addr->sin_port = htons(port); 188: </pre>	
	污染路径: expanded from macro 'AF_INET' expanded from macro 'PF_INET'	
205	缺陷发生位置: /source/util/src/tlruCache.c	行号: 351
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 349: shard->strictCapacity = strict; 350: shard->highPriPoolRatio = highPriPoolRatio; 351: shard->highPriPoolCapacity = 0; 352: 353: shard->usage = 0; </pre>	
	污染路径: 无	
206	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 656
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 654: } 655: if (field->max > (RAND_MAX >> 1)) { 656: field->max = RAND_MAX >> 1; 657: } 658: intTmp = field->min; </pre>	

	污染路径: expanded from macro 'RAND_MAX'	
207	缺陷发生位置: /source/util/src/tcompare.c	行号: 1383
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 1381: } 1382: taosHashSetFreeFp(sRegexCache.regexHash, regexCacheFree); 1383: sRegexCache.regexCacheTmr = taosTmrInit(0, 0, 0, "REGEXCACHE"); 1384: if (sRegexCache.regexCacheTmr == NULL) { 1385: uError("failed to create regex cache check timer"); </pre>	
	污染路径: 无	
208	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 616
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 614: // current.c.json exists 615: 616: fs->etype = TSDB_FEDIT_COMMIT; 617: if (rollback) { 618: code = abort_edit(fs); </pre>	
	污染路径: 无	
209	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 269
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 267: strncpy(configDir, argv[++i], PATH_MAX); 268: } else if (strcmp(argv[i], "-g") == 0) { 269: g_stConflInfo.showMsgFlag = atol(argv[++i]); 270: } else if (strcmp(argv[i], "-r") == 0) { 271: g_stConflInfo.showRowFlag = atol(argv[++i]); </pre>	
	污染路径: 无	

210	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	行号: 395
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 393: 394: /* Remember that we've scanned as far as the end of the buffer. */ 395: fs->point_offs = fs->p - fs->buf; 396: } 397:	
	污染路径: 无	
211	缺陷发生位置: /tools/taos-tools/src/benchTmq.c	行号: 342
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 340: 341: tmqThreadInfo * pThreadInfo = infos + i; 342: pThreadInfo->totalMsgs = 0; 343: pThreadInfo->totalRows = 0; 344: pThreadInfo->id = i;	
	污染路径: 无	
212	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 609
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 607: cell->newState = 0; 608: cell->symbol = symbolValue; 609: cell->nbBits = 0; 610: 611: return 0;	
	污染路径: 无	
213	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1475

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1473: { FSEv06_DTableHeader DTableH; 1474: DTableH.tableLog = (U16)tableLog; 1475: DTableH.fastMode = 1; 1476: { S16 const largeLimit= (S16)(1 << (tableLog-1)); 1477: U32 s;	
	污染路径: 无	
214	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v02.c	1292
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1290: DTableH->fastMode = 0; 1291: 1292: cell->newState = 0; 1293: cell->symbol = symbolValue; 1294: cell->nbBits = 0;	
	污染路径: 无	
215	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbSnapshot.c	145
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 143: 144: // tomb 145: config.type = TSDB_ITER_TYPE_DATA_TOMB; 146: config.dataReader = reader->dataReader; 147:	
	污染路径: 无	
216	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/dictBuilder/cover.c	814
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	812: if (!best->dict) { 813: best->compressedSize = ERROR(GENERIC); 814: best->dictSize = 0; 815: return; 816: }	
	污染路径: 无	
217	缺陷发生位置: /source/dnode/vnode/src/bse/bseMgt.c	行号: 588
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 586: int32_t lino = 0; 587: 588: pBatchMgt->pBse = pBse; 589: 590: pBatchMgt->pBatchList = taosArrayInit(2, sizeof(SBseBatch *));	
	污染路径: 无	
218	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 724
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 722: QUERY_CHECK_CODE(code, lino, _end); 723: if (block->info.id.groupId == grpSortOpInfo->currGroupId) { 724: grpSortOpInfo->childOpStatus = CHILD_OP_SAME_GROUP; 725: *ppBlock = block; 726: } else {	
	污染路径: 无	
219	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 1195
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1193: TAOS_CHECK_GOTO(transInitBuffer(&conn->readBuf),	

	NULL, _failed); 1194: 1195: conn->hostThrd = pThrd; 1196: conn->seq = 0; 1197:	
	污染路径: 无	
220	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 529
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 527: 528: hol->entries = malloc (sizeof (struct hol_entry) * hol->num_entries); 529: hol->short_options = malloc (num_short_options + 1); 530: 531: assert (hol->entries && hol->short_options);	
	污染路径: 无	
221	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 105
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 103: int8_t c9; 104: char c10[32]; 105: } v = {0}; 106: TAOS_BIND params[11]; 107: params[0].buffer_type = TSDB_DATA_TYPE_TIMESTAMP;	
	污染路径: 无	
222	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 130
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	128: { 129: /* Reset hooks */ 130: global_hooks.allocate = malloc; 131: global_hooks.deallocate = free; 132: global_hooks.reallocate = realloc;	
	污染路径: 无	
223	缺陷发生位置: /docs/examples/c/prepare.c	行号: 202
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 200: stmt = taos_stmt_init(taos); 201: taos_stmt_prepare(stmt, "SELECT * FROM m1 WHERE v1 > ? AND v2 < ?", 0); 202: v.v1 = 5; 203: v.v2 = 15; 204: taos_stmt_bind_param(stmt, params + 2);	
	污染路径: 无	
224	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 189
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 187: gzipStream.avail_in = (uLong)srcLen; 188: gzipStream.next_out = (Bytef*)pDest; 189: gzipStream.avail_out = (uLong)(destLen); 190: 191: while (gzipStream.avail_in != 0 && gzipStream.total_out < (uLong)(destLen)) {	
	污染路径: 无	
225	缺陷发生位置: /source/libs/parser/src/parInsertSql.c	行号: 4846
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 4844: parser->delimiter = CSV_DEFAULT_DELIMITER; 4845: parser->quote = CSV_QUOTE_SINGLE; // Default to	

	single quote for TDengine compatibility 4846: parser->escape = CSV_ESCAPE_CHAR; 4847: parser->allowNewlineInField = true; 4848:	
	污染路径: expanded from macro 'CSV_ESCAPE_CHAR'	
226	缺陷发生位置: /docs/examples/c/prepare.c	行号: 179
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 177: params[j].is_null = ((i == j) ? &is_null : 0); 178: } 179: v.b = (int8_t)i % 2; 180: v.v1 = (int8_t)i; 181: v.v2 = (int16_t)(i * 2);	
	污染路径: 无	
227	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 1139
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1137: code = tsdbSttFileReaderOpen(pStt->fname, &readerConfig, &reader); 1138: if (code != 0) { 1139: result->action = TSDB_REPAIR_ACTION_DROP; 1140: tsdbRepairSetReason(&result->reason, "open_stt_reader_failed"); 1141: tsdbWarn("vgId:%d fid:%d level:%d failed to open stt file:%s, downgrade action to %s since %s", vgId, pFileSet->fid,	
	污染路径: 无	
228	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 94
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 92: params[i+0].buffer_type = TSDB_DATA_TYPE_TIMESTAMP;	

	93: params[i+0].buffer_length = sizeof(int64_t); 94: params[i+0].buffer = &v->ts[j*rowsOfPerColumn]; 95: params[i+0].length = NULL; 96: params[i+0].is_null = no_null;	
	污染路径: 无	
229	缺陷发生位置: /source/libs/tdb/test/tdbBtreeToStackTest.cpp	行号: 25
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 23: 24: pPool->prev = pPool->next = pPool; 25: pPool->size = 0; 26: 27: return pPool;	
	污染路径: 无	
230	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 731
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 729: void generateErrorSQL(BindData *data, int32_t tblIdx) { 730: int32_t len = 0; 731: data->sql = taosMemoryCalloc(1, 1024); 732: 733: switch(tblIdx) {	
	污染路径: 无	
231	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 1010
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1008: return ERROR(GENERIC); 1009: } 1010: data->ctx = &ctx; 1011: data->best = &best; 1012: data->dictBufferCapacity = dictBufferCapacity;	

	污染路径: 无	
232	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 76
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 74: static struct argp argp2 = 75: { 76: opt2, NULL, "args doc2", "doc2", children2, NULL, NULL 77: }; 78:	
	污染路径: 无	
233	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 39
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 37: char msg[256] = {0}; 38: SSmlMsgBuf msgBuf; 39: msgBuf.buf = msg; 40: msgBuf.len = 256; 41: SSmlLineInfo elements = {0};	
	污染路径: 无	
234	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 706
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 704: DStatePtr->state = BIT_readBits(bitD, DTableH.tableLog); 705: BIT_reloadDStream(bitD); 706: DStatePtr->table = dt + 1; 707: } 708:	
	污染路径:	

	无	
235	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v04.c	行号： 1282
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 1280: 1281: DTableH->tableLog = 0; 1282: DTableH->fastMode = 0; 1283: 1284: cell->newState = 0;	
	污染路径： 无	
236	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号： 343
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 341: 342: if (idstr != NULL) { 343: p->idstr = taosStrdup(idstr); 344: TSDB_CHECK_NULL(p->idstr, code, lino, _end, terrno); 345: }	
	污染路径： 无	
237	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号： 509
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 507: assert (hol); 508: 509: hol->num_entries = 0; 510: hol->clusters = 0; 511:	
	污染路径： 无	
238	缺陷发生位置： /test/new_test_framework/script/api/stmt2-test.c	行号： 747

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 745: void generateErrorSQL(BindData *data, int32_t tblIdx) { 746: int32_t len = 0; 747: data->sql = taosMemoryCalloc(1, 1024); 748: 749: switch (tblIdx) {	
	污染路径: 无	
239	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v04.c	1284
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1282: DTableH->fastMode = 0; 1283: 1284: cell->newState = 0; 1285: cell->symbol = symbolValue; 1286: cell->nbBits = 0;	
	污染路径: 无	
240	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/common/fse_decompress.c	110
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 108: { FSE_DTableHeader DTableH; 109: DTableH.tableLog = (U16)tableLog; 110: DTableH.fastMode = 1; 111: { S16 const largeLimit= (S16)(1 << (tableLog-1)); 112: U32 s;	
	污染路径: 无	
241	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchCsv.c	127
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 125: return -1;	

	<pre> 126: } 127: g_arguments->output_path = g_arguments- >output_path_buf; 128: } 129: </pre>	
	污染路径: 无	
242	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 118
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 116: symbolNext[s] = 1; 117: } else { 118: if (normalizedCounter[s] >= largeLimit) DTableH.fastMode=0; 119: symbolNext[s] = normalizedCounter[s]; 120: } } } </pre>	
	污染路径: 无	
243	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 398
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 396: bitD->start = (const char*)srcBuffer; 397: bitD->ptr = bitD->start; 398: bitD->bitContainer = *(const BYTE*)(bitD->start); 399: switch(srcSize) 400: { </pre>	
	污染路径: 无	
244	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 214
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 212: v.c6[j] = (char)('a'); 213: } 214: v.c7 = 32767; </pre>	

	215: v.c8 = 127; 216: v.c9 = 1;	
	污染路径: 无	
245	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 253
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 251: return terrno; 252: } 253: pEncoder->pBuf = pNewBuf; 254: } 255:	
	污染路径: 无	
246	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 5066
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 5064: 5065: pInfo->accountId = pScanPhyNode->accountId; 5066: pInfo->pUser = taosStrdup((void*)pUser); 5067: QUERY_CHECK_NULL(pInfo->pUser, code, lino, _error, terrno); 5068: pInfo->privInfo = pScanPhyNode->privInfo;	
	污染路径: 无	
247	缺陷发生位置: /examples/c/prepare.c	行号: 203
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 201: taos_stmt_prepare(stmt, "SELECT * FROM m1 WHERE v1 > ? AND v2 < ?", 0); 202: v.v1 = 5; 203: v.v2 = 15; 204: taos_stmt_bind_param(stmt, params + 2); 205: if (taos_stmt_execute(stmt) != 0) {	

	污染路径: 无	
248	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 532
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 530: task->cancel = cancel; 531: task->arg = arg; 532: task->state = EVA_TASK_STATE_WAITTING; 533: task->numWait = 0; 534: (void)taosThreadCondInit(&task->waitCond, NULL);	
	污染路径: 无	
249	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 1122
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1120: tsDumpInfo *dump_info = (tsDumpInfo *)param; 1121: if (num_of_rows > 0) { 1122: dump_info->numOfRows = num_of_rows; 1123: if (dump_info->numOfAllRows < dump_info->resShowMaxNum) { 1124: if (dump_info->vertical) {	
	污染路径: 无	
250	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1290
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1288: 1289: DTableH->tableLog = 0; 1290: DTableH->fastMode = 0; 1291: 1292: cell->newState = 0;	
	污染路径: 无	

251	缺陷发生位置: /source/os/src/osSocket.c	行号: 191
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 189: 190: bzero((char *)&serverAdd, sizeof(serverAdd)); 191: serverAdd.sin_family = AF_INET; 192: #ifdef WINDOWS 193: serverAdd.sin_addr.s_addr = INADDR_ANY;	
	污染路径: expanded from macro 'AF_INET' expanded from macro 'PF_INET'	
252	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 279
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 277: params[i+0].buffer_type = TSDB_DATA_TYPE_TIMESTAMP; 278: params[i+0].buffer_length = sizeof(int64_t); 279: params[i+0].buffer = &v->ts[j*rowsOfPerColumn]; 280: params[i+0].length = NULL; 281: params[i+0].is_null = no_null;	
	污染路径: 无	
253	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 169
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 167: if (pSortNode->calcGroupId) { 168: int32_t keyLen; 169: SSortOpGroupIdCalc* pGroupIdCalc = pInfo->pGroupIdCalc = taosMemoryCalloc(1, sizeof(SSortOpGroupIdCalc)); 170: if (!pGroupIdCalc) { 171: code = terrno;	
	污染路径: 无	

254	缺陷发生位置： /utils/test/c/tmqDemo.c	行号： 170
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 168: } 169: 170: g_stConflInfo.totalRowsOfT2 = g_stConflInfo.totalRowsOfPerTbl * g_stConflInfo.ratio; 171: 172: #if 0	
	污染路径： 无	
255	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v05.c	行号： 1385
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 1383: 1384: DTableH->tableLog = 0; 1385: DTableH->fastMode = 0; 1386: 1387: cell->newState = 0;	
	污染路径： 无	
256	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v05.c	行号： 1384
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 1382: FSEv05_decode_t* const cell = (FSEv05_decode_t*)dPtr; 1383: 1384: DTableH->tableLog = 0; 1385: DTableH->fastMode = 0; 1386:	
	污染路径： 无	
257	缺陷发生位置： /tools/taos-tools/src/benchInsertMix.c	行号： 225

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 223: 224: if(stb->partialColNum > 0 && stb->partialColNum < MAX_BATCOLS) { 225: info->nBatCols = stb->partialColNum; 226: int j = 0; 227: for (int i = stb->partialColFrom; i < stb->partialColFrom + stb->partialColNum; i++) {	
	污染路径: 无	
258	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchCommandOpt.c	58
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 56: type[pmatch[1].rm_eo - pmatch[1].rm_so] = '\0'; 57: strncpy(length, token + pmatch[2].rm_so, pmatch[2].rm_eo - pmatch[2].rm_so); 58: field->type = convertStringToDatatype(type, 0, NULL); 59: field->length = atoi(length); 60: regfree(®ex);	
259	污染路径: 无	
	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchJsonOpt.c	1046
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1044: } 1045: superTable->autoTblCreating = false; 1046: superTable->batchTblCreatingNum = DEFAULT_CREATE_BATCH; 1047: superTable->batchTblCreatingNumbers = NULL; 1048: superTable->batchTblCreatingIntervals = NULL;	
260	污染路径: expanded from macro 'DEFAULT_CREATE_BATCH'	
	缺陷发生位置:	行号:

	/contrib/TSZ/zstd/dictBuilder/cover.c		739
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: 737: (void)ZSTD_pthread_mutex_init(&best->mutex, NULL); 738: (void)ZSTD_pthread_cond_init(&best->cond, NULL); 739: best->liveJobs = 0; 740: best->dict = NULL; 741: best->dictSize = 0;		
	污染路径: 无		
261	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp		行号: 61
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: 59: } 60: 61: pMem->size = sizeof(*pMem) + size; 62: pMem->next = pPool->next; 63: pMem->prev = pPool;		
	污染路径: 无		
262	缺陷发生位置: /test/new_test_framework/script/api/apitest.c		行号: 226
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: 224: params[0].buffer_type = TSDB_DATA_TYPE_TIMESTAMP; 225: params[0].buffer_length = sizeof(v.ts); 226: params[0].buffer = &v.ts; 227: params[0].length = &bintLen; 228: params[0].is_null = NULL;		
	污染路径: 无		
263	缺陷发生位置: /tools/taos-tools/deps/toolsc.Json/src/toolsc.Json.c		行号: 142
	步骤描述: 结构体变量的初始化要一致.		
	代码片段:		

	140: } 141: 142: global_hooks.deallocate = free; 143: if (hooks->free_fn != NULL) 144: {	
	污染路径: 无	
264	缺陷发生位置: /utlis/test/c/tmqSim.c	行号: 271
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 269: g_stConflInfo.showMsgFlag = atol(argv[++i]); 270: } else if (strcmp(argv[i], "-r") == 0) { 271: g_stConflInfo.showRowFlag = atol(argv[++i]); 272: } else if (strcmp(argv[i], "-s") == 0) { 273: g_stConflInfo.saveRowFlag = atol(argv[++i]);	
	污染路径: 无	
265	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 497
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 495: 496: if (sem->count <= 0) { 497: struct timespec ts = {0}; 498: if (clock_gettime(CLOCK_MONOTONIC, &ts) == -1) { 499: code = TAOS_SYSTEM_ERROR(ERRNO);	
	污染路径: 无	
266	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 384
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 382: pCol->dtype = taosRand() % TSDB_DATA_TYPE_MAX; 383: pCol->len = qptGetColumnRandLen(pCol->dtype); 384: pCol->inUse = 0; 385: pCol->hasIndex = COLUMN_TYPE_COLUMN ==	

	colType ? false : QPT_RAND_BOOL_V; 386: pCol->isPrimTs = false;	
	污染路径: 无	
267	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 822
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 820: pInfo->currGroupId = pInfo->prefetchedSortInput->info.id.groupId; 821: pInfo->currBaseGId = pInfo->prefetchedSortInput->info.id.baseGId; 822: pInfo->childOpStatus = CHILD_OP_NEW_GROUP; 823: code = beginSortGroup(pOperator); 824: QUERY_CHECK_CODE(code, lino, _end);	
	污染路径: 无	
268	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 263
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 261: obj.url = (char*)malloc(obj.urlLen); 262: snprintf(obj.url, obj.urlLen, "node%d:6050", i); 263: obj.createTime = 1000000 + i; 264: obj.updateTime = 2000000 + i; 265:	
	污染路径: 无	
269	缺陷发生位置: /source/util/src/tlruCache.c	行号: 114
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 112: static int taosLRUEntryTableInit(SLRUEntryTable *table, int maxUpperHashBits) { 113: table->lengthBits = 16; 114: table->list = taosMemoryCalloc(1 << table->lengthBits, sizeof(SLRUEntry *));	

	115: if (!table->list) { 116: TAOS_RETURN(terrno);	
	污染路径: 无	
270	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 639
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 637: write_meta->stb = stb; 638: write_meta->start_ts = stb->startTimestamp; 639: write_meta->end_ts = stb->startTimestamp + stb->timestamp_step * stb->insertRows; 640: write_meta->ts_step = stb->timestamp_step * stb->insertRows; 641: write_meta->interlace_step = stb-> >timestamp_step * stb->interlaceRows;	
	污染路径: 无	
271	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttDaemon.c	行号: 222
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 220: } 221: msgInfo->rpcType = MQTTD_RPC_RETRIVE_FUNC; 222: msgInfo->param = mqtt; 223: if (uv_sem_init(&msgInfo->resultSem, 0) != 0) { 224: taosMemoryFree(msgInfo);	
	污染路径: 无	
272	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 408
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 406: TSDB_CHECK_CODE(code, lino, _error); 407: 408: pIter->pTableIter = pTableIter; 409: pIter->index++;	

	410: return code;	
	污染路径: 无	
273	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 110
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 108: } ThreadParam; 109: 110: static struct timeval global_start_time = {.tv_sec = 0, .tv_usec = 0}; 111: static struct timeval global_end_time = {.tv_sec = 0, .tv_usec = 0}; 112:	
	污染路径: 无	
274	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 119
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 117: bufPool->bufferSize = 64 KB; 118: bufPool->totalBuffers = maxNbBuffers; 119: bufPool->nbBuffers = 0; 120: bufPool->cMem = cMem; 121: return bufPool;	
	污染路径: 无	
275	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 218
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 216: 217: static const struct argp argp_default_argp = 218: {argp_default_options, &argp_default_parser, NULL, NULL, NULL, NULL, "libc"}; 219:	

	220:	
	污染路径: 无	
276	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 163
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 161: g_stConflInfo.showMsgFlag = atol(argv[++i]); 162: } else if (strcmp(argv[i], "-sim") == 0) { 163: g_stConflInfo.simCase = atol(argv[++i]); 164: } else { 165: printf("%s unknow para: %s %s", GREEN, argv[++i], NC);	
	污染路径: 无	
277	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp1.c	行号: 92
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 90: static struct argp argp = 91: { 92: test_options, parse_opt 93: }; 94:	
	污染路径: 无	
278	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 634
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 632: static int csvInitWriteMeta(SDataBase* db, SSuperTable* stb, CsvWriteMeta* write_meta) { 633: write_meta->naming_type = csvGetFileNamingType(stb); 634: write_meta->total_threads = 1; 635: write_meta->csv_header_length = 0;	

	636: write_meta->db = db;	
	污染路径: 无	
279	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 895
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 893: DStatePtr->state = BIT_readBits(bitD, DTableH.tableLog); 894: BIT_reloadDStream(bitD); 895: DStatePtr->table = dt + 1; 896: } 897:	
	污染路径: 无	
280	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 550
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 548: ctx->samples = samples; 549: ctx->samplesSizes = samplesSizes; 550: ctx->nbSamples = nbSamples; 551: /* Partial suffix array */ 552: ctx->suffixSize = totalSamplesSize - MAX(d, sizeof(U64)) + 1;	
	污染路径: 无	
281	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 59
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 57: strncpy(length, token + pmatch[2].rm_so, pmatch[2].rm_eo - pmatch[2].rm_so); 58: field->type = convertStringToDatatype(type, 0, NULL); 59: field->length = atoi(length); 60: regfree(®ex);	

	61: goto SET_PROPS;	
	污染路径: 无	
282	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号: 198
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 196: case 'f': 197: if (arg) 198: params->foonly = atoi (arg); 199: else 200: params->foonly = params->foonly_default;	
	污染路径: 无	
283	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 88
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 86: return false; 87: } 88: _size = nRead; 89: _s = fstSliceCreate((uint8_t*)buf, _size); 90: _fst = fstCreate(&_s);	
	污染路径: 无	
284	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 503
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 501: char buf[64] = {0}; 502: SSmlMsgBuf msg = {0}; 503: msg.buf = buf; 504: msg.len = 64; 505: kv.value = "3.2e-900";	
	污染路径:	

	无	
285	缺陷发生位置： /tools/taos-tools/src/benchTmq.c	行号： 343
	步骤描述： 结构体变量的初始化要一致。	
	代码片段： 341: tmqThreadInfo * pThreadInfo = infos + i; 342: pThreadInfo->totalMsgs = 0; 343: pThreadInfo->totalRows = 0; 344: pThreadInfo->id = i; 345:	
	污染路径： 无	
286	缺陷发生位置： /source/client/src/clientTmq.c	行号： 519
	步骤描述： 结构体变量的初始化要一致。	
	代码片段： 517: } 518: taosMemoryFree(conf->user); 519: conf->user = tmp; 520: return TMQ_CONF_OK; 521: }	
	污染路径： 无	
287	缺陷发生位置： /docs/examples/c/prepare.c	行号： 72
	步骤描述： 结构体变量的初始化要一致。	
	代码片段： 70: char blob[80]; 71: int8_t varbin[16]; 72: } v = {0}; 73: 74: int32_t boolLen = sizeof(int8_t);	
	污染路径： 无	
288	缺陷发生位置： /source/dnode/vnode/src/bse/bseTable.c	行号： 2168
	步骤描述：	

	结构体变量的初始化要一致.	
	代码片段: 2166: return terrno; 2167: } 2168: p->pTableMeta = pMeta; 2169: 2170: p->blockCap = pMeta->blockCap;	
	污染路径: 无	
289	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.c	行号: 52
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 50: return terrno; 51: } 52: p->pBse = pBse; 53: p->pHashObj = taosHashInit(64, taosGetDefaultHashFunction(TSDB_DATA_TYPE_BIGINT), false, HASH_ENTRY_LOCK); 54: if (p->pHashObj == NULL) {	
	污染路径: 无	
290	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 108
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 106: // other 107: field->type = convertStringToDatatype(token, 0, NULL); 108: field->length = convertTypeToLength(field->type); 109: 110: SET_PROPS:	
	污染路径: 无	
291	缺陷发生位置: /source/libs/sync/test/syncRefTest.cpp	行号: 56
	步骤描述: 结构体变量的初始化要一致.	

	代码片段: 54: TD_ALWAYS_ASSERT(pObj != NULL); 55: 56: pObj->data = &g; 57: snprintf(pObj->name, sizeof(pObj->name), "%s", "hello"); 58:	
	污染路径: 无	
292	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 150
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 148: 149: // Set pBse pointer for encryption/decryption 150: pBlkWrapper->pBse = p->pBse; 151: 152: code = tableLoadBlock(p->pDataFile, pHandle, pBlkWrapper);	
293	污染路径: 无	
	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1109
	步骤描述: 结构体变量的初始化要一致.	
294	代码片段: 1107: pThreadInfo->end_table_to = i < mod ? tableFrom + div : tableFrom + div - 1; 1108: tableFrom = pThreadInfo->end_table_to + 1; 1109: pThreadInfo->tables_created = 0; 1110: debugPrint("div table by thread. i=%d from= %"PRId64" to=%"PRId64" ntable=%"PRId64"\n", i, pThreadInfo->start_table_from, 1111: pThreadInfo->end_table_to, pThreadInfo->ntables);	
	污染路径: 无	
294	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 901

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 899: bitD->start = (const char*)srcBuffer; 900: bitD->ptr = bitD->start; 901: bitD->bitContainer = *(const BYTE*)(bitD->start); 902: switch(srcSize) 903: {	
	污染路径: 无	
295	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchQuery.c	572
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 570: pthreadInfo->threadID = i; 571: pthreadInfo->start_table_from = tableFrom; 572: pthreadInfo->ntables = i < b ? a + 1 : a; 573: pthreadInfo->end_table_to = 574: i < b ? tableFrom + a : tableFrom + a - 1;	
	污染路径: 无	
296	缺陷发生位置:	行号:
	/tools/lastqps-test/taos_query_a_simple.c	299
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 297: params[i].taos = taos; 298: if (query_mode == 2) { 299: params[i].sql = g_sql_statements[i]; 300: } else { 301: params[i].sql = (const char*)sql;	
	污染路径: 无	
297	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	128
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 126: obj.status = (char*)malloc(obj.statusLen);	

	127: strcpy(obj.status, "online"); 128: obj.createTime = 1234567890; 129: obj.updateTime = 1234567900; 130:	
	污染路径: 无	
298	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshot.c	行号: 135
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 133: if (reader->dataReader) { 134: // data 135: config.type = TSDB_ITER_TYPE_DATA; 136: config.dataReader = reader->dataReader; 137:	
	污染路径: 无	
299	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 61
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 59: } 60: 61: pMem->size = sizeof(*pMem) + size; 62: pMem->next = pPool->next; 63: pMem->prev = pPool;	
	污染路径: 无	
300	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 216
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 214: v.c7 = 32767; 215: v.c8 = 127; 216: v.c9 = 1; 217: strcpy(v.c10, "一二三四五六七八"); 218: c10len=strlen(v.c10);	

	污染路径: 无	
301	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 23
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 21: 22: pPool->prev = pPool->next = pPool; 23: pPool->size = 0; 24: 25: return pPool;	
	污染路径: 无	
302	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 373
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 371: TSDB_CHECK_CODE(code, lino, _error); 372: 373: plter->pTablelter = pTablelter; 374: 375: plter->index++;	
	污染路径: 无	
303	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 5835
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 5833: TSDB_CHECK_NULL(pReader->status.pLDataalterArray, code, lino, _end, terrno); 5834: 5835: pReader->flag = READER_STATUS_SUSPEND; 5836: pReader->info.execMode = pCond->notLoadData ? READER_EXEC_ROWS : READER_EXEC_DATA; 5837: pReader->info.cacheSttStatis = pCond->cacheSttStatis;	

	污染路径: 无	
304	缺陷发生位置: /test/new_test_framework/script/api/stmt-crash.c	行号: 74
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 72: char blob[80]; 73: int8_t varbin[16]; 74: } v = {0}; 75: 76: int32_t boolLen = sizeof(int8_t);	
	污染路径: 无	
305	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 911
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 909: hol->entries = more->entries; 910: hol->short_options = more->short_options; 911: more->num_entries = 0; /* Mark MORE's fields as invalid. */ 912: } 913: else	
	污染路径: 无	
306	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 118
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 116: return terrno; 117: } 118: pEncoder->pBuf = pNewBuf; 119: pEncoder->allocSize = pEncoder->allocSize * 2; 120: }	
	污染路径: 无	

307	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 752
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 750: initOperatorCostInfo(pOperator); 751: 752: pMlExtInfo->pExtW = taosMemoryCalloc(1, sizeof(SExternalWindowOperator)); 753: if (!pMlExtInfo->pExtW) { 754: code = terrno;	
	污染路径: 无	
308	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 250
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 248: 249: static const struct argp argp_version_argp = 250: {argp_version_options, &argp_version_parser, NULL, NULL, NULL, NULL, "libc"}; 251: ? 252: /* Returns the offset into the getopt long options array LONG_OPTIONS of a	
	污染路径: 无	
309	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1291
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1289: 1290: DTableH->tableLog = 0; 1291: DTableH->fastMode = 0; 1292: 1293: cell->newState = 0;	
	污染路径: 无	

310	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 282
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 280: stbInfo->iface = TAOSC_IFACE; 281: } 282: g_arguments->iface = stbInfo->iface; 283: break; 284:	
	污染路径: 无	
311	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1291
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1289: 1290: /* Report an error for each rule that can never be reduced. */ 1291: for(rp=lemp->rule; rp; rp=rp->next) rp->canReduce = LEMON_FALSE; 1292: for(i=0; i<lemp->nstate; i++){ 1293: struct action *ap;	
	污染路径: 无	
312	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 161
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 159: g_stConflInfo.startTimestamp = atol(argv[++i]); 160: } else if (strcmp(argv[i], "-g") == 0) { 161: g_stConflInfo.showMsgFlag = atol(argv[++i]); 162: } else if (strcmp(argv[i], "-sim") == 0) { 163: g_stConflInfo.simCase = atol(argv[++i]);	
	污染路径: 无	
313	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 3065

	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 3063: param.joinType = JOIN_TYPE_INNER; 3064: param.subType = JOIN_STYPE_NONE; 3065: param.cond = TEST_ON_COND; 3066: param.asc = true; 3067: param.grpJoin = false;	
	污染路径: 无	
314	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v07.c	1503
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1501: symbolNext[s] = 1; 1502: } else { 1503: if (normalizedCounter[s] >= largeLimit) DTableH.fastMode=0; 1504: symbolNext[s] = normalizedCounter[s]; 1505: } } }	
	污染路径: 无	
315	缺陷发生位置:	行号:
	/source/libs/parser/test/parTestUtil.cpp	78
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 76: 77: void useDb(const string& acctId, const string& db) { 78: caseEnv_.acctId_ = acctId; 79: caseEnv_.db_ = db; 80: }	
	污染路径: 无	
316	缺陷发生位置:	行号:
	/source/libs/tdb/test/tdbPageDefragmentTest.cpp	23
	步骤描述: 结构体变量的初始化要一致.	
	代码片段:	

	21: 22: pPool->prev = pPool->next = pPool; 23: pPool->size = 0; 24: 25: return pPool;	
	污染路径: 无	
317	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1550
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1548: FSEv07_decode_t* const cell = (FSEv07_decode_t*)dPtr; 1549: 1550: DTableH->tableLog = 0; 1551: DTableH->fastMode = 0; 1552:	
	污染路径: 无	
318	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 111
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 109: 110: static struct timeval global_start_time = {.tv_sec = 0, .tv_usec = 0}; 111: static struct timeval global_end_time = {.tv_sec = 0, .tv_usec = 0}; 112: 113: static double calculate_qps(int count, double duration) {	
	污染路径: 无	
319	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 430
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 428: // link to list	

	429: if(prev) { 430: prev->next = node; 431: } 432:	
	污染路径: 无	
320	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 851
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 849: json_object_foreach(element, key, value) { 850: if (0 == strcmp(key, "version")) { 851: recordSchema->version = json_integer_value(value); 852: } else if (0 == strcmp(key, "name")) { 853: tstrncpy(recordSchema->stbName, json_string_value(value), RECORD_NAME_LEN - 1);	
	污染路径: 无	
321	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 5775
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 5773: capacity, pResBlock, idstr); 5774: TSDB_CHECK_CODE(code, lino, _end); 5775: pReader->step = EXTERNAL_ROWS_PREV; 5776: 5777: if (order == TSDB_ORDER_ASC) {	
	污染路径: 无	
322	缺陷发生位置: /utls/test/c/varbinary_test.c	行号: 422
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 420: int8_t b; 421: int8_t varbin[16]; 422: } v = {0};	

	423: 424: int32_t boolLen = sizeof(int8_t);	
	污染路径: 无	
323	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 605
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 603: 604: DTableH->tableLog = 0; 605: DTableH->fastMode = 0; 606: 607: cell->newState = 0;	
	污染路径: 无	
324	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 145
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 143: g_stConflInfo.numOfThreads = atoi(argv[++i]); 144: } else if (strcmp(argv[i], "-n") == 0) { 145: g_stConflInfo.numOfTables = atoll(argv[++i]); 146: } else if (strcmp(argv[i], "-v") == 0) { 147: g_stConflInfo.numOfVgroups = atoi(argv[++i]);	
	污染路径: 无	
325	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 1088
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1086: DEBUGLOG(3, "ZDICT_trainFromBuffer"); 1087: memset(¶ms, 0, sizeof(params)); 1088: params.d = 8; 1089: params.steps = 4; 1090: /* Default to level 6 since no compression level information is available */	

	污染路径: 无	
326	缺陷发生位置: /test/new_test_framework/script/api/stmtQuery.c	行号: 96
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 94: params[0].buffer_type = TSDB_DATA_TYPE_BINARY; 95: params[0].buffer_length = strlen(condBuf) + 1; 96: params[0].buffer = condBuf; 97: params[0].length = (int*)¶ms[0].buffer_length; 98: params[0].is_null = NULL;	
	污染路径: 无	
327	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttDaemon.c	行号: 410
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 408: void *inputBuf = taosMemoryRealloc(ctx->inputBuf, ctx->inputCap); 409: if (inputBuf) { 410: ctx->inputBuf = inputBuf; 411: buf->base = ctx->inputBuf + ctx->inputLen; 412: buf->len = ctx->inputCap - ctx->inputLen;	
	污染路径: 无	
328	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 256
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 254: g_stConflInfo.producers = 0; 255: 256: g_stConflInfo.nowTime = taosGetTimestampMs(); 257: 258: for (int32_t i = 1; i < argc; i++) {	
	污染路径: 无	
329	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchInsertMix.c		231
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: <pre> 229: } 230: } else { 231: info->nBatCols = RD(max) + 1; 232: // random select cnt elements from max 233: randomFillCols(info->batCols, max, info->nBatCols); </pre>		
	污染路径: expanded from macro 'RD'		
330	缺陷发生位置:		行号:
	/test/new_test_framework/script/api/batchprepare.c		867
	步骤描述: 结构体变量的初始化要一致.		
	代码片段: <pre> 865: case TSDB_DATA_TYPE_INT: 866: pBase[bindIdx].buffer_length = sizeof(int32_t); 867: pBase[bindIdx].buffer = data->intData + rowIdx; 868: pBase[bindIdx].length = NULL; 869: pBase[bindIdx].is_null = data->isNull ? (data->isNull + rowIdx) : NULL; </pre>		
	污染路径: 无		
	缺陷发生位置:		行号:
	/source/dnode/vnode/src/tsdb/tsdbFS2.c		629
	步骤描述: 结构体变量的初始化要一致.		
331	代码片段: <pre> 627: } else if (taosCheckExistFile(mCurrent)) { 628: // current.m.json exists 629: fs->etype = TSDB_FEDIT_MERGE; 630: code = abort_edit(fs); 631: TSDB_CHECK_CODE(code, lino, _exit); </pre>		
	污染路径: 无		
	缺陷发生位置:		行号:
	/source/libs/executor/src/tsort.c		337
332	步骤描述: 结构体变量的初始化要一致.		

	代码片段: 335: 336: if (idstr != NULL) { 337: pSortHandle->idStr = taosStrdup(idstr); 338: QUERY_CHECK_NULL(pSortHandle->idStr, code, lino, _err, terrno); 339: }	
	污染路径: 无	
333	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 3093
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 3091: param.joinType = JOIN_TYPE_INNER; 3092: param.subType = JOIN_STYPE_NONE; 3093: param.cond = TEST_FULL_COND; 3094: param.asc = true; 3095: param.grpJoin = false;	
	污染路径: 无	
334	缺陷发生位置: /source/libs/tdb/test/tdbExOVFLTest.cpp	行号: 23
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 21: 22: pPool->prev = pPool->next = pPool; 23: pPool->size = 0; 24: 25: return pPool;	
	污染路径: 无	
335	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 864
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 862: case TSDB_DATA_TYPE_BOOL: 863: // pBase[bindIdx].buffer_length = sizeof(bool);	

	864: pBase[bindIdx].buffer = data->boolData + rowIdx; 865: pBase[bindIdx].length = NULL; 866: pBase[bindIdx].is_null = data->isNull ? (data->isNull + rowIdx) : NULL;	
	污染路径: 无	
336	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 6374
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 6372: move to the MAIN scan and prepare for it 6373: */ 6374: pReader->step = EXTERNAL_ROWS_MAIN; 6375: pReader->currentStepDone = false; 6376:	
	污染路径: 无	
337	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 59
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 57: static struct argp argp4 = 58: { 59: opt4, NULL, "args doc4", "doc4", NULL, NULL, NULL 60: }; 61:	
	污染路径: 无	
338	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1663
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1661: args.oldval = &old_usespid; 1662: args.oldlenp = &old_len; 1663: args.newval = &new_usespid; 1664: args.newlen = new_len;	

	1665:	
	污染路径: 无	
339	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 219
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 217: } 218: } 219: fs->point_col = pad; 220: } 221:	
	污染路径: 无	
340	缺陷发生位置: /source/libs/parser/src/parInsertSql.c	行号: 4845
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 4843: // Set default CSV format 4844: parser->delimiter = CSV_DEFAULT_DELIMITER; 4845: parser->quote = CSV_QUOTE_SINGLE; // Default to single quote for TDengine compatibility 4846: parser->escape = CSV_ESCAPE_CHAR; 4847: parser->allowNewlineInField = true;	
	污染路径: expanded from macro 'CSV_QUOTE_SINGLE'	
341	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 1500
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1498: TSDB_CHECK_CODE(code, lino, _error); 1499: 1500: p->pSubMgt = retentionMgt; 1501: 1502: code = tableReaderOpen(timestamp, &p->pTableReader, retentionMgt->pReaderMgt);	

	污染路径: 无

6.2.1.4 WK4755 变量初始化禁止隐含依赖于系统的缺省值

缺陷数量： 59 个		
缺陷描述： 变量初始化禁止隐含依赖于系统的缺省值。		
修复意见： 无		
1	缺陷发生位置： /test/new_test_framework/script/api/stmt2-test.c	行号： 1181
	步骤描述： 禁止使用系统缺省值初始化变量 'row' .	
	代码片段： 1179: 1180: void bpFetchRows(TAOS_RES *result, bool print, int32_t *rows) { 1181: TAOS_ROW row; 1182: int num_fields = taos_num_fields(result); 1183: TAOS_FIELD *fields = taos_fetch_fields(result);	
	污染路径： 无	
2	缺陷发生位置： /contrib/TSZ/zstd/common/entropy_common.c	行号： 69
	步骤描述： 禁止使用系统缺省值初始化变量 'threshold' .	
	代码片段： 67: int nbBits; 68: int remaining; 69: int threshold; 70: U32 bitStream; 71: int bitCount;	
	污染路径： 无	
3	缺陷发生位置： /contrib/TSZ/zstd/common/fse_decompress.c	行号： 108

	步骤描述: 禁止使用系统缺省值初始化变量 'DTableH' .	
	代码片段: 106: 107: /* Init, lay down lowprob symbols */ 108: { FSE_DTableHeader DTableH; 109: DTableH.tableLog = (U16)tableLog; 110: DTableH.fastMode = 1;	
	污染路径: 无	
4	缺陷发生位置: /contrib/lemon/lemon.c	行号: 117
	步骤描述: 禁止使用系统缺省值初始化变量 'p' .	
	代码片段: 115: static void *lemon_realloc(void *pOld, size_t nNew){ 116: void *pNew; 117: MemChunk *p; 118: if(pOld==0) return lemon_malloc(nNew); 119: p = (MemChunk*)pOld;	
	污染路径: 无	
5	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 379
	步骤描述: 禁止使用系统缺省值初始化变量 'systemTime' .	
	代码片段: 377: 378: static int64_t taosGetTimestampUs() { 379: struct timeval systemTime; 380: taosGetTimeOfDay(&systemTime); 381: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec;	
	污染路径: 无	
6	缺陷发生位置: /contrib/lemon/lemon.c	行号: 116
	步骤描述: 禁止使用系统缺省值初始化变量 'pNew' .	
	代码片段:	

	114: } 115: static void *lemon_realloc(void *pOld, size_t nNew){ 116: void *pNew; 117: MemChunk *p; 118: if(pOld==0) return lemon_malloc(nNew);	
	污染路径: 无	
7	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/getopt.h	行号: 78
	步骤描述: 禁止使用系统缺省值初始化变量 'opterr' .	
	代码片段: 76: for unrecognized options. */ 77: 78: extern int opterr; 79: 80: /* Set to an option character which was unrecognized. */	
	污染路径: 无	
8	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 218
	步骤描述: 禁止使用系统缺省值初始化变量 'new_text' .	
	代码片段: 216: help_filter (int key, const char *text, void *input) 217: { 218: char *new_text; 219: struct params *params = input; 220:	
	污染路径: 无	
9	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 230
	步骤描述: 禁止使用系统缺省值初始化变量 'U1n' .	
	代码片段: 228: const int *p1, const int *p2, 229: int depth) {	

	230: const unsigned char *U1, *U2, *U1n, *U2n; 231: 232: for(U1 = T + depth + *p1,	
	污染路径: 无	
10	缺陷发生位置: /contrib/lemon/lemon.c	行号: 171
	步骤描述: 禁止使用系统缺省值初始化变量 'j' .	
	代码片段: 169: } 170: static int lemon_vsprintf(char *str, const char *zFormat, va_list ap){ 171: int i, j, k, c; 172: int nUsed = 0; 173: const char *z;	
	污染路径: 无	
11	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 195
	步骤描述: 禁止使用系统缺省值初始化变量 'y' .	
	代码片段: 193: int 194: ss_isqrt(int x) { 195: int y, e; 196: 197: if(x >= (SS_BLOCKSIZE * SS_BLOCKSIZE)) { return SS_BLOCKSIZE; }	
	污染路径: 无	
12	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 195
	步骤描述: 禁止使用系统缺省值初始化变量 'e' .	
	代码片段: 193: int 194: ss_isqrt(int x) { 195: int y, e; 196:	

	197: if(x >= (SS_BLOCKSIZE * SS_BLOCKSIZE)) { return SS_BLOCKSIZE; }	
	污染路径: 无	
13	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 638
	步骤描述: 禁止使用系统缺省值初始化变量 'words' .	
	代码片段: 636: std::array<Word, sizeof(Int) / sizeof(Word)> to_words(Int x) noexcept 637: { 638: std::array<Word, sizeof(Int) / sizeof(Word)> words; 639: std::memcpy(&words, &x, sizeof(x)); 640: return words;	
	污染路径: 无	
14	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号: 168
	步骤描述: 禁止使用系统缺省值初始化变量 'buf' .	
	代码片段: 166: popt (int key, char *arg) 167: { 168: char buf[10]; 169: if (isprint (key)) 170: sprintf (buf, "%c", key);	
	污染路径: 无	
15	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 564
	步骤描述: 禁止使用系统缺省值初始化变量 'p' .	
	代码片段: 562: constexpr int num_words = sizeof(uint<N>) / sizeof(uint64_t); 563: 564: uint<N> p; 565: auto pw = as_words(p);	

	566: auto uw = as_words(u);	
	污染路径: 无	
16	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/getopt.h	行号: 59
	步骤描述: 禁止使用系统缺省值初始化变量 'optarg' .	
	代码片段: 57: each non-option ARGV-element is returned here. */ 58: 59: extern char *optarg; 60: 61: /* Index in ARGV of the next element to be scanned.	
	污染路径: 无	
17	缺陷发生位置: /contrib/lemon/lemon.c	行号: 171
	步骤描述: 禁止使用系统缺省值初始化变量 'i' .	
	代码片段: 169: } 170: static int lemon_vsprintf(char *str, const char *zFormat, va_list ap){ 171: int i, j, k, c; 172: int nUsed = 0; 173: const char *z;	
	污染路径: 无	
18	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 129
	步骤描述: 禁止使用系统缺省值初始化变量 'i' .	
	代码片段: 127: U32 s, position = 0; 128: for (s=0; s<maxSV1; s++) { 129: int i; 130: for (i=0; i<normalizedCounter[s]; i++) { 131: tableDecode[position].symbol = (FSE_FUNCTION_TYPE)s;	

	污染路径: 无	
19	缺陷发生位置: /contrib/TSZ/zstd/common/threading.c	行号: 18
	步骤描述: 禁止使用系统缺省值初始化变量 'g_ZSTD_threading_useles_symbol' .	
	代码片段: 16: 17: /* create fake symbol to avoid empty trnaslation unit warning */ 18: int g_ZSTD_threading_useles_symbol; 19: 20: #if defined(ZSTD_MULTITHREAD) && defined(_WIN32)	
	污染路径: 无	
20	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1165
	步骤描述: 禁止使用系统缺省值初始化变量 'row' .	
	代码片段: 1163: 1164: void bpFetchRows(TAOS_RES *result, bool print, int32_t *rows) { 1165: TAOS_ROW row; 1166: int num_fields = taos_num_fields(result); 1167: TAOS_FIELD *fields = taos_fetch_fields(result);	
	污染路径: 无	
21	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 394
	步骤描述: 禁止使用系统缺省值初始化变量 'systemTime' .	
	代码片段: 392: 393: static int64_t taosGetTimestampMs() { 394: struct timeval systemTime; 395: taosGetTimeOfDay(&systemTime); 396: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec / 1000;	

	污染路径: 无	
22	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 230
	步骤描述: 禁止使用系统缺省值初始化变量 'U1' .	
	代码片段: 228: const int *p1, const int *p2, 229: int depth) { 230: const unsigned char *U1, *U2, *U1n, *U2n; 231: 232: for(U1 = T + depth + *p1,	
	污染路径: 无	
23	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 70
	步骤描述: 禁止使用系统缺省值初始化变量 'bpTs' .	
	代码片段: 68: char *bpTbPrefix = "t"; 69: int32_t bpDefaultStbId = 1; 70: int64_t bpTs; 71: 72: //char *operatorList[] = {">", ">=", "<", "<=", "=", "<>", "in", "not in"};	
	污染路径: 无	
24	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 65
	步骤描述: 禁止使用系统缺省值初始化变量 'bpTs' .	
	代码片段: 63: char *bpTbPrefix = "t"; 64: int32_t bpDefaultStbId = 1; 65: int64_t bpTs; 66: 67: // char *operatorList[] = {">", ">=", "<", "<=", "=", "<>", "in", "not in"};	
	污染路径: 无	

25	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 68
	步骤描述: 禁止使用系统缺省值初始化变量 'remaining' .	
	代码片段: 66: const BYTE* ip = istart; 67: int nbBits; 68: int remaining; 69: int threshold; 70: U32 bitStream;	
	污染路径: 无	
26	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-pv.c	行号: 25
	步骤描述: 禁止使用系统缺省值初始化变量 'argp_program_version' .	
	代码片段: 23: print this this string followed by a newline and exit (unless the 24: ARGP_NO_EXIT flag is used). Overridden by ARGP_PROGRAM_VERSION_HOOK. */ 25: const char *argp_program_version;	
	污染路径: 无	
27	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/getopt.h	行号: 82
	步骤描述: 禁止使用系统缺省值初始化变量 'optopt' .	
	代码片段: 80: /* Set to an option character which was unrecognized. */ 81: 82: extern int optopt; 83: 84: #ifndef __need_getopt	
	污染路径: 无	
28	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 67

	步骤描述: 禁止使用系统缺省值初始化变量 'nbBits' .	
	代码片段: 65: const BYTE* const iend = istart + hbSize; 66: const BYTE* ip = istart; 67: int nbBits; 68: int remaining; 69: int threshold;	
	污染路径: 无	
29	缺陷发生位置:	行号:
	/source/libs/decimal/src/detail/intx/intx.hpp	540
	步骤描述: 禁止使用系统缺省值初始化变量 'p' .	
	代码片段: 538: constexpr int num_words = sizeof(uint<N>) / sizeof(uint64_t); 539: 540: uint<2 * N> p; 541: auto pw = as_words(p); 542: auto uw = as_words(x);	
30	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/decimal/src/detail/intx/intx.hpp	706
	步骤描述: 禁止使用系统缺省值初始化变量 'na' .	
31	代码片段: 704: auto* v = as_words(denominator); 705: 706: normalized_div_args<IntT::num_bits> na; 707: auto* un = as_words(na.numerator); 708: auto* vn = as_words(na.divisor);	
	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/dictBuilder/divsufsort.c	230
	步骤描述: 禁止使用系统缺省值初始化变量 'U2' .	
	代码片段:	

	228: const int *p1, const int *p2, 229: int depth) { 230: const unsigned char *U1, *U2, *U1n, *U2n; 231: 232: for(U1 = T + depth + *p1,	
	污染路径: 无	
32	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1184
	步骤描述: 禁止使用系统缺省值初始化变量 'temp' .	
	代码片段: 1182: int num_fields = taos_num_fields(result); 1183: TAOS_FIELD *fields = taos_fetch_fields(result); 1184: char temp[256]; 1185: 1186: // fetch the records row by row	
	污染路径: 无	
33	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/getopt.h	行号: 73
	步骤描述: 禁止使用系统缺省值初始化变量 'optind' .	
	代码片段: 71: how much of ARGV has been scanned so far. */ 72: 73: extern int optind; 74: 75: /* Callers store zero here to inhibit the error message `getopt' prints	
	污染路径: 无	
34	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1000
	步骤描述: 禁止使用系统缺省值初始化变量 'local_error' .	
	代码片段: 998: if (value != NULL) 999: {	

	1000: error local_error; 1001: local_error.json = (const unsigned char*)value; 1002: local_error.position = 0;	
	污染路径: 无	
35	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 373
	步骤描述: 禁止使用系统缺省值初始化变量 'systemTime' .	
	代码片段: 371: 372: static int64_t taosGetTimestampMs() { 373: struct timeval systemTime; 374: taosGetTimeOfDay(&systemTime); 375: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000;	
	污染路径: 无	
36	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 105
	步骤描述: 禁止使用系统缺省值初始化变量 'tv' .	
	代码片段: 103: 104: static uint64_t now() { 105: struct timeval tv; 106: gettimeofday(&tv, NULL); 107:	
	污染路径: 无	
37	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-ba.c	行号: 26
	步骤描述: 禁止使用系统缺省值初始化变量 'argp_program_bug_address' .	
	代码片段: 24: messages), embedded in a sentence that says something like `Report bugs to 25: ADDR.'. */ 26: const char *argp_program_bug_address;	

	污染路径: 无	
38	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1168
	步骤描述: 禁止使用系统缺省值初始化变量 'temp' .	
	代码片段: 1166: int num_fields = taos_num_fields(result); 1167: TAOS_FIELD *fields = taos_fetch_fields(result); 1168: char temp[256]; 1169: 1170: // fetch the records row by row	
	污染路径: 无	
39	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 188
	步骤描述: 禁止使用系统缺省值初始化变量 'buf' .	
	代码片段: 186: _argp_fmtstream_update (argp_fmtstream_t fs) 187: { 188: char *buf, *nl; 189: size_t len; 190:	
	污染路径: 无	
40	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 98
	步骤描述: 禁止使用系统缺省值初始化变量 '__old' .	
	代码片段: 96: argp_fmtstream_set_rmargin (argp_fmtstream_t __fs, size_t __rmargin) 97: { 98: size_t __old; 99: if ((size_t) (__fs->p - __fs->buf) > __fs->point_offs) 100: _argp_fmtstream_update (__fs);	

	污染路径: 无	
41	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 110
	步骤描述: 禁止使用系统缺省值初始化变量 '__old' .	
	代码片段: 108: argp_fmtstream_set_wmargin (argp_fmtstream_t __fs, size_t __wmargin) 109: { 110: size_t __old; 111: if ((size_t) (__fs->p - __fs->buf) > __fs->point_offs) 112: _argp_fmtstream_update (__fs);	
	污染路径: 无	
42	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 400
	步骤描述: 禁止使用系统缺省值初始化变量 'systemTime' .	
	代码片段: 398: 399: static int64_t taosGetTimestampUs() { 400: struct timeval systemTime; 401: taosGetTimeOfDay(&systemTime); 402: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec;	
	污染路径: 无	
43	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 420
	步骤描述: 禁止使用系统缺省值初始化变量 's' .	
	代码片段: 418: auto y = as_words(b); 419: 420: uint<N> s; 421: auto z = as_words(s); 422:	
	污染路径:	

	无	
44	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 140
	步骤描述: 禁止使用系统缺省值初始化变量 'fs' .	
	代码片段: 138: size_t lmargin, size_t rmargin, ssize_t wmargin) 139: { 140: argp_fmtstream_t fs; 141: 142: fs = (struct argp_fmtstream *) malloc (sizeof (struct argp_fmtstream));	
	污染路径: 无	
45	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 86
	步骤描述: 禁止使用系统缺省值初始化变量 '__old' .	
	代码片段: 84: argp_fmtstream_set_lmargin (argp_fmtstream_t __fs, size_t __lmargin) 85: { 86: size_t __old; 87: if ((size_t) (__fs->p - __fs->buf) > __fs->point_offs) 88: _argp_fmtstream_update (__fs);	
	污染路径: 无	
46	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 127
	步骤描述: 禁止使用系统缺省值初始化变量 's' .	
	代码片段: 125: { U32 const tableMask = tableSize-1; 126: U32 const step = FSE_TABLESTEP(tableSize); 127: U32 s, position = 0; 128: for (s=0; s<maxSV1; s++) { 129: int i;	

	污染路径: 无	
47	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 222
	步骤描述: 禁止使用系统缺省值初始化变量 'number_c_string' .	
	代码片段: 220: double number = 0; 221: unsigned char *after_end = NULL; 222: unsigned char number_c_string[64]; 223: unsigned char decimal_point = get_decimal_point(); 224: size_t i = 0;	
	污染路径: 无	
48	缺陷发生位置: /test/new_test_framework/script/api/stmtQuery.c	行号: 93
	步骤描述: 禁止使用系统缺省值初始化变量 'params' .	
	代码片段: 91: 92: char* condBuf = "2 or c1 > 0"; 93: TAOS_MULTI_BIND params[1]; 94: params[0].buffer_type = TSDB_DATA_TYPE_BINARY; 95: params[0].buffer_length = strlen(condBuf) + 1;	
	污染路径: 无	
49	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 441
	步骤描述: 禁止使用系统缺省值初始化变量 'number_buffer' .	
	代码片段: 439: int length = 0; 440: size_t i = 0; 441: unsigned char number_buffer[26]; /* temporary buffer to print the number into */ 442: unsigned char decimal_point = get_decimal_point(); 443: double test;	
	污染路径: 无	
50	缺陷发生位置:	行号:

	/tools/taos-tools/deps/toolscJson/src/toolscJson.c	443
	步骤描述: 禁止使用系统缺省值初始化变量 'test' .	
	代码片段: 441: unsigned char number_buffer[26]; /* temporary buffer to print the number into */ 442: unsigned char decimal_point = get_decimal_point(); 443: double test; 444: 445: if (output_buffer == NULL)	
51	污染路径: 无	
	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 70
	步骤描述: 禁止使用系统缺省值初始化变量 'bitStream' .	
52	代码片段: 68: int remaining; 69: int threshold; 70: U32 bitStream; 71: int bitCount; 72: unsigned charnum = 0;	
	污染路径: 无	
	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号: 244
53	步骤描述: 禁止使用系统缺省值初始化变量 'params' .	
	代码片段: 242: main (int argc, char **argv) 243: { 244: struct params params; 245: params.foonly = 0; 246: params.foonly_default = random ();	
	污染路径: 无	
53	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 54
	步骤描述:	

	禁止使用系统缺省值初始化变量 'buf' .	
	代码片段: 52: static void func(T t) { 53: uint64_t val = 198; 54: char buf[1024]; 55: void * pBuf = buf; 56:	
	污染路径: 无	
54	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmtQuery.c	26
	步骤描述: 禁止使用系统缺省值初始化变量 'row' .	
	代码片段: 24: void check_result(TAOS_RES *result, int expected) { 25: int rows = 0; 26: TAOS_ROW row; 27: while ((row = taos_fetch_row(result))) { 28: rows++; 	
55	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/common/entropy_common.c	71
	步骤描述: 禁止使用系统缺省值初始化变量 'bitCount' .	
56	代码片段: 69: int threshold; 70: U32 bitStream; 71: int bitCount; 72: unsigned charnum = 0; 73: int previous0 = 0;	
	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/common/fse_decompress.c	112
	步骤描述: 禁止使用系统缺省值初始化变量 's' .	
	代码片段: 110: DTableH.fastMode = 1; 111: { S16 const largeLimit= (S16)(1 << (tableLog-1));	

	112: U32 s; 113: for (s=0; s<maxSV1; s++) { 114: if (normalizedCounter[s]==-1) {	
	污染路径: 无	
57	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 66
	步骤描述: 禁止使用系统缺省值初始化变量 'version' .	
	代码片段: 64: cJSON_PUBLIC(const char*) tools_cJSON_Version(void) 65: { 66: static char version[40]; 67: snprintf(version, sizeof(version), "%i.%i.%i", CJSON_VERSION_MAJOR, CJSON_VERSION_MINOR, CJSON_VERSION_PATCH); 68:	
	污染路径: 无	
58	缺陷发生位置: /contrib/lemon/lemon.c	行号: 90
	步骤描述: 禁止使用系统缺省值初始化变量 'p' .	
	代码片段: 88: */ 89: static void *lemon_malloc(size_t nByte){ 90: MemChunk *p; 91: if(nByte<0) return 0; 92: p = malloc(nByte + sizeof(MemChunk));	
	污染路径: 无	
59	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 97
	步骤描述: 禁止使用系统缺省值初始化变量 'symbolNext' .	
	代码片段: 95: void* const tdPtr = dt+1; /* because *dt is unsigned, 32-bits aligned on 32-bits */ 96: FSE_DECODE_TYPE* const tableDecode = (FSE_DECODE_TYPE*) (tdPtr);	

	97: U16 symbolNext[FSE_MAX_SYMBOL_VALUE+1]; 98: 99: U32 const maxSV1 = maxSymbolValue + 1;
	污染路径: 无

6.2.1.5 WK4786 类中所有成员变量必须在构造函数中初始化

缺陷数量：103 个		
缺陷描述： 类中所有成员变量必须在构造函数中初始化		
修复意见： 无		
1	缺陷发生位置：	行号：
	/source/dnode/mnode/impl/test/xnode/xnodeEncodeDecodeTest.cpp	359
	步骤描述： constructor does not initialize these fields: jid, tid, xnodeId, createTime, updateTime	
	代码片段： <pre> 357: TEST_F(MndTestXnodeEncodeDecode, test_xnode_job_basic_encode_decode) { 358: // Test basic XNode Job encode/decode 359: struct TestJobObj { 360: int32_t jid; 361: int32_t tid; </pre>	
	污染路径： 无	
2	缺陷发生位置：	行号：
	/source/libs/index/test/fstUT.cc	69
	步骤描述： constructor does not initialize these fields: _w, _fst, _s, _wc	
	代码片段： <pre> 67: class FstReadMemory { 68: public: 69: FstReadMemory(size_t size) { 70: _wc = idxFileCtxCreate(TFILE, tindex, true, 64 * 1024); 71: _w = idxFileCreate(_wc); </pre>	
	污染路径：	

	无	
3	缺陷发生位置： /source/libs/executor/test/queryPlanTests.cpp	行号： 108
	步骤描述： constructor does not initialize these fields: vnodeNum, vgld, epSet	
	代码片段： 106: 107: 108: typedef struct { 109: int32_t vnodeNum; 110: int32_t vgld;	
	污染路径： 无	
4	缺陷发生位置： /source/libs/executor/test/joinTests.cpp	行号： 58
	步骤描述： constructor does not initialize these fields: maxResRows, maxResBlkRows, totalResRows, useMSecs, pHistory	
	代码片段： 56: } SJoinTestResInfo; 57: 58: typedef struct { 59: int32_t maxResRows; 60: int32_t maxResBlkRows;	
	污染路径： 无	
5	缺陷发生位置： /source/libs/index/test/jsonUT.cc	行号： 47
	步骤描述： constructor does not initialize these fields: opts, index	
	代码片段： 45: } 46: } 47: class JsonEnv : public ::testing::Test { 48: protected: 49: virtual void SetUp() {	
	污染路径： 无	
6	缺陷发生位置： /source/os/test/osThreadTests.cpp	行号： 54

	步骤描述: uninitialized record type: 'para'	
	代码片段: 52: 53: int32_t policy; 54: struct sched_param para; 55: taosThreadGetSchedParam(thread, &policy, ¶); 56: taosThreadGetSchedParam(thread, NULL, ¶);	
	污染路径: 无	
7	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 22
	步骤描述: constructor does not initialize these fields: src, rsIt	
	代码片段: 20: #include "indexFstDfa.h" 21: 22: class UtilEnv : public ::testing::Test { 23: protected: 24: virtual void SetUp() {	
	污染路径: 无	
8	缺陷发生位置: /source/libs/transport/test/transUT.cpp	行号: 45
	步骤描述: constructor does not initialize these fields: sem, rpclnit_, transCli, resp	
	代码片段: 43: // client process; 44: static void processResp(void *parent, SRpcMsg *pMsg, SEpSet *pEpSet); 45: class Client { 46: public: 47: void Init(int nThread) {	
	污染路径: 无	
9	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 76
	步骤描述: constructor does not initialize these fields: _w, _fst, _s, _wc	
	代码片段:	

	74: class FstReadMemory { 75: public: 76: FstReadMemory(size_t size) { 77: _wc = idxFileCtxCreate(TFILE, TD_TMP_DIR_PATH "tindex", true, 64 * 1024); 78: _w = idxFileCreate(_wc);	
	污染路径: 无	
10	缺陷发生位置: /source/libs/parser/test/mockCatalogService.cpp	行号: 97
	步骤描述: constructor does not initialize these fields: builder_, meta_, udf_, index_, dnode_, dbCfg_	
	代码片段: 95: static const int32_t numOfDataTypes = sizeof(tDataTypes) / sizeof(tDataTypes[0]); 96: 97: MockCatalogServiceImpl() : id_(1), havaCache_(true) {} 98: 99: ~MockCatalogServiceImpl() {	
	污染路径: 无	
11	缺陷发生位置: /source/libs/planner/test/planTestUtil.cpp	行号: 260
	步骤描述: constructor does not initialize these fields: sql_, pQuery_	
	代码片段: 258: SQuery* pQuery_ 259: 260: stmtEnv() : pQuery_(nullptr) {} 261: ~stmtEnv() { qDestroyQuery(pQuery_); } 262: };	
	污染路径: 无	
12	缺陷发生位置: /source/libs/transport/test/transUTIPv6.cpp	行号: 100
	步骤描述: constructor does not initialize these fields: rpclnit_, transSrv	
	代码片段: 98: class Server {	

	99: public: 100: Server() { 101: memcpy(tsTempDir, TD_TMP_DIR_PATH, strlen(TD_TMP_DIR_PATH)); 102: memset(&rpclnit_, 0, sizeof(rpclnit_));	
	污染路径: 无	
13	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 16
	步骤描述: constructor does not initialize these fields: pathName	
	代码片段: 14: SWalSyncInfo syncMeta = {0}; 15: 16: class WalCleanEnv : public ::testing::Test { 17: protected: 18: static void SetUpTestCase() {	
	污染路径: 无	
14	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeEncodeDecodeTest.cpp	行号: 293
	步骤描述: constructor does not initialize these fields: tid, name, xnodeId, agentId, createTime, updateTime	
	代码片段: 291: // In production, use actual SXnodeTaskObj structure from mndDef.h 292: 293: struct TestTaskObj { 294: int32_t tid; 295: char name[TSDB_TABLE_NAME_LEN];	
	污染路径: 无	
15	缺陷发生位置: /source/libs/transport/test/transUT2.cpp	行号: 133
	步骤描述: constructor does not initialize these fields: rpclnit_, transSrv	
	代码片段: 131: class Server { 132: public:	

	133: Server() { 134: memcpy(tsTempDir, TD_TMP_DIR_PATH, strlen(TD_TMP_DIR_PATH)); 135: memset(&rpclnit_, 0, sizeof(rpclnit_));	
	污染路径: 无	
16	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 956
	步骤描述: uninitialized record type: 'tm'	
	代码片段: 954: 955: TEST(timezoneCase, mktime_rz_Test){ 956: struct tm tm; 957: time_t t; 958:	
	污染路径: 无	
17	缺陷发生位置: /source/dnode/vnode/test/tsdbSmaTest.cpp	行号: 38
	步骤描述: constructor does not initialize these fields: info, rollup, type, nBSmaCols, pBSmaCols	
	代码片段: 36: 37: TEST(testCase, unionEncodeDecodeTest) { 38: typedef struct { 39: union { 40: uint8_t info;	
	污染路径: 无	
18	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 1212
	步骤描述: uninitialized record type: 'tm_val'	
	代码片段: 1210: { 1211: time_t t = (time_t)(winter_result / 1000); 1212: struct tm tm_val; 1213: taosLocalTime(&t, &tm_val, NULL, 0, ny);	

	1214: std::cout << "Winter ts truncated to: "	
	污染路径: 无	
19	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 1005
	步骤描述: uninitialized record type: 'tm1'	
	代码片段: 1003: for (int j = 0; j < cnt; ++j) { 1004: time_t t = time_winter - j; 1005: struct tm tm1; 1006: ASSERT (taosLocalTime(&t, &tm1, NULL, 0, sp)); 1007: }	
	污染路径: 无	
20	缺陷发生位置: /source/common/test/ttimeNaturalUnitsTest.cpp	行号: 32
	步骤描述: constructor does not initialize these fields: tz	
	代码片段: 30: */ 31: 32: class TimeNaturalUnitsTest : public ::testing::Test { 33: protected: 34: void SetUp() override {	
	污染路径: 无	
21	缺陷发生位置: /source/util/test/memPoolTest.cpp	行号: 155
	步骤描述: constructor does not initialize these fields: taskId, taskExecLock, destoryed, poolMaxUsedSize, poolTotalUsedSize, stat, memIdx, pMemList, taskFreed, lastAct	
	代码片段: 153: } SMPTestMemInfo; 154: 155: typedef struct { 156: uint64_t taskId; 157: SRWLatch taskExecLock;	
	污染路径:	

	无	
22	缺陷发生位置： /source/libs/parser/test/mockCatalogService.cpp	行号： 846
	步骤描述： constructor does not initialize these fields: impl_	
	代码片段： 844: }; 845: 846: MockCatalogService::MockCatalogService() : impl_(new MockCatalogServiceImpl()) {} 847: 848: MockCatalogService::~~MockCatalogService() {}	
	污染路径： 无	
23	缺陷发生位置： /source/os/test/osTimeTests.cpp	行号： 103
	步骤描述： uninitialized record type: 'tmInfo'	
	代码片段： 101: // Test 1: Test when both timep and result are not NULL 102: time_t timep = 1617531000; // 2021-04-04 18:10:00 103: struct tm tmInfo; 104: ASSERT_NE(taosGmTimeR(&timep, &tmInfo), nullptr); 105:	
	污染路径： 无	
24	缺陷发生位置： /source/client/test/connectOptionsTest.cpp	行号： 933
	步骤描述： uninitialized record type: 'tm'	
	代码片段： 931: 932: TEST(timezoneCase, mktime_Test){ 933: struct tm tm; 934: time_t t; 935:	
	污染路径： 无	
25	缺陷发生位置： /source/common/test/commonTests.cpp	行号： 1171

	步骤描述: uninitialized record type: 'tm_val'	
	代码片段: 1169: { 1170: time_t t; 1171: struct tm tm_val; 1172: 1173: // Verify winter day start is indeed midnight EST	
	污染路径: 无	
26	缺陷发生位置: /source/libs/new-stream/test/streamTriggerTaskTest.cpp	行号: 175
	步骤描述: uninitialized record type: 'tm_ekey'	
	代码片段: 173: // Verify ekey 174: time_t t_ekey = win.ekey / precisionFactor; 175: struct tm tm_ekey; 176: taosLocalTime(&t_ekey, &tm_ekey, NULL, 0, NULL); 177:	
	污染路径: 无	
27	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 33
	步骤描述: constructor does not initialize these fields: size, capacity, data	
	代码片段: 31: typedef struct SBlockData SBlockData; 32: 33: typedef struct { 34: int32_t size; 35: int32_t capacity;	
	污染路径: 无	
28	缺陷发生位置: /source/libs/transport/test/transUTIPv6.cpp	行号: 42
	步骤描述: constructor does not initialize these fields: sem, rpclnit_, transCli, resp	
	代码片段: 40: static void processResp(void *parent, SRpcMsg *pMsg,	

	SEpSet *pEpSet); 41: 42: class Client { 43: public: 44: void Init(int nThread, const char *lbl = "client") {	
	污染路径: 无	
29	缺陷发生位置: /source/libs/index/test/fstUT.cc	行号: 202
	步骤描述: constructor does not initialize these fields: fst	
	代码片段: 200: FstReadMemory* fr; 201: }; 202: class FstEnv : public ::testing::Test { 203: protected: 204: virtual void SetUp() {	
	污染路径: 无	
30	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 60
	步骤描述: constructor does not initialize these fields: suid, uid, nRow, aUid, aVersion, aTSKEY	
	代码片段: 58: } SBrinRecord; 59: 60: struct SBlockData { 61: int64_t suid; 62: int64_t uid;	
	污染路径: 无	
31	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 78
	步骤描述: constructor does not initialize these fields: pathName	
	代码片段: 76: }; 77: 78: class WalKeepEnv : public ::testing::Test { 79: protected:	

	80: static void SetUpTestCase() {	
	污染路径: 无	
32	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 126
	步骤描述: constructor does not initialize these fields: uid, suid, tblType, colNum, tagNum, pkNum, tblName, tblAlias, pColList, pTagList, pColTagList	
	代码片段: 124: } SQPTCol; 125: 126: typedef struct { 127: int64_t uid; 128: int64_t suid;	
	污染路径: 无	
33	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 116
	步骤描述: constructor does not initialize these fields: pathName	
	代码片段: 114: }; 115: 116: class WalRetentionEnv : public ::testing::Test { 117: protected: 118: static void SetUpTestCase() {	
	污染路径: 无	
34	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 104
	步骤描述: constructor does not initialize these fields: dec_	
	代码片段: 102: public: 103: friend Numeric64; 104: Numeric128() { dec_ = {0}; } 105: Numeric128(const Numeric128& r) = default; 106: int32_t fromStr(const string& str, uint8_t prec, uint8_t scale) {	

	污染路径: 无	
35	缺陷发生位置: /source/dnode/mnode/impl/test/sdb/sdbTest.cpp	行号: 72
	步骤描述: constructor does not initialize these fields: key, v8, v16, v32, v64, vstr, unused	
	代码片段: 70: } SStrObj; 71: 72: typedef struct SI32Obj { 73: int32_t key; 74: int8_t v8;	
	污染路径: 无	
36	缺陷发生位置: /source/libs/tdb/test/tdbBtreeToStackTest.cpp	行号: 124
	步骤描述: constructor does not initialize these fields: root, keyLen, valLen, pPager, kcmp, pageSize, maxLocal, minLocal, maxLeaf, minLeaf, info, tname, pBuf	
	代码片段: 122: }; 123: 124: struct SBTree { 125: SPgno root; 126: int keyLen;	
	污染路径: 无	
37	缺陷发生位置: /source/libs/tss/test/mock.cpp	行号: 20
	步骤描述: constructor does not initialize these fields: type	
	代码片段: 18: 19: // mocked shared storage instance, for testing. 20: typedef struct { 21: // [type] is inherited from SSharedStorage, it must be the first member 22: // to allow casting to SSharedStorage.	
	污染路径: 无	

38	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 44
	步骤描述: constructor does not initialize these fields: suid, uid, minVer, maxVer, blockOffset, smaOffset, blockSize, blockKeySize, smaSize, numRows, count	
	代码片段: 42: } STsdbRowKey; 43: 44: typedef struct { 45: int64_t suid; 46: int64_t uid;	
	污染路径: 无	
39	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeEncodeDecodeTest.cpp	行号: 15
	步骤描述: constructor does not initialize these fields: id, urlLen, url, statusLen, status, createTime, updateTime	
	代码片段: 13: 14: // Simplified XNode structures for testing 15: typedef struct SXnodeObj { 16: int32_t id; 17: int32_t urlLen;	
	污染路径: 无	
40	缺陷发生位置: /source/common/test/ttimeNaturalUnitsTest.cpp	行号: 115
	步骤描述: uninitialized record type: 'tm'	
	代码片段: 113: 114: time_t t = result / 1000; 115: struct tm tm; 116: taosLocalTime(&t, &tm, NULL, 0, NULL); 117: std::cout << "Original timestamp: " << ts << ", Aligned timestamp: " << result << std::endl;	
	污染路径: 无	
41	缺陷发生位置:	行号:

	/source/libs/index/test/indexTests.cc		359
	步骤描述: constructor does not initialize these fields: path_, fileName_, writer_, reader_		
	代码片段: 357: class TFileObj { 358: public: 359: TFileObj(const std::string& path = TD_TMP_DIR_PATH "tindex", const std::string& colName = "voltage") 360: : path_(path), colName_(colName) { 361: colld_ = 10;		
	污染路径: 无		
42	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp		行号: 48
	步骤描述: constructor does not initialize these fields: pathName		
	代码片段: 46: }; 47: 48: class WalCleanDeleteEnv : public ::testing::Test { 49: protected: 50: static void SetUpTestCase() {		
	污染路径: 无		
43	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp		行号: 167
	步骤描述: constructor does not initialize these fields: prec_, scale_		
	代码片段: 165: 166: template <int BitNum> 167: class Numeric { 168: static_assert(BitNum == 64 BitNum == 128, "only support Numeric64 and Numeric128"); 169: using Type = typename NumericType<BitNum>::Type;		
	污染路径: 无		
44	缺陷发生位置: /source/libs/parser/test/parTestUtil.cpp		行号: 113
	步骤描述:		

	constructor does not initialize these fields: sql_, expect_, checkFunc_	
	代码片段: 111: }; 112: 113: struct stmtEnv { 114: string sql_; 115: array<char, 1024> msgBuf_;	
	污染路径: 无	
45	缺陷发生位置: /source/libs/tdb/test/tdbBtreeToStackTest.cpp	行号: 119
	步骤描述: constructor does not initialize these fields: pEnv, pBt	
	代码片段: 117: } 118: 119: struct STTB { 120: TDB *pEnv; 121: SBTree *pBt;	
	污染路径: 无	
46	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 39
	步骤描述: constructor does not initialize these fields: key, version	
	代码片段: 37: } TFileSetArray; 38: 39: typedef struct { 40: SRowKey key; 41: int64_t version;	
	污染路径: 无	
47	缺陷发生位置: /source/libs/new-stream/test/streamTriggerTaskTest.cpp	行号: 162
	步骤描述: uninitialized record type: 'tm_skey'	
	代码片段: 160: // Verify skey 161: time_t t_skey = win.skey / precisionFactor;	

	162: struct tm tm_skey; 163: taosLocalTime(&t_skey, &tm_skey, NULL, 0, NULL); 164:	
	污染路径: 无	
48	缺陷发生位置: /source/common/test/ttimeNaturalUnitsTest.cpp	行号: 92
	步骤描述: uninitialized record type: 'tm'	
	代码片段: 90: // Convert result to struct tm to verify 91: time_t t = result / 1000; 92: struct tm tm; 93: taosLocalTime(&t, &tm, NULL, 0, NULL); 94: std::cout << "Original timestamp: " << ts << ", Aligned timestamp: " << result << std::endl;	
	污染路径: 无	
49	缺陷发生位置: /source/client/test/stmtTest.cpp	行号: 368
	步骤描述: constructor does not initialize these fields: c1, c2, c3, c4, c5, c6, c7, c8, c9, c10	
	代码片段: 366: } v = {1591060628000, 1, 2, 3.0, 4.0, "abcdef", 5, 6, 7, "ijnop"}; 367: 368: struct { 369: int32_t c1; 370: int32_t c2;	
	污染路径: 无	
50	缺陷发生位置: /source/common/test/tmsgTest.cpp	行号: 39
	步骤描述: constructor does not initialize these fields: type, rspType	
	代码片段: 37: }; 38: 39: typedef struct { 40: string name;	

	41: string rspName;	
	污染路径: 无	
51	缺陷发生位置: /source/libs/sync/test/syncRefTest.cpp	行号: 22
	步骤描述: constructor does not initialize these fields: rid, data, name, counter	
	代码片段: 20: int g = 100; 21: 22: typedef struct SyncObj { 23: int64_t rid; 24: void *data;	
	污染路径: 无	
52	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 1232
	步骤描述: uninitialized record type: 'tm_val'	
	代码片段: 1230: { 1231: time_t t = (time_t)(summer_result / 1000); 1232: struct tm tm_val; 1233: taosLocalTime(&t, &tm_val, NULL, 0, ny); 1234: std::cout << "Summer ts truncated to: "	
	污染路径: 无	
53	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 52
	步骤描述: constructor does not initialize these fields: _b, _wc	
	代码片段: 50: class FstWriter { 51: public: 52: FstWriter() { 53: _wc = idxFileCtxCreate(TFILE, TD_TMP_DIR_PATH "tindex", false, 64 * 1024 * 1024); 54: _b = fstBuilderCreate(NULL, 0);	
	污染路径:	

	无	
54	缺陷发生位置： /source/libs/index/test/fstTest.cc	行号： 57
	步骤描述： constructor does not initialize these fields: _w, _fst, _s, _wc	
	代码片段： <pre> 55: class FstReadMemory { 56: public: 57: FstReadMemory(int32_t size, const std::string& fileName = TD_TMP_DIR_PATH "tindex.tindex") { 58: _wc = idxFileCtxCreate(TFILE, fileName.c_str(), true, 64 * 1024); 59: _w = idxFileCreate(_wc); </pre>	
	污染路径： 无	
55	缺陷发生位置： /source/libs/executor/test/joinTests.cpp	行号： 197
	步骤描述： constructor does not initialize these fields: res	
	代码片段： <pre> 195: } SJoinTestCtx; 196: 197: typedef struct { 198: SJoinTestResInfo res; 199: SJoinTestCtx ctx; </pre>	
	污染路径： 无	
56	缺陷发生位置： /source/libs/decimal/test/decimalTest.cpp	行号： 1211
	步骤描述： constructor does not initialize these fields: type_a, type_b, type_out, op_type	
	代码片段： <pre> 1209: }; 1210: struct DecimalRetTypeCheckContent { 1211: DecimalRetTypeCheckContent(const SDataType& a, const SDataType& b, const SDataType& out, EOperatorType op) 1212: : type_a(a), type_b(b), type_out(out), op_type(op) {} 1213: SDataType type_a; </pre>	
	污染路径：	

	无	
57	缺陷发生位置: /source/libs/parser/test/parTestUtil.cpp	行号: 110
	步骤描述: constructor does not initialize these fields: acctId_, user_, db_, numOfLimitSql_	
	代码片段: <pre> 108: int32_t numOfLimitSql_; 109: 110: caseEnv() : user_("wangxiaoyu"), numOfSkipSql_(0) {} 111: }; 112: </pre>	
	污染路径: 无	
58	缺陷发生位置: /source/os/test/osThreadTests.cpp	行号: 139
	步骤描述: uninitialized record type: 'para'	
	代码片段: <pre> 137: 138: int32_t policy; 139: struct sched_param para; 140: taosThreadGetSchedParam(tid2, &policy, &para); 141: taosThreadGetSchedParam(tid2, NULL, &para); </pre>	
	污染路径: 无	
59	缺陷发生位置: /source/libs/planner/test/planTestUtil.cpp	行号: 264
	步骤描述: constructor does not initialize these fields: ast_, prepareAst_, boundAst_, rawLogicPlan_, optimizedLogicPlan_, splitLogicPlan_, scaledLogicPlan_, physiPlan_, physiSubplans_	
	代码片段: <pre> 262: }; 263: 264: struct stmtRes { 265: string ast_; 266: string prepareAst_; </pre>	
	污染路径: 无	
60	缺陷发生位置:	行号:

	/source/common/test/ttimeNaturalUnitsTest.cpp	163
	步骤描述: uninitialized record type: 'tm'	
	代码片段: 161: 162: time_t t = result / 1000; 163: struct tm tm; 164: taosLocalTime(&t, &tm, NULL, 0, NULL); 165: std::cout << "Original timestamp: " << ts << ", Aligned timestamp: " << result << std::endl;	
	污染路径: 无	
61	缺陷发生位置: /source/dnode/mnode/impl/test/sdb/sdbTest.cpp	行号: 62
	步骤描述: constructor does not initialize these fields: key, v8, v16, v32, v64, vstr, unused	
	代码片段: 60: } SMnode; 61: 62: typedef struct SStrObj { 63: char key[24]; 64: int8_t v8;	
	污染路径: 无	
62	缺陷发生位置: /source/libs/parser/test/mockCatalogService.cpp	行号: 80
	步骤描述: constructor does not initialize these fields: colId_	
	代码片段: 78: } 79: 80: TableBuilder(STableMeta* schemaMeta) : colId_(1), rowsize_(0), meta_(new MockTableMeta()) { 81: meta_>schema = schemaMeta; 82: }	
	污染路径: 无	
63	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 114
	步骤描述:	

	constructor does not initialize these fields: type, name, dtype, len, inUse, hasIndex, isPrimTs, isPk, colType	
	代码片段: 112: } SQPTVnodeParam; 113: 114: typedef struct { 115: int32_t type; 116: char name[TSDB_COL_NAME_LEN];	
	污染路径: 无	
64	缺陷发生位置: /source/libs/transport/test/transUTIPv6.cpp	行号: 180
	步骤描述: constructor does not initialize these fields: cli, srv	
	代码片段: 178: } 179: 180: class IPv6DualStackTest : public ::testing::Test { 181: protected: 182: virtual void SetUp() {	
	污染路径: 无	
65	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 116
	步骤描述: constructor does not initialize these fields: filter, asc, grpJoin, leftMaxRows, leftMaxGrpRows, rightMaxRows, rightMaxGrpRows, blkRows, colCond, joinType, subType, asofOpType, jLimit, winStartOffset, winEndOffset, inGrpId, leftTotalRows, rightTotalRows, blkRowSize, inputStat, colEqNum, colEqList, colOnNum, colOnList, leftFilterNum, leftFilterColList, rightFilterNum, rightFilterColList, keyInSlotIdx, keyOutSlotIdx, keyColOffset, resColNum, resColInSlot, resColList, resColOffset, resColSize, resColBuf, colRowDataBufSize, colRowDataBuf, colRowOffset, curTs, curKeyOffset, grpOffset, leftBlkReadIdx, leftBlkList, rightBlkReadIdx, rightBlkList, resRows, leftColOnly, rightColOnly, jtResRows, pJoinOp, loopIdx, rightFinMatchNum, rightFinMatch, inColOffset, inColSize, inColBuf, leftRowsList, rightRowsList, rightFilterOut, startTsUs	
	代码片段: 114: 115: 116: typedef struct { 117: bool filter;	

	118: bool asc;	
	污染路径: 无	
66	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 102
	步骤描述: constructor does not initialize these fields: precision, dbName	
	代码片段: 100: } SQTPPlanParam; 101: 102: typedef struct { 103: uint8_t precision; 104: char dbName[TSDB_DB_NAME_LEN];	
	污染路径: 无	
67	缺陷发生位置: /source/libs/index/test/fstUtilUT.cc	行号: 39
	步骤描述: constructor does not initialize these fields: set	
	代码片段: 37: }; 38: 39: class FstSparseSetEnv : public ::testing::Test { 40: protected: 41: virtual void SetUp() { set = sparSetCreate(256); }	
	污染路径: 无	
68	缺陷发生位置: /source/dnode/mnode/impl/test/sdb/sdbTest.cpp	行号: 82
	步骤描述: constructor does not initialize these fields: key, v8, v16, v32, v64, vstr, unused	
	代码片段: 80: } SI32Obj; 81: 82: typedef struct SI64Obj { 83: int64_t key; 84: int8_t v8;	
	污染路径: 无	

69	缺陷发生位置: /source/util/test/memPoolTest.cpp	行号: 124
	步骤描述: constructor does not initialize these fields: retired, errCode, memInfo, pCtx, lock, destroyed, pSessions, initDone	
	代码片段: 122: } SMPTCaseParam; 123: 124: typedef struct SMPTJobInfo { 125: int8_t retired; 126: int32_t errCode;	
	污染路径: 无	
70	缺陷发生位置: /source/libs/transport/test/transUT2.cpp	行号: 47
	步骤描述: constructor does not initialize these fields: sem, rpcInit_, transCli, resp	
	代码片段: 45: // client process; 46: static void processResp(void *parent, SRpcMsg *pMsg, SEpSet *pEpSet); 47: class Client { 48: public: 49: void Init(int nThread) {	
	污染路径: 无	
71	缺陷发生位置: /source/client/test/stmtTest.cpp	行号: 67
	步骤描述: constructor does not initialize these fields: ts, current, voltage, phase	
	代码片段: 65: } 66: 67: typedef struct { 68: int64_t ts; 69: float current;	
	污染路径: 无	
72	缺陷发生位置: /source/libs/new-stream/test/dataSinkTest.cpp	行号: 887

	步骤描述: constructor does not initialize these fields: groupID, wstart, wend, pBlock	
	代码片段: 885: const int taskPerProducer = 10000; 886: 887: struct Task { 888: int64_t groupID; 889: TSKEY wstart;	
	污染路径: 无	
73	缺陷发生位置: /source/client/test/stmtTest.cpp	行号: 355
	步骤描述: constructor does not initialize these fields: c1, c2, c3, c4, c5, c6, c7, c8, c9, c10	
	代码片段: 353: 354: uintptr_t c10len = 0; 355: struct { 356: int64_t c1; 357: int32_t c2;	
	污染路径: 无	
74	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 81
	步骤描述: constructor does not initialize these fields: dec_	
	代码片段: 79: public: 80: friend class Numeric128; 81: Numeric64() { dec_ = {0}; } 82: int32_t fromStr(const string& str, uint8_t prec, uint8_t scale) { 83: return decimal64FromStr(str.c_str(), str.size(), prec, scale, &dec_);	
	污染路径: 无	
75	缺陷发生位置: /source/libs/index/test/fstTest.cc	行号: 27
	步骤描述: constructor does not initialize these fields: _b, _wc	

	代码片段: 25: class FstWriter { 26: public: 27: FstWriter() { 28: taosRemoveFile(fileName.c_str()); 29: _wc = idxFileCtxCreate(TFILE, fileName.c_str(), false, 64 * 1024 * 1024); 	
	污染路径: 无	
76	缺陷发生位置: /source/libs/planner/test/planTestUtil.cpp	行号: 252
	步骤描述: constructor does not initialize these fields: acctId_, user_, db_, numOfLimitSql_	
	代码片段: 250: int32_t numOfLimitSql_; 251: 252: caseEnv() : numOfSkipSql_(0) {} 253: }; 254:	
	污染路径: 无	
77	缺陷发生位置: /source/os/test/osTimeTests.cpp	行号: 38
	步骤描述: uninitialized record type: 'result'	
	代码片段: 36: // Test 1: Test when both timep and result are not NULL 37: time_t timep = 1617531000; // 2021-04-04 18:10:00 38: struct tm result; 39: struct tm *local_time = taosLocalTime(&timep, &result, NULL, 0, NULL); 40: ASSERT_NE(local_time, nullptr);	
	污染路径: 无	
78	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 202
	步骤描述: constructor does not initialize these fields: joinType, subType, asofOp, jLimit, cond, filter, asc, grpJoin, timetruncate, pTask	
	代码片段:	

	200: } SJoinTestHistory; 201: 202: typedef struct { 203: EJoinType joinType; 204: EJoinSubType subType;	
	污染路径: 无	
79	缺陷发生位置: /source/util/test/memPoolTest.cpp	行号: 150
	步骤描述: constructor does not initialize these fields: p, size	
	代码片段: 148: } SMPTestCtrl; 149: 150: typedef struct { 151: void* p; 152: int64_t size;	
	污染路径: 无	
80	缺陷发生位置: /source/libs/index/test/indexBench.cc	行号: 48
	步骤描述: constructor does not initialize these fields: terms	
	代码片段: 46: } 47: 48: struct WriteBatch { 49: SIndexMultiTerm *terms; 50: };	
	污染路径: 无	
81	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 35
	步骤描述: constructor does not initialize these fields: info	
	代码片段: 33: class DebugInfo { 34: public: 35: DebugInfo(const char* str) : info(str) { 36: std::cout << "-----" << info << "\t"	

	37: << "begin"	
	污染路径: 无	
82	缺陷发生位置: /source/libs/index/test/indexBench.cc	行号: 53
	步骤描述: constructor does not initialize these fields: path, opts, index	
	代码片段: 51: class Idx { 52: public: 53: Idx(int _cacheSize = 1024 * 1024 * 4, const char *_path = "tindex") { 54: opts.cacheSize = _cacheSize; 55: path += TD_TMP_DIR_PATH;	
	污染路径: 无	
83	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 676
	步骤描述: constructor does not initialize these fields: numerator_ex, num_divisor_words, num_numerator_words, shift	
	代码片段: 674: { 675: template <unsigned N> 676: struct normalized_div_args 677: { 678: uint<N> divisor;	
	污染路径: 无	
84	缺陷发生位置: /source/util/test/memPoolTest.cpp	行号: 116
	步骤描述: constructor does not initialize these fields: jobNum, sessionNum, memSize, jobQuotaRetire, poolRetire	
	代码片段: 114: }; 115: 116: typedef struct { 117: int32_t jobNum; 118: int32_t sessionNum;	

	污染路径: 无	
85	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 91
	步骤描述: constructor does not initialize these fields: path, pNode, name	
	代码片段: 89: } EFCurrentT; 90: 91: struct STsdb { 92: char *path; 93: SVnode *pVnode;	
	污染路径: 无	
86	缺陷发生位置: /source/libs/parser/test/parTestUtil.cpp	行号: 68
	步骤描述: constructor does not initialize these fields: res_	
	代码片段: 66: class ParserTestBaseImpl { 67: public: 68: ParserTestBaseImpl(ParserTestBase* pBase) : pBase_(pBase), sqlNo_(0), sqlNum_(0) { 69: int32_t code = qlInitKeywordsTable(); 70: TD_ALWAYS_ASSERT(TSDB_CODE_SUCCESS == code);	
	污染路径: 无	
87	缺陷发生位置: /source/client/test/stmt2Test.cpp	行号: 60
	步骤描述: constructor does not initialize these fields: async_affected_rows, sem	
	代码片段: 58: } 59: 60: typedef struct AsyncArgs { 61: int async_affected_rows; 62: tsem_t sem;	
	污染路径: 无	

88	缺陷发生位置: /source/util/test/timerTest.cpp	行号: 142
	步骤描述: uninitialized record type: 'tv'	
	代码片段: 140: #else 141: static bool adjustSystemTime(int64_t offsetMs) { 142: struct timeval tv; 143: if (gettimeofday(&tv, NULL) != 0) return false; 144:	
	污染路径: 无	
89	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 86
	步骤描述: constructor does not initialize these fields: type, param	
	代码片段: 84: 85: 86: typedef struct { 87: ENodeType type; 88: void* param;	
	污染路径: 无	
90	缺陷发生位置: /source/dnode/mnode/impl/test/sdb/sdbTest.cpp	行号: 54
	步骤描述: constructor does not initialize these fields: v100, v200, insertTimes, deleteTimes, pSdb	
	代码片段: 52: }; 53: 54: typedef struct SMnode { 55: int32_t v100; 56: int32_t v200;	
	污染路径: 无	
91	缺陷发生位置: /source/libs/index/test/fstUT.cc	行号: 41
	步骤描述:	

	constructor does not initialize these fields: _b, _wc	
	代码片段: 39: class FstWriter { 40: public: 41: FstWriter() { 42: _wc = idxFileCtxCreate(TFILE, tindex, false, 64 * 1024 * 1024); 43: _b = fstBuilderCreate(_wc, 0);	
	污染路径: 无	
92	缺陷发生位置: /source/common/test/ttimeNaturalUnitsTest.cpp	行号: 138
	步骤描述: uninitialized record type: 'tm'	
	代码片段: 136: 137: time_t t = result / 1000; 138: struct tm tm; 139: taosLocalTime(&t, &tm, NULL, 0, NULL); 140: std::cout << "Original timestamp: " << ts << ", Aligned timestamp: " << result << std::endl;	
	污染路径: 无	
93	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 290
	步骤描述: constructor does not initialize these fields: path, opts, index	
	代码片段: 288: } 289: } 290: class IndexEnv : public ::testing::Test { 291: protected: 292: virtual void SetUp() {	
	污染路径: 无	
94	缺陷发生位置: /source/libs/index/test/fstUtilUT.cc	行号: 21
	步骤描述: constructor does not initialize these fields: builder	
	代码片段:	

	19: #include "tskiplist.h" 20: #include "tutil.h" 21: class FstUtilEnv : public ::testing::Test { 22: protected: 23: virtual void SetUp() {	
	污染路径: 无	
95	缺陷发生位置: /source/libs/executor/test/executorTests.cpp	行号: 57
	步骤描述: constructor does not initialize these fields: totalPages, current, startVal, type, numOfRowsPerPage, numOfCols, tsStart, pBlock	
	代码片段: 55: } 56: 57: typedef struct SDummyInputInfo { 58: int32_t totalPages; // numOfPages 59: int32_t current;	
	污染路径: 无	
96	缺陷发生位置: /source/util/test/bufferTest.cpp	行号: 5
	步骤描述: constructor does not initialize these fields: value1, value2, value3	
	代码片段: 3: #include "tbuffer.h" 4: 5: typedef struct { 6: int8_t value1; 7: int32_t value2;	
	污染路径: 无	
97	缺陷发生位置: /source/util/test/memPoolTest.cpp	行号: 137
	步骤描述: constructor does not initialize these fields: taskActTimes, caseLoopTimes, jobExecTimes, jobNum, jobTaskNum, maxSingleAllocSize, printExecDetail, printInputRow, lockDbg	
	代码片段: 135:	

	136: 137: typedef struct { 138: int32_t taskActTimes; 139: int32_t caseLoopTimes;	
	污染路径: 无	
98	缺陷发生位置: /source/libs/index/test/fstUtilUT.cc	行号: 32
	步骤描述: constructor does not initialize these fields: regex	
	代码片段: 30: }; 31: 32: class FstRegexEnv : public ::testing::Test { 33: protected: 34: virtual void SetUp() { regex = regexCreate("test"); }	
	污染路径: 无	
99	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 991
	步骤描述: uninitialized record type: 'tm1'	
	代码片段: 989: for (int j = 0; j < cnt; ++j) { 990: time_t t = time_winter - j; 991: struct tm tm1; 992: ASSERT (taosLocalTime(&t, &tm1, NULL, 0, NULL)); 993: }	
	污染路径: 无	
100	缺陷发生位置: /source/libs/transport/test/transUT.cpp	行号: 121
	步骤描述: constructor does not initialize these fields: rpclnit_, transSrv	
	代码片段: 119: class Server { 120: public: 121: Server() { 122: memcpy(tsTempDir, TD_TMP_DIR_PATH, strlen(TD_TMP_DIR_PATH));	

	123: memset(&rpclnit_, 0, sizeof(rpclnit_));	
	污染路径: 无	
101	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 154
	步骤描述: constructor does not initialize these fields: pathName	
	代码片段: 152: }; 153: 154: class WalSkipLevel : public ::testing::Test { 155: protected: 156: static void SetUpTestCase() {	
	污染路径: 无	
102	缺陷发生位置: /source/util/test/heapTest.cpp	行号: 8
	步骤描述: constructor does not initialize these fields: data, node	
	代码片段: 6: using namespace std; 7: 8: typedef struct TNode { 9: int32_t data; 10: HeapNode node;	
	污染路径: 无	
103	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 388
	步骤描述: constructor does not initialize these fields: prec_, scale_	
	代码片段: 386: 387: template <int BitNum2> 388: Numeric(const Numeric<BitNum2>& num2) { 389: Numeric(); 390: *this = num2;	
	污染路径: 无	

6.2.1.6 WK4113 类成员变量未初始化

缺陷数量： 19 个		
缺陷描述： C++类中的成员变量未在构造函数中初始化，或在某些条件分支下未初始化。		
修复意见： 在 C++类的构造方法中初始化成员变量。		
1	缺陷发生位置：	行号：
	/source/common/test/ttimeNaturalUnitsTest.cpp	92
	步骤描述： uninitialized record type: 'tm'	
	代码片段： 90: // Convert result to struct tm to verify 91: time_t t = result / 1000; 92: struct tm tm; 93: taosLocalTime(&t, &tm, NULL, 0, NULL); 94: std::cout << "Original timestamp: " << ts << ", Aligned timestamp: " << result << std::endl;	
2	污染路径：	
	无	
	缺陷发生位置：	行号：
	/source/common/test/ttimeNaturalUnitsTest.cpp	163
2	步骤描述： uninitialized record type: 'tm'	
	代码片段： 161: 162: time_t t = result / 1000; 163: struct tm tm; 164: taosLocalTime(&t, &tm, NULL, 0, NULL); 165: std::cout << "Original timestamp: " << ts << ", Aligned timestamp: " << result << std::endl;	
	污染路径：	
	无	
3	缺陷发生位置：	行号：
	/source/libs/new-stream/test/streamTriggerTaskTest.cpp	175
3	步骤描述： uninitialized record type: 'tm_ekey'	

	代码片段: 173: // Verify ekey 174: time_t t_ekey = win.ekey / precisionFactor; 175: struct tm tm_ekey; 176: taosLocalTime(&t_ekey, &tm_ekey, NULL, 0, NULL); 177:	
	污染路径: 无	
4	缺陷发生位置: /source/os/test/osThreadTests.cpp	行号: 54
	步骤描述: uninitialized record type: 'para'	
	代码片段: 52: 53: int32_t policy; 54: struct sched_param para; 55: taosThreadGetSchedParam(thread, &policy, ¶); 56: taosThreadGetSchedParam(thread, NULL, ¶);	
	污染路径: 无	
5	缺陷发生位置: /source/os/test/osTimeTests.cpp	行号: 38
	步骤描述: uninitialized record type: 'result'	
	代码片段: 36: // Test 1: Test when both timep and result are not NULL 37: time_t timep = 1617531000; // 2021-04-04 18:10:00 38: struct tm result; 39: struct tm *local_time = taosLocalTime(&timep, &result, NULL, 0, NULL); 40: ASSERT_NE(local_time, nullptr);	
	污染路径: 无	
6	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 1005
	步骤描述: uninitialized record type: 'tm1'	
	代码片段: 1003: for (int j = 0; j < cnt; ++j) { 1004: time_t t = time_winter - j;	

	1005: struct tm tm1; 1006: ASSERT (taosLocalTime(&t, &tm1, NULL, 0, sp)); 1007: }	
	污染路径: 无	
7	缺陷发生位置: /source/common/test/ttimeNaturalUnitsTest.cpp	行号: 138
	步骤描述: uninitialized record type: 'tm'	
	代码片段: 136: 137: time_t t = result / 1000; 138: struct tm tm; 139: taosLocalTime(&t, &tm, NULL, 0, NULL); 140: std::cout << "Original timestamp: " << ts << ", Aligned timestamp: " << result << std::endl;	
	污染路径: 无	
8	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 991
	步骤描述: uninitialized record type: 'tm1'	
	代码片段: 989: for (int j = 0; j < cnt; ++j) { 990: time_t t = time_winter - j; 991: struct tm tm1; 992: ASSERT (taosLocalTime(&t, &tm1, NULL, 0, NULL)); 993: }	
	污染路径: 无	
9	缺陷发生位置: /source/common/test/ttimeNaturalUnitsTest.cpp	行号: 224
	步骤描述: uninitialized record type: 'tm'	
	代码片段: 222: // Verify it's January 1st 223: time_t t = result1 / 1000; 224: struct tm tm; 225: taosLocalTime(&t, &tm, NULL, 0, NULL); 226: EXPECT_EQ(tm.tm_mon, 0); // January	

	污染路径: 无	
10	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 1212
	步骤描述: uninitialized record type: 'tm_val'	
	代码片段: 1210: { 1211: time_t t = (time_t)(winter_result / 1000); 1212: struct tm tm_val; 1213: taosLocalTime(&t, &tm_val, NULL, 0, ny); 1214: std::cout << "Winter ts truncated to: "	
	污染路径: 无	
11	缺陷发生位置: /source/os/test/osTimeTests.cpp	行号: 103
	步骤描述: uninitialized record type: 'tmInfo'	
	代码片段: 101: // Test 1: Test when both timep and result are not NULL 102: time_t timep = 1617531000; // 2021-04-04 18:10:00 103: struct tm tmInfo; 104: ASSERT_NE(taosGmTimeR(&timep, &tmInfo), nullptr); 105:	
	污染路径: 无	
12	缺陷发生位置: /source/util/test/timerTest.cpp	行号: 142
	步骤描述: uninitialized record type: 'tv'	
	代码片段: 140: #else 141: static bool adjustSystemTime(int64_t offsetMs) { 142: struct timeval tv; 143: if (gettimeofday(&tv, NULL) != 0) return false; 144:	
	污染路径: 无	
13	缺陷发生位置:	行号:

	/source/client/test/connectOptionsTest.cpp		956
	步骤描述: uninitialized record type: 'tm'		
	代码片段: 954: 955: TEST(timezoneCase, mktime_rz_Test){ 956: struct tm tm; 957: time_t t; 958:		
	污染路径: 无		
14	缺陷发生位置: /source/os/test/osThreadTests.cpp		行号: 139
	步骤描述: uninitialized record type: 'para'		
	代码片段: 137: 138: int32_t policy; 139: struct sched_param para; 140: taosThreadGetSchedParam(tid2, &policy, ¶); 141: taosThreadGetSchedParam(tid2, NULL, ¶);		
	污染路径: 无		
15	缺陷发生位置: /source/client/test/connectOptionsTest.cpp		行号: 933
	步骤描述: uninitialized record type: 'tm'		
	代码片段: 931: 932: TEST(timezoneCase, mktime_Test){ 933: struct tm tm; 934: time_t t; 935:		
	污染路径: 无		
16	缺陷发生位置: /source/common/test/commonTests.cpp		行号: 1171
	步骤描述: uninitialized record type: 'tm_val'		
	代码片段:		

	1169: { 1170: time_t t; 1171: struct tm tm_val; 1172: 1173: // Verify winter day start is indeed midnight EST	
	污染路径: 无	
17	缺陷发生位置: /source/libs/new-stream/test/streamTriggerTaskTest.cpp	行号: 162
	步骤描述: uninitialized record type: 'tm_skey'	
	代码片段: 160: // Verify skey 161: time_t t_skey = win.skey / precisionFactor; 162: struct tm tm_skey; 163: taosLocalTime(&t_skey, &tm_skey, NULL, 0, NULL); 164:	
	污染路径: 无	
18	缺陷发生位置: /source/common/test/ttimeNaturalUnitsTest.cpp	行号: 115
	步骤描述: uninitialized record type: 'tm'	
	代码片段: 113: 114: time_t t = result / 1000; 115: struct tm tm; 116: taosLocalTime(&t, &tm, NULL, 0, NULL); 117: std::cout << "Original timestamp: " << ts << " Aligned timestamp: " << result << std::endl;	
	污染路径: 无	
19	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 1232
	步骤描述: uninitialized record type: 'tm_val'	
	代码片段: 1230: { 1231: time_t t = (time_t)(summer_result / 1000); 1232: struct tm tm_val;	

	1233: taosLocalTime(&t, &tm_val, NULL, 0, ny); 1234: std::cout << "Summer ts truncated to: "
	污染路径: 无

7 C/C++缺陷类型-空指针解引用

7.1 分项汇总表

序号	C/C++缺陷类型-空指针解引用	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4691	动态分配的指针第一次使用前必须进行是否为 NULL 的判别	100	严重
2	WK4692	空指针必须使用 NULL,禁止使用整数 0	104	严重
3	WK4690	动态分配的指针变量定义时如未被分配空间必须初始化为 NULL	24	严重
4	WK4871	宏 NULL 应是唯一允许的整数空指针常量形式	104	严重

7.2 缺陷/安全漏洞详解

7.2.1 严重类

7.2.1.1 WK4691 动态分配的指针第一次使用前必须进行是否为

NULL 的判别

缺陷数量：100 个		
缺陷描述： 动态分配的指针第一次使用前必须进行是否为 NULL 的判别。		
修复意见： 无		
1	缺陷发生位置： /test/new_test_framework/script/api/stmt2-nohole.c	行号： 109
	步骤描述： 动态分配的指针 t32_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段： 107: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 108: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 109: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 110: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 111: int32_t* float_len = malloc(sizeof(int32_t) * 10);	
	污染路径： 无	
2	缺陷发生位置： /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号： 25
	步骤描述： 动态分配的指针 pRaw 第一次使用前必须进行是否为 NULL 的判别	
	代码片段： 23: // Mock SDB functions 24: SSdbRaw* sdbAllocRaw(int type, int version, int dataLen) { 25: SSdbRaw* pRaw = (SSdbRaw*)malloc(sizeof(SSdbRaw)); 26: memset(pRaw, 0, sizeof(SSdbRaw));	

	27: pRaw->sver = version;	
	污染路径: 无	
3	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 236
	步骤描述: 动态分配的指针 t8_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 234: } v; 235: 236: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 237: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 238: int32_t* t32_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
4	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 5050
	步骤描述: 动态分配的指针 array_u8 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 5048: } else { 5049: if (TSDB_DATA_TYPE_INT == field->array_type) { 5050: uint8_t *array_u8 = malloc(sizeof(uint8_t)); 5051: TOOLS_ASSERT(array_u8); 5052: *array_u8 = 0;	
	污染路径: 无	
5	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 27
	步骤描述: 动态分配的指针 out 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 25: { 26: *outSize = dataLength*sizeof(double); 27: unsigned char* out = (unsigned char*)malloc(dataLength*sizeof(double)); 28: memcpy(out, data, dataLength*sizeof(double)); 29: return out;	

	污染路径: 无	
6	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 111
	步骤描述: 动态分配的指针 float_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 109: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 110: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 111: int32_t* float_len = malloc(sizeof(int32_t) * 10); 112: int32_t* double_len = malloc(sizeof(int32_t) * 10); 113: int32_t* bin_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
7	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_leader10002.c	行号: 104
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 102: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 103: pFsm->apply = fsmApplyCb; 104: Tsdb *tsdb = malloc(sizeof(*tsdb)); 105: pFsm->data = tsdb; 106:	
	污染路径: 无	
8	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 86
	步骤描述: 动态分配的指针 b 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 84: // col params 85: int64_t** ts = (int64_t**)malloc(CTB_NUMS * sizeof(int64_t)); 86: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 87: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int)); 88: int* b_len = (int*)malloc(ROW_NUMS * sizeof(int));	
	污染路径: 无	
9	缺陷发生位置:	行号:

	/contrib/test/traft/cluster/node10002_restart.c		97
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别		
	代码片段: 95: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 96: pFsm->apply = fsmApplyCb; 97: Tsdb *tsdb = malloc(sizeof(*tsdb)); 98: pFsm->data = tsdb; 99:		
	污染路径: 无		
10	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c		行号: 439
	步骤描述: 动态分配的指针 L 第一次使用前必须进行是否为 NULL 的判别		
	代码片段: 437: else if(nodeCount<=65536) 438: { 439: unsigned short* L = (unsigned short*)malloc(nodeCount*sizeof(unsigned short)); 440: memset(L, 0, nodeCount*sizeof(unsigned short)); 441: unsigned short* R = (unsigned short*)malloc(nodeCount*sizeof(unsigned short));		
	污染路径: 无		
11	缺陷发生位置: /docs/examples/c-ws-new/stmt2_insert_demo.c		行号: 72
	步骤描述: 动态分配的指针 location_len 第一次使用前必须进行是否为 NULL 的判别		
	代码片段: 70: 71: char *location = (char *)malloc(20 * sizeof(char)); 72: int *location_len = (int *)malloc(sizeof(int)); 73: *location_len = sprintf(location, "location_%d", i); 74:		
	污染路径: 无		
12	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c		行号: 416
	步骤描述:		

	动态分配的指针 C 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 414: unsigned char* R = (unsigned char*)malloc(nodeCount*sizeof(unsigned char)); 415: memset(R, 0, nodeCount*sizeof(unsigned char)); 416: unsigned int* C = (unsigned int*)malloc(nodeCount*sizeof(unsigned int)); 417: memset(C, 0, nodeCount*sizeof(unsigned int)); 418: unsigned char* t = (unsigned char*)malloc(nodeCount*sizeof(unsigned char)); </pre>	
	污染路径: 无	
13	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 85
	步骤描述: 动态分配的指针 b 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 83: // col params 84: int64_t** ts = (int64_t**)malloc(CTB_NUMS * sizeof(int64_t*)); 85: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 86: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int)); 87: int* b_len = (int*)malloc(ROW_NUMS * sizeof(int)); </pre>	
	污染路径: 无	
14	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 89
	步骤描述: 动态分配的指针 t16_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 87: 88: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 89: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 90: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 91: int32_t* t64_len = malloc(sizeof(int32_t) * 10); </pre>	
	污染路径: 无	
15	缺陷发生位置: /contrib/TSZ/sz/src/szd_float.c	行号: 129
	步骤描述:	

	动态分配的指针 types 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 127: //data = (float*)malloc(sizeof(float)*dataSeriesLength); // comment by tickduan 128: 129: int* types = (int*)malloc(dataSeriesLength*sizeof(int)); 130: 131: if (tdps->ifAdtFse == false) {	
	污染路径: 无	
16	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/api_with_reqid_test.c	277
	步骤描述: 动态分配的指针 t16_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 275: 276: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 277: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 278: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 279: int32_t* t64_len = malloc(sizeof(int32_t) * 10);	
17	污染路径: 无	
	缺陷发生位置:	行号:
	/docs/examples/c/stmt2_insert_demo.c	157
	步骤描述: 动态分配的指针 phase 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 155: float *current = (float *)malloc(NUM_OF_ROWS * sizeof(float)); 156: int *voltage = (int *)malloc(NUM_OF_ROWS * sizeof(int)); 157: float *phase = (float *)malloc(NUM_OF_ROWS * sizeof(float)); 158: if (ts == NULL current == NULL voltage == NULL phase == NULL) { 159: (void)fprintf(stderr, "malloc column buffers failed (i=%d)\n", i);	
18	污染路径: 无	
	缺陷发生位置:	行号:

	/tools/taos-tools/src/taosdump.c	5011
	步骤描述: 动态分配的指针 array_u16 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 5009: } else { 5010: if (TSDB_DATA_TYPE_INT == field->array_type) { 5011: uint16_t *array_u16 = malloc(sizeof(uint16_t)); 5012: TOOLS_ASSERT(array_u16); 5013: *array_u16 = 0; </pre>	
	污染路径: 无	
19	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 280
	步骤描述: 动态分配的指针 float_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 278: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 279: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 280: int32_t* float_len = malloc(sizeof(int32_t) * 10); 281: int32_t* double_len = malloc(sizeof(int32_t) * 10); 282: int32_t* bin_len = malloc(sizeof(int32_t) * 10); </pre>	
	污染路径: 无	
20	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 87
	步骤描述: 动态分配的指针 blob_len3 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 85: int32_t* blob_len = malloc(sizeof(int32_t) * 10); 86: int32_t* blob_len2 = malloc(sizeof(int32_t) * 10); 87: int32_t* blob_len3 = malloc(sizeof(int32_t) * 10); 88: int32_t* blob_len4 = malloc(sizeof(int32_t) * 10); 89: int32_t* blob_len5 = malloc(sizeof(int32_t) * 10); </pre>	
	污染路径: 无	
21	缺陷发生位置: /contrib/TSZ/sz/src/sz_float.c	行号: 114
	步骤描述: 动态分配的指针 type 第一次使用前必须进行是否为 NULL 的判别	
	代码片段:	

	112: 113: // 1 type 114: int* type = (int*) malloc(dataLength*sizeof(int)); 115: float* spaceFillingValue = oriData; // 116: // 2 lead	
	污染路径: 无	
22	缺陷发生位置: /docs/examples/c/stmt2_insert_demo.c	行号: 156
	步骤描述: 动态分配的指针 voltage 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 154: int64_t *ts = (int64_t *)malloc(NUM_OF_ROWS * sizeof(int64_t)); 155: float *current = (float *)malloc(NUM_OF_ROWS * sizeof(float)); 156: int *voltage = (int *)malloc(NUM_OF_ROWS * sizeof(int)); 157: float *phase = (float *)malloc(NUM_OF_ROWS * sizeof(float)); 158: if (ts == NULL current == NULL voltage == NULL phase == NULL) {	
	污染路径: 无	
23	缺陷发生位置: /docs/examples/c/stmt2_insert_demo.c	行号: 154
	步骤描述: 动态分配的指针 ts 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 152: } 153: 154: int64_t *ts = (int64_t *)malloc(NUM_OF_ROWS * sizeof(int64_t)); 155: float *current = (float *)malloc(NUM_OF_ROWS * sizeof(float)); 156: int *voltage = (int *)malloc(NUM_OF_ROWS * sizeof(int));	
	污染路径: 无	
24	缺陷发生位置:	行号:

	/test/new_test_framework/script/api/apitest.c	406
	步骤描述: 动态分配的指针 t8_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 404: } v; 405: 406: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 407: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 408: int32_t* t32_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
25	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 4946
	步骤描述: 动态分配的指针 array_u64 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 4944: } else { 4945: if (TSDB_DATA_TYPE_BIGINT == field->array_type) { 4946: uint64_t *array_u64 = malloc(sizeof(uint64_t)); 4947: TOOLS_ASSERT(array_u64); 4948: *array_u64 = 0;	
	污染路径: 无	
26	缺陷发生位置: /contrib/test/craft/raftServer.c	行号: 43
	步骤描述: 动态分配的指针 tmp 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 41: } 42: 43: char* tmp = (char*)malloc(strlen(str) + 1); 44: strcpy(tmp, str); 45: char* context;	
	污染路径: 无	
27	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 4920
	步骤描述: 动态分配的指针 array_u64 第一次使用前必须进行是否为 NULL 的判别	

	代码片段: <pre> 4918: } else { 4919: if (TSDB_DATA_TYPE_BIGINT == field->array_type) { 4920: uint64_t *array_u64 = malloc(sizeof(uint64_t)); 4921: TOOLS_ASSERT(array_u64); 4922: *array_u64 = 0; </pre>	
	污染路径: 无	
28	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 91
	步骤描述: 动态分配的指针 t64_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 89: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 90: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 91: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 92: int32_t* float_len = malloc(sizeof(int32_t) * 10); 93: int32_t* double_len = malloc(sizeof(int32_t) * 10); </pre>	
29	污染路径: 无	
	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 84
	步骤描述: 动态分配的指针 t64_len 第一次使用前必须进行是否为 NULL 的判别	
30	代码片段: <pre> 82: } v; 83: 84: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 85: int32_t* blob_len = malloc(sizeof(int32_t) * 10); 86: int32_t* blob_len2 = malloc(sizeof(int32_t) * 10); </pre>	
	污染路径: 无	
	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeRealFunctionsTest.cpp	行号: 180
	步骤描述: 动态分配的指针 oldStr 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 178: int32_t oldLen = 10; </pre>	

	179: int32_t newLen = 20; 180: char* oldStr = (char*)malloc(oldLen); 181: char* newStr = (char*)malloc(newLen); 182: strcpy(oldStr, "old");	
	污染路径: 无	
31	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 276
	步骤描述: 动态分配的指针 params 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 274: pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; 275: 276: ThreadParam* params = malloc(thread_count * sizeof(ThreadParam)); 277: pthread_t* threads = malloc(thread_count * sizeof(pthread_t)); 278: double* thread_qps = malloc(thread_count * sizeof(double));	
	污染路径: 无	
32	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 2633
	步骤描述: 动态分配的指针 address 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 2631: void* ZSTDv07_defaultAllocFunction(void* opaque, size_t size) 2632: { 2633: void* address = malloc(size); 2634: (void)opaque; 2635: /* printf("alloc %p, %d opaque=%p \n", address, (int)size, opaque); */	
	污染路径: 无	
33	缺陷发生位置: /contrib/TSZ/sz/src/szd_double.c	行号: 122
	步骤描述: 动态分配的指针 type 第一次使用前必须进行是否为 NULL 的判别	

	代码片段: <pre> 120: /*data = (double*)malloc(sizeof(double)*dataSeriesLength); comment by tickduan 121: 122: int* type = (int*)malloc(dataSeriesLength*sizeof(int)); 123: 124: if (tdps->ifAdtFse == false) { </pre>	
	污染路径: 无	
34	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 502
	步骤描述: 动态分配的指针 suffix0 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 500: U32 minRatio, U32 notificationLevel) 501: { 502: int* const suffix0 = (int*)malloc((bufferSize+2)*sizeof(*suffix0)); 503: int* const suffix = suffix0+1; 504: U32* reverseSuffix = (U32*)malloc((bufferSize)*sizeof(*reverseSuffix)); </pre>	
	污染路径: 无	
35	缺陷发生位置: /docs/examples/c-ws-new/stmt2_insert_demo.c	行号: 66
	步骤描述: 动态分配的指针 gid 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 64: 65: // Allocate memory for tags data 66: int *gid = (int *)malloc(sizeof(int)); 67: int *gid_len = (int *)malloc(sizeof(int)); 68: *gid = i; </pre>	
	污染路径: 无	
36	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 278
	步骤描述: 动态分配的指针 t32_len 第一次使用前必须进行是否为 NULL 的判别	

	代码片段: 276: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 277: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 278: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 279: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 280: int32_t* float_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
37	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10000.c	行号: 94
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 92: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 93: pFsm->apply = fsmApplyCb; 94: Tsdb *tsdb = malloc(sizeof(*tsdb)); 95: pFsm->data = tsdb; 96:	
	污染路径: 无	
38	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 240
	步骤描述: 动态分配的指针 float_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 238: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 239: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 240: int32_t* float_len = malloc(sizeof(int32_t) * 10); 241: int32_t* double_len = malloc(sizeof(int32_t) * 10); 242: int32_t* bin_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
39	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 85
	步骤描述: 动态分配的指针 b 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 83: // col params 84: int64_t** ts = (int64_t**)malloc(CTB_NUMS * sizeof(int64_t*));	

	85: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 86: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int)); 87: int* b_len = (int*)malloc(ROW_NUMS * sizeof(int));	
	污染路径: 无	
40	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 86
	步骤描述: 动态分配的指针 blob_len2 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 84: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 85: int32_t* blob_len = malloc(sizeof(int32_t) * 10); 86: int32_t* blob_len2 = malloc(sizeof(int32_t) * 10); 87: int32_t* blob_len3 = malloc(sizeof(int32_t) * 10); 88: int32_t* blob_len4 = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
41	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 36
	步骤描述: 动态分配的指针 pRow 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 34: 35: SSdbRow* sdbAllocRow(int size) { 36: SSdbRow* pRow = (SSdbRow*)malloc(sizeof(SSdbRow)); 37: memset(pRow, 0, sizeof(SSdbRow)); 38: return pRow;	
	污染路径: 无	
42	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 279
	步骤描述: 动态分配的指针 t64_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 277: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 278: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 279: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 280: int32_t* float_len = malloc(sizeof(int32_t) * 10); 281: int32_t* double_len = malloc(sizeof(int32_t) * 10);	

	污染路径: 无	
43	缺陷发生位置: /docs/examples/c-ws-new/stmt2_insert_demo.c	行号: 71
	步骤描述: 动态分配的指针 location 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 69: *gid_len = sizeof(int); 70: 71: char *location = (char *)malloc(20 * sizeof(char)); 72: int *location_len = (int *)malloc(sizeof(int)); 73: *location_len = sprintf(location, "location_%d", i);	
	污染路径: 无	
44	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 238
	步骤描述: 动态分配的指针 t32_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 236: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 237: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 238: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 239: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 240: int32_t* float_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
45	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 921
	步骤描述: 动态分配的指针 entries 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 919: struct hol_entry *e; 920: unsigned num_entries = hol->num_entries + more->num_entries; 921: struct hol_entry *entries = 922: malloc (num_entries * sizeof (struct hol_entry)); 923: unsigned hol_so_len = strlen (hol->short_options);	
	污染路径: 无	

46	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 237
	步骤描述: 动态分配的指针 t16_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 235: 236: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 237: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 238: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 239: int32_t* t64_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
47	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 74
	步骤描述: 动态分配的指针 tbs 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 72: 73: // tbname 74: char** tbs = (char**)malloc(CTB_NUMS * sizeof(char*)); 75: for (int i = 0; i < CTB_NUMS; i++) { 76: tbs[i] = (char*)malloc(sizeof(char) * 20);	
	污染路径: 无	
48	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 412
	步骤描述: 动态分配的指针 L 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 410: if(nodeCount<=256) 411: { 412: unsigned char* L = (unsigned char*)malloc(nodeCount*sizeof(unsigned char)); 413: memset(L, 0, nodeCount*sizeof(unsigned char)); 414: unsigned char* R = (unsigned char*)malloc(nodeCount*sizeof(unsigned char));	
	污染路径: 无	
49	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeRealFunctionsTest.cpp	行号: 231

	步骤描述: 动态分配的指针 str2 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 229: 230: char* str1 = (char*)malloc(len1); 231: char* str2 = (char*)malloc(len2); 232: char* str3 = (char*)malloc(len3); 233:	
	污染路径: 无	
50	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 85
	步骤描述: 动态分配的指针 blob_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 83: 84: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 85: int32_t* blob_len = malloc(sizeof(int32_t) * 10); 86: int32_t* blob_len2 = malloc(sizeof(int32_t) * 10); 87: int32_t* blob_len3 = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
51	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 110
	步骤描述: 动态分配的指针 t64_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 108: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 109: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 110: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 111: int32_t* float_len = malloc(sizeof(int32_t) * 10); 112: int32_t* double_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
52	缺陷发生位置: /docs/examples/c-ws-new/stmt2_insert_demo.c	行号: 82
	步骤描述: 动态分配的指针 ts 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 80: (*params)[i] = (TAOS_STMT2_BIND *)malloc(4 *	

	sizeof(TAOS_STMT2_BIND)); 81: 82: int64_t *ts = (int64_t *)malloc(NUM_OF_ROWS * sizeof(int64_t)); 83: float *current = (float *)malloc(NUM_OF_ROWS * sizeof(float)); 84: int *voltage = (int *)malloc(NUM_OF_ROWS * sizeof(int));	
	污染路径: 无	
53	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 276
	步骤描述: 动态分配的指针 t8_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 274: } v; 275: 276: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 277: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 278: int32_t* t32_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
54	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 407
	步骤描述: 动态分配的指针 t16_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 405: 406: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 407: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 408: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 409: int32_t* t64_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
55	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 410
	步骤描述: 动态分配的指针 float_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 408: int32_t* t32_len = malloc(sizeof(int32_t) * 10);	

	409: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 410: int32_t* float_len = malloc(sizeof(int32_t) * 10); 411: int32_t* double_len = malloc(sizeof(int32_t) * 10); 412: int32_t* bin_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
56	缺陷发生位置: /test/new_test_framework/script/api/insertBlob.c	行号: 58
	步骤描述: 动态分配的指针 qstr 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 56: 57: int main(int argc, char *argv[]) { 58: char *qstr = (char *)malloc(BLOB_LEN); 59: 60: // connect to server	
	污染路径: 无	
57	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 35
	步骤描述: 动态分配的指针 tmp 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 33: 34: void createCtb(TAOS* taos, const char* tname) { 35: char* tmp = (char*)malloc(sizeof(char) * 100); 36: sprintf(tmp, "create table db.%s using db.stb tags(0, 'after')", tname); 37: do_query(taos, tmp);	
	污染路径: 无	
58	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeRealFunctionsTest.cpp	行号: 230
	步骤描述: 动态分配的指针 str1 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 228: int32_t len3 = 15; 229: 230: char* str1 = (char*)malloc(len1); 231: char* str2 = (char*)malloc(len2);	

	232: char* str3 = (char*)malloc(len3);	
	污染路径: 无	
59	缺陷发生位置: /contrib/test/traft/cluster/node10001_restart.c	行号: 97
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 95: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 96: pFsm->apply = fsmApplyCb; 97: Tsdb *tsdb = malloc(sizeof(*tsdb)); 98: pFsm->data = tsdb; 99:	
	污染路径: 无	
60	缺陷发生位置: /contrib/test/traft/cluster/node10000_restart.c	行号: 97
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 95: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 96: pFsm->apply = fsmApplyCb; 97: Tsdb *tsdb = malloc(sizeof(*tsdb)); 98: pFsm->data = tsdb; 99:	
	污染路径: 无	
61	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 107
	步骤描述: 动态分配的指针 type 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 105: computeReqLength_double(realPrecision, radExpo, &reqLength, &medianValue); 106: 107: int* type = (int*) malloc(dataLength*sizeof(int)); 108: double* spaceFillingValue = oriData; // 109:	
	污染路径:	

	无	
62	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 73
	步骤描述: 动态分配的指针 tbs 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 71: 72: // tbname 73: char** tbs = (char**)malloc(CTB_NUMS * sizeof(char*)); 74: for (int i = 0; i < CTB_NUMS; i++) { 75: tbs[i] = (char*)malloc(sizeof(char) * 20);	
	污染路径: 无	
63	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 239
	步骤描述: 动态分配的指针 t64_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 237: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 238: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 239: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 240: int32_t* float_len = malloc(sizeof(int32_t) * 10); 241: int32_t* double_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
64	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 73
	步骤描述: 动态分配的指针 tbs 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 71: 72: // tbname 73: char** tbs = (char**)malloc(CTB_NUMS * sizeof(char*)); 74: for (int i = 0; i < CTB_NUMS; i++) { 75: tbs[i] = (char*)malloc(sizeof(char) * 20);	
	污染路径: 无	
65	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 35
	步骤描述:	

	动态分配的指针 tmp 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 33: 34: void createCtb(TAOS* taos, const char* tname) { 35: char* tmp = (char*)malloc(sizeof(char) * 100); 36: sprintf(tmp, "create table db.%s using db.stb tags(0, 'after')", tname); 37: do_query(taos, tmp);	
	污染路径: 无	
66	缺陷发生位置:	行号:
	/tools/taos-tools/src/taosdump.c	4985
	步骤描述: 动态分配的指针 array_u16 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 4983: } else { 4984: if (TSDB_DATA_TYPE_INT == field->array_type) { 4985: uint16_t *array_u16 = malloc(sizeof(uint16_t)); 4986: TOOLS_ASSERT(array_u16); 4987: *array_u16 = 0;	
	污染路径: 无	
67	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmt2-nohole.c	107
	步骤描述: 动态分配的指针 t8_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 105: } v; 106: 107: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 108: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 109: int32_t* t32_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
68	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmt2-nchar.c	88
	步骤描述: 动态分配的指针 blob_len4 第一次使用前必须进行是否为 NULL 的判别	

	代码片段: 86: int32_t* blob_len2 = malloc(sizeof(int32_t) * 10); 87: int32_t* blob_len3 = malloc(sizeof(int32_t) * 10); 88: int32_t* blob_len4 = malloc(sizeof(int32_t) * 10); 89: int32_t* blob_len5 = malloc(sizeof(int32_t) * 10); 90:	
	污染路径: 无	
69	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 84
	步骤描述: 动态分配的指针 ts 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 82: for (int r = 0; r < CYC_NUMS; r++) { 83: // col params 84: int64_t** ts = (int64_t**)malloc(CTB_NUMS * sizeof(int64_t*)); 85: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 86: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int));	
	污染路径: 无	
70	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 414
	步骤描述: 动态分配的指针 R 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 412: unsigned char* L = (unsigned char*)malloc(nodeCount*sizeof(unsigned char)); 413: memset(L, 0, nodeCount*sizeof(unsigned char)); 414: unsigned char* R = (unsigned char*)malloc(nodeCount*sizeof(unsigned char)); 415: memset(R, 0, nodeCount*sizeof(unsigned char)); 416: unsigned int* C = (unsigned int*)malloc(nodeCount*sizeof(unsigned int));	
	污染路径: 无	
71	缺陷发生位置: /test/new_test_framework/script/api/insert_stb.c	行号: 156
	步骤描述: 动态分配的指针 qstr 第一次使用前必须进行是否为 NULL 的判别	

	代码片段: 154: exit(1); 155: } 156: char* qstr = malloc(1024*1024); 157: { 158: printf("test insert into tb using stb\n\n");	
	污染路径: 无	
72	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 180
	步骤描述: 动态分配的指针 vgld1 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 178: int rtGetTablesRouteInfo(TAOS * taos) { 179: char *table = {0}; 180: int *vgld1 = malloc(rtTables * sizeof(int)); 181: int vgld2 = 0; 182: char sql[1024] = {0};	
	污染路径: 无	
73	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 924
	步骤描述: 动态分配的指针 short_options 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 922: malloc (num_entries * sizeof (struct hol_entry)); 923: unsigned hol_so_len = strlen (hol->short_options); 924: char *short_options = 925: malloc (hol_so_len + strlen (more->short_options) + 1); 926:	
	污染路径: 无	
74	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 86
	步骤描述: 动态分配的指针 ts_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 84: int64_t** ts = (int64_t**)malloc(CTB_NUMS *	

	<pre>sizeof(int64_t*)); 85: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 86: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int)); 87: int* b_len = (int*)malloc(ROW_NUMS * sizeof(int)); 88: for (int i = 0; i < ROW_NUMS; i++) {</pre>	
	污染路径: 无	
75	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 85
	步骤描述: 动态分配的指针 ts 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre>83: for (int r = 0; r < CYC_NUMS; r++) { 84: // col params 85: int64_t** ts = (int64_t**)malloc(CTB_NUMS * sizeof(int64_t*)); 86: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 87: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int));</pre>	
	污染路径: 无	
76	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeRealFunctionsTest.cpp	行号: 181
	步骤描述: 动态分配的指针 newStr 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre>179: int32_t newLen = 20; 180: char* oldStr = (char*)malloc(oldLen); 181: char* newStr = (char*)malloc(newLen); 182: strcpy(oldStr, "old"); 183: strcpy(newStr, "new_string");</pre>	
	污染路径: 无	
77	缺陷发生位置: /contrib/test/traft/cluster/node10000.c	行号: 97
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre>95: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 96: pFsm->apply = fsmApplyCb; 97: Tsdb *tsdb = malloc(sizeof(*tsdb));</pre>	

	98: pFsm->data = tsdb; 99:	
	污染路径: 无	
78	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 575
	步骤描述: 动态分配的指针 cl 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 573: struct hol_cluster *parent, const struct argp *argp) 574: { 575: struct hol_cluster *cl = malloc (sizeof (struct hol_cluster)); 576: if (cl) 577: {	
	污染路径: 无	
79	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 277
	步骤描述: 动态分配的指针 threads 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 275: 276: ThreadParam* params = malloc(thread_count * sizeof(ThreadParam)); 277: pthread_t* threads = malloc(thread_count * sizeof(pthread_t)); 278: double* thread_qps = malloc(thread_count * sizeof(double)); 279:	
	污染路径: 无	
80	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 18
	步骤描述: 动态分配的指针 tmp 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 16: 17: void parseAddr(const char *addr, char *host, int len,	

	uint32_t *port) { 18: char* tmp = (char*)malloc(strlen(addr) + 1); 19: strcpy(tmp, addr); 20:	
	污染路径: 无	
81	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10001.c	行号: 94
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 92: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 93: pFsm->apply = fsmApplyCb; 94: Tsdb *tsdb = malloc(sizeof(*tsdb)); 95: pFsm->data = tsdb; 96:	
	污染路径: 无	
82	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 409
	步骤描述: 动态分配的指针 t64_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 407: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 408: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 409: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 410: int32_t* float_len = malloc(sizeof(int32_t) * 10); 411: int32_t* double_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
83	缺陷发生位置: /contrib/test/traft/cluster/node10001.c	行号: 97
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 95: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 96: pFsm->apply = fsmApplyCb; 97: Tsdb *tsdb = malloc(sizeof(*tsdb)); 98: pFsm->data = tsdb; 99:	

	污染路径: 无	
84	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 108
	步骤描述: 动态分配的指针 t16_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 106: 107: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 108: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 109: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 110: int32_t* t64_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
85	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 86
	步骤描述: 动态分配的指针 ts_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 84: int64_t** ts = (int64_t**)malloc(CTB_NUMS * sizeof(int64_t*)); 85: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 86: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int)); 87: int* b_len = (int*)malloc(ROW_NUMS * sizeof(int)); 88: for (int i = 0; i < ROW_NUMS; i++) {	
	污染路径: 无	
86	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 408
	步骤描述: 动态分配的指针 t32_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 406: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 407: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 408: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 409: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 410: int32_t* float_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	

87	缺陷发生位置: /contrib/test/traft/cluster/node10002.c	行号: 97
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 95: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 96: pFsm->apply = fsmApplyCb; 97: Tsdb *tsdb = malloc(sizeof(*tsdb)); 98: pFsm->data = tsdb; 99:	
	污染路径: 无	
88	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 84
	步骤描述: 动态分配的指针 ts 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 82: for (int r = 0; r < CYC_NUMS; r++) { 83: // col params 84: int64_t** ts = (int64_t**)malloc(CTB_NUMS * sizeof(int64_t*)); 85: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 86: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int));	
	污染路径: 无	
89	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 88
	步骤描述: 动态分配的指针 t8_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 86: } v; 87: 88: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 89: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 90: int32_t* t32_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
90	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 278
	步骤描述:	

	动态分配的指针 thread_qps 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 276: ThreadParam* params = malloc(thread_count * sizeof(ThreadParam)); 277: pthread_t* threads = malloc(thread_count * sizeof(pthread_t)); 278: double* thread_qps = malloc(thread_count * sizeof(double)); 279: 280: if (query_mode == 2) { </pre>	
	污染路径: 无	
91	缺陷发生位置:	行号:
	/docs/examples/c-ws-new/stmt2_insert_demo.c	67
	步骤描述: 动态分配的指针 gid_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 65: // Allocate memory for tags data 66: int *gid = (int *)malloc(sizeof(int)); 67: int *gid_len = (int *)malloc(sizeof(int)); 68: *gid = i; 69: *gid_len = sizeof(int); </pre>	
92	污染路径: 无	
	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/test/xnode/xnodeRealFunctionsTest.cpp	207
	步骤描述: 动态分配的指针 oldStr 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 205: int32_t oldLen = 10; 206: int32_t newLen = 0; 207: char* oldStr = (char*)malloc(oldLen); 208: char* newStr = nullptr; 209: strcpy(oldStr, "old"); </pre>	
93	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/test/traft/single_node/singleNode.c	97
	步骤描述: 动态分配的指针 tsdb 第一次使用前必须进行是否为 NULL 的判别	

	代码片段: 95: struct raft_fsm *pFsm = malloc(sizeof(*pFsm)); 96: pFsm->apply = fsmApplyCb; 97: Tsdb *tsdb = malloc(sizeof(*tsdb)); 98: pFsm->data = tsdb; 99:	
	污染路径: 无	
94	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 92
	步骤描述: 动态分配的指针 float_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 90: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 91: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 92: int32_t* float_len = malloc(sizeof(int32_t) * 10); 93: int32_t* double_len = malloc(sizeof(int32_t) * 10); 94: int32_t* bin_len = malloc(sizeof(int32_t) * 10);	
	污染路径: 无	
95	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 39
	步骤描述: 动态分配的指针 tmp 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 37: int parseCommand(const char* str, char* token1, char* token2, char* token3, int len) 38: { 39: char* tmp = (char*)malloc(strlen(str) + 1); 40: strcpy(tmp, str); 41:	
	污染路径: 无	
96	缺陷发生位置: /docs/examples/c/stmt2_insert_demo.c	行号: 155
	步骤描述: 动态分配的指针 current 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 153: 154: int64_t *ts = (int64_t *)malloc(NUM_OF_ROWS *	

	<pre> sizeof(int64_t)); 155: float *current = (float *)malloc(NUM_OF_ROWS * sizeof(float)); 156: int *voltage = (int *)malloc(NUM_OF_ROWS * sizeof(int)); 157: float *phase = (float *)malloc(NUM_OF_ROWS * sizeof(float)); </pre>	
	污染路径: 无	
97	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 90
	步骤描述: 动态分配的指针 t32_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 88: int32_t* t8_len = malloc(sizeof(int32_t) * 10); 89: int32_t* t16_len = malloc(sizeof(int32_t) * 10); 90: int32_t* t32_len = malloc(sizeof(int32_t) * 10); 91: int32_t* t64_len = malloc(sizeof(int32_t) * 10); 92: int32_t* float_len = malloc(sizeof(int32_t) * 10); </pre>	
	污染路径: 无	
98	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 418
	步骤描述: 动态分配的指针 t 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: <pre> 416: unsigned int* C = (unsigned int*)malloc(nodeCount*sizeof(unsigned int)); 417: memset(C, 0, nodeCount*sizeof(unsigned int)); 418: unsigned char* t = (unsigned char*)malloc(nodeCount*sizeof(unsigned char)); 419: memset(t, 0, nodeCount*sizeof(unsigned char)); 420: </pre>	
	污染路径: 无	
99	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 39
	步骤描述: 动态分配的指针 tmp 第一次使用前必须进行是否为 NULL 的判别	
	代码片段:	

	37: 38: void createCtb(TAOS* taos, const char* tname) { 39: char* tmp = (char*)malloc(sizeof(char) * 100); 40: sprintf(tmp, "create table db.%s using db.stb tags(0, 'after')", tname); 41: do_query(taos, tmp);	
	污染路径: 无	
100	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 87
	步骤描述: 动态分配的指针 ts_len 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 85: int64_t** ts = (int64_t**)malloc(CTB_NUMS * sizeof(int64_t*)); 86: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 87: int* ts_len = (int*)malloc(ROW_NUMS * sizeof(int)); 88: int* b_len = (int*)malloc(ROW_NUMS * sizeof(int)); 89: for (int i = 0; i < ROW_NUMS; i++) {	
	污染路径: 无	

7.2.1.2 WK4692 空指针必须使用 NULL,禁止使用整数 0

缺陷数量: 104 个		
缺陷描述: 空指针必须使用 NULL,禁止使用整数 0。		
修复意见: 无		
1	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 676
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 674: { 675: return hol_entry_short_iterate (entry, until_short, 676: entry->argp->argp_domain, 0);	

	677: } 678:	
	污染路径: 无	
2	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1310
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1308: } else { 1309: code = pAPI->metaFn.resumeTableMetaCursor(pInfo->pCur, 0, 0); 1310: if (code != 0) { 1311: pAPI->metaFn.closeTableMetaCursor(pInfo->pCur); 1312: pInfo->pCur = NULL;	
	污染路径: 无	
3	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 103
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 101: 102: remainder = pBuf->numOfTables % pBuf->numPerBucket; 103: if (remainder > 0) { 104: TSDB_CHECK_CONDITION(taosArrayGetSize(pBuf->pData) > 0, code, lino, _end, TSDB_CODE_INVALID_PARA); 105: px = taosArrayPop(pBuf->pData);	
	污染路径: 无	
4	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1478
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1476: colDataSetNNULL(pOutputCol, 0, numOfRows); 1477: } else { 1478: for (; i >= 0 && i < numOfRows; i += step, output += 1) {	

	1479: if (IS_HELPER_NULL(pLeftCol, i)) { 1480: colDataSetNULL(pOutputCol, i);	
	污染路径: 无	
5	缺陷发生位置: /source/libs/parser/test/parTestMain.cpp	行号: 91
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 89: {"limitSql", required_argument, NULL, 'i'}, 90: {"log", required_argument, NULL, 'l'}, 91: {0, 0, 0, 0} 92: }; 93: // clang-format on	
	污染路径: 无	
6	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 53
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 51: {"records", 'n', "NUMBER", 0, 52: "Number of records for each table, default is 10000."}, 53: {0}}; 54: 55: static error_t parse_opt(int key, char *arg, struct argp_state *state) {	
	污染路径: 无	
7	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2421
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2419: //120table 60 record each table 2420: int sql_perf1(TAOS *taos) { 2421: char *sql[3000] = {0}; 2422: TAOS_RES *result; 2423:	

	污染路径: 无	
8	缺陷发生位置: /docs/examples/c-ws/sml_insert_demo.c	行号: 97
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 95: 96: // opentsdb json protocol 97: char *jsons[1] = {0}; 98: // allocate memory for json data. can not use static memory. 99: totalLines = 0;	
	污染路径: 无	
9	缺陷发生位置: /utils/test/c/createTable.c	行号: 288
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 286: taos_close(con); 287: taosMemoryFree(qstr); 288: return 0; 289: } 290:	
	污染路径: 无	
10	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1697
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1695: case TSDB_DATA_TYPE_VARBINARY: 1696: case TSDB_DATA_TYPE_GEOMETRY: 1697: if (bufSize < 0) { 1698: return TSDB_CODE_TSC_INVALID_VALUE; 1699: }	
	污染路径: 无	
11	缺陷发生位置:	行号:

	/tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	157
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 155: { 156: free (fs); 157: fs = 0; 158: } 159: else </pre>	
	污染路径: 无	
12	缺陷发生位置: /source/libs/scalar/src/scvector.c	行号: 1321
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 1319: colDataSetNNULL(pOutputCol, 0, numOfRows); 1320: } else { 1321: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1322: if (IS_HELPER_NULL(pLeftCol, i)) { 1323: colDataSetNULL(pOutputCol, i); </pre>	
	污染路径: 无	
13	缺陷发生位置: /utils/test/c/sml_test.c	行号: 232
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 230: "34}}", {"metric\":"sys.cpu.nice","\timestamp\:1662344045,\value\ ":9,\tags\":{"host\":"web02","\dc\:4}}]"}]; 231: 232: char *sql1[1] = {0}; 233: for (int i = 0; i < 1; i++) { 234: sql1[i] = taosMemoryCalloc(1, 1024); </pre>	
	污染路径: 无	
14	缺陷发生位置:	行号:

	/docs/examples/c/schemaless.c	47
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 45: (void)taos_select_db(taos, "db"); 46: 47: time_t ct = time(0); 48: int64_t ts = ct * 1000; 49: char* lineFormat = "sta %d,t0=true,t1=127i8,t2=32767i16,t3= %di32,t4=9223372036854775807i64,t9=11.12345f32,t10=22 .123456789f64,t11=\"binaryTagValue\",t12=L\"ncharTagValue\ \" c0=true,c1=127i8,c2=32767i16,c3=2147483647i32,c4=9223 372036854775807i64,c5=254u8,c6=32770u16,c7=21474836 99u32,c8=9223372036854775899u64,c9=11.12345f32,c10= 22.123456789f64,c11=\"binaryValue\",c12=L\"ncharValue\ \" %\" PRIu64;	
	污染路径: 无	
15	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2817
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2815: tfree(fileList[i]); 2816: } 2817: tfree(fileList); 2818: } 2819:	
	污染路径: expanded from macro 'tfree'	
16	缺陷发生位置: /tools/taos-tools/deps/toolsc.Json/src/toolsc.Json.c	行号: 1027
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1025: cJSON_PUBLIC(tools_cJSON *) tools_cJSON_Parse(const char *value) 1026: { 1027: return tools_cJSON_ParseWithOpts(value, 0, 0);	

	1028: } 1029:	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 95
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 93: 94: // opentsdb json protocol 95: char *jsons[1] = {0}; 96: // allocate memory for json data. can not use static memory. 97: size_t size = 1024;	
	污染路径: 无	
18	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 136
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 134: static const struct argp_option argp_default_options[] = 135: { 136: {"help", '?', 0, 0, N_("Give this help list"), -1}, 137: {"usage", OPT_USAGE, 0, 0, N_("Give a short usage message")}, 138: {"program-name", OPT_PROGNAME, "NAME", OPTION_HIDDEN, N_("Set the program name")},	
	污染路径: 无	
19	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 137
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 135: { 136: {"help", '?', 0, 0, N_("Give this help list"), -1},	

	137: {"usage", OPT_USAGE, 0, 0, N_("Give a short usage message")}, 138: {"program-name",OPT_PROGNAME,"NAME", OPTION_HIDDEN, N_("Set the program name")}, 139: {"HANG", OPT_HANG, "SECS", OPTION_ARG_OPTIONAL OPTION_HIDDEN,	
	污染路径: 无	
20	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号: 79
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 77: 78: { 0, 0, 0, 0, "Some more nested options:", 10}, 79: {"subopt3", 'p', 0, 0, "Nested option 3"}, 80: 81: {"subopt4", 'q', 0, 0, "Nested option 4", 1},	
	污染路径: 无	
21	缺陷发生位置: /utils/test/c/sml_test.c	行号: 261
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 259: }; 260: 261: char *sql3[1] = {0}; 262: for (int i = 0; i < 1; i++) { 263: sql3[i] = taosMemoryCalloc(1, 1024);	
	污染路径: 无	
22	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 2092
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2090: SExtWinTimeWindow* pWin = NULL; 2091: int32_t idx = taosArraySearchIdx(pWins, tsCol,	

	extWinTsWinCompare, TD_EQ); 2092: if (idx >= 0) { 2093: for (int i = idx - 1; i >= 0; --i) { 2094: pWin = TARRAY_GET_ELEM(pWins, i);	
	污染路径: 无	
23	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 75
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 73: struct argp_option sub_options[] = 74: { 75: {"subopt1", 's', 0, 0, "Nested option 1"}, 76: {"subopt2", 'S', 0, 0, "Nested option 2"}, 77:	
	污染路径: 无	
24	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 1808
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1806: static int32_t extWinGetMultiTbWinFromTs(SOperatorInfo* pOperator, SArray* pWins, int64_t* tsCol, int64_t rowNum, int32_t* startPos) { 1807: int32_t idx = taosArraySearchIdx(pWins, tsCol, extWinTsWinCompare, TD_EQ); 1808: if (idx >= 0) { 1809: *startPos = 0; 1810: return idx;	
	污染路径: 无	
25	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 1004
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段:	

	1002: *pRecv = p->pBuf; 1003: *len = p->nBuf; 1004: p->pBuf = 0; 1005: p->nBuf = 0; 1006: } else {	
	污染路径: 无	
26	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 271
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 269: int32_t num) { 270: int32_t code = TSDB_CODE_SUCCESS; 271: if (num > 0) { 272: int32_t start = pEncoder->offset; 273: pEncoder->offset += sizeof(STlv);	
	污染路径: 无	
27	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 2660
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2658: tools_cJSON *a_element = NULL; 2659: tools_cJSON *b_element = NULL; 2660: tools_cJSON_ArrayForEach(a_element, a) 2661: { 2662: /* TODO This has O(n^2) runtime, which is horrible! */	
	污染路径: expanded from macro 'tools_cJSON_ArrayForEach' expanded from macro 'NULL'	
28	缺陷发生位置: /source/util/test/decompressTest.cpp	行号: 22
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 20: ONE_STAGE_COMP, NULL, 0); 21:	

	22: char* decompOutput[10 * 8] = {0}; 23: 24: tsDecompressTimestamp(pOutput, len, 10, decompOutput, sizeof(int64_t) * 10, ONE_STAGE_COMP, NULL, 0);	
	污染路径: 无	
29	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2521
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2519: //small record size 2520: int sql_s_perf1(TAOS *taos) { 2521: char *sql[3000] = {0}; 2522: TAOS_RES *result; 2523:	
	污染路径: 无	
30	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 168
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 166: {"escape-character", 'E', 0, 0, BENCH_ESCAPE}, 167: {"chinese", 'C', 0, 0, BENCH_CHINESE}, 168: {"normal-table", 'N', 0, 0, BENCH_NORMAL}, 169: {"random", 'M', 0, 0, BENCH_RANDOM}, 170: {"aggr-func", 'x', 0, 0, BENCH_AGGR},	
	污染路径: 无	
31	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 428
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 426: taosArrayClear(pldx); 427: int32_t newSize = (int32_t)taosArrayGetSize(pNewIdx); 428: if (newSize > 0) { 429: void* dest = taosArrayReserve(pldx, newSize);	

	430: TSDB_CHECK_NULL(dest, code, lino, _exit, terrno);	
	污染路径: 无	
32	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 74
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 72: int32_t qwtTestQueryQueueNum = 0; 73: SRWLatch qwtTestQueryQueueLock = 0; 74: struct SRpcMsg *qwtTestQueryQueue[qwtTestQueryQueueSize] = {0}; 75: 76: int32_t qwtTestFetchQueueRIdx = 0;	
	污染路径: 无	
33	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 129
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 127: } 128: 129: if (remainder > 0) { 130: p = taosMemoryCalloc(remainder, sizeof(STableBlockScanInfo)); 131: TSDB_CHECK_NULL(p, code, lino, _end, terrno);	
	污染路径: 无	
34	缺陷发生位置: /contrib/test/sqlite/sqliteTest.c	行号: 22
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 20: int main(int argc, char const *argv[]) { 21: sqlite3 *db; 22: char * err_msg = 0; 23: 24: int rc = sqlite3_open("test.db", &db);	

	污染路径: 无	
35	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 81
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 79: {"subopt3", 'p', 0, 0, "Nested option 3"}, 80: 81: {"subopt4", 'q', 0, 0, "Nested option 4", 1}, 82: 83: {0}	
	污染路径: 无	
36	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1107
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1105: CJSON_PUBLIC(char *) tools_cJSON_PrintBuffered(const tools_cJSON *item, int prebuffer, tools_cJSON_bool fmt) 1106: { 1107: printbuffer p = { 0, 0, 0, 0, 0, 0, { 0, 0, 0 } }; 1108: 1109: if (prebuffer < 0)	
	污染路径: 无	
37	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1269
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1267: while (name) { 1268: if (0 == strcmp(name, dbName)) { 1269: tfree(dupSeq); 1270: return 0; 1271: }	
	污染路径: expanded from macro 'tfree'	

38	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 267
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 265: 266: static void ttq_signal_setup(void) { 267: struct sigaction sa = {0}; 268: 269: sa.sa_handler = ttq_handle_sigint;	
	污染路径: 无	
39	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 839
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 837: } else { 838: code = pAPI->metaFn.resumeTableMetaCursor(pInfo->pCur, 0, 0); 839: if (code != 0) { 840: pAPI->metaFn.closeTableMetaCursor(pInfo->pCur); 841: pInfo->pCur = NULL;	
	污染路径: 无	
40	缺陷发生位置: /contrib/lemon/lemon.c	行号: 75
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 73: ** Global linked list of all memory allocations. 74: */ 75: static MemChunk *memChunkList = 0; 76: 77: /*	
	污染路径: 无	
41	缺陷发生位置: /source/util/test/decompressTest.cpp	行号: 49
	步骤描述:	

	空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 47: NULL, 0); 48: 49: decompOutput[4 * 8] = {0}; 50: tsDecompressTimestamp(pOutput, len, 4, decompOutput, sizeof(int64_t) * 4, ONE_STAGE_COMP, NULL, 0); 51:	
	污染路径: 无	
42	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 713
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 711: } 712: 713: return 0; 714: } 715:	
	污染路径: 无	
43	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 274
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 272: } 273: 274: pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; 275: 276: ThreadParam* params = malloc(thread_count * sizeof(ThreadParam));	
	污染路径: expanded from macro 'PTHREAD_MUTEX_INITIALIZER' expanded from macro '___PTHREAD_MUTEX_INITIALIZER'	
44	缺陷发生位置:	行号:

	无	
47	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1705
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1703: break; 1704: case TSDB_DATA_TYPE_NCHAR: 1705: if (bufSize < 0) { 1706: return TSDB_CODE_TSC_INVALID_VALUE; 1707: }	
	污染路径: 无	
48	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 983
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 981: static int32_t extWinGetMultiTbWinFromTs(SOperatorInfo* pOperator, SExternalWindowOperator* pExtW, int64_t* tsCol, int64_t rowNum, int32_t* startPos) { 982: int32_t idx = taosArraySearchIdx(pExtW->pWins, tsCol, extWinTsWinCompare, TD_EQ); 983: if (idx >= 0) { 984: *startPos = 0; 985: return idx;	
	污染路径: 无	
49	缺陷发生位置: /contrib/lemon/lemon.c	行号: 93
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 91: if(nByte<0) return 0; 92: p = malloc(nByte + sizeof(MemChunk)); 93: if(p==0){ 94: fprintf(stderr, "Out of memory. Failed to allocate %lld bytes.\n", 95: (long long int)nByte);	

	污染路径: 无	
50	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1347
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1345: colDataSetNNULL(pOutputCol, 0, numOfRows); 1346: } else { 1347: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1348: if (IS_HELPER_NULL(pLeftCol, i)) { 1349: colDataSetNULL(pOutputCol, i);	
	污染路径: 无	
51	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2815
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2813: 2814: for (int64_t i = 0; i < count; i++) { 2815: tfree(fileList[i]); 2816: } 2817: tfree(fileList);	
	污染路径: expanded from macro 'tfree'	
52	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1505
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1503: colDataSetNNULL(pOutputCol, 0, numOfRows); 1504: } else { 1505: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1506: if (IS_HELPER_NULL(pLeftCol, i)) { 1507: colDataSetNULL(pOutputCol, i);	
	污染路径: 无	

53	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 166
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 164: {"binwidth", 'w', "NUMBER", 0, BENCH_WIDTH}, 165: {"table-prefix", 'm', "TABLE_PREFIX", 0, BENCH_PREFIX}, 166: {"escape-character", 'E', 0, 0, BENCH_ESCAPE}, 167: {"chinese", 'C', 0, 0, BENCH_CHINESE}, 168: {"normal-table", 'N', 0, 0, BENCH_NORMAL},	
	污染路径: 无	
54	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 183
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 181: int vgl2 = 0; 182: char sql[1024] = {0}; 183: const char *tbs[RT_TABLE_NUM] = {0}; 184: 185: for (int32_t i = 0; i < rtTables; ++i) {	
	污染路径: 无	
55	缺陷发生位置: /contrib/lemon/lempar.c	行号: 270
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 268: yyTraceFILE = TraceFILE; 269: yyTracePrompt = zTracePrompt; 270: if(yyTraceFILE==0) yyTracePrompt = 0; 271: else if(yyTracePrompt==0) yyTraceFILE = 0; 272: }	
	污染路径: 无	
56	缺陷发生位置: /examples/c/schemaless.c	行号: 47
	步骤描述:	

	空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 45: (void)taos_select_db(taos, "db"); 46: 47: time_t ct = time(0); 48: int64_t ts = ct * 1000; 49: char* lineFormat = "sta %d,t0=true,t1=127i8,t2=32767i16,t3= %di32,t4=9223372036854775807i64,t9=11.12345f32,t10=22 .123456789f64,t11=\"binaryTagValue\",t12=L\"ncharTagValue\ \" c0=true,c1=127i8,c2=32767i16,c3=2147483647i32,c4=9223 372036854775807i64,c5=254u8,c6=32770u16,c7=21474836 99u32,c8=9223372036854775899u64,c9=11.12345f32,c10= 22.123456789f64,c11=\"binaryValue\",c12=L\"ncharValue\ \"%\" PRIu64;	
	污染路径: 无	
57	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 224
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 222: { 223: {"version", 'V', 0, 0, N_("Print program version"), - 1}, 224: {0, 0} 225: }; 226:	
58	污染路径: 无	
	缺陷发生位置: /utils/test/c/sml_test.c	行号: 289
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 287: "[{\"metric\": \"test_us\", \"timestamp\": {\"value\": 1626006833639, \"type\": \"ms\"}, \"value\": true, \"tags\": {\"t0\": true}}, {\"metric\": \"test_us\", \"timestamp\": {\"value\": 1626006833638, \"type\": \"ms\"}, \"value\":	

	false, \"tags\": {\"t0\": true}}]}\" 288: }; 289: char *sql5[1] = {0}; 290: for (int i = 0; i < 1; i++) { 291: sql5[i] = taosMemoryCalloc(1, 1024);	
	污染路径: 无	
59	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 223
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 221: static const struct argp_option argp_version_options[] = 222: { 223: {"version", 'V', 0, 0, N_("Print program version"), - 1}, 224: {0, 0} 225: };	
	污染路径: 无	
60	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 1042
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1040: 1041: void initResultSizeInfo(SResultInfo* pResultInfo, int32_t numOfRows) { 1042: if (numOfRows == 0) { 1043: numOfRows = 4096; 1044: }	
	污染路径: 无	
61	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 510
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段:	

	508: 509: hol->num_entries = 0; 510: hol->clusters = 0; 511: 512: if (opts)	
	污染路径: 无	
62	缺陷发生位置: /source/libs/planner/test/planTestMain.cpp	行号: 89
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 87: {"queryPolicy", required_argument, NULL, 'q'}, 88: {"useNodeAllocator", required_argument, NULL, 'a'}, 89: {0, 0, 0, 0} 90: }; 91: // clang-format on	
	污染路径: 无	
63	缺陷发生位置: /contrib/lemon/lempar.c	行号: 245
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 243: #ifndef NDEBUG 244: #include <stdio.h> 245: static FILE *yyTraceFILE = 0; 246: static char *yyTracePrompt = 0; 247: #endif /* NDEBUG */	
	污染路径: 无	
64	缺陷发生位置: /utils/test/c/sml_test.c	行号: 191
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 189: }; 190: 191: char *sql3[1] = {0}; 192: for (int i = 0; i < 1; i++) {	

	193: sql3[i] = taosMemoryCalloc(1, 1024);	
	污染路径: 无	
65	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter_custom.c	行号: 181
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 179: memset(name, 0, size + 1); 180: memcpy(name, metric->name, size); 181: char* arr[2] = {0}; //arr[0] is table name, arr[1] is metric name 182: taos_monitor_split_str((char**)&arr, name, ":"); 183:	
	污染路径: 无	
66	缺陷发生位置: /utils/tsim/test/simTests.cpp	行号: 36
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 34: int32_t ret = 0; 35: int32_t argc = 3; 36: char *argv[4] = {0}; 37: 38: argc = 3;	
	污染路径: 无	
67	缺陷发生位置: /contrib/lemon/lemon.c	行号: 118
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 116: void *pNew; 117: MemChunk *p; 118: if(pOld==0) return lemon_malloc(nNew); 119: p = (MemChunk*)pOld; 120: p--;	
	污染路径:	

	无	
68	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1097
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1095: for (int i = 0; i < g_args.dumpDbCount; i++) 1096: tfree(g_dbInfos[i]); 1097: tfree(g_dbInfos); 1098: } 1099:	
	污染路径: expanded from macro 'tfree'	
69	缺陷发生位置: /utils/test/c/sml_test.c	行号: 162
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 160: "["metric\":"syscpu.nice\","timestamp\:1662344045,\"value\":9,\"tags\":{\"host\":"web02\","dc\:4}}]"; 161: 162: char *sql1[1] = {0}; 163: for (int i = 0; i < 1; i++) { 164: sql1[i] = taosMemoryCalloc(1, 1024);	
	污染路径: 无	
70	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 4049
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 4047: mInfo("vgId:%d, build compact vnode config req", pVgroup->vgId); 4048: int32_t contLen = tSerializeSCompactVnodeReq(NULL, 0, &compactReq); 4049: if (contLen < 0) { 4050: terrno = TSDB_CODE_OUT_OF_MEMORY; 4051: return NULL;	
	污染路径:	

	无	
71	缺陷发生位置: /contrib/lemon/lemon.c	行号: 91
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 89: static void *lemon_malloc(size_t nByte){ 90: MemChunk *p; 91: if(nByte<0) return 0; 92: p = malloc(nByte + sizeof(MemChunk)); 93: if(p==0){	
	污染路径: 无	
72	缺陷发生位置: /source/util/test/hashTest.cpp	行号: 158
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 156: int32_t code = 0; 157: int32_t num = 0; 158: TESTSTRUCT data = {0}; 159: const char* str1 = "abcdefg"; 160: const char* str2 = "aaaaaaa";	
	污染路径: 无	
73	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 179
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 177: 178: int rtGetTablesRouteInfo(TAOS *taos) { 179: char *table = {0}; 180: int *vgld1 = malloc(rtTables * sizeof(int)); 181: int vgld2 = 0;	
	污染路径: 无	
74	缺陷发生位置: /source/util/test/decompressTest.cpp	行号: 18
	步骤描述:	

	空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 16: 1700000500, 1700000600, 1700000700, 1700000800, 1700000900}; 17: 18: char* pOutput[10 * sizeof(int64_t)] = {0}; 19: int32_t len = tsCompressTimestamp(tsList, sizeof(tsList), sizeof(tsList) / sizeof(tsList[0]), pOutput, 10, 20: ONE_STAGE_COMP, NULL, 0);	
	污染路径: 无	
75	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2661
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2659: // small record size 2660: int sql_s_perf1(TAOS *taos) { 2661: char *sql[3000] = {0}; 2662: TAOS_RES *result; 2663:	
	污染路径: 无	
76	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2560
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2558: // 120table 60 record each table 2559: int sql_perf1(TAOS *taos) { 2560: char *sql[3000] = {0}; 2561: TAOS_RES *result; 2562:	
	污染路径: 无	
77	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 76
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	

	代码片段: 74: { 75: {"subopt1", 's', 0, 0, "Nested option 1"}, 76: {"subopt2", 'S', 0, 0, "Nested option 2"}, 77: 78: { 0, 0, 0, 0, "Some more nested options:", 10},	
	污染路径: 无	
78	缺陷发生位置: /source/common/src/tanalytics.c	行号: 921
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 919: const char *taosAnalysisAlgoType(EAnalyAlgoType algoType) { return 0; } 920: EAnalyAlgoType taosAnalAlgoInt(const char *algoName) { return 0; } 921: const char *taosAnalAlgoUrlStr(EAnalyAlgoType algoType) { return 0; } 922: 923: int64_t taosAnalysisParseTimeout(SHashObj *pHashMap, const char *id) { return 0; }	
	污染路径: 无	
79	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1096
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1094: if (g_dbInfos == NULL) return; 1095: for (int i = 0; i < g_args.dumpDbCount; i++) 1096: tfree(g_dbInfos[i]); 1097: tfree(g_dbInfos); 1098: }	
	污染路径: expanded from macro 'tfree'	
80	缺陷发生位置: /source/util/src/tlruccache.c	行号: 737
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	

	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 166: const char *_l_value = taos_metric_formatter_dump(self->formatter); 167: if (_l_value == NULL) { 168: TAOS_METRIC_SAMPLE_FROM_LABELS_HANDLE_UNLOCK(); 169: } 170:	
	污染路径: expanded from macro 'TAOS_METRIC_SAMPLE_FROM_LABELS_HANDLE_UNLOCK' expanded from macro 'NULL'	
84	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 901
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 899: cl_end = &(*cl_end)->next; 900: *cl_end = more->clusters; 901: more->clusters = 0; 902: 903: /* Merge entries. */	
	污染路径: 无	
85	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 177
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 175: } 176: 177: z_stream gzipStream = {0}; 178: gzipStream.zalloc = (alloc_func)0; 179: gzipStream.zfree = (free_func)0;	
	污染路径: 无	
86	缺陷发生位置:	行号:

	/contrib/lemon/lempar.c	307
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 305: if(p->yystack==p->yystk0){ 306: pNew = YYREALLOC(0, newSize*sizeof(pNew[0])); 307: if(pNew==0) return 1; 308: memcpy(pNew, p->yystack, oldSize*sizeof(pNew[0])); 309: }else{	
	污染路径: 无	
87	缺陷发生位置: /source/common/src/tanalytics.c	行号: 919
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 917: void taosAnalyBufDestroy(SAnalyticBuf *pBuf) {} 918: 919: const char *taosAnalysisAlgoType(EAnalyAlgoType algoType) { return 0; } 920: EAnalyAlgoType taosAnalAlgoInt(const char *algoName) { return 0; } 921: const char *taosAnalAlgoUrlStr(EAnalyAlgoType algoType) { return 0; }	
	污染路径: 无	
88	缺陷发生位置: /contrib/lemon/lempar.c	行号: 271
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 269: yyTracePrompt = zTracePrompt; 270: if(yyTraceFILE==0) yyTracePrompt = 0; 271: else if(yyTracePrompt==0) yyTraceFILE = 0; 272: } 273: #endif /* NDEBUG */	
	污染路径: 无	
89	缺陷发生位置: /contrib/lemon/lempar.c	行号: 246

	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 244: #include <stdio.h> 245: static FILE *yyTraceFILE = 0; 246: static char *yyTracePrompt = 0; 247: #endif /* NDEBUG */ 248:	
	污染路径: 无	
90	缺陷发生位置:	行号:
	/source/libs/scalar/src/scvector.c	1610
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1608: colDataSetNNULL(pOutputCol, 0, numOfRows); 1609: } else { 1610: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1611: if (IS_HELPER_NULL(pLeftCol, i)) { 1612: colDataSetNULL(pOutputCol, i);	
91	污染路径: 无	
	缺陷发生位置:	行号:
	/tools/taos-tools/deps/toolscJson/src/toolscJson.c	947
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
92	代码片段: 945: CJSON_PUBLIC(tools_cJSON *) tools_cJSON_ParseWithOpts(const char *value, const char **return_parse_end, tools_cJSON_bool require_null_terminated) 946: { 947: parse_buffer buffer = { 0, 0, 0, 0, { 0, 0, 0 } }; 948: tools_cJSON *item = NULL; 949:	
	污染路径: 无	
	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchSys.c	169
步骤描述:		

	空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 167: {"chinese", 'C', 0, 0, BENCH_CHINESE}, 168: {"normal-table", 'N', 0, 0, BENCH_NORMAL}, 169: {"random", 'M', 0, 0, BENCH_RANDOM}, 170: {"aggr-func", 'x', 0, 0, BENCH_AGGR}, 171: {"answer-yes", 'y', 0, 0, BENCH_YES},	
	污染路径: 无	
93	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 167
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 165: {"table-prefix", 'm', "TABLE_PREFIX", 0, BENCH_PREFIX}, 166: {"escape-character", 'E', 0, 0, BENCH_ESCAPE}, 167: {"chinese", 'C', 0, 0, BENCH_CHINESE}, 168: {"normal-table", 'N', 0, 0, BENCH_NORMAL}, 169: {"random", 'M', 0, 0, BENCH_RANDOM},	
94	污染路径: 无	
	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1106
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
95	代码片段: 1104: } else { 1105: code = pAPI->metaFn.resumeTableMetaCursor(pInfo->pCur, 0, 0); 1106: if (code != 0) { 1107: pAPI->metaFn.closeTableMetaCursor(pInfo->pCur); 1108: pInfo->pCur = NULL;	
	污染路径: 无	
	缺陷发生位置: /docs/examples/c/sml_insert_demo.c	行号: 105
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段:	

	103: 104: // opentsdb json protocol 105: char *jsons[1] = {0}; 106: // allocate memory for json data. can not use static memory. 107: size_t size = 1024;	
	污染路径: 无	
96	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 344
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 342: const char* passwd = "taosdata"; 343: #ifndef WINDOWS 344: argp_parse(&argp, argc, argv, 0, 0, NULL); 345: #endif 346: TAOS* taos = taos_connect(host, user, passwd, "", 0);	
	污染路径: 无	
97	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 1261
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1259: SExtWinTimeWindow* pWin = NULL; 1260: int32_t idx = taosArraySearchIdx(pExtW->pWins, tsCol, extWinTsWinCompare, TD_EQ); 1261: if (idx >= 0) { 1262: for (int i = idx - 1; i >= 0; --i) { 1263: pWin = TARRAY_GET_ELEM(pExtW->pWins, i);	
	污染路径: 无	
98	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 80
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 78: int32_t qwtTestFetchQueueNum = 0; 79: SRWLatch qwtTestFetchQueueLock = 0;	

	80: struct SRpcMsg *qwtTestFetchQueue[qwtTestFetchQueueSize] = {0}; 81: 82: int32_t qwtTestSinkBlockNum = 0;	
	污染路径: 无	
99	缺陷发生位置: /contrib/lemon/lemon.c	行号: 570
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 568: /* Allocate a new parser action */ 569: static struct action *Action_new(void){ 570: static struct action *actionfreelist = 0; 571: struct action *newaction; 572:	
	污染路径: 无	
100	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 3300
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 3298: if (state->pFileSet != state->pr->pCurFileSet) { 3299: SDataFileReaderConfig conf = {.tsdb = pTsdB, .szPage = pTsdB->pVnode->config.tsdbPageSize}; 3300: const char *fileName[4] = {0}; 3301: if (pFileObj[0] != NULL) { 3302: conf.files[0].file = *pFileObj[0]->f;	
	污染路径: 无	
101	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 688
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 686: if (opt->name && ovisible (opt)) 687: return opt->name; 688: return 0;	

	689: } 690:	
	污染路径: 无	
102	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 308
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 306: v.ts += 1; 307: for (int j = 1; j < 10; ++j) { 308: params[j].is_null = ((i == j) ? &is_null : 0); 309: } 310: v.b = (int8_t)i % 2;	
	污染路径: 无	
103	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 141
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 139: {"HANG", OPT_HANG, "SECS", OPTION_ARG_OPTIONAL OPTION_HIDDEN, 140: N_("Hang for SECS seconds (default 3600)"), 141: {0, 0} 142: }; 143:	
	污染路径: 无	
104	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 154
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 152: {"angle-step", 'H', "NUMBER", 0, ANGLE_STEP}, 153: {"start-timestamp", 's', "NUMBER", 0, BENCH_START_TIMESTAMP}, 154: {"supplement-insert", 'U', 0, 0, BENCH_SUPPLEMENT}, 155: {"interlace-rows", 'B', "NUMBER", 0,	

	BENCH_INTERLACE}, 156: {"rec-per-req", 'r', "NUMBER", 0, BENCH_BATCH},
	污染路径: 无

7.2.1.3 WK4690 动态分配的指针变量定义时如未被分配空间必须初始化为 NULL

缺陷数量： 24 个		
缺陷描述： 动态分配的指针变量定义时如未被分配空间必须初始化为 NULL。		
修复意见： 无		
1	缺陷发生位置： /test/new_test_framework/script/api/stmt2-test.c	行号： 2561
	步骤描述： 动态分配的指针变量 result 定义时如未被分配空间必须初始化为 NULL	
	代码片段： 2559: int sql_perf1(TAOS *taos) { 2560: char *sql[3000] = {0}; 2561: TAOS_RES *result; 2562: 2563: for (int i = 0; i < 3000; i++) {	
	污染路径： 无	
2	缺陷发生位置： /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号： 255
	步骤描述： 动态分配的指针变量 i 定义时如未被分配空间必须初始化为 NULL	
	代码片段： 253: ss_insertionsort(const unsigned char *T, const int *PA, 254: int *first, int *last, int depth) { 255: int *i, *j; 256: int t; 257: int r;	

	污染路径: 无	
3	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 285
	步骤描述: 动态分配的指针变量 p 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 283: Then insert a line break. */ 284: 285: char *p, *nextline; 286: int i; 287:	
	污染路径: 无	
4	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 188
	步骤描述: 动态分配的指针变量 buf 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 186: _argp_fmtstream_update (argp_fmtstream_t fs) 187: { 188: char *buf, *nl; 189: size_t len; 190:	
	污染路径: 无	
5	缺陷发生位置: /contrib/lemon/lemon.c	行号: 571
	步骤描述: 动态分配的指针变量 newaction 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 569: static struct action *Action_new(void){ 570: static struct action *actionfreelist = 0; 571: struct action *newaction; 572: 573: if(actionfreelist==0){	
	污染路径: 无	

6	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 230
	步骤描述: 动态分配的指针变量 U1n 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 228: const int *p1, const int *p2, 229: int depth) { 230: const unsigned char *U1, *U2, *U1n, *U2n; 231: 232: for(U1 = T + depth + *p1,	
	污染路径: 无	
7	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2422
	步骤描述: 动态分配的指针变量 result 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 2420: int sql_perf1(TAOS *taos) { 2421: char *sql[3000] = {0}; 2422: TAOS_RES *result; 2423: 2424: for (int i = 0; i < 3000; i++) {	
	污染路径: 无	
8	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2783
	步骤描述: 动态分配的指针变量 result 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 2781: 2782: void prepare(TAOS *taos, int32_t colNum, int32_t *colList, int prepareStb) { 2783: TAOS_RES *result; 2784: int code; 2785: char createDbSql[128] = {0};	
	污染路径: 无	
9	缺陷发生位置: /contrib/lemon/lemon.c	行号: 116
	步骤描述:	

	动态分配的指针变量 pNew 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 114: } 115: static void *lemon_realloc(void *pOld, size_t nNew){ 116: void *pNew; 117: MemChunk *p; 118: if(pOld==0) return lemon_malloc(nNew);	
	污染路径: 无	
10	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2470
	步骤描述: 动态分配的指针变量 result 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 2468: int sql_perf_s1(TAOS *taos) { 2469: char **sql = calloc(1, sizeof(char*) * 360000); 2470: TAOS_RES *result; 2471: 2472: for (int i = 0; i < 360000; i++) {	
	污染路径: 无	
11	缺陷发生位置: /contrib/lemon/lemon.c	行号: 117
	步骤描述: 动态分配的指针变量 p 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 115: static void *lemon_realloc(void *pOld, size_t nNew){ 116: void *pNew; 117: MemChunk *p; 118: if(pOld==0) return lemon_malloc(nNew); 119: p = (MemChunk*)pOld;	
	污染路径: 无	
12	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2923
	步骤描述: 动态分配的指针变量 result 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 2921: 2922: void prepare(TAOS *taos, int32_t colNum, int32_t	

	<pre> *colList, int prepareStb) { 2923: TAOS_RES *result; 2924: int code; 2925: char createDbSql[128] = {0}; </pre>	
	污染路径: 无	
13	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2608
	步骤描述: 动态分配的指针变量 result 定义时如未被分配空间必须初始化为 NULL	
	代码片段: <pre> 2606: int sql_perf_s1(TAOS *taos) { 2607: char **sql = calloc(1, sizeof(char *) * 360000); 2608: TAOS_RES *result; 2609: 2610: for (int i = 0; i < 360000; i++) { </pre>	
	污染路径: 无	
14	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 218
	步骤描述: 动态分配的指针变量 new_text 定义时如未被分配空间必须初始化为 NULL	
	代码片段: <pre> 216: help_filter (int key, const char *text, void *input) 217: { 218: char *new_text; 219: struct params *params = input; 220: </pre>	
	污染路径: 无	
15	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 230
	步骤描述: 动态分配的指针变量 U2n 定义时如未被分配空间必须初始化为 NULL	
	代码片段: <pre> 228: const int *p1, const int *p2, 229: int depth) { 230: const unsigned char *U1, *U2, *U1n, *U2n; 231: </pre>	

	232: for(U1 = T + depth + *p1,	
	污染路径: 无	
16	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 285
	步骤描述: 动态分配的指针变量 nextline 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 283: Then insert a line break. */ 284: 285: char *p, *nextline; 286: int i; 287:	
	污染路径: 无	
17	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2662
	步骤描述: 动态分配的指针变量 result 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 2660: int sql_s_perf1(TAOS *taos) { 2661: char *sql[3000] = {0}; 2662: TAOS_RES *result; 2663: 2664: for (int i = 0; i < 3000; i++) {	
	污染路径: 无	
18	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 230
	步骤描述: 动态分配的指针变量 U1 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 228: const int *p1, const int *p2, 229: int depth) { 230: const unsigned char *U1, *U2, *U1n, *U2n; 231: 232: for(U1 = T + depth + *p1,	
	污染路径:	

	无	
19	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 430
	步骤描述: 动态分配的指针变量 new_buf 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 428: size_t old_size = fs->end - fs->buf; 429: size_t new_size = old_size + amount; 430: char *new_buf; 431: 432: if (new_size < old_size ! (new_buf = realloc (fs->buf, new_size)))	
	污染路径: 无	
20	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 230
	步骤描述: 动态分配的指针变量 U2 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 228: const int *p1, const int *p2, 229: int depth) { 230: const unsigned char *U1, *U2, *U1n, *U2n; 231: 232: for(U1 = T + depth + *p1,	
	污染路径: 无	
21	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2522
	步骤描述: 动态分配的指针变量 result 定义时如未被分配空间必须初始化为 NULL	
	代码片段: 2520: int sql_s_perf1(TAOS *taos) { 2521: char *sql[3000] = {0}; 2522: TAOS_RES *result; 2523: 2524: for (int i = 0; i < 3000; i++) {	
	污染路径: 无	
22	缺陷发生位置:	行号:

	/contrib/lemon/lemon.c		90
	步骤描述: 动态分配的指针变量 p 定义时如未被分配空间必须初始化为 NULL		
	代码片段: 88: */ 89: static void *lemon_malloc(size_t nByte){ 90: MemChunk *p; 91: if(nByte<0) return 0; 92: p = malloc(nByte + sizeof(MemChunk));		
	污染路径: 无		
23	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c		行号: 188
	步骤描述: 动态分配的指针变量 nl 定义时如未被分配空间必须初始化为 NULL		
	代码片段: 186: _argp_fmtstream_update (argp_fmtstream_t fs) 187: { 188: char *buf, *nl; 189: size_t len; 190:		
	污染路径: 无		
24	缺陷发生位置: /contrib/lemon/lemon.c		行号: 173
	步骤描述: 动态分配的指针变量 z 定义时如未被分配空间必须初始化为 NULL		
	代码片段: 171: int i, j, k, c; 172: int nUsed = 0; 173: const char *z; 174: char zTemp[50]; 175: str[0] = 0;		
	污染路径: 无		

7.2.1.4 WK4871 宏 NULL 应是唯一允许的整数空指针常量形式

缺陷数量：104 个		
缺陷描述： 如果出现在以下任何上下文中，则值为 0 的整数常量表达式应从宏 NULL 的展开中导出： · 作为分配给指针的值； · 作为 ==或! =的操作数另一个操作数是指针的运算符； · 作为? 的第二个操作数：第三个操作数是指针的运算符； · 作为? 的第三个操作数：第二个操作数是指针的运算符。 忽略空格和任何周围的括号，任何此类整数常量表达式都应表示 NULL 的整个扩展。 注意：允许使用 (void*) 0 形式的空指针常量，无论它是否从 null 扩展而来。 理论基础 使用 NULL 而不是 0 可以清楚地表明需要一个空指针常量。		
修复意见： 无		
1	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号： 55
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 53: } 54: 55: if (remainder > 0) { 56: p = taosMemoryCalloc(remainder, sizeof(STableBlockScanInfo)); 57: TSDB_CHECK_NULL(p, code, lino, _end, terrno);	
	污染路径： 无	
2	缺陷发生位置： /source/common/src/tanalytics.c	行号： 919
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 917: void taosAnalyBufDestroy(SAnalyticBuf *pBuf) {} 918: 919: const char *taosAnalysisAlgoType(EAnalyAlgoType algoType) { return 0; } 920: EAnalyAlgoType taosAnalAlgoInt(const char *algoName) { return 0; } 921: const char *taosAnalAlgoUrlStr(EAnalyAlgoType algoType) { return 0; }	
	污染路径：	

	无	
3	缺陷发生位置： /test/new_test_framework/script/api/batchprepare.c	行号： 2421
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 2419: //120table 60 record each table 2420: int sql_perf1(TAOS *taos) { 2421: char *sql[3000] = {0}; 2422: TAOS_RES *result; 2423:	
	污染路径： 无	
4	缺陷发生位置： /utils/test/c/createTable.c	行号： 288
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 286: taos_close(con); 287: taosMemoryFree(qstr); 288: return 0; 289: } 290:	
	污染路径： 无	
5	缺陷发生位置： /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号： 1107
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 1105: cJSON_PUBLIC(char *) tools_cJSON_PrintBuffered(const tools_cJSON *item, int prebuffer, tools_cJSON_bool fmt) 1106: { 1107: printbuffer p = { 0, 0, 0, 0, 0, 0, { 0, 0, 0 } }; 1108: 1109: if (prebuffer < 0)	
	污染路径： 无	
6	缺陷发生位置： /utils/test/c/sml_test.c	行号： 191

	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 189: }; 190: 191: char *sql3[1] = {0}; 192: for (int i = 0; i < 1; i++) { 193: sql3[i] = taosMemoryCalloc(1, 1024);	
	污染路径: 无	
7	缺陷发生位置:	行号:
	/utils/test/c/sml_test.c	289
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 287: "[{\"metric\": \"test_us\", \"timestamp\": {\"value\": 1626006833639, \"type\": \"ms\"}, \"value\": true, \"tags\": {\"t0\": true}}, {\"metric\": \"test_us\", \"timestamp\": {\"value\": 1626006833638, \"type\": \"ms\"}, \"value\": false, \"tags\": {\"t0\": true}}]" 288: }; 289: char *sql5[1] = {0}; 290: for (int i = 0; i < 1; i++) { 291: sql5[i] = taosMemoryCalloc(1, 1024);	
	污染路径: 无	
8	缺陷发生位置:	行号:
	/source/libs/executor/src/externalwindowoperator.c	2092
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2090: SExtWinTimeWindow* pWin = NULL; 2091: int32_t idx = taosArraySearchIdx(pWins, tsCol, extWinTsWinCompare, TD_EQ); 2092: if (idx >= 0) { 2093: for (int i = idx - 1; i >= 0; --i) { 2094: pWin = TARRAY_GET_ELEM(pWins, i);	
	污染路径: 无	
9	缺陷发生位置:	行号:

	/source/libs/scalar/src/sclvector.c		1610
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0		
	代码片段: 1608: colDataSetNNULL(pOutputCol, 0, numOfRows); 1609: } else { 1610: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1611: if (IS_HELPER_NULL(pLeftCol, i)) { 1612: colDataSetNULL(pOutputCol, i);		
	污染路径: 无		
10	缺陷发生位置: /source/libs/monitorfw/src/taos_metric.c		行号: 162
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0		
	代码片段: 160: r = taos_metric_formatter_load_l_value(self->formatter, self->name, NULL, self->label_key_count, self->label_keys, label_values); 161: if (r) { 162: TAOS_METRIC_SAMPLE_FROM_LABELS_HANDLE_UNLOCK(); 163: } 164:		
	污染路径: expanded from macro 'TAOS_METRIC_SAMPLE_FROM_LABELS_HANDLE_UNLOCK' expanded from macro 'NULL'		
11	缺陷发生位置: /source/util/test/decompressTest.cpp		行号: 22
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0		
	代码片段: 20: ONE_STAGE_COMP, NULL, 0); 21: 22: char* decompOutput[10 * 8] = {0}; 23: 24: tsDecompressTimestamp(pOutput, len, 10, decompOutput, sizeof(int64_t) * 10, ONE_STAGE_COMP, NULL, 0);		

	污染路径: 无	
12	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2521
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2519: //small record size 2520: int sql_s_perfl(TAOS *taos) { 2521: char *sql[3000] = {0}; 2522: TAOS_RES *result; 2523:	
	污染路径: 无	
13	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 74
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 72: int32_t qwtTestQueryQueueNum = 0; 73: SRWLatch qwtTestQueryQueueLock = 0; 74: struct SRpcMsg *qwtTestQueryQueue[qwtTestQueryQueueSize] = {0}; 75: 76: int32_t qwtTestFetchQueueRIdx = 0;	
	污染路径: 无	
14	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1096
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1094: if (g_dbInfos == NULL) return; 1095: for (int i = 0; i < g_args.dumpDbCount; i++) 1096: tfree(g_dbInfos[i]); 1097: tfree(g_dbInfos); 1098: }	
	污染路径: expanded from macro 'tfree'	

15	缺陷发生位置: /contrib/lemon/lempar.c	行号: 271
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 269: yyTracePrompt = zTracePrompt; 270: if(yyTraceFILE==0) yyTracePrompt = 0; 271: else if(yyTracePrompt==0) yyTraceFILE = 0; 272: } 273: #endif /* NDEBUG */	
	污染路径: 无	
16	缺陷发生位置: /contrib/lemon/lempar.c	行号: 270
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 268: yyTraceFILE = TraceFILE; 269: yyTracePrompt = zTracePrompt; 270: if(yyTraceFILE==0) yyTracePrompt = 0; 271: else if(yyTracePrompt==0) yyTraceFILE = 0; 272: }	
	污染路径: 无	
17	缺陷发生位置: /utils/test/c/sml_test.c	行号: 261
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 259: }; 260: 261: char *sql3[1] = {0}; 262: for (int i = 0; i < 1; i++) { 263: sql3[i] = taosMemoryCalloc(1, 1024);	
	污染路径: 无	
18	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1027
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	

	代码片段: 1025: cJSON_PUBLIC(tools_cJSON *) tools_cJSON_Parse(const char *value) 1026: { 1027: return tools_cJSON_ParseWithOpts(value, 0, 0); 1028: } 1029:	
	污染路径: 无	
19	缺陷发生位置: /utils/test/c/sml_test.c	行号: 232
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 230: "34}}, {"metric\":\"sys.cpu.nice\",\"timestamp\":1662344045,\"value\":9,\"tags\":{\"host\":\"web02\",\"dc\":4}}]"; 231: 232: char *sql1[1] = {0}; 233: for (int i = 0; i < 1; i++) { 234: sql1[i] = taosMemoryCalloc(1, 1024);	
	污染路径: 无	
20	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 137
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 135: { 136: {"help", '?', 0, 0, N_("Give this help list"), -1}, 137: {"usage", OPT_USAGE, 0, 0, N_("Give a short usage message")}, 138: {"program-name", OPT_PROGNAME, "NAME", OPTION_HIDDEN, N_("Set the program name")}, 139: {"HANG", OPT_HANG, "SECS", OPTION_ARG_OPTIONAL OPTION_HIDDEN,	
	污染路径: 无	
21	缺陷发生位置:	行号:

	/tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c		901
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0		
	代码片段: 899: cl_end = &(*cl_end)->next; 900: *cl_end = more->clusters; 901: more->clusters = 0; 902: 903: /* Merge entries. */		
	污染路径: 无		
22	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c		行号: 1137
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0		
	代码片段: 1135: CJSON_PUBLIC(tools_cJSON_bool) tools_cJSON_PrintPreallocated(tools_cJSON *item, char *buf, const int len, const tools_cJSON_bool fmt) 1136: { 1137: printbuffer p = { 0, 0, 0, 0, 0, 0, { 0, 0, 0 } }; 1138: 1139: if ((len < 0) (buf == NULL))		
	污染路径: 无		
23	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c		行号: 1705
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0		
	代码片段: 1703: break; 1704: case TSDB_DATA_TYPE_NCHAR: 1705: if (bufSize < 0) { 1706: return TSDB_CODE_TSC_INVALID_VALUE; 1707: }		
	污染路径: 无		
24	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/		行号: 76

	test/argp-test.c	
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 74: { 75: {"subopt1", 's', 0, 0, "Nested option 1"}, 76: {"subopt2", 'S', 0, 0, "Nested option 2"}, 77: 78: { 0, 0, 0, 0, "Some more nested options:", 10},	
	污染路径: 无	
25	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1505
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1503: colDataSetNNULL(pOutputCol, 0, numOfRows); 1504: } else { 1505: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1506: if (IS_HELPER_NULL(pLeftCol, i)) { 1507: colDataSetNULL(pOutputCol, i);	
	污染路径: 无	
26	缺陷发生位置: /contrib/lemon/lempar.c	行号: 307
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 305: if(p->yystack==p->yystk0){ 306: pNew = YYREALLOC(0, newSize*sizeof(pNew[0])); 307: if(pNew==0) return 1; 308: memcpy(pNew, p->yystack, oldSize*sizeof(pNew[0])); 309: }else{	
	污染路径: 无	
27	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 1261
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	

	代码片段: 1259: SExtWinTimeWindow* pWin = NULL; 1260: int32_t idx = taosArraySearchIdx(pExtW->pWins, tsCol, extWinTsWinCompare, TD_EQ); 1261: if (idx >= 0) { 1262: for (int i = idx - 1; i >= 0; --i) { 1263: pWin = TARRAY_GET_ELEM(pExtW->pWins, i);	
	污染路径: 无	
28	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 4049
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 4047: mInfo("vgId:%d, build compact vnode config req", pVgroup->vgId); 4048: int32_t contLen = tSerializeSCompactVnodeReq(NULL, 0, &compactReq); 4049: if (contLen < 0) { 4050: terrno = TSDB_CODE_OUT_OF_MEMORY; 4051: return NULL;	
	污染路径: 无	
29	缺陷发生位置: /source/libs/scalar/src/scVector.c	行号: 1478
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1476: colDataSetNNULL(pOutputCol, 0, numOfRows); 1477: } else { 1478: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1479: if (IS_HELPER_NULL(pLeftCol, i)) { 1480: colDataSetNULL(pOutputCol, i);	
	污染路径: 无	
30	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 224
	步骤描述:	

	空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 222: { 223: {"version", 'V', 0, 0, N_("Print program version"), -1}, 224: {0, 0} 225: }; 226:	
	污染路径: 无	
31	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 688
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 686: if (opt->name && ovisible (opt)) 687: return opt->name; 688: return 0; 689: } 690:	
	污染路径: 无	
32	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 167
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 165: {"table-prefix", 'm', "TABLE_PREFIX", 0, BENCH_PREFIX}, 166: {"escape-character", 'E', 0, 0, BENCH_ESCAPE}, 167: {"chinese", 'C', 0, 0, BENCH_CHINESE}, 168: {"normal-table", 'N', 0, 0, BENCH_NORMAL}, 169: {"random", 'M', 0, 0, BENCH_RANDOM},	
	污染路径: 无	
33	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 308
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	

	代码片段: 306: v.ts += 1; 307: for (int j = 1; j < 10; ++j) { 308: params[j].is_null = ((i == j) ? &is_null : 0); 309: } 310: v.b = (int8_t)i % 2;	
	污染路径: 无	
34	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 177
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 175: } 176: 177: z_stream gzipStream = {0}; 178: gzipStream.zalloc = (alloc_func)0; 179: gzipStream.zfree = (free_func)0;	
	污染路径: 无	
35	缺陷发生位置: /contrib/lemon/lempar.c	行号: 246
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 244: #include <stdio.h> 245: static FILE *yyTraceFILE = 0; 246: static char *yyTracePrompt = 0; 247: #endif /* NDEBUG */ 248:	
	污染路径: 无	
36	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 157
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 155: { 156: free (fs);	

	157: fs = 0; 158: } 159: else	
	污染路径: 无	
37	缺陷发生位置: /source/common/src/tanalytics.c	行号: 921
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 919: const char *taosAnalysisAlgoType(EAnalyAlgoType algoType) { return 0; } 920: EAnalyAlgoType taosAnalAlgoInt(const char *algoName) { return 0; } 921: const char *taosAnalAlgoUrlStr(EAnalyAlgoType algoType) { return 0; } 922: 923: int64_t taosAnalysisParseTimeout(SHashObj *pHashMap, const char *id) { return 0; }	
	污染路径: 无	
38	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1310
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1308: } else { 1309: code = pAPI->metaFn.resumeTableMetaCursor(pInfo->pCur, 0, 0); 1310: if (code != 0) { 1311: pAPI->metaFn.closeTableMetaCursor(pInfo->pCur); 1312: pInfo->pCur = NULL;	
	污染路径: 无	
39	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 428
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 426: taosArrayClear(pldx);	

	427: int32_t newSize = (int32_t)taosArrayGetSize(pNewIdx); 428: if (newSize > 0) { 429: void* dest = taosArrayReserve(pIdx, newSize); 430: TSDB_CHECK_NULL(dest, code, lino, _exit, terno);	
	污染路径: 无	
40	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 75
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 73: struct argp_option sub_options[] = 74: { 75: {"subopt1", 's', 0, 0, "Nested option 1"}, 76: {"subopt2", 'S', 0, 0, "Nested option 2"}, 77:	
	污染路径: 无	
41	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 271
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 269: int32_t num) { 270: int32_t code = TSDB_CODE_SUCCESS; 271: if (num > 0) { 272: int32_t start = pEncoder->offset; 273: pEncoder->offset += sizeof(STlv);	
	污染路径: 无	
42	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1097
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1095: for (int i = 0; i < g_args.dumpDbCount; i++) 1096: tfree(g_dbInfos[i]); 1097: tfree(g_dbInfos); 1098: }	

	170:	
	污染路径: expanded from macro 'TAOS_METRIC_SAMPLE_FROM_LABELS_HANDLE_UNLOCK' expanded from macro 'NULL'	
46	缺陷发生位置: /source/util/src/tlru_cache.c	行号: 737
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 735: } 736: taosMemoryFree(cache->shards); 737: cache->shards = 0; 738: } 739:	
	污染路径: 无	
47	缺陷发生位置: /contrib/test/sqlite/sqliteTest.c	行号: 22
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 20: int main(int argc, char const *argv[]) { 21: sqlite3 *db; 22: char * err_msg = 0; 23: 24: int rc = sqlite3_open("test.db", &db);	
	污染路径: 无	
48	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 53
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 51: {"records", 'n', "NUMBER", 0, 52: "Number of records for each table, default is 10000."}, 53: {0}}; 54: 55: static error_t parse_opt(int key, char *arg, struct argp_state *state) {	

	污染路径: 无	
49	缺陷发生位置: /docs/examples/c-ws/sml_insert_demo.c	行号: 97
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 95: 96: // opentsdb json protocol 97: char *jsons[1] = {0}; 98: // allocate memory for json data. can not use static memory. 99: totalLines = 0;	
	污染路径: 无	
50	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 676
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 674: { 675: return hol_entry_short_iterate (entry, until_short, 676: entry->argp->argp_domain, 0); 677: } 678:	
	污染路径: 无	
51	缺陷发生位置: /source/libs/parser/test/parTestMain.cpp	行号: 91
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 89: {"limitSql", required_argument, NULL, 'i'}, 90: {"log", required_argument, NULL, 'l'}, 91: {0, 0, 0, 0} 92: }; 93: // clang-format on	
	污染路径:	

	无	
52	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2560
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2558: // 120table 60 record each table 2559: int sql_perfl(TAOS *taos) { 2560: char *sql[3000] = {0}; 2561: TAOS_RES *result; 2562:	
	污染路径: 无	
53	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 183
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 181: int vgl2 = 0; 182: char sql[1024] = {0}; 183: const char *tbs[RT_TABLE_NUM] = {0}; 184: 185: for (int32_t i = 0; i < rtTables; ++i) {	
	污染路径: 无	
54	缺陷发生位置: /contrib/lemon/lemon.c	行号: 93
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 91: if(nByte<0) return 0; 92: p = malloc(nByte + sizeof(MemChunk)); 93: if(p==0){ 94: fprintf(stderr, "Out of memory. Failed to allocate %lld bytes.\n", 95: (long long int)nByte);	
	污染路径: 无	
55	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1347

	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1345: colDataSetNNULL(pOutputCol, 0, numOfRows); 1346: } else { 1347: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1348: if (IS_HELPER_NULL(pLeftCol, i)) { 1349: colDataSetNULL(pOutputCol, i);	
	污染路径: 无	
56	缺陷发生位置:	行号:
	/source/libs/planner/test/planTestMain.cpp	89
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0 代码片段: 87: {"queryPolicy", required_argument, NULL, 'q'}, 88: {"useNodeAllocator", required_argument, NULL, 'a'}, 89: {0, 0, 0, 0} 90: }; 91: // clang-format on	
	污染路径: 无	
57	缺陷发生位置:	行号:
	/examples/c/schemaless.c	47
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0 代码片段: 45: (void)taos_select_db(taos, "db"); 46: 47: time_t ct = time(0); 48: int64_t ts = ct * 1000; 49: char* lineFormat = "sta %d,t0=true,t1=127i8,t2=32767i16,t3= %di32,t4=9223372036854775807i64,t9=11.12345f32,t10=22 .123456789f64,t11=\"binaryTagValue\",t12=L\"ncharTagValue\ " c0=true,c1=127i8,c2=32767i16,c3=2147483647i32,c4=9223 372036854775807i64,c5=254u8,c6=32770u16,c7=21474836 99u32,c8=9223372036854775899u64,c9=11.12345f32,c10= 22.123456789f64,c11=\"binaryValue\",c12=L\"ncharValue\"	

	19: int32_t len = tsCompressTimestamp(tsList, sizeof(tsList), sizeof(tsList) / sizeof(tsList[0]), pOutput, 10, 20: ONE_STAGE_COMP, NULL, 0);	
	污染路径: 无	
61	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 78
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 76: {"subopt2", 'S', 0, 0, "Nested option 2"}, 77: 78: { 0, 0, 0, 0, "Some more nested options:", 10}, 79: {"subopt3", 'p', 0, 0, "Nested option 3"}, 80:	
	污染路径: 无	
62	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2817
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2815: tfree(fileList[i]); 2816: } 2817: tfree(fileList); 2818: } 2819:	
	污染路径: expanded from macro 'tfree'	
63	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 839
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 837: } else { 838: code = pAPI->metaFn.resumeTableMetaCursor(pInfo->pCur, 0, 0); 839: if (code != 0) { 840: pAPI->metaFn.closeTableMetaCursor(pInfo->pCur);	

	841: pInfo->pCur = NULL;	
	污染路径: 无	
64	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1269
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1267: while (name) { 1268: if (0 == strcmp(name, dbName)) { 1269: tfree(dupSeq); 1270: return 0; 1271: }	
	污染路径: expanded from macro 'tfree'	
65	缺陷发生位置: /utils/tsim/test/simTests.cpp	行号: 36
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 34: int32_t ret = 0; 35: int32_t argc = 3; 36: char *argv[4] = {0}; 37: 38: argc = 3;	
	污染路径: 无	
66	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 267
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 265: 266: static void ttq_signal_setup(void) { 267: struct sigaction sa = {0}; 268: 269: sa.sa_handler = ttq_handle_sigint;	
	污染路径: 无	

67	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号： 129
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 127: } 128: 129: if (remainder > 0) { 130: p = taosMemoryCalloc(remainder, sizeof(STableBlockScanInfo)); 131: TSDB_CHECK_NULL(p, code, lino, _end, terrno);	
	污染路径： 无	
68	缺陷发生位置： /source/libs/monitorfw/src/taos_metric_formatter_custom.c	行号： 181
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 179: memset(name, 0, size + 1); 180: memcpy(name, metric->name, size); 181: char* arr[2] = {0}; //arr[0] is table name, arr[1] is metric name 182: taos_monitor_split_str((char**)&arr, name, ":"); 183:	
	污染路径： 无	
69	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号： 713
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 711: } 712: 713: return 0; 714: } 715:	
	污染路径： 无	
70	缺陷发生位置：	行号：

	/tools/taos-tools/src/benchSys.c	166
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 164: {"binwidth", 'w', "NUMBER", 0, BENCH_WIDTH}, 165: {"table-prefix", 'm', "TABLE_PREFIX", 0, BENCH_PREFIX}, 166: {"escape-character", 'E', 0, 0, BENCH_ESCAPE}, 167: {"chinese", 'C', 0, 0, BENCH_CHINESE}, 168: {"normal-table", 'N', 0, 0, BENCH_NORMAL},	
	污染路径: 无	
71	缺陷发生位置: /contrib/lemon/lempar.c	行号: 245
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 243: #ifndef NDEBUG 244: #include <stdio.h> 245: static FILE *yyTraceFILE = 0; 246: static char *yyTracePrompt = 0; 247: #endif /* NDEBUG */	
	污染路径: 无	
72	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 95
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 93: 94: // opentsdb json protocol 95: char *jsons[1] = {0}; 96: // allocate memory for json data. can not use static memory. 97: size_t size = 1024;	
	污染路径: 无	
73	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 136

	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 134: static const struct argp_option argp_default_options[] = 135: { 136: {"help", '?', 0, 0, N_("Give this help list"), -1}, 137: {"usage", OPT_USAGE, 0, 0, N_("Give a short usage message")}, 138: {"program-name", OPT_PROGNAME, "NAME", OPTION_HIDDEN, N_("Set the program name")},	
	污染路径: 无	
74	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 1808
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1806: static int32_t extWinGetMultiTbWinFromTs(SOperatorInfo* pOperator, SArray* pWins, int64_t* tsCol, int64_t rowNum, int32_t* startPos) { 1807: int32_t idx = taosArraySearchIdx(pWins, tsCol, extWinTsWinCompare, TD_EQ); 1808: if (idx >= 0) { 1809: *startPos = 0; 1810: return idx;	
	污染路径: 无	
75	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 344
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 342: const char* passwd = "taosdata"; 343: #ifndef WINDOWS 344: argp_parse(&argp, argc, argv, 0, 0, NULL); 345: #endif 346: TAOS* taos = taos_connect(host, user, passwd, "", 0);	
	污染路径: 无	

76	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 274
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 272: } 273: 274: pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; 275: 276: ThreadParam* params = malloc(thread_count * sizeof(ThreadParam));	
	污染路径: expanded from macro 'PTHREAD_MUTEX_INITIALIZER' expanded from macro '__PTHREAD_MUTEX_INITIALIZER'	
77	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 510
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 508: 509: hol->num_entries = 0; 510: hol->clusters = 0; 511: 512: if (opts)	
	污染路径: 无	
78	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 179
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 177: 178: int rtGetTablesRouteInfo(TAOS * taos) { 179: char *table = {0}; 180: int *vgId1 = malloc(rtTables * sizeof(int)); 181: int vgId2 = 0;	
	污染路径: 无	

79	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 141
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 139: {"HANG", OPT_HANG, "SECS", OPTION_ARG_OPTIONAL OPTION_HIDDEN, 140: N_("Hang for SECS seconds (default 3600)"), 141: {0, 0} 142: }; 143:	
	污染路径: 无	
80	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号: 81
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 79: {"subopt3", 'p', 0, 0, "Nested option 3"}, 80: 81: {"subopt4", 'q', 0, 0, "Nested option 4", 1}, 82: 83: {0}	
	污染路径: 无	
81	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 168
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 166: {"escape-character", 'E', 0, 0, BENCH_ESCAPE}, 167: {"chinese", 'C', 0, 0, BENCH_CHINESE}, 168: {"normal-table", 'N', 0, 0, BENCH_NORMAL}, 169: {"random", 'M', 0, 0, BENCH_RANDOM}, 170: {"aggr-func", 'x', 0, 0, BENCH_AGGR},	
	污染路径: 无	
82	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchSys.c	154
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 152: {"angle-step", 'H', "NUMBER", 0, ANGLE_STEP}, 153: {"start-timestamp", 's', "NUMBER", 0, BENCH_START_TIMESTAMP}, 154: {"supplement-insert", 'U', 0, 0, BENCH_SUPPLEMENT}, 155: {"interlace-rows", 'B', "NUMBER", 0, BENCH_INTERLACE}, 156: {"rec-per-req", 'r', "NUMBER", 0, BENCH_BATCH},	
	污染路径: 无	
83	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 103
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 101: 102: remainder = pBuf->numOfTables % pBuf->numPerBucket; 103: if (remainder > 0) { 104: TSDB_CHECK_CONDITION(taosArrayGetSize(pBuf->pData) > 0, code, lino, _end, TSDB_CODE_INVALID_PARA); 105: px = taosArrayPop(pBuf->pData);	
	污染路径: 无	
84	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 1042
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1040: 1041: void initResultSizeInfo(SResultInfo* pResultInfo, int32_t numOfRows) { 1042: if (numOfRows == 0) { 1043: numOfRows = 4096; 1044: }	
	污染路径: 无	

85	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 983
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 981: static int32_t extWinGetMultiTbWinFromTs(SOperatorInfo* pOperator, SExternalWindowOperator* pExtW, int64_t* tsCol, int64_t rowNum, int32_t* startPos) { 982: int32_t idx = taosArraySearchIdx(pExtW->pWins, tsCol, extWinTsWinCompare, TD_EQ); 983: if (idx >= 0) { 984: *startPos = 0; 985: return idx; </pre>	
	污染路径: 无	
86	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 223
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 221: static const struct argp_option argp_version_options[] = 222: { 223: {"version", 'V', 0, 0, N_("Print program version"), - 1}, 224: {0, 0} 225: }; </pre>	
	污染路径: 无	
87	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2661
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 2659: // small record size 2660: int sql_s_perf1(TAOS *taos) { 2661: char *sql[3000] = {0}; 2662: TAOS_RES *result; 2663: </pre>	

	污染路径: 无	
88	缺陷发生位置: /docs/examples/c/sml_insert_demo.c	行号: 105
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 103: 104: // opentsdb json protocol 105: char *jsons[1] = {0}; 106: // allocate memory for json data. can not use static memory. 107: size_t size = 1024;	
	污染路径: 无	
89	缺陷发生位置: /source/libs/monitorfw/src/taos_metric.c	行号: 178
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 176: if (r) { 177: taos_free((void *)l_value); 178: TAOS_METRIC_SAMPLE_FROM_LABELS_HANDLE_UNLOCK(); 179: } 180: }	
	污染路径: expanded from macro 'TAOS_METRIC_SAMPLE_FROM_LABELS_HANDLE_UNLOCK' expanded from macro 'NULL'	
90	缺陷发生位置: /source/util/test/hashTest.cpp	行号: 158
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 156: int32_t code = 0; 157: int32_t num = 0; 158: TESTSTRUCT data = {0}; 159: const char* str1 = "abcdefg"; 160: const char* str2 = "aaaaaaa";	

	污染路径: 无	
91	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1106
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1104: } else { 1105: code = pAPI->metaFn.resumeTableMetaCursor(pInfo->pCur, 0, 0); 1106: if (code != 0) { 1107: pAPI->metaFn.closeTableMetaCursor(pInfo->pCur); 1108: pInfo->pCur = NULL;	
	污染路径: 无	
92	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 947
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 945: CJSON_PUBLIC(tools_cJSON *) tools_cJSON_ParseWithOpts(const char *value, const char **return_parse_end, tools_cJSON_bool require_null_terminated) 946: { 947: parse_buffer buffer = { 0, 0, 0, 0, { 0, 0, 0 } }; 948: tools_cJSON *item = NULL; 949:	
	污染路径: 无	
93	缺陷发生位置: /docs/examples/c/schemaless.c	行号: 47
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 45: (void)taos_select_db(taos, "db"); 46: 47: time_t ct = time(0); 48: int64_t ts = ct * 1000; 49: char* lineFormat = "sta %d,t0=true,t1=127i8,t2=32767i16,t3= %di32,t4=9223372036854775807i64,t9=11.12345f32,t10=22	

	.123456789f64,t11=\"binaryTagValue\",t12=L\"ncharTagValue\" c0=true,c1=127i8,c2=32767i16,c3=2147483647i32,c4=9223 372036854775807i64,c5=254u8,c6=32770u16,c7=21474836 99u32,c8=9223372036854775899u64,c9=11.12345f32,c10= 22.123456789f64,c11=\"binaryValue\",c12=L\"ncharValue\" %\" PRId64;	
	污染路径: 无	
94	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1697
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1695: case TSDB_DATA_TYPE_VARBINARY: 1696: case TSDB_DATA_TYPE_GEOMETRY: 1697: if (bufSize < 0) { 1698: return TSDB_CODE_TSC_INVALID_VALUE; 1699: }	
	污染路径: 无	
95	缺陷发生位置: /contrib/lemon/lemon.c	行号: 75
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 73: ** Global linked list of all memory allocations. 74: */ 75: static MemChunk *memChunkList = 0; 76: 77: /*	
	污染路径: 无	
96	缺陷发生位置: /contrib/lemon/lemon.c	行号: 91
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 89: static void *lemon_malloc(size_t nByte){ 90: MemChunk *p;	

	91: if(nByte<0) return 0; 92: p = malloc(nByte + sizeof(MemChunk)); 93: if(p==0){	
	污染路径: 无	
97	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 79
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 77: 78: { 0, 0, 0, 0, "Some more nested options:", 10}, 79: {"subopt3", 'p', 0, 0, "Nested option 3"}, 80: 81: {"subopt4", 'q', 0, 0, "Nested option 4", 1},	
	污染路径: 无	
98	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 80
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 78: int32_t qwtTestFetchQueueNum = 0; 79: SRWLatch qwtTestFetchQueueLock = 0; 80: struct SRpcMsg *qwtTestFetchQueue[qwtTestFetchQueueSize] = {0}; 81: 82: int32_t qwtTestSinkBlockNum = 0;	
	污染路径: 无	
99	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 1004
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1002: *pRecv = p->pBuf; 1003: *len = p->nBuf; 1004: p->pBuf = 0; 1005: p->nBuf = 0;	

	1006: } else {	
	污染路径: 无	
100	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 3300
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 3298: if (state->pFileSet != state->pr->pCurFileSet) { 3299: SDataFileReaderConfig conf = {.tsdb = pTsdB, .szPage = pTsdB->pVnode->config.tsdbPageSize}; 3300: const char *fileName[4] = {0}; 3301: if (pFileObj[0] != NULL) { 3302: conf.files[0].file = *pFileObj[0]->f;	
	污染路径: 无	
101	缺陷发生位置: /contrib/lemon/lemon.c	行号: 570
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 568: /* Allocate a new parser action */ 569: static struct action *Action_new(void){ 570: static struct action *actionfreelist = 0; 571: struct action *newaction; 572:	
	污染路径: 无	
102	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2815
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2813: 2814: for (int64_t i = 0; i < count; i++) { 2815: tfree(fileList[i]); 2816: } 2817: tfree(fileList);	
	污染路径:	

	expanded from macro 'tfree'	
103	缺陷发生位置: /contrib/lemon/lemon.c	行号: 118
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 116: void *pNew; 117: MemChunk *p; 118: if(pOld==0) return lemon_malloc(nNew); 119: p = (MemChunk*)pOld; 120: p--;	
	污染路径: 无	
104	缺陷发生位置: /source/libs/scalar/src/scivector.c	行号: 1321
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1319: colDataSetNNULL(pOutputCol, 0, numOfRows); 1320: } else { 1321: for (; i >= 0 && i < numOfRows; i += step, output += 1) { 1322: if (IS_HELPER_NULL(pLeftCol, i)) { 1323: colDataSetNULL(pOutputCol, i);	
	污染路径: 无	

8 C/C++缺陷类型-数组问题

8.1 分项汇总表

序号	C/C++缺陷类型-数组问题	缺陷/安全漏洞名称	数量	缺陷级别
----	----------------	-----------	----	------

1	WK4672	数组定义禁止没有显式的边界限定	338	严重
---	--------	-----------------	-----	----

8.2 缺陷/安全漏洞详解

8.2.1 严重类

8.2.1.1 WK4672 数组定义禁止没有显式的边界限定

缺陷数量：338 个		
缺陷描述： 数组定义禁止没有显式的边界限定。		
修复意见： 无		
1	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号： 86
	步骤描述： 数组 sub_args_doc 定义的没有显式的边界限定	
	代码片段： 84: }; 85: 86: static const char sub_args_doc[] = "STRING...\n-"; 87: static const char sub_doc[] = "\vThis is the doc string from the sub-arg-parser."; 88:	
	污染路径： 无	
2	缺陷发生位置： /source/util/test/regexTest.cpp	行号： 66
	步骤描述： 数组 s1 定义的没有显式的边界限定	
	代码片段： 64: TEST(testCase, regexCacheTest1) { 65: int times = 100000; 66: char s1[] = "abc";	

	67: auto start = std::chrono::high_resolution_clock::now(); 68:	
	污染路径: 无	
3	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 31
	步骤描述: 数组 keywordTable 定义的没有显式的边界限定	
	代码片段: 29: // clang-format off 30: // keywords in sql string 31: static SKeyword keywordTable[] = { 32: {"ACCOUNT", TK_ACCOUNT}, 33: {"ACCOUNTS", TK_ACCOUNTS},	
	污染路径: 无	
4	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 3103
	步骤描述: 数组 sample_labels 定义的没有显式的边界限定	
	代码片段: 3101: if (code == 0 && tsEnableMonitor && tsMonitorFqdn[0] != 0 && tsMonitorPort != 0 && pSubmitRsp->affectedRows > 0 && 3102: strlen(RPC_MSG_USER(pOriginalMsg)) > 0 && tsInsertCounter != NULL) { 3103: const char *sample_labels[] = {VNODE_METRIC_TAG_VALUE_INSERT_AFFECTED_ROWS, 3104: pVnode->monitor.strClusterId, 3105: pVnode->monitor.strDnodeId,	
	污染路径: 无	
5	缺陷发生位置: /utils/test/c/tmq_taosx_ci.c	行号: 746
	步骤描述: 数组 result 定义的没有显式的边界限定	
	代码片段: 744: } 745: } else { 746: char* result[] = {	

	747: <pre>{\"type\": \"create\", \"tableType\": \"normal\", \"tableName\": \"tb1\", \"columns\": [{\"name\": \"ts\",</pre> 748: \"type\":	
	污染路径: 无	
6	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 572
	步骤描述: 数组 ERR_strings 定义的没有显式的边界限定	
	代码片段: <pre>570: #define ERROR_CONVERTTOSTRING(String) #String, 571: #define ERROR_GENERATE_STRING(EXPR) ERROR_CONVERTTOSTRING(EXPR) 572: static const char* ERR_strings[] = { ERROR_LIST(ERROR_GENERATE_STRING) }; 573: 574: ERR_STATIC unsigned ERR_isError(size_t code) { return (code > ERROR(maxCode)); }</pre>	
	污染路径: 无	
7	缺陷发生位置: /contrib/test/cos/main.c	行号: 13
	步骤描述: 数组 TEST_COS_ENDPOINT 定义的没有显式的边界限定	
	代码片段: <pre>11: // static char TEST_COS_ENDPOINT[] = "cos.ap- guangzhou.myqcloud.com"; 12: // static char TEST_COS_ENDPOINT[] = "http://oss-cn- beijing.aliyuncs.com"; 13: static char TEST_COS_ENDPOINT[] = "http://cos.ap- beijing.myqcloud.com"; 14: // 数据万象的访问域名，详情请参见 https://cloud.tencent.com/document/product/460/31066 文档 15: static char TEST_CI_ENDPOINT[] = "https://ci.ap- guangzhou.myqcloud.com";</pre>	
	污染路径: 无	
8	缺陷发生位置: /source/dnode/mnode/impl/test/show/show.cpp	行号: 57

	步骤描述: 数组 passwd 定义的没有显式的边界限定	
	代码片段: 55: 56: TEST_F(MndTestShow, 03_ShowMsg_Conn) { 57: char passwd[] = "taosdata"; 58: SConnectReq connectReq = {0}; 59: char secretEncrypt[sizeof(connectReq.passwd)+1] = {0};	
	污染路径: 无	
9	缺陷发生位置:	行号:
	/source/common/src/tdataformat.c	6715
	步骤描述: 数组 tColDataCalcSMA 定义的没有显式的边界限定	
	代码片段: 6713: } 6714: 6715: void (*tColDataCalcSMA[])(SColData *pColData, SColumnDataAgg *pAggs) = { 6716: NULL, 6717: tColDataCalcSMABool, // TSDB_DATA_TYPE_BOOL	
10	污染路径: 无	
	缺陷发生位置:	行号:
	/tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	220
	步骤描述: 数组 uparam_names 定义的没有显式的边界限定	
	代码片段: 218: 219: /* The name-field mappings we know about. */ 220: static const struct uparam_name uparam_names[] = 221: { 222: { "dup-args", true, offsetof (struct uparams, dup_args) },	
11	污染路径: 无	
	缺陷发生位置:	行号:

	/test/new_test_framework/script/api/stmt2-nohole.c	233
	步骤描述: 数组 blob_str 定义的没有显式的边界限定	
	代码片段: 231: "一二三四五", "一二三四", "一二三", "一二", "一", 232: }; 233: const char* blob_str[] = { 234: "一", "一二", "一二三", "一二三四", "一二三四五", 235: "一二三四五六", "一二三四五六七", "一二三四五六七八", "一二三四五六七八九", "一二三四五六七八九十",	
	污染路径: 无	
12	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 635
	步骤描述: 数组 params2 定义的没有显式的边界限定	
	代码片段: 633: }; 634: 635: struct insert_params params2[] = { 636: {"UTC+9", "ntb", "2023-09-16 08:00:00", "2023-09-16 08:00:00-01:00"}, 637: {"UTC+9", "ctb1", "2023-09-16 08:00:00", "2023-09-16 11:00:00+02:00"},	
	污染路径: 无	
13	缺陷发生位置: /source/client/wrapper/src/wrapperVariable.c	行号: 94
	步骤描述: 数组 fp_taos_is_null_by_column 定义的没有显式的边界限定	
	代码片段: 92: void (*fp_taos_stop_query)(TAOS_RES *res) = NULL; 93: bool (*fp_taos_is_null)(TAOS_RES *res, int32_t row, int32_t col) = NULL; 94: int (*fp_taos_is_null_by_column)(TAOS_RES *res, int columnIndex, bool result[], int *rows) = NULL; 95: bool (*fp_taos_is_update_query)(TAOS_RES *res) = NULL; 96: int (*fp_taos_fetch_block)(TAOS_RES *res, TAOS_ROW *rows) = NULL;	

	污染路径: 无	
14	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 1291
	步骤描述: 数组 count_digits 定义的没有显式的边界限定	
	代码片段: 1289: EOperatorType op = OP_TYPE_DIV; 1290: std::vector<SDataType> out_types; 1291: auto count_digits = [](uint64_t v) { return std::floor(std::log10(v) + 1); }; 1292: std::vector<SDataType> equivalent_decimal_types; 1293: // #define TSDB_DATA_TYPE_NULL 0 // 1 bytes	
	污染路径: 无	
15	缺陷发生位置: /source/os/test/osEnvTests.cpp	行号: 135
	步骤描述: 数组 argv 定义的没有显式的边界限定	
	代码片段: 133: char *argv[] = {(char *)"/usr/local/bin/taosd", NULL}; 134: #else 135: char *argv[] = {(char *)"/usr/bin/taosd", NULL}; 136: #endif 137: osSetProcPath(1, argv);	
	污染路径: 无	
16	缺陷发生位置: /source/common/src/tglobal.c	行号: 2934
	步骤描述: 数组 debugOptions 定义的没有显式的边界限定	
	代码片段: 2932: 2933: { // 'bool/int32_t/int64_t/float/double' variables with general modification function 2934: static OptionNameAndVar debugOptions[] = { 2935: {"dDebugFlag", &dDebugFlag}, {"vDebugFlag", &vDebugFlag}, {"mDebugFlag", &mDebugFlag},	

	2936: {"wDebugFlag", &wDebugFlag}, {"azDebugFlag", &azDebugFlag}, {"sDebugFlag", &sDebugFlag},	
	污染路径: 无	
17	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 2734
	步骤描述: 数组 dec64table 定义的没有显式的边界限定	
	代码片段: 2732: { 2733: static const int dec32table[] = {0, 1, 2, 1, 4, 4, 4, 4}; /* added */ 2734: static const int dec64table[] = {8, 8, 8, 7, 8, 9,10,11}; /* subtracted */ 2735: const BYTE* const ostart = op; 2736: BYTE* const oLitEnd = op + sequence.litLength;	
	污染路径: 无	
18	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 3408
	步骤描述: 数组 dec32table 定义的没有显式的边界限定	
	代码片段: 3406: if (sequence.offset < 8) { 3407: /* close range match, overlap */ 3408: static const U32 dec32table[] = { 0, 1, 2, 1, 4, 4, 4, 4 }; /* added */ 3409: static const int dec64table[] = { 8, 8, 8, 7, 8, 9,10,11 }; /* subtracted */ 3410: int const sub2 = dec64table[sequence.offset];	
	污染路径: 无	
19	缺陷发生位置: /source/libs/azure/test/azExceptionTest.cpp	行号: 114
	步骤描述: 数组 object_name 定义的没有显式的边界限定	
	代码片段: 112: } 113:	

	114: const char object_name[] = "azut.bin"; 115: char path[PATH_MAX] = {0}; 116: char path_download[PATH_MAX] = {0};	
	污染路径: 无	
20	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 89
	步骤描述: 数组 doc 定义的没有显式的边界限定	
	代码片段: 87: 88: /* Program documentation. */ 89: static char doc[] = ""; 90: /* "Argp example #4 -- a program with somewhat more complicated\" */ 91: /* options\" */	
	污染路径: 无	
21	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/tst-argp2.c	行号: 22
	步骤描述: 数组 opt1 定义的没有显式的边界限定	
	代码片段: 20: #include <argp.h> 21: 22: static const struct argp_option opt1[] = 23: { 24: { "opt1", '1', "NUMBER", 0, "Option 1" },	
	污染路径: 无	
22	缺陷发生位置: /source/util/test/regexTest.cpp	行号: 181
	步骤描述: 数组 s1 定义的没有显式的边界限定	
	代码片段: 179: int times = 100; 180: int count = 1000; 181: char s1[] = "abc%*"; 182: char s2[] = "abc";	

	183: auto start = std::chrono::high_resolution_clock::now();	
	污染路径: 无	
23	缺陷发生位置: /source/util/src/tbase64.c	行号: 109
	步骤描述: 数组 tbase64_encoding_table 定义的没有显式的边界限定	
	代码片段: 107: } 108: 109: static char tbase64_encoding_table[] = 110: "ABCDEFGHIJKLMNOPQRSTUVWXYZ" 111: "abcdefghijklmnopqrstuvwxyz"	
	污染路径: 无	
24	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 96
	步骤描述: 数组 jtSubTypeStr 定义的没有显式的边界限定	
	代码片段: 94: char* jtColCondStr[] = {"", "NO COND", "EQ COND", "ON COND", "FULL COND"}; 95: char* jtJoinTypeStr[] = {"INNER", "LEFT", "RIGHT", "FULL"}; 96: char* jtSubTypeStr[] = {"", "OUTER", "SEMI", "ANTI", "ASOF", "WINDOW"}; 97: 98: int64_t TIMESTAMP_FILTER_VALUE = 10000000000;	
	污染路径: 无	
25	缺陷发生位置: /utils/test/c/sml_test.c	行号: 38
	步骤描述: 数组 sql 定义的没有显式的边界限定	
	代码片段: 36: taos_free_result(pRes); 37: 38: const char *sql[] = { 39: "readings,name=truck_0,fleet=South,driver=Trish,model=H-	

	2,device_version=v2.3 " 40: "load_capacity=1500,fuel_capacity=150,nominal_fuel_consumption=12,latitude=52.31854,longitude=4.72037,elevation="	
	污染路径: 无	
26	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 212
	步骤描述: 数组 transAsyncHandle 定义的没有显式的边界限定	
	代码片段: 210: static void uvHandleFreeById(SSvrRespMsg* pMsg, SWorkThrd* thrd); 211: static void uvHandleReloadTlsConfig(SSvrRespMsg* pMsg, SWorkThrd* thrd); 212: static void (*transAsyncHandle[])(SSvrRespMsg* msg, SWorkThrd* thrd) = { 213: uvHandleResp, uvHandleQuit, uvHandleRelease, uvHandleRegister, 214: uvHandleUpdate, uvHandleFreeById, uvHandleReloadTlsConfig};	
	污染路径: 无	
27	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 183
	步骤描述: 数组 BIT_mask 定义的没有显式的边界限定	
	代码片段: 181: 182: /*===== Local Constants =====*/ 183: static const unsigned BIT_mask[] = { 184: 0, 1, 3, 7, 0xF, 0x1F, 185: 0x3F, 0x7F, 0xFF, 0x1FF, 0x3FF, 0x7FF,	
	污染路径: 无	
28	缺陷发生位置: /contrib/test/cos/main.c	行号: 28
	步骤描述: 数组 TEST_BUCKET_NAME 定义的没有显式的边界限定	
	代码片段:	

	26: // static char TEST_BUCKET_NAME[] = "<bucketname- appid>"; 27: // static char TEST_BUCKET_NAME[] = "test-bucket-119"; 28: static char TEST_BUCKET_NAME[] = "test0711- 1309024725"; 29: // 对象拥有者，比如用户 UIN: 100000000001 30: static char TEST_UIN[] = "<Uin>"; // your uin	
	污染路径: 无	
29	缺陷发生位置: /utlis/test/c/tmq_taosx_ci.c	行号: 727
	步骤描述: 数组 result 定义的没有显式的边界限定	
	代码片段: 725: if (g_conf.snapShot) { 726: if (g_conf.subTable) { 727: char* result[] = { 728: "{"type\":"create\","tableType\":"super\","tableName\":"me ters_summary\","columns\":[{"name\":"_ " 729: "wstart\","type\":"9\","isPrimarykey\":false,"encode\":"delta- i\","compress\":"lz4\","level\":"medium\"}"	
	污染路径: 无	
30	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 128
	步骤描述: 数组 lines 定义的没有显式的边界限定	
	代码片段: 126: static void verifySchemaLess(TAOS* taos) { 127: TAOS_RES* res = NULL; 128: char* lines[] = { 129: "st,t1=3i64,t2=4f64,t3=L\"t3\" c1=3i64,c3=L\"passit\",c2=false,c4=4f64 1626006833639000000", 130: "st,t1=4i64,t3=L\"t4\",t2=5f64,t4=5f64 c1=3i64,c3=L\"passitagin\",c2=true,c4=5f64,c5=5f64 1626006833640000000",	
	污染路径:	

	无	
31	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1656
	步骤描述: 数组 name 定义的没有显式的边界限定	
	代码片段: 1654: size_t new_len = sizeof(new_usespid); 1655: 1656: int name[] = {CTL_KERN, KERN_CORE_USES_PID}; 1657: 1658: (void)memset(&args, 0, sizeof(struct __sysctl_args));	
	污染路径: 无	
32	缺陷发生位置: /source/libs/azure/test/azTest.cpp	行号: 323
	步骤描述: 数组 object_name_arr 定义的没有显式的边界限定	
	代码片段: 321: } 322: 323: const char *object_name_arr[] = {object_name}; 324: code = azDeleteObjects(object_name_arr, 1); 325: GTEST_ASSERT_EQ(code, 0);	
	污染路径: 无	
33	缺陷发生位置: /source/libs/tfs/test/tfsTest.cpp	行号: 157
	步骤描述: 数组 p45 定义的没有显式的边界限定	
	代码片段: 155: EXPECT_NE(taosDirExist(ap3), 1); 156: 157: char p45[] = "p5"; 158: char p44[] = "p4"; 159: char p4[] = "p4/p2/p1/p0";	
	污染路径: 无	
34	缺陷发生位置: /source/client/src/clientMonitorSql.c	行号: 47
	步骤描述:	

	数组 selectMonitorLabelValues 定义的没有显式的边界限定	
	代码片段: <pre> 45: uError("failed to generate clusterId:%" PRIu64, clusterId); 46: } 47: const char* selectMonitorLabelValues[] = {clusterIdStr, typeStr, user, monitorResultStr(result)}; 48: monitorCounterInc(clusterId, selectMonitorName, selectMonitorLabelValues); 49: } </pre>	
	污染路径: 无	
35	缺陷发生位置:	行号:
	/source/os/test/osFileTests.cpp	328
	步骤描述: 数组 buf3 定义的没有显式的边界限定	
	代码片段: <pre> 326: char buf1[] = "Hello "; 327: char buf2[] = "World "; 328: char buf3[] = "!\n"; 329: 330: iov[0].iov_base = buf1; </pre>	
	污染路径: 无	
36	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v06.c	3409
	步骤描述: 数组 dec64table 定义的没有显式的边界限定	
	代码片段: <pre> 3407: /* close range match, overlap */ 3408: static const U32 dec32table[] = { 0, 1, 2, 1, 4, 4, 4, 4 }; /* added */ 3409: static const int dec64table[] = { 8, 8, 8, 7, 8, 9,10,11 }; /* subtracted */ 3410: int const sub2 = dec64table[sequence.offset]; 3411: op[0] = match[0]; </pre>	
	污染路径: 无	
37	缺陷发生位置:	行号:
	/source/libs/scalar/test/scalar/scalarTests.cpp	3211

	步骤描述: 数组 val_float 定义的没有显式的边界限定	
	代码片段: 3209: 3210: // FLOAT 3211: float val_float[] = {13.00, 14.00, 15.00}; 3212: type = TSDB_DATA_TYPE_FLOAT; 3213: code = scltMakeDataBlock(&pInput, type, 0, rowNum, false);	
	污染路径: 无	
38	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 270
	步骤描述: 数组 tsMaxKeyByPrecision 定义的没有显式的边界限定	
	代码片段: 268: } 269: 270: extern int64_t tsMaxKeyByPrecision[]; 271: static int32_t vnodePreProcessSubmitTbData(SVnode *pVnode, SDecoder *pCoder, int64_t btimeMs, int64_t ctimeMs) { 272: int32_t code = 0;	
	污染路径: 无	
39	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 3294
	步骤描述: 数组 sample_labels 定义的没有显式的边界限定	
	代码片段: 3292: if (tsEnableMonitor && tsMonitorFqdn[0] != 0 && tsMonitorPort != 0 && pSubmitRsp->affectedRows > 0 && 3293: strlen(RPC_MSG_USER(pOriginalMsg)) > 0 && tsInsertCounter != NULL) { 3294: const char *sample_labels[] = {VNODE_METRIC_TAG_VALUE_INSERT_AFFECTED_ROWS, 3295: pVnode->monitor.strClusterId, 3296: pVnode->monitor.strDnodeId,	
	污染路径: 无	

40	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 159
	步骤描述: 数组 args_doc 定义的没有显式的边界限定	
	代码片段: 157: }; 158: 159: static const char args_doc[] = "STRING"; 160: static const char doc[] = "Test program for argp." 161: "\vThis doc string comes after the options."	
	污染路径: 无	
41	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 26
	步骤描述: 数组 taosUnRefHandle 定义的没有显式的边界限定	
	代码片段: 24: 25: void (*taosRefHandle[])(void* handle) = {transRefSrvHandle, transRefCliHandle}; 26: void (*taosUnRefHandle[])(void* handle) = {transUnrefSrvHandle, NULL}; 27: 28: int (*transReleaseHandle[])(void* handle, int32_t status) = {transReleaseSrvHandle, transReleaseCliHandle};	
	污染路径: 无	
42	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 713
	步骤描述: 数组 msgTypeList 定义的没有显式的边界限定	
	代码片段: 711: 712: int32_t qptGetRandSubplanMsgType() { 713: int32_t msgTypeList[] = {TDMT_VND_DELETE, TDMT_SCH_MERGE_QUERY, TDMT_SCH_QUERY, TDMT_VND_SUBMIT}; 714: 715: return QPT_CORRECT_HIGH_PROB() ? msgTypeList[taosRand()] %	

	(sizeof(msgTypeList)/sizeof(msgTypeList[0])) : taosRand();	
	污染路径: 无	
43	缺陷发生位置: /source/dnode/mnode/impl/src/mndUser.c	行号: 865
	步骤描述: 数组 weekdays 定义的没有显式的边界限定	
	代码片段: 863: } 864: 865: static const char *weekdays[] = {"SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT"}; 866: 867: static int32_t convertTimeRangesToStr(SUserObj *pUser, char **buf) {	
	污染路径: 无	
44	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 365
	步骤描述: 数组 tsCpuPeriodFile 定义的没有显式的边界限定	
	代码片段: 363: static char tsSysNetFile[] = "/proc/net/dev"; 364: static char tsSysCpuFile[] = "/proc/stat"; 365: static char tsCpuPeriodFile[] = "/sys/fs/cgroup/cpu/cpu.cfs_period_us"; 366: static char tsCpuQuotaFile[] = "/sys/fs/cgroup/cpu/cpu.cfs_quota_us"; 367: static char tsProcCpuFile[25] = {0};	
	污染路径: 无	
45	缺陷发生位置: /contrib/lemon/lemon.c	行号: 3747
	步骤描述: 数组 templatenamename 定义的没有显式的边界限定	
	代码片段: 3745: PRIVATE FILE *tplt_open(struct lemon *lemp) 3746: { 3747: static char templatenamename[] = "lempar.c"; 3748: char buf[1000];	

	3749: FILE *in;	
	污染路径: 无	
46	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 343
	步骤描述: 数组 tb_actions 定义的没有显式的边界限定	
	代码片段: 341: }; 342: 343: char* tb_actions[] = { 344: "add column ", "modify column ", "drop column ", "rename column ", "add tag ", 345: "modify tag ", "drop tag ", "rename tag ", "set tag ",	
	污染路径: 无	
47	缺陷发生位置: /source/dnode/mnode/impl/test/profile/profile.cpp	行号: 72
	步骤描述: 数组 passwd 定义的没有显式的边界限定	
	代码片段: 70: 71: TEST_F(MndTestProfile, 02_ConnectMsg_NotExistDB) { 72: char passwd[] = "taosdata"; 73: SConnectReq connectReq = {0}; 74: char secretEncrypt[sizeof(connectReq.passwd)+1] = {0};	
	污染路径: 无	
48	缺陷发生位置: /source/libs/command/src/explain.c	行号: 92
	步骤描述: 数组 targetName 定义的没有显式的边界限定	
	代码片段: 90: 91: char* qExplainGetTimerangeTargetStr(int32_t target) { 92: static char* targetName[] = {"", "Left", "Right", "Left/Right"}; 93: if (target <= 0 target > 3) {	

	94: return "Unknown";	
	污染路径: 无	
49	缺陷发生位置: /source/util/test/decompressTest.cpp	行号: 173
	步骤描述: 数组 end 定义的没有显式的边界限定	
	代码片段: 171: 172: const char* alg[] = {"disabled", "lz4", "zlib", "zstd", "tsz", "xz"}; 173: const char* end[] = {"disabled", "simppe8b", "delta", "test", "test"}; 174: void compressImplTestByAlg(void* pVal, int8_t type, int32_t num, uint32_t cmprAlg) { 175: {	
	污染路径: 无	
50	缺陷发生位置: /tools/taos-tools/src/benchDataGeometry.c	行号: 36
	步骤描述: 数组 GeoInfo 定义的没有显式的边界限定	
	代码片段: 34: 35: // should be ordered by (baseLen + minCoordNum * CCoordSize), ASC 36: const SGeoSubTypeInfo GeoInfo[] = { 37: {"POINT(", ")", 5, 1, 1, false}, {"LINESTRING(", ")", 9, 2, 4094, false}, {"POLYGON((" , ")", 13, 3, 4094, true)}}; 38:	
	污染路径: 无	
51	缺陷发生位置: /source/os/src/osSystem.c	行号: 212
	步骤描述: 数组 allowedCmds 定义的没有显式的边界限定	
	代码片段: 210: // 允许的命令白名单，用于防止命令注入 211: static bool isCommandAllowed(const char* cmd) { 212: const char* allowedCmds[] = {"taos", "taosd",	

	"taosdump", "taosBenchmark", "taosAdapter", "taosKeeper", NULL}; 213: for (const char** p = allowedCmds; *p != NULL; p++) { 214: size_t cmdLen = strlen(*p);	
	污染路径: 无	
52	缺陷发生位置: /docs/examples/c/line_example.c	行号: 30
	步骤描述: 数组 lines 定义的没有显式的边界限定	
	代码片段: 28: executeSQL(taos, "CREATE DATABASE test"); 29: executeSQL(taos, "USE test"); 30: char *lines[] = {"meters,location=California.LosAngeles,groupid=2 current=11.8,voltage=221,phase=0.28 1648432611249", 31: "meters,location=California.LosAngeles,groupid=2 current=13.4,voltage=223,phase=0.29 1648432611250", 32: "meters,location=California.LosAngeles,groupid=3 current=10.8,voltage=223,phase=0.29 1648432611249",	
	污染路径: 无	
53	缺陷发生位置: /source/util/src/tcompression.c	行号: 358
	步骤描述: 数组 bit_per_integer 定义的没有显式的边界限定	
	代码片段: 356: // Selector value: 0 1 2 3 4 5 6 7 8 9 10 11 357: // 12 13 14 15 358: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 359: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 360: char bit_to_selector[] = {0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 11, 12, 12, 12, 13, 13, 13, 13,	
	污染路径: 无	

54	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 412
	步骤描述: 数组 payload 定义的没有显式的边界限定	
	代码片段: <pre> 410: ASSERT_GE(fd, 0); 411: 412: static const char payload[] = "0123456789abcdef"; 413: ssize_t written = write(fd, payload, sizeof(payload) - 1); 414: ASSERT_EQ(written, static_cast<ssize_t>(sizeof(payload) - 1)); </pre>	
	污染路径: 无	
55	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 3632
	步骤描述: 数组 dec64table 定义的没有显式的边界限定	
	代码片段: <pre> 3630: /* close range match, overlap */ 3631: static const U32 dec32table[] = { 0, 1, 2, 1, 4, 4, 4, 4 }; /* added */ 3632: static const int dec64table[] = { 8, 8, 8, 7, 8, 9,10,11 }; /* subtracted */ 3633: int const sub2 = dec64table[sequence.offset]; 3634: op[0] = match[0]; </pre>	
	污染路径: 无	
56	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1735
	步骤描述: 数组 options 定义的没有显式的边界限定	
	代码片段: <pre> 1733: static int printPP = 0; 1734: 1735: static struct s_options options[] = { 1736: {OPT_FLAG, "b", (char*)&basisflag, "Print only the basis in report."}, 1737: {OPT_FLAG, "c", (char*)&compress, "Don't compress the action table."}, </pre>	

	污染路径: 无	
57	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 13231
	步骤描述: 数组 rollupFuncs 定义的没有显式的边界限定	
	代码片段: 13229: 13230: static bool validRollupFunc(const char* pFunc) { 13231: static const char* rollupFuncs[] = {"avg", "sum", "min", "max", "last", "first"}; 13232: static const int32_t numOfRollupFuncs = (sizeof(rollupFuncs) / sizeof(char*)); 13233: for (int i = 0; i < numOfRollupFuncs; ++i) {	
	污染路径: 无	
58	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 895
	步骤描述: 数组 pScopeStr 定义的没有显式的边界限定	
	代码片段: 893: 894: TEST(TaosSetSlowLogScopeTest, OthersScopeInput) { 895: char pScopeStr[] = "others"; 896: int32_t scope = 0; 897: int32_t result = taosSetSlowLogScope2(pScopeStr, &scope);	
	污染路径: 无	
59	缺陷发生位置: /utils/test/c/tmq_td38404.c	行号: 27
	步骤描述: 数组 result2 定义的没有显式的边界限定	
	代码片段: 25: 26: char* result1 = "{"type\":"create\","tableType\":"super\","tableName\":"s5 466\","columns\": [{"name\":"ts\","type\":9,"isPrimarykey\":false,"encode\":" delta-i\","compress\":"lz4\","level\":"medium"}, {"name\":"c1\","type\":4,"isPrimarykey\":false,"encode\":"s	

	<pre>imple8b\","compress\":"lz4\","level\":"medium"}, {"name\":"c2\","type\:4,\"isPrimarykey\":false,\"encode\":"s imple8b\","compress\":"lz4\","level\":"medium"}],\"tags\": [{"name\":"t\","type\:8,\"length\:32}, {"name\":"t2\","type\:10,\"length\:4}]]"; 27: char* result2 = "{\"type\":"create\", \"tableType\":"child\", \"tableName\":"t1\", \"using\":"s5466\", \"tagNum\:2, \"tags\": [{"name\":"t\","type\:8,\"value\":"\\\\\\\\\"}, {"name\":"t2\","type\:10,\"value\":"\\\\\\\\\"}], \"createList\": []}"; 28: 29: static TAOS* use_db() {</pre>	
	污染路径: 无	
60	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 630
	步骤描述: 数组 params1 定义的没有显式的边界限定	
	代码片段: <pre>628: }; 629: 630: struct insert_params params1[] = { 631: {"UTC", "ntb", "2023-09-16 17:00:00", "2023- 09-16 17:00:00+08:00"}, 632: {"UTC", "ctb1", "2023-09-16 17:00:00", "2023- 09-16 17:00:00+08:00"},</pre>	
	污染路径: 无	
61	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 96
	步骤描述: 数组 parseLocaltimeFp 定义的没有显式的边界限定	
	代码片段: <pre>94: // static int32_t parseLocaltimeWithDst(char* timestr, int64_t* time, int32_t timePrec, char delim); 95: 96: static int32_t (*parseLocaltimeFp[]) (char* timestr, int64_t* time, int32_t timePrec, char delim) = { 97: parseLocaltime, 98: // parseLocaltimeWithDst</pre>	

	污染路径: 无	
62	缺陷发生位置: /source/libs/txnode/src/txnodeMgmt.c	行号: 282
	步骤描述: 数组 envXnoded 定义的没有显式的边界限定	
	代码片段: 280: xnodedLogLevel, xnodedClusterId); 281: 282: char *envXnoded[] = {xnodedCfgDir, xnodedLogDir, dnodedEnvItem, 283: xnodedUserPass, xnodedToken, xnodedClusterId, 284: xnodedPipeSocket, xnodedLogLevel, NULL};	
	污染路径: 无	
63	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 49
	步骤描述: 数组 options 定义的没有显式的边界限定	
	代码片段: 47: #define OPT_TESTDIR 1001 48: 49: static struct option options[] = 50: { 51: #ifdef CMDLINE_OPTIONS	
	污染路径: 无	
64	缺陷发生位置: /source/client/test/session.cpp	行号: 594
	步骤描述: 数组 tz_test 定义的没有显式的边界限定	
	代码片段: 592: } 593: 594: char* tz_test[] = { 595: "2023-09-16 17:00:00+", "2023-09-16 17:00:00+a", "2023-09-16 17:00:00+8", "2023-09-16 17:00:00+832",	

	596: "2023-09-16 17:00:00+8323", "2023-09-16 17:00:00+:", "2023-09-16 17:00:00+8:", "2023-09-16 17:00:00++:",	
	污染路径: 无	
65	缺陷发生位置: /source/libs/parser/test/parTestMain.cpp	行号: 85
	步骤描述: 数组 long_options 定义的没有显式的边界限定	
	代码片段: 83: const char* optstring = ""; 84: // clang-format off 85: static struct option long_options[] = { 86: {"dump", no_argument, NULL, 'd'}, 87: {"async", required_argument, NULL, 'a'},	
	污染路径: 无	
66	缺陷发生位置: /source/util/test/cfgTest.cpp	行号: 297
	步骤描述: 数组 envCmd 定义的没有显式的边界限定	
	代码片段: 295: EXPECT_EQ(cfgAddString(pConfig, "test_string", "4", 0, 6, 0, 0), 0); 296: 297: const char *envCmd[] = {"test_bool=1", "test_int32=2", "test_int64=3", "test_float=4", "test_string=5", NULL}; 298: 299: ASSERT_EQ(cfgLoad(pConfig, CFG_STYPE_ENV_CMD, envCmd), TSDB_CODE_SUCCESS);	
	污染路径: 无	
67	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1421
	步骤描述: 数组 ops 定义的没有显式的边界限定	
	代码片段: 1419: int32_t code = TSDB_CODE_SUCCESS; 1420:	

	1421: SMetaTableOp ops[] = { 1422: {META_ENTRY_TABLE, META_TABLE_OP_INSERT}, // 1423: {META_SCHEMA_TABLE, META_TABLE_OP_UPDATA}, // TODO: need to be insert	
	污染路径: 无	
68	缺陷发生位置: /docs/examples/c/multi_bind_example.c	行号: 77
	步骤描述: 数组 current 定义的没有显式的边界限定	
	代码片段: 75: // values to bind 76: int64_t ts[] = {1648432611249, 1648432611749}; 77: float current[] = {10.3, 12.6}; 78: int voltage[] = {219, 218}; 79: float phase[] = {0.31, 0.33};	
	污染路径: 无	
69	缺陷发生位置: /docs/examples/c/multi_bind_example.c	行号: 78
	步骤描述: 数组 voltage 定义的没有显式的边界限定	
	代码片段: 76: int64_t ts[] = {1648432611249, 1648432611749}; 77: float current[] = {10.3, 12.6}; 78: int voltage[] = {219, 218}; 79: float phase[] = {0.31, 0.33}; 80: // is_null array	
	污染路径: 无	
70	缺陷发生位置: /source/util/src/tcompression.c	行号: 489
	步骤描述: 数组 bit_per_integer 定义的没有显式的边界限定	
	代码片段: 487: 488: // Selector value: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 489: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60};	

	490: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 491:	
	污染路径: 无	
71	缺陷发生位置: /tools/shell/test/shellTest.cpp	行号: 159
	步骤描述: 数组 cases 定义的没有显式的边界限定	
	代码片段: 157: 158: TEST(shellUtil, trimStr) { 159: struct Case { const char* in; const char* expect; } cases[] = { 160: {"3.0.8.0 trial ", "3.0.8.0"}, // remove "trial" and trailing spaces 161: {"3.0.8.0 trial", "3.0.8.0"}, // remove "trial"	
	污染路径: 无	
72	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 779
	步骤描述: 数组 file 定义的没有显式的边界限定	
	代码片段: 777: char *pCont = tjsonToString(json); 778: int32_t contLen = strlen(pCont); 779: char file[] = "sdb.json"; 780: TdFilePtr pFile = taosOpenFile(file, TD_FILE_CREATE TD_FILE_WRITE TD_FILE_TRUNC TD_FILE_WRITE_THROUGH); 781: if (pFile == NULL) {	
	污染路径: 无	
73	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 161
	步骤描述: 数组 arr2 定义的没有显式的边界限定	
	代码片段: 159: } 160: 161: uint64_t arr2[] = {8, 10, 12, 13};	

	162: f = (SArray *)taosArrayGetP(src, 1); 163: for (int i = 0; i < sizeof(arr2) / sizeof(arr2[0]); i++) {	
	污染路径: 无	
74	缺陷发生位置: /source/common/test/tmsgTest.cpp	行号: 109
	步骤描述: 数组 endsWith 定义的没有显式的边界限定	
	代码片段: 107: } 108: 109: auto endsWith = [](const string& str, const string& suffix) { 110: if (str.length() < suffix.length()) { 111: return false;	
	污染路径: 无	
75	缺陷发生位置: /source/libs/monitor/src/monFramework.c	行号: 146
	步骤描述: 数组 dnodes_log_sample_labels 定义的没有显式的边界限定	
	代码片段: 144: 145: int32_t dnodes_log_label_count = 4; 146: const char *dnodes_log_sample_labels[] = {"cluster_id", "dnode_id", "dnode_ep", "data_dir_name"}; 147: char *dnodes_log_gauges[] = {DNODE_LOG_AVAIL, DNODE_LOG_USED, DNODE_LOG_TOTAL}; 148: for(int32_t i = 0; i < 3; i++){	
	污染路径: 无	
76	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 216
	步骤描述: 数组 fstStateStr 定义的没有显式的边界限定	
	代码片段: 214: { .state = EmptyFinal, .val = 0b00000000 } }; 215: // debug 216: static const char* fstStateStr[] = {"ONE_TRANS_NEXT",	

	"ONE_TRANS", "ANY_TRANS", "EMPTY_FINAL"}; 217: 218: FstState fstStateCreate(State state) {	
	污染路径: 无	
77	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 2860
	步骤描述: 数组 ppDownstreams 定义的没有显式的边界限定	
	代码片段: 2858: SOperatorInfo* pDownstreams[2]; 2859: createDummyDownstreamOperators(2, pDownstreams); 2860: SOperatorInfo* ppDownstreams[] = {pDownstreams[0], pDownstreams[1]}; 2861: int32_t code = createMergeJoinOperatorInfo(ppDownstreams, 2, pNode, pTask, &jtCtx.pJoinOp); 2862: ASSERT_TRUE(NULL != jtCtx.pJoinOp);	
	污染路径: 无	
78	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 51
	步骤描述: 数组 joinType 定义的没有显式的边界限定	
	代码片段: 49: 50: char* getJoinTypeString(EJoinType type) { 51: static char* joinType[] = { "", "INNER", "LEFT", "RIGHT", "FULL"}; 52: return joinType[type]; 53: }	
	污染路径: 无	
79	缺陷发生位置: /utils/test/c/sml_test.c	行号: 90
	步骤描述: 数组 sql1 定义的没有显式的边界限定	
	代码片段: 88: // sql[2] = taosMemoryCalloc(1, 128);	

	89: // sql[3] = taosMemoryCalloc(1, 128); 90: const char *sql1[] = {"sys.if.bytes.out 1479496100 1.3E0 host=web01 interface=eth0", 91: "sys.if.bytes.out 1479496101 1.3E1 interface=eth0 host=web01 ", 92: "sys.if.bytes.out 1479496102 1.3E3 network=tcp",	
	污染路径: 无	
80	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 102
	步骤描述: 数组 jtFilterValue 定义的没有显式的边界限定	
	代码片段: 100: int64_t BIGINT_FILTER_VALUE = 3000000000000000; 101: 102: int64_t jtFilterValue[] = {TIMESTAMP_FILTER_VALUE, INT_FILTER_VALUE, INT_FILTER_VALUE, BIGINT_FILTER_VALUE}; 103: 104: bool jtErrorRerun = false;	
	污染路径: 无	
81	缺陷发生位置: /source/common/test/dataformatTest.cpp	行号: 523
	步骤描述: 数组 stbName 定义的没有显式的边界限定	
	代码片段: 521: 522: TEST(testCase, StreamWithoutDotInStbName2) { 523: char stbName[] = ""; 524: char ctbName[TSDB_TABLE_NAME_LEN] = {0}; 525: uint64_t groupId = 12345;	
	污染路径: 无	
82	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 221
	步骤描述: 数组 argp_version_options 定义的没有显式的边界限定	
	代码片段:	

	219: 220: 221: static const struct argp_option argp_version_options[] = 222: { 223: {"version", 'V', 0, 0, N_("Print program version"), -1}, 	
	污染路径: 无	
83	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 3189
	步骤描述: 数组 val_tinyint 定义的没有显式的边界限定	
	代码片段: 3187: 3188: // TINYINT 3189: int8_t val_tinyint[] = {13, 14, 15}; 3190: type = TSDB_DATA_TYPE_TINYINT; 3191: code = scltMakeDataBlock(&plInput, type, 0, rowNum, false);	
	污染路径: 无	
84	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 164
	步骤描述: 数组 data 定义的没有显式的边界限定	
	代码片段: 162: 163: TEST(testCase, smlParseCols_Error_Test) { 164: const char *data[] = {"st,t=1 c=\"89sd 1626006833639000000", // binary, nchar 165: "st,t=1 c=j\"89sd\" 1626006833639000000", 166: "st,t=1 c=\"89sd\"k 1626006833639000000",	
	污染路径: 无	
85	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 277
	步骤描述: 数组 keywords 定义的没有显式的边界限定	

	代码片段: 275: 276: // where keyword 277: char* keywords[] = { 278: "where ", "and ", "asc ", "desc ", "from ", "fill(", "limit ", 279: "interval(", "order by ", "order by ", "offset ", "or ", "group by ", "now()",	
	污染路径: 无	
86	缺陷发生位置: /source/client/wrapper/src/wrapperVariable.c	行号: 123
	步骤描述: 数组 fp_taos_get_tables_vgld 定义的没有显式的边界限定	
	代码片段: 121: int (*fp_taos_get_db_route_info)(TAOS *taos, const char *db, TAOS_DB_ROUTE_INFO *dbInfo) = NULL; 122: int (*fp_taos_get_table_vgld)(TAOS *taos, const char *db, const char *table, int *vgld) = NULL; 123: int (*fp_taos_get_tables_vgld)(TAOS *taos, const char *db, const char *table[], int tableNum, int *vgld) = NULL; 124: 125: int (*fp_taos_load_table_info)(TAOS *taos, const char *tableNameList) = NULL;	
	污染路径: 无	
87	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1467
	步骤描述: 数组 ops 定义的没有显式的边界限定	
	代码片段: 1465: int32_t code = TSDB_CODE_SUCCESS; 1466: 1467: SMetaTableOp ops[] = { 1468: {META_ENTRY_TABLE, META_TABLE_OP_INSERT}, // 1469: {META_UID_IDX, META_TABLE_OP_INSERT}, //	
	污染路径: 无	
88	缺陷发生位置: /utils/test/c/tmq_get_meta_json.c	行号: 153

	步骤描述: 数组 sql2 定义的没有显式的边界限定	
	代码片段: 151: 152: // check tag name duplication 153: const char *sql2[] = { 154: "alarm_record,t=34t,tag=alarm_recordr3331 uid=\"2+100012+303fe9b5\",deviceId=\"2iiiiii\",numId=\"100 012\" 1732527119497", 155: };	
	污染路径: 无	
89	缺陷发生位置: /source/client/inc/clientStmt.h	行号: 164
	步骤描述: 数组 gStmtStatusStr 定义的没有显式的边界限定	
	代码片段: 162: } STscStmt; 163: 164: extern char *gStmtStatusStr[]; 165: 166: #define STMT_LOG_SEQ(n) \ 167: #endif	
	污染路径: 无	
90	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 79
	步骤描述: 数组 telnets 定义的没有显式的边界限定	
	代码片段: 77: 78: // opentsdb telnet protocol 79: char *telnets[] = {telnet_demo}; 80: result = taos_schemaless_insert(taos, telnets, 1, TSDB_SML_TELNET_PROTOCOL, TSDB_SML_TIMESTAMP_MILLI_SECONDS); 81: code = taos_errno(result);	
	污染路径: 无	
91	缺陷发生位置:	行号:

	/source/common/src/ttime.c	891
	步骤描述: 数组 daysOfMonth 定义的没有显式的边界限定	
	代码片段: 889: tm.tm_year = y; 890: tm.tm_mon = m; 891: int daysOfMonth[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}; 892: if (taosIsLeapYear(1900 + tm.tm_year)) { 893: daysOfMonth[1] = 29;	
	污染路径: 无	
92	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 1316
	步骤描述: 数组 sql 定义的没有显式的边界限定	
	代码片段: 1314: 1315: #ifndef TD_ASTR 1316: char sql[] = "show grants"; 1317: 1318: TAOS_RES *tres = taos_query(shell.conn, sql);	
	污染路径: 无	
93	缺陷发生位置: /source/libs/tfs/test/tfsTest.cpp	行号: 158
	步骤描述: 数组 p44 定义的没有显式的边界限定	
	代码片段: 156: 157: char p45[] = "p5"; 158: char p44[] = "p4"; 159: char p4[] = "p4/p2/p1/p0"; 160: char ap4[128] = {0};	
	污染路径: 无	
94	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 521
	步骤描述: 数组 largeVglds 定义的没有显式的边界限定	

	代码片段: 519: TEST_F(MetricsTest, LargeVgldRange) { 520: // Test with large vgld values 521: int32_t largeVglds[] = {999999, 1000000, 2147483647}; // Including INT32_MAX 522: 523: for (int32_t i = 0; i < 3; i++) {	
	污染路径: 无	
95	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 926
	步骤描述: 数组 configDir 定义的没有显式的边界限定	
	代码片段: 924: extern STmqMetaInfo g_tmqInfo; 925: extern bool g_fail; 926: extern char configDir[]; 927: extern tools_cJSON * root; 928: extern uint64_t g_memoryUsage;	
	污染路径: 无	
96	缺陷发生位置: /source/client/test/session.cpp	行号: 623
	步骤描述: 数组 params1 定义的没有显式的边界限定	
	代码片段: 621: }; 622: 623: struct insert_params params1[] = { 624: {"UTC", "ntb", "2023-09-16 17:00:00", "2023-09-16 17:00:00+08:00"}, 625: {"UTC", "ctb1", "2023-09-16 17:00:00", "2023-09-16 17:00:00+08:00"},	
	污染路径: 无	
97	缺陷发生位置: /source/libs/tmqtt/mgmt/src/tmqttMgmt.c	行号: 86
	步骤描述: 数组 argsMqtttd 定义的没有显式的边界限定	
	代码片段:	

	84: snprintf(dnodeId, sizeof(dnodeId), "%d", pData->dnodeId); 85: 86: char *argsMqtttd[] = {path, "-c", configDir, "-d", dnodeId, NULL}; 87: options.args = argsMqtttd; 88: options.file = path;	
	污染路径: 无	
98	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 203
	步骤描述: 数组 lines 定义的没有显式的边界限定	
	代码片段: 201: int code = 0; 202: 203: char* lines[] = { 204: "st,t1=3i64,t2=4f64,t3=\"t3\"" c1=3i64,c3=L\"passit\",c2=false,c4=4f64 1626006833639000000ns", 205: "st,t1=4i64,t3=\"t4\",t2=5f64,t4=5f64" c1=3i64,c3=L\"passitagin\",c2=true,c4=5f64,c5=5f64 1626006833640000000ns",	
	污染路径: 无	
99	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 38
	步骤描述: 数组 locations_sml 定义的没有显式的边界限定	
	代码片段: 36: "California.SantaClara", "California.Cupertino"}; 37: 38: const char* locations_sml[] = { 39: "California.SanFrancisco", "California.LosAngles", 40: "California.SanDiego", "California.SanJose",	
	污染路径: 无	
100	缺陷发生位置: /source/util/test/talgoTest.cpp	行号: 65
	步骤描述:	

	数组 arr 定义的没有显式的边界限定	
	代码片段: <pre> 63: start_time = clock(); 64: for (int i = 0; i < 10000; i++) { 65: TestStruct arr[] = {{4, 2.5}, {3, 6}, {2, 1.5}, {3, 2}, {1, 3.5}, {3, 5}}; 66: taosMergeSort(arr, 6, sizeof(TestStruct), cmpFunc); 67: }</pre>	
	污染路径: 无	
101	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchCommandOpt.c	28
	步骤描述: 数组 g_aggreFunc 定义的没有显式的边界限定	
	代码片段: <pre> 26: "first(current)", 27: "last(current)"}; 28: char *g_aggreFunc[] = {"*", "count(*)", "avg(C0)", "sum(C0)", 29: "max(C0)", "min(C0)", "first(C0)", "last(C0)"}; 30:</pre>	
	污染路径: 无	
102	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/src/mndEncryptAlgr.c	309
	步骤描述: 数组 setFpArr 定义的没有显式的边界限定	
	代码片段: <pre> 307: STrans *pTrans = NULL; 308: 309: mndSetEncryptAlgrFp setFpArr[] = {mndSetSM4EncryptAlgr 310: #if defined(TD_ENTERPRISE) && defined(LINUX) 311: ,</pre>	
	污染路径: 无	
103	缺陷发生位置:	行号:
	/source/libs/planner/src/planOptimizer.c	10553
	步骤描述:	

	数组 optimizeRuleSet 定义的没有显式的边界限定	
	代码片段: 10551: 10552: // clang-format off 10553: static const SOptimizeRule optimizeRuleSet[] = { 10554: { .pName = "RewriteTail", .optimizeFunc = rewriteTailOptimize }, 10555: { .pName = "RewriteUnique", .optimizeFunc = rewriteUniqueOptimize },	
	污染路径: 无	
104	缺陷发生位置: /contrib/lemon/lempar.c	行号: 715
	步骤描述: 数组 yyRuleInfoNRhs 定义的没有显式的边界限定	
	代码片段: 713: /* For rule J, yyRuleInfoNRhs[J] contains the negative of the number 714: ** of symbols on the right-hand side of that rule. */ 715: static const signed char yyRuleInfoNRhs[] = { 716: %% 717: };	
105	污染路径: 无	
	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 40
	步骤描述: 数组 opt4 定义的没有显式的边界限定	
106	代码片段: 38: }; 39: 40: static const struct argp_option opt4[] = 41: { 42: { "opt4", '4', "NUMBER", 0, "Option 4" },	
	污染路径: 无	
	缺陷发生位置: /source/common/test/dataformatTest.cpp	行号: 503
	步骤描述:	

	数组 stbName 定义的没有显式的边界限定	
	代码片段: 501: 502: TEST(testCase, StreamWithStbName) { 503: char stbName[] = "1.table.stb"; 504: char ctbName[TSDB_TABLE_NAME_LEN] = {0}; 505: uint64_t groupId = 12345;	
	污染路径: 无	
107	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 2872
	步骤描述: 数组 dec64table 定义的没有显式的边界限定	
	代码片段: 2870: { 2871: static const int dec32table[] = { 0, 1, 2, 1, 4, 4, 4, 4 }; /* added */ 2872: static const int dec64table[] = { 8, 8, 8, 7, 8, 9,10,11 }; /* subtracted */ 2873: BYTE* const oLitEnd = op + sequence.litLength; 2874: const size_t sequenceLength = sequence.litLength + sequence.matchLength;	
	污染路径: 无	
108	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 3093
	步骤描述: 数组 dec64table 定义的没有显式的边界限定	
	代码片段: 3091: { 3092: static const int dec32table[] = {0, 1, 2, 1, 4, 4, 4, 4}; /* added */ 3093: static const int dec64table[] = {8, 8, 8, 7, 8, 9,10,11}; /* subtracted */ 3094: const BYTE* const ostart = op; 3095: BYTE* const oLitEnd = op + sequence.litLength;	
	污染路径: 无	
109	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 26

	步骤描述:	
	数组 succMessage 定义的没有显式的边界限定	
	代码片段: 24: 25: char resEncodingChunk[] = "Encoding: chunked"; 26: char succMessage[] = "succ"; 27: char resHttp[] = "HTTP/1.1 "; 28: char resHttpOk[] = "HTTP/1.1 200 OK";	
110	污染路径:	
	无	
	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/batchprepare.c	17
	步骤描述:	
	数组 shortColList 定义的没有显式的边界限定	
	代码片段: 15: #define TIME_PRECISION_NANO 2 16: 17: int32_t shortColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_INT}; 18: int32_t fullColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_BOOL, TSDB_DATA_TYPE_TINYINT, TSDB_DATA_TYPE_UTINYINT, TSDB_DATA_TYPE_SMALLINT, TSDB_DATA_TYPE_USMALLINT, TSDB_DATA_TYPE_INT, TSDB_DATA_TYPE_UINT, TSDB_DATA_TYPE_BIGINT, TSDB_DATA_TYPE_UBIGINT, TSDB_DATA_TYPE_FLOAT, TSDB_DATA_TYPE_DOUBLE, TSDB_DATA_TYPE_BINARY, TSDB_DATA_TYPE_NCHAR}; 19: int32_t bindColTypeList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_NCHAR};	
111	污染路径:	
	无	
	缺陷发生位置:	行号:
	/tools/taos-tools/src/taosdump.c	106
	步骤描述:	
	数组 options 定义的没有显式的边界限定	
	代码片段: 104: 105: /* The options we understand. */ 106: static struct argp_option options[] = { 107: // connection option 108: {"host", 'h', "HOST", 0,	

	污染路径: 无	
112	缺陷发生位置: /source/libs/monitor/src/monMain.c	行号: 141
	步骤描述: 数组 sample_labels 定义的没有显式的边界限定	
	代码片段: 139: taos_counter_t *counter = NULL; 140: int32_t label_count = 7; 141: const char *sample_labels[] = {VNODE_METRIC_TAG_NAME_SQL_TYPE, VNODE_METRIC_TAG_NAME_CLUSTER_ID, VNODE_METRIC_TAG_NAME_DNODE_ID, VNODE_METRIC_TAG_NAME_DNODE_EP, VNODE_METRIC_TAG_NAME_VGROUP_ID, VNODE_METRIC_TAG_NAME_USERNAME, 142: 143:	
	污染路径: 无	
113	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 3216
	步骤描述: 数组 dec32table 定义的没有显式的边界限定	
	代码片段: 3214: const BYTE* const base, const BYTE* const vBase, const BYTE* const dictEnd) 3215: { 3216: static const int dec32table[] = { 0, 1, 2, 1, 4, 4, 4, 4 }; /* added */ 3217: static const int dec64table[] = { 8, 8, 8, 7, 8, 9,10,11 }; /* subtracted */ 3218: BYTE* const oLitEnd = op + sequence.litLength;	
	污染路径: 无	
114	缺陷发生位置: /contrib/lemon/lemon.c	行号: 2070
	步骤描述: 数组 emsg 定义的没有显式的边界限定	
	代码片段:	

	2068: } 2069: 2070: static char emsg[] = "Command line syntax error: "; 2071: 2072: /*	
	污染路径: 无	
115	缺陷发生位置: /source/client/src/clientMonitorSlow.c	行号: 23
	步骤描述: 数组 slowQueryLabels 定义的没有显式的边界限定	
	代码片段: 21: const char* slowQueryHelp = "slow query log when cost over than config duration"; 22: const int slowQueryLabelCount = 4; 23: const char* slowQueryLabels[] = {"cluster_id", "username", "result", "duration"}; 24: 25: const int64_t usInSeconds = 1000 * 1000;	
	污染路径: 无	
116	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 20
	步骤描述: 数组 g_aggreFuncDemo 定义的没有显式的边界限定	
	代码片段: 18: extern char g_configDir[MAX_PATH_LEN]; 19: 20: char *g_aggreFuncDemo[] = {"*", 21: "count(*)", 22: "avg(current)",	
	污染路径: 无	
117	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 434
	步骤描述: 数组 validVglds 定义的没有显式的边界限定	
	代码片段: 432: 433: // Create valid vgroups hash with only some of the	

	vgroups 434: int32_t validVgIds[] = {100, 200, 204}; // 202 is missing 435: SHashObj *pValidVgroups = CreateValidVgroupsHash(validVgIds, 3); 436:	
	污染路径: 无	
118	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 585
	步骤描述: 数组 keys 定义的没有显式的边界限定	
	代码片段: 583: if (!cmd) return NULL; 584: 585: const char *keys[] = { "FROM", "USING" }; 586: const int kcnt = sizeof(keys) / sizeof(keys[0]); 587:	
	污染路径: 无	
119	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1273
	步骤描述: 数组 ops 定义的没有显式的边界限定	
	代码片段: 1271: int32_t code = TSDB_CODE_SUCCESS; 1272: 1273: SMetaTableOp ops[] = { 1274: {META_ENTRY_TABLE, META_TABLE_OP_INSERT}, // 1275: {META_SCHEMA_TABLE, META_TABLE_OP_UPDATA}, // TODO: need to be insert	
	污染路径: 无	
120	缺陷发生位置: /docs/examples/c/multi_bind_example.c	行号: 79
	步骤描述: 数组 phase 定义的没有显式的边界限定	
	代码片段: 77: float current[] = {10.3, 12.6}; 78: int voltage[] = {219, 218};	

	79: float phase[] = {0.31, 0.33}; 80: // is_null array 81: char is_null[2] = {0};	
	污染路径: 无	
121	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 2499
	步骤描述: 数组 splitRuleSet 定义的没有显式的边界限定	
	代码片段: 2497: 2498: // clang-format off 2499: static const SSplitRule splitRuleSet[] = { 2500: {.pName = "SuperTableSplit", .splitFunc = stableSplit}, 2501: {.pName = "SingleTableJoinSplit", .splitFunc = singleTableJoinSplit},	
	污染路径: 无	
122	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 170
	步骤描述: 数组 filterDict 定义的没有显式的边界限定	
	代码片段: 168: static int32_t sysFilte__Type(void* arg, SNode* pNode, SArray* result); 169: 170: const SStabFltFuncDef filterDict[] = { 171: {.name = "table_name", .chkFunc = sysChkFilter__TableName, .fltFunc = sysFilte__TableName}, 172: {.name = "db_name", .chkFunc = sysChkFilter__DBName, .fltFunc = sysFilte__DbName},	
	污染路径: 无	
123	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 28
	步骤描述: 数组 opt2 定义的没有显式的边界限定	
	代码片段:	

	26: }; 27: 28: static const struct argp_option opt2[] = 29: { 30: { "opt2", '2', "NUMBER", 0, "Option 2" },	
	污染路径: 无	
124	缺陷发生位置: /source/util/src/ttimer.c	行号: 100
	步骤描述: 数组 wheels 定义的没有显式的边界限定	
	代码片段: 98: static uintptr_t nextTimerId = 0; 99: 100: static time_wheel_t wheels[] = { 101: {.resolution = MSECONDS_PER_TICK, .size = 4096}, 102: {.resolution = 1000, .size = 1024},	
	污染路径: 无	
125	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 28
	步骤描述: 数组 transReleaseHandle 定义的没有显式的边界限定	
	代码片段: 26: void (*taosUnRefHandle[])(void* handle) = {transUnrefSrvHandle, NULL}; 27: 28: int (*transReleaseHandle[])(void* handle, int32_t status) = {transReleaseSrvHandle, transReleaseCliHandle}; 29: 30: static int32_t transValidLocalFqdn(const char* localFqdn, SlpAddr* addr) {	
	污染路径: 无	
126	缺陷发生位置: /source/util/test/cacheTest.cpp	行号: 14
	步骤描述: 数组 data1 定义的没有显式的边界限定	
	代码片段: 12:	

	13: const char* key1 = "test1"; 14: char data1[] = "test11"; 15: 16: char* cachedObj = (char*)taosCachePut(tscMetaCache, key1, strlen(key1), data1, strlen(data1) + 1, 1);	
	污染路径: 无	
127	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 106
	步骤描述: 数组 sysTableShowAdapter 定义的没有显式的边界限定	
	代码片段: 104: 105: // clang-format off 106: static const SSysTableShowAdapter sysTableShowAdapter[] = { 107: { 108: .showType = QUERY_NODE_SHOW_DNODES_STMT,	
	污染路径: 无	
128	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 920
	步骤描述: 数组 g_aggreFuncDemo 定义的没有显式的边界限定	
	代码片段: 918: 919: /* ***** Global variables ***** */ 920: extern char * g_aggreFuncDemo[]; 921: extern char * g_aggreFunc[]; 922: extern SArguments * g_arguments;	
	污染路径: 无	
129	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 10
	步骤描述: 数组 DBPath 定义的没有显式的边界限定	
	代码片段: 8: 9: // const char DBPath[] = "/tmp/rocksdb_c_simple_example"; 10: const char DBPath[] = "rocksdb_c_simple_example";	

	11: const char DBBackupPath[] = "/tmp/rocksdb_c_simple_example_backup"; 12:	
	污染路径: 无	
130	缺陷发生位置: /source/dnode/mnode/impl/test/profile/profile.cpp	行号: 34
	步骤描述: 数组 passwd 定义的没有显式的边界限定	
	代码片段: 32: connectReq.pid = 1234; 33: 34: char passwd[] = "taosdata"; 35: char secretEncrypt[sizeof(connectReq.passwd)+1] = {0}; 36: taosEncryptPass_c((uint8_t*)passwd, strlen(passwd), secretEncrypt);	
	污染路径: 无	
131	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 20
	步骤描述: 数组 optrIdxList 定义的没有显式的边界限定	
	代码片段: 18: int32_t fullColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_BOOL, TSDB_DATA_TYPE_TINYINT, TSDB_DATA_TYPE_UTINYINT, TSDB_DATA_TYPE_SMALLINT, TSDB_DATA_TYPE_USMALLINT, TSDB_DATA_TYPE_INT, TSDB_DATA_TYPE_UINT, TSDB_DATA_TYPE_BIGINT, TSDB_DATA_TYPE_UBIGINT, TSDB_DATA_TYPE_FLOAT, TSDB_DATA_TYPE_DOUBLE, TSDB_DATA_TYPE_BINARY, TSDB_DATA_TYPE_NCHAR}; 19: int32_t bindColTypeList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_NCHAR}; 20: int32_t optrIdxList[] = {5, 11}; 21: 22: typedef struct {	
	污染路径: 无	
132	缺陷发生位置:	行号:

	/source/libs/executor/test/joinTests.cpp	95
	步骤描述: 数组 jtJoinTypeStr 定义的没有显式的边界限定	
	代码片段: 93: 94: char* jtColCondStr[] = {"", "NO COND", "EQ COND", "ON COND", "FULL COND"}; 95: char* jtJoinTypeStr[] = {"INNER", "LEFT", "RIGHT", "FULL"}; 96: char* jtSubTypeStr[] = {"", "OUTER", "SEMI", "ANTI", "ASOF", "WINDOW"}; 97:	
	污染路径: 无	
133	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 903
	步骤描述: 数组 pScopeStr 定义的没有显式的边界限定	
	代码片段: 901: 902: TEST(TaosSetSlowLogScopeTest, NoneScopeInput) { 903: char pScopeStr[] = "none"; 904: int32_t scope = 0; 905: int32_t result = taosSetSlowLogScope2(pScopeStr, &scope);	
	污染路径: 无	
134	缺陷发生位置: /source/os/test/osMathTests.cpp	行号: 52
	步骤描述: 数组 arr 定义的没有显式的边界限定	
	代码片段: 50: TEST(osMathTests, taosSort) { 51: // Create an array of test data 52: TestStruct arr[] = {{4, 2.5}, {2, 1.5}, {1, 3.5}, {3, 4.5}}; 53: 54: // Sort the array using taosSort	
	污染路径: 无	

	污染路径: 无	
138	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 135
	步骤描述: 数组 options 定义的没有显式的边界限定	
	代码片段: 133: #define OPT_SESS 2 134: 135: struct argp_option options[] = 136: { 137: {"pid", 'p', "PID", 0, "List the process PID"},	
	污染路径: 无	
139	缺陷发生位置: /source/libs/metrics/src/metrics.c	行号: 165
	步骤描述: 数组 label_values 定义的没有显式的边界限定	
	代码片段: 163: snprintf(dnodeIdStr, sizeof(dnodeIdStr), "%d", dnodeId); 164: snprintf(vgIdStr, sizeof(vgIdStr), "%d", vgId); 165: const char *label_values[] = {VNODE_WRITE_METRIC, clusterIdStr, dnodeIdStr, 166: dnodeEp ? dnodeEp : "", vgIdStr, dbname ? dbname : ""}; 167:	
	污染路径: 无	
140	缺陷发生位置: /tools/taos-tools/inc/benchLocations.h	行号: 19
	步骤描述: 数组 locations_chinese 定义的没有显式的边界限定	
	代码片段: 17: #define __BENCH_LOCATIONS__ 18: 19: const char* locations_chinese[] = {"旧金山","洛杉矶","圣地亚哥", "圣何塞","帕洛阿尔托","坎贝尔", 20: "山景城","森尼韦尔","圣克拉拉","库比蒂诺"}; 21:	

	污染路径: 无	
141	缺陷发生位置: /source/libs/txnode/src/txnodeMgmt.c	行号: 222
	步骤描述: 数组 argsXnoded 定义的没有显式的边界限定	
	代码片段: 220: xndInfo("xnode mgmt spawn xnoded path: %s", path); 221: // char *argsXnoded[] = {path, "-c", configDir, "-d", dnodeld, NULL}; 222: char *argsXnoded[] = {path, NULL}; 223: options.args = argsXnoded; 224: options.file = path;	
	污染路径: 无	
142	缺陷发生位置: /source/libs/tfs/test/tfsTest.cpp	行号: 130
	步骤描述: 数组 p1 定义的没有显式的边界限定	
	代码片段: 128: ASSERT_NE(pTfs, nullptr); 129: 130: char p1[] = "p1"; 131: char ap1[128] = {0}; 132: snprintf(ap1, 128, "%s%s%s", root, TD_DIRSEP, p1);	
	污染路径: 无	
143	缺陷发生位置: /source/util/test/regexTest.cpp	行号: 141
	步骤描述: 数组 s2 定义的没有显式的边界限定	
	代码片段: 139: int times = 100000; 140: char s1[] = "abc%*"; 141: char s2[] = "abc"; 142: auto start = std::chrono::high_resolution_clock::now(); 143:	
	污染路径: 无	

144	缺陷发生位置: /source/client/src/clientSml.c	行号: 110
	步骤描述: 数组 smlToMilli 定义的没有显式的边界限定	
	代码片段: 108: #define IS_COMMENT(protocol, data) (protocol == TSDB_SML_LINE_PROTOCOL && data == '#') 109: 110: int64_t smlToMilli[] = {3600000LL, 60000LL, 1000LL}; 111: int64_t smlFactorNS[] = {NANOSECOND_PER_MSEC, NANOSECOND_PER_USEC, 1}; 112: int64_t smlFactorS[] = {1000LL, 1000000LL, 1000000000LL};	
	污染路径: 无	
145	缺陷发生位置: /source/util/src/tconfig.c	行号: 1171
	步骤描述: 数组 name 定义的没有显式的边界限定	
	代码片段: 1169: int32_t code = 0; 1170: if (string[0] == 'T' && string[1] == 'Z' && string[2] == '=') { 1171: char name[] = "timezone"; 1172: char *value = string + 3; 1173:	
	污染路径: 无	
146	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 366
	步骤描述: 数组 tsCpuQuotaFile 定义的没有显式的边界限定	
	代码片段: 364: static char tsSysCpuFile[] = "/proc/stat"; 365: static char tsCpuPeriodFile[] = "/sys/fs/cgroup/cpu/cpu.cfs_period_us"; 366: static char tsCpuQuotaFile[] = "/sys/fs/cgroup/cpu/cpu.cfs_quota_us"; 367: static char tsProcCpuFile[25] = {0}; 368: static char tsProcMemFile[25] = {0};	

	污染路径: 无	
147	缺陷发生位置: /source/client/test/session.cpp	行号: 905
	步骤描述: 数组 test_tz 定义的没有显式的边界限定	
	代码片段: 903: time_t time_summer = 1731323281 - 120 * 24 * 60 * 60; 904: 905: struct test_times { 906: const char* name; 907: time_t t;	
	污染路径: 无	
148	缺陷发生位置: /source/common/src/tpriv.c	行号: 29
	步骤描述: 数组 __privObjInfo 定义的没有显式的边界限定	
	代码片段: 27: } SPrivObjInfo; 28: 29: static const SPrivObjInfo __privObjInfo[] = { 30: {"CLUSTER", 0}, {"NODE", 0}, {"DATABASE", 0}, {"TABLE", 1}, {"FUNCTION", 0}, {"INDEX", 1}, 31: {"VIEW", 1}, {"USER", 0}, {"ROLE", 0}, {"RSMA", 1}, {"TSMA", 1}, {"TOPIC", 1},	
	污染路径: 无	
149	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 540
	步骤描述: 数组 reciprocal_table 定义的没有显式的边界限定	
	代码片段: 538: 539: /// Reciprocal lookup table. 540: constexpr uint16_t reciprocal_table[] = {REPEAT256()}; 541: 542: #undef REPEAT4	
	污染路径: 无	

150	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 348
	步骤描述: 数组 user_actions 定义没有显式的边界限定	
	代码片段: 346: }; 347: 348: char* user_actions[] = {"pass ", "enable ", "sysinfo ", "createdb "}; 349: 350: char* tb_options[] = {"comment ", "watermark ", "max_delay ", "ttl ", "rollup(", "sma(", "virtual 1"};	
	污染路径: 无	
151	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 97
	步骤描述: 数组 args_doc 定义没有显式的边界限定	
	代码片段: 95: 96: /* A description of the arguments we accept. */ 97: static char args_doc[] = "dbname [tbname ...]\n-- databases db1,db2,... \n" 98: "--all-databases\n-i inpath\n-o outpath"; 99:	
	污染路径: 无	
152	缺陷发生位置: /source/common/src/ttime.c	行号: 1485
	步骤描述: 数组 weekDays 定义没有显式的边界限定	
	代码片段: 1483: } TSFormatNode; 1484: 1485: static const char* const weekDays[] = {"Sunday", "Monday", "Tuesday", "Wednesday", 1486: "Thursday", "Friday", "Saturday", "NULL"}; 1487: static const char* const shortWeekDays[] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "NULL"};	

	污染路径: 无	
153	缺陷发生位置: /source/client/src/clientStmt.c	行号: 10
	步骤描述: 数组 gStmtStatusStr 定义的没有显式的边界限定	
	代码片段: 8: #include "parser.h" 9: 10: char* gStmtStatusStr[] = {"unknown", "init", "prepare", "settbyname", "settags", 11: "fetchFields", "bind", "bindCol", "addBatch", "exec"}; 12:	
	污染路径: 无	
154	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp1.c	行号: 32
	步骤描述: 数组 test_options 定义的没有显式的边界限定	
	代码片段: 30: 31: 32: static const struct argp_option test_options[] = 33: { 34: { NULL, 0, NULL, 0, "\	
	污染路径: 无	
155	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 38
	步骤描述: 数组 gCurrentFname 定义的没有显式的边界限定	
	代码片段: 36: } STFileHash; 37: 38: static const char *gCurrentFname[] = { 39: [TSDB_FCURRENT] = "current.json", 40: [TSDB_FCURRENT_C] = "current.c.json",	
	污染路径:	

	无	
156	缺陷发生位置： /source/libs/index/test/utilUT.cc	行号： 130
	步骤描述： 数组 arr1 定义的没有显式的边界限定	
	代码片段： 128: clearFinalArray(rsIt); 129: 130: uint64_t arr1[] = {2, 3, 4, 5}; 131: SArray * f = (SArray *)taosArrayGetP(src, 0); 132: for (int i = 0; i < sizeof(arr1) / sizeof(arr1[0]); i++) {	
	污染路径： 无	
157	缺陷发生位置： /source/dnode/mnode/impl/test/xnode/xnodeAgentTest.cpp	行号： 30
	步骤描述： 数组 MNDXNODE_DEFAULT_SECRET 定义的没有显式的边界限定	
	代码片段： 28: } agentTokenField; 29: 30: const unsigned char MNDXNODE_DEFAULT_SECRET[] = {126, 222, 130, 137, 43, 122, 41, 173, 144, 146, 116, 31: 138, 153, 244, 251, 99, 50, 55, 140, 238, 218, 232, 32: 15, 161, 226, 54, 130, 40, 211, 234, 111, 171};	
	污染路径： 无	
158	缺陷发生位置： /source/libs/parser/src/parTranslator.c	行号： 13242
	步骤描述： 数组 aggrRollupFuncs 定义的没有显式的边界限定	
	代码片段： 13240: 13241: static bool aggrRollupFunc(const char* pFunc) { 13242: static const char* aggrRollupFuncs[] = {"avg", "sum"}; 13243: static const int32_t numOfAggrRollupFuncs = (sizeof(aggrRollupFuncs) / sizeof(char*)); 13244: for (int i = 0; i < numOfAggrRollupFuncs; ++i) {	

	污染路径: 无	
159	缺陷发生位置: /source/client/src/clientSml.c	行号: 111
	步骤描述: 数组 smlFactorNS 定义的没有显式的边界限定	
	代码片段: 109: 110: int64_t smlToMilli[] = {3600000LL, 60000LL, 1000LL}; 111: int64_t smlFactorNS[] = {NANOSECOND_PER_MSEC, NANOSECOND_PER_USEC, 1}; 112: int64_t smlFactorS[] = {1000LL, 1000000LL, 1000000000LL}; 113:	
	污染路径: 无	
160	缺陷发生位置: /source/libs/tfs/test/tfsTest.cpp	行号: 137
	步骤描述: 数组 p2 定义的没有显式的边界限定	
	代码片段: 135: EXPECT_EQ(taosDirExist(ap1), 1); 136: 137: char p2[] = "p2"; 138: char ap2[128] = {0}; 139: snprintf(ap2, 128, "%s%s%s", root, TD_DIRSEP, p2);	
	污染路径: 无	
161	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 56
	步骤描述: 数组 joinSType 定义的没有显式的边界限定	
	代码片段: 54: 55: char* getJoinSTypeString(EJoinSubType type) { 56: static char* joinSType[] = {"", "", "OUTER", "SEMI", "ANTI", "ANY", "ASOF", "WINDOW"}; 57: return joinSType[type]; 58: }	
	污染路径:	

	无	
162	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 3092
	步骤描述: 数组 dec32table 定义的没有显式的边界限定	
	代码片段: 3090: BYTE* const base, BYTE* const oend) 3091: { 3092: static const int dec32table[] = {0, 1, 2, 1, 4, 4, 4, 4}; /* added */ 3093: static const int dec64table[] = {8, 8, 8, 7, 8, 9,10,11}; /* subtracted */ 3094: const BYTE* const ostart = op;	
	污染路径: 无	
163	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1754
	步骤描述: 数组 dec64table 定义的没有显式的边界限定	
	代码片段: 1752: { 1753: static const int dec32table[] = {0, 1, 2, 1, 4, 4, 4, 4}; /* added */ 1754: static const int dec64table[] = {8, 8, 8, 7, 8, 9,10,11}; /* subtracted */ 1755: const BYTE* const ostart = op; 1756: const size_t litLength = sequence.litLength;	
	污染路径: 无	
164	缺陷发生位置: /source/os/test/osFileTests.cpp	行号: 327
	步骤描述: 数组 buf2 定义的没有显式的边界限定	
	代码片段: 325: TaosIOVec iov[3]; 326: char buf1[] = "Hello "; 327: char buf2[] = "World "; 328: char buf3[] = "!\n"; 329:	

	污染路径: 无	
165	缺陷发生位置: /utils/test/c/sml_test.c	行号: 137
	步骤描述: 数组 sql3 定义的没有显式的边界限定	
	代码片段: 135: taos_free_result(pRes); 136: 137: const char *sql3[] = {"sysif.bytes.out 1479496300 1.1E0 interface=eth0 host=web01"}; 138: pRes = taos_schemaless_insert(taos, (char **)sql3, sizeof(sql3) / sizeof(sql3[0]), TSDB_SML_TELNET_PROTOCOL, 139: TSDB_SML_TIMESTAMP_NANO_SECONDS);	
	污染路径: 无	
166	缺陷发生位置: /tools/shell/test/shellTest.cpp	行号: 82
	步骤描述: 数组 s0 定义的没有显式的边界限定	
	代码片段: 80: 81: // str false 82: const char *s0[] = { 83: "create table st(ts)", 84: "create table st(ts timestamp, age) ",	
	污染路径: 无	
167	缺陷发生位置: /source/util/test/cacheTest.cpp	行号: 21
	步骤描述: 数组 data2 定义的没有显式的边界限定	
	代码片段: 19: printf("obj is still valid: %s\n", cachedObj); 20: 21: char data2[] = "test22"; 22: taosCacheRelease(tscMetaCache, (void**)&cachedObj, false); 23:	

	污染路径: 无	
168	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 887
	步骤描述: 数组 pScopeStr 定义的没有显式的边界限定	
	代码片段: 885: 886: TEST(TaosSetSlowLogScopeTest, InsertScopeInput) { 887: char pScopeStr[] = "insert"; 888: int32_t scope = 0; 889: int32_t result = taosSetSlowLogScope2(pScopeStr, &scope);	
	污染路径: 无	
169	缺陷发生位置: /source/os/test/osFileTests.cpp	行号: 326
	步骤描述: 数组 buf1 定义的没有显式的边界限定	
	代码片段: 324: 325: TaosIOVec iov[3]; 326: char buf1[] = "Hello "; 327: char buf2[] = "World "; 328: char buf3[] = "!\n";	
	污染路径: 无	
170	缺陷发生位置: /docs/examples/c-ws/sml_insert_demo.c	行号: 66
	步骤描述: 数组 lines 定义的没有显式的边界限定	
	代码片段: 64: 65: // influxdb line protocol 66: char *lines[] = {line_demo}; 67: int totalLines = 0; 68: result = ws_schemaless_insert_raw(taos, line_demo, strlen(line_demo), &totalLines, WS_TSDB_SML_LINE_PROTOCOL,	
	污染路径:	

	无	
171	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 497
	步骤描述: 数组 isIdChar 定义的没有显式的边界限定	
	代码片段: 495: // clang-format on 496: 497: static const char isIdChar[] = { 498: /* x0 x1 x2 x3 x4 x5 x6 x7 x8 x9 xA xB xC xD xE xF */ 499: 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* 0x */	
	污染路径: 无	
172	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1230
	步骤描述: 数组 ops 定义的没有显式的边界限定	
	代码片段: 1228: int32_t code = TSDB_CODE_SUCCESS; 1229: 1230: SMetaTableOp ops[] = { 1231: {META_ENTRY_TABLE, META_TABLE_OP_INSERT}, // 1232: {META_SCHEMA_TABLE, META_TABLE_OP_UPDATA}, // TODO: here should be insert	
	污染路径: 无	
173	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 23
	步骤描述: 数组 taosCloseHandle 定义的没有显式的边界限定	
	代码片段: 21: transInitClient}; 22: 23: void (*taosCloseHandle[])(void* arg) = {transCloseServer, transCloseClient}; 24: 25: void (*taosRefHandle[])(void* handle) = {transRefSrvHandle, transRefCliHandle};	
	污染路径:	

	无	
174	缺陷发生位置: /source/libs/parser/src/parAstCreator.c	行号: 7531
	步骤描述: 数组 prefix 定义的没有显式的边界限定	
	代码片段: 7529: 7530: static bool validateNotifyUrl(const char* url) { 7531: const char* prefix[] = {"ws://", "wss://"}; 7532: const char* host = NULL; 7533:	
	污染路径: 无	
175	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 63
	步骤描述: 数组 lines 定义的没有显式的边界限定	
	代码片段: 61: 62: // influxdb line protocol 63: char *lines[] = {line_demo}; 64: result = taos_schemaless_insert(taos, lines, 1, TSDB_SML_LINE_PROTOCOL, TSDB_SML_TIMESTAMP_MILLI_SECONDS); 65: code = taos_errno(result);	
	污染路径: 无	
176	缺陷发生位置: /source/libs/metrics/src/metrics.c	行号: 80
	步骤描述: 数组 write_labels 定义的没有显式的边界限定	
	代码片段: 78: 79: // Initialize global shared write metrics counters 80: const char *write_labels[] = {"metric_type", "cluster_id", "dnode_id", "dnode_ep", "vgroup_id", "database_name"}; 81: write_total_requests = taos_collector_registry_must_register_metric(82: taos_counter_new(WRITE_TOTAL_REQUESTS, "Total write requests", 6, write_labels));	

	污染路径: 无	
177	缺陷发生位置: /utils/test/c/tmq_get_meta_json.c	行号: 142
	步骤描述: 数组 sql1 定义的没有显式的边界限定	
	代码片段: 140: 141: // check tag name duplication 142: const char *sql1[] = { 143: "alarm_record,tag=alarm_record1 uid=\"2+100012+303fe9b5\",deviceId=\"2\",alarmId=\"10001 2\" 1732527119493", 144: };	
	污染路径: 无	
178	缺陷发生位置: /source/client/src/clientStmt2.c	行号: 9
	步骤描述: 数组 gStmt2StatusStr 定义的没有显式的边界限定	
	代码片段: 7: #include "clientStmt2.h" 8: 9: char* gStmt2StatusStr[] = {"unknown", "init", "prepare", "settbname", "settags", 10: "fetchFields", "bind", "bindCol", "addBatch", "exec"}; 11:	
	污染路径: 无	
179	缺陷发生位置: /source/util/test/stringTest.cpp	行号: 131
	步骤描述: 数组 a1 定义的没有显式的边界限定	
	代码片段: 129: memset(t, 1, tListLen(t)); 130: 131: char a1[] = "AB.C"; 132: EXPECT_TRUE(strnchr(a1, '.', strlen(a1), true) != NULL); 133:	

	污染路径: 无	
180	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 419
	步骤描述: 数组 activeVglds 定义的没有显式的边界限定	
	代码片段: 417: TEST_F(MetricsTest, CleanupExpiredMetrics_BasicFunctionality) { 418: // Add metrics for vgroups 100, 200, 202, 204 419: int32_t activeVglds[] = {100, 200, 202, 204}; 420: for (int32_t i = 0; i < 4; i++) { 421: SRawWriteMetrics rawMetrics = {0};	
	污染路径: 无	
181	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 830
	步骤描述: 数组 bad_key_err 定义的没有显式的边界限定	
	代码片段: 828: with each option. */ 829: { 830: static const char bad_key_err[] = 831: N_("PROGRAM ERROR) Option should have been recognized!?"); 832: if (group_key == 0)	
	污染路径: 无	
182	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 20
	步骤描述: 数组 taosInitHandle 定义的没有显式的边界限定	
	代码片段: 18: 19: #ifndef TD_ASTRARPC 20: void* (*taosInitHandle[])(SlpAddr* addr, char* label, int32_t numThreads, void* fp, void* plnit) = {transInitServer, 21: transInitClient};	

	22:	
	污染路径: 无	
183	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 2733
	步骤描述: 数组 dec32table 定义的没有显式的边界限定	
	代码片段: 2731: BYTE* const base, BYTE* const oend) 2732: { 2733: static const int dec32table[] = {0, 1, 2, 1, 4, 4, 4, 4}; /* added */ 2734: static const int dec64table[] = {8, 8, 8, 7, 8, 9,10,11}; /* subtracted */ 2735: const BYTE* const ostart = op;	
	污染路径: 无	
184	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 364
	步骤描述: 数组 tsSysCpuFile 定义的没有显式的边界限定	
	代码片段: 362: static pid_t tsProcId; 363: static char tsSysNetFile[] = "/proc/net/dev"; 364: static char tsSysCpuFile[] = "/proc/stat"; 365: static char tsCpuPeriodFile[] = "/sys/fs/cgroup/cpu/cpu.cfs_period_us"; 366: static char tsCpuQuotaFile[] = "/sys/fs/cgroup/cpu/cpu.cfs_quota_us";	
	污染路径: 无	
185	缺陷发生位置: /contrib/lemon/lempar.c	行号: 278
	步骤描述: 数组 yyTokenName 定义的没有显式的边界限定	
	代码片段: 276: /* For tracing shifts, the names of all terminals and nonterminals 277: ** are required. The following table supplies these	

	names */ 278: static const char *const yyTokenName[] = { 279: %% 280: };	
	污染路径: 无	
186	缺陷发生位置: /test/new_test_framework/script/api/stmt-insert-dupkeys.c	行号: 192
	步骤描述: 数组 phase0 定义的没有显式的边界限定	
	代码片段: 190: float current0[] = {10.1f, 10.2f, 10.3f}; 191: int voltage0[] = {216, 217, 218}; 192: float phase0[] = {0.31f, 0.32f, 0.33f}; 193: insertData(taos, ts0, current0, voltage0, phase0); 194:	
	污染路径: 无	
187	缺陷发生位置: /source/client/test/session.cpp	行号: 941
	步骤描述: 数组 users 定义的没有显式的边界限定	
	代码片段: 939: int32_t expect = 1000; 940: 941: char* users[] = {"test1", "test2", "test3", "test4", "test5"}; 942: 943: for (int32_t i = 0; i < sizeof(users) / sizeof(users[0]); i+ +) {	
	污染路径: 无	
188	缺陷发生位置: /source/util/test/talgoTest.cpp	行号: 25
	步骤描述: 数组 arr 定义的没有显式的边界限定	
	代码片段: 23: TEST(utilTest, taosMSort) { 24: // Create an array of test data 25: TestStruct arr[] = {{4, 2.5}, {3, 6}, {2, 1.5}, {3, 2}, {1,	

	3.5}, {3, 5}}; 26: 27: // Sort the array using taosSort	
	污染路径: 无	
189	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 48
	步骤描述: 数组 JSON_COLUMN 定义的没有显式的边界限定	
	代码片段: 46: #define INDEX_DATA_NULL_STR_L "null" 47: 48: char JSON_COLUMN[] = "JSON"; 49: char JSON_VALUE_DELIM = '&'; 50:	
	污染路径: 无	
190	缺陷发生位置: /utlis/test/c/tmq_td37436.c	行号: 26
	步骤描述: 数组 result1 定义的没有显式的边界限定	
	代码片段: 24: #include "types.h" 25: 26: char* result1 = "{\"type\": \"create\", \"tableType\": \"normal\", \"tableName\": \"stream_query\", \"columns\": [{\"name\": \"ts\", \"type\": 9, \"isPrimarykey\": false, \"encode\": \"delta-i\", \"compress\": \"lz4\", \"level\": \"medium\" }, { \"name\": \"id\", \"type\": 4, \"isPrimarykey\": false, \"encode\": \"simple8b\", \"compress\": \"lz4\", \"level\": \"medium\" }], \"tags\": [] }"; 27: char* result2 = "{\"type\": \"create\", \"tableType\": \"normal\", \"tableName\": \"stream_out\", \"columns\": [{\"name\": \"_twstart\", \"type\": 9, \"isPrimarykey\": false, \"encode\": \"delta-i\", \"compress\": \"lz4\", \"level\": \"medium\" }, { \"name\": \"avg(id)\", \"type\": 7, \"isPrimarykey\": false, \"encode\": \"bss\", \"compress\": \"lz4\", \"level\": \"medium\" }], \"tags\": [] }"; 28:	

	污染路径: 无	
191	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 251
	步骤描述: 数组 cliAsyncHandle 定义的没有显式的边界限定	
	代码片段: 249: static void (*cliDealFunc[])(queue* h, SCLiThrd* pThrd) = {cliDoReq, cliDoBatchReq}; 250: 251: static void (*cliAsyncHandle[])(SCLiThrd* pThrd, SCLiReq* pReq) = { 252: cliHandleReq, cliHandleQuit, cliHandleRelease, cliHandleRegister, 253: cliHandleUpdate, cliHandleFreeByld, cliHandleReloadTlsConfig};	
	污染路径: 无	
192	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 3289
	步骤描述: 数组 val_tinyint 定义的没有显式的边界限定	
	代码片段: 3287: 3288: // TINYINT 3289: int8_t val_tinyint[] = {13, 14, 15}; 3290: type = TSDB_DATA_TYPE_TINYINT; 3291: code = scltMakeDataBlock(&pInput, type, 0, rowNum, false);	
	污染路径: 无	
193	缺陷发生位置: /source/util/test/talgoTest.cpp	行号: 74
	步骤描述: 数组 arr 定义的没有显式的边界限定	
	代码片段: 72: start_time = clock(); 73: for (int i = 0; i < 10000; i++) { 74: TestStruct arr[] = {{4, 2.5}, {3, 6}, {2, 1.5}, {3, 2}, {1, 3.5}, {3, 5}};	

	75: taosSort(arr, 6, sizeof(TestStruct), cmpFunc); 76: }	
	污染路径: 无	
194	缺陷发生位置: /source/common/src/tdataformat.c	行号: 4233
	步骤描述: 数组 tColDataGetValueImpl 定义的没有显式的边界限定	
	代码片段: 4231: } 4232: } 4233: static void (*tColDataGetValueImpl[])(SColData *pColData, int32_t iVal, SColVal *pColVal) = { 4234: NULL, // 0 4235: tColDataGetValue1, // HAS_NONE	
	污染路径: 无	
195	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 229
	步骤描述: 数组 blob2_str 定义的没有显式的边界限定	
	代码片段: 227: char* blob2_buffer = v.blob2; 228: char* blob_buffer = v.blob; 229: const char* blob2_str[] = { 230: "一二三四五六七十九八", "一二三四五六七十九", "一二三四五六七十", "一二三四五六七", "一二三四五六", 231: "一二三四五", "一二三四", "一二三", "一二", "一",	
	污染路径: 无	
196	缺陷发生位置: /source/common/test/dataformatTest.cpp	行号: 513
	步骤描述: 数组 stbName 定义的没有显式的边界限定	
	代码片段: 511: 512: TEST(testCase, StreamWithoutDotInStbName) { 513: char stbName[] = "table"; 514: char ctbName[TSDB_TABLE_NAME_LEN] = {0};	

	515: uint64_t groupId = 12345;	
	污染路径: 无	
197	缺陷发生位置: /source/common/src/tcol.c	行号: 21
	步骤描述: 数组 supportedEncode 定义的没有显式的边界限定	
	代码片段: 19: #include "tutil.h" 20: 21: const char* supportedEncode[] = { 22: TSDB_COLUMN_ENCODE_SIMPLE8B, TSDB_COLUMN_ENCODE_XOR, TSDB_COLUMN_ENCODE_RLE, 23: TSDB_COLUMN_ENCODE_DELTAD, TSDB_COLUMN_ENCODE_DISABLED, TSDB_COLUMN_ENCODE_BYTE_STREAM_SPLIT,	
	污染路径: 无	
198	缺陷发生位置: /test/new_test_framework/script/api/stmt-insert-dupkeys.c	行号: 189
	步骤描述: 数组 ts0 定义的没有显式的边界限定	
	代码片段: 187: 188: // initial insert, expect insert 3 rows 189: int64_t ts0[] = {1648432611234, 1648432611345, 1648432611456}; 190: float current0[] = {10.1f, 10.2f, 10.3f}; 191: int voltage0[] = {216, 217, 218};	
	污染路径: 无	
199	缺陷发生位置: /source/util/src/tcompression.c	行号: 360
	步骤描述: 数组 bit_to_selector 定义的没有显式的边界限定	
	代码片段: 358: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 359: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15,	

	12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 360: char bit_to_selector[] = {0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 11, 12, 12, 12, 13, 13, 13, 13, 13, 361: 14, 14, 14, 14, 14, 14, 14, 14, 14, 14, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 362: 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15};	
	污染路径: 无	
200	缺陷发生位置: /source/dnode/mnode/impl/inc/mndStream.h	行号: 58
	步骤描述: 数组 gMndStreamState 定义的没有显式的边界限定	
	代码片段: 56: #define MND_STM_STATE_NORMAL 2 57: 58: static const char* gMndStreamState[] = {"X", "W", "N"}; 59: 60:	
	污染路径: 无	
201	缺陷发生位置: /utls/test/c/tmq_get_meta_json.c	行号: 129
	步骤描述: 数组 sql 定义的没有显式的边界限定	
	代码片段: 127: 128: // check column name duplication 129: const char *sql[] = { 130: "alarm_record,tag=alarm_record uid=\"3+8001+c939604c\",deviceId=\"3\" 1732527117484", 131: };	
	污染路径: 无	
202	缺陷发生位置: /source/client/test/session.cpp	行号: 628
	步骤描述: 数组 params2 定义的没有显式的边界限定	
	代码片段: 626: };	

	627: 628: struct insert_params params2[] = { 629: {"UTC+9", "ntb", "2023-09-16 08:00:00", "2023-09-16 08:00:00-01:00"}, 630: {"UTC+9", "ctb1", "2023-09-16 08:00:00", "2023-09-16 11:00:00+02:00"},	
	污染路径: 无	
203	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1753
	步骤描述: 数组 dec32table 定义的没有显式的边界限定	
	代码片段: 1751: BYTE* const base, BYTE* const oend) 1752: { 1753: static const int dec32table[] = {0, 1, 2, 1, 4, 4, 4, 4}; /* added */ 1754: static const int dec64table[] = {8, 8, 8, 7, 8, 9,10,11}; /* subtracted */ 1755: const BYTE* const ostart = op;	
	污染路径: 无	
204	缺陷发生位置: /source/util/test/cacheTest.cpp	行号: 44
	步骤描述: 数组 key5 定义的没有显式的边界限定	
	代码片段: 42: TD_ALWAYS_ASSERT(d == NULL); 43: 44: char key5[] = "test5"; 45: char data5[] = "data5kkkkk"; 46: cachedObj2 = (char*)taosCachePut(tscMetaCache, key5, strlen(key5), data5, strlen(data5) + 1, 20);	
	污染路径: 无	
205	缺陷发生位置: /source/common/src/ttime.c	行号: 1487
	步骤描述: 数组 shortWeekDays 定义的没有显式的边界限定	

	代码片段: 1485: static const char* const weekdays[] = {"Sunday", "Monday", "Tuesday", "Wednesday", 1486: "Thursday", "Friday", "Saturday", "NULL"}; 1487: static const char* const shortWeekdays[] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "NULL"}; 1488: static const char* const fullMonths[] = {"January", "February", "March", "April", "May", "June", "July", 1489: "August", "September", "October", "November", "December", NULL};	
	污染路径: 无	
206	缺陷发生位置: /source/os/src/osLocale.c	行号: 61
	步骤描述: 数组 charsetRep 定义的没有显式的边界限定	
	代码片段: 59: return NULL; 60: } 61: CharsetPair charsetRep[] = { 62: {"utf8", "UTF-8"}, 63: {"936", "CP936"},	
	污染路径: 无	
207	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 73
	步骤描述: 数组 sub_options 定义的没有显式的边界限定	
	代码片段: 71: const char *argp_program_version = "argp-test 1.0"; 72: ? 73: struct argp_option sub_options[] = 74: { 75: {"subopt1", 's', 0, 0, "Nested option 1"},	
	污染路径: 无	
208	缺陷发生位置: /contrib/lemon/lempar.c	行号: 286

	步骤描述: 数组 yyRuleName 定义的没有显式的边界限定	
	代码片段: 284: /* For tracing reduce actions, the names of all rules are required. 285: */ 286: static const char *const yyRuleName[] = { 287: %% 288: };	
	污染路径: 无	
209	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 556
	步骤描述: 数组 f 定义的没有显式的边界限定	
	代码片段: 554: } 555: 556: auto f = [](TTB *pDb, int nData) { 557: TBC *pDBC; 558: void *pKey = NULL;	
	污染路径: 无	
210	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 46
	步骤描述: 数组 opt5 定义的没有显式的边界限定	
	代码片段: 44: }; 45: 46: static const struct argp_option opt5[] = 47: { 48: { "opt5", '5', "NUMBER", 0, "Option 5" },	
	污染路径: 无	
211	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 40
	步骤描述: 数组 operInfo 定义的没有显式的边界限定	

	代码片段: 38: #define BP_BIND_TYPE_STR(t) (((t) == BP_BIND_COL) ? "column" : "tag") 39: 40: OperInfo operInfo[] = { 41: {">", 2, false}, 42: {">=", 2, false},	
	污染路径: 无	
212	缺陷发生位置: /contrib/test/azure/main.cpp	行号: 70
	步骤描述: 数组 blobContent 定义的没有显式的边界限定	
	代码片段: 68: 69: std::string blobName = "blob.txt"; 70: uint8_t blobContent[] = "Hello Azure!"; 71: // Create the block blob client 72: BlockBlobClient blobClient = containerClient.GetBlockBlobClient(blobName);	
	污染路径: 无	
213	缺陷发生位置: /source/libs/tcs/test/tcsTest.cpp	行号: 140
	步骤描述: 数组 object_name 定义的没有显式的边界限定	
	代码片段: 138: } 139: 140: const char object_name[] = "tcsut.bin"; 141: char path[PATH_MAX] = {0}; 142: char path_download[PATH_MAX] = {0};	
	污染路径: 无	
214	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 23
	步骤描述: 数组 bindColTypeList 定义的没有显式的边界限定	
	代码片段: 21: TSDB_DATA_TYPE_UBIGINT,	

	TSDB_DATA_TYPE_FLOAT, TSDB_DATA_TYPE_DOUBLE, 22: TSDB_DATA_TYPE_BINARY, TSDB_DATA_TYPE_NCHAR}; 23: int32_t bindColTypeList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_NCHAR}; 24: int32_t optrIdxList[] = {5, 11}; 25:	
	污染路径: 无	
215	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 864
	步骤描述: 数组 test_tz 定义的没有显式的边界限定	
	代码片段: 862: time_t time_summer = 1731323281 - 120 * 24 * 60 * 60; 863: 864: struct test_times 865: { 866: const char *name;	
	污染路径: 无	
216	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 136
	步骤描述: 数组 arr2 定义的没有显式的边界限定	
	代码片段: 134: } 135: 136: uint64_t arr2[] = {1, 2, 3, 5}; 137: f = (SArray *)taosArrayGetP(src, 1); 138: for (int i = 0; i < sizeof(arr2) / sizeof(arr2[0]); i++) {	
	污染路径: 无	
217	缺陷发生位置: /source/util/test/regexTest.cpp	行号: 140
	步骤描述: 数组 s1 定义的没有显式的边界限定	
	代码片段: 138: TEST(testCase, regexCacheTest3) { 139: int times = 100000;	

	140: char s1[] = "abc%*"; 141: char s2[] = "abc"; 142: auto start = std::chrono::high_resolution_clock::now();	
	污染路径: 无	
218	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 44
	步骤描述: 数组 operInfo 定义的没有显式的边界限定	
	代码片段: 42: #define BP_BIND_TYPE_STR(t) (((t) == BP_BIND_COL) ? "column" : "tag") 43: 44: OperInfo operInfo[] = { 45: {">", 2, false}, {">=", 2, false}, {"<", 2, false}, {"<=", 2, false}, 46: {"=", 2, false}, {"<>", 2, false}, {"in", 2, true}, {"not in", 2, true},	
	污染路径: 无	
219	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 259
	步骤描述: 数组 qptPlans 定义的没有显式的边界限定	
	代码片段: 257: 258: 259: SQPTPlan qptPlans[] = { 260: {QUERY_NODE_PHYSICAL_PLAN_TAG_SCAN, QPT_PLAN_PHYSIC, "tagScan", qptCreateTagScanPhysiNode}, 261: {QUERY_NODE_PHYSICAL_PLAN_TABLE_SCAN, QPT_PLAN_PHYSIC, "tableScan", qptCreateTableScanPhysiNode},	
	污染路径: 无	
220	缺陷发生位置: /source/os/src/osSocket.c	行号: 769
	步骤描述: 数组 ret 定义的没有显式的边界限定	
	代码片段:	

	767: return 0; 768: #else 769: int32_t ret = setsockopt(fd, IPPROTO_TCP, TCP_QUICKACK, (int[]){1}, sizeof(int)); 770: if (ret < 0) { 771: terrno = TAOS_SYSTEM_ERROR(ERRNO);	
	污染路径: 无	
221	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 61
	步骤描述: 数组 g_avro_codec 定义的没有显式的边界限定	
	代码片段: 59: static int dumpExtraInfo(void **taos_v, FILE *fp); 60: 61: char *g_avro_codec[] = { 62: "null", 63: "deflate",	
	污染路径: 无	
222	缺陷发生位置: /utls/test/c/tmq_taosx_ci.c	行号: 862
	步骤描述: 数组 result 定义的没有显式的边界限定	
	代码片段: 860: if (g_conf.subTable) { 861: } else { 862: char* result[] = { 863: "{"type\":"create\","tableType\":"normal\","tableName\":"tb 1\","columns\":[{"name\":"ts\"," 864: "type\":"	
	污染路径: 无	
223	缺陷发生位置: /source/util/test/stringTest.cpp	行号: 16
	步骤描述: 数组 t2 定义的没有显式的边界限定	
	代码片段: 14: EXPECT_STRCASEEQ(t1, "abc");	

	15: 16: char t2[] = "\"abc\""; 17: len = strdequote(t2); 18:	
	污染路径: 无	
224	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 45
	步骤描述: 数组 offlineReason 定义的没有显式的边界限定	
	代码片段: 43: #define TSDB_DNODE_RESERVE_SIZE 40 44: 45: static const char *offlineReason[] = { 46: "", 47: "status msg timeout",	
	污染路径: 无	
225	缺陷发生位置: /source/common/test/cosTest.cpp	行号: 113
	步骤描述: 数组 object_name 定义的没有显式的边界限定	
	代码片段: 111: const long objectSize = 65 * 1024 * 1024; 112: char const *objectName = "cosut.bin"; 113: const char object_name[] = "cosut.bin"; 114: 115: EXPECT_EQ(std::string(object_name), objectName);	
	污染路径: 无	
226	缺陷发生位置: /docs/examples/c/json_protocol_example.c	行号: 38
	步骤描述: 数组 lines 定义的没有显式的边界限定	
	代码片段: 36: "\"California.LosAngeles\", \"groupid\": 1}}]"; 37: 38: char *lines[] = {line}; 39: TAOS_RES *res = taos_schemaless_insert(taos, lines, 1, TSDB_SML_JSON_PROTOCOL,	

	TSDB_SML_TIMESTAMP_NOT_CONFIGURED); 40: if (taos_errno(res) != 0) {	
	污染路径: 无	
227	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 94
	步骤描述: 数组 jtColCondStr 定义的没有显式的边界限定	
	代码片段: <pre> 92: int32_t jtInputColType[MAX_SLOT_NUM] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_INT, TSDB_DATA_TYPE_INT, TSDB_DATA_TYPE_BIGINT}; 93: 94: char* jtColCondStr[] = {"", "NO COND", "EQ COND", "ON COND", "FULL COND"}; 95: char* jtJoinTypeStr[] = {"INNER", "LEFT", "RIGHT", "FULL"}; 96: char* jtSubTypeStr[] = {"", "OUTER", "SEMI", "ANTI", "ASOF", "WINDOW"}; </pre>	
	污染路径: 无	
228	缺陷发生位置: /tools/taos-tools/src/benchSys.c	行号: 139
	步骤描述: 数组 bench_options 定义的没有显式的边界限定	
	代码片段: <pre> 137: const char * argp_program_bug_address = CUS_EMAIL; 138: 139: static struct argp_option bench_options[] = { 140: {"file", 'f', "FILE", 0, BENCH_FILE, 0}, 141: {"config-dir", 'c', "CONFIG_DIR", 0, BENCH_CFG_DIR, 1}, </pre>	
	污染路径: 无	
229	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/bug-argp1.c	行号: 8
	步骤描述: 数组 test_options 定义的没有显式的边界限定	

	代码片段: 6: 7: 8: static const struct argp_option test_options[] = 9: { 10: { NULL, 'a', NULL, OPTION_DOC, NULL },	
	污染路径: 无	
230	缺陷发生位置: /contrib/test/cos/main.c	行号: 23
	步骤描述: 数组 TEST_APPID 定义的没有显式的边界限定	
	代码片段: 21: // static char TEST_APPID[] = "<APPID>"; // your appid 22: // static char TEST_APPID[] = "119"; // your appid 23: static char TEST_APPID[] = "1309024725"; // your appid 24: // the cos bucket name, syntax: [bucket]-[appid], for example: mybucket-1253666666, 可在 25: // https://console.cloud.tencent.com/cos5/bucket 查看 static char TEST_BUCKET_NAME[] = "<bucketname-appid>";	
	污染路径: 无	
231	缺陷发生位置: /docs/examples/c/sml_insert_demo.c	行号: 89
	步骤描述: 数组 telnets 定义的没有显式的边界限定	
	代码片段: 87: 88: // opentsdb telnet protocol 89: char *telnets[] = {telnet_demo}; 90: result = taos_schemaless_insert(taos, telnets, 1, TSDB_SML_TELNET_PROTOCOL, TSDB_SML_TIMESTAMP_MILLI_SECONDS); 91: code = taos_errno(result);	
	污染路径: 无	
232	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 62
	步骤描述: 数组 shellCommands 定义的没有显式的边界限定	

	代码片段: 60: } SWords; 61: 62: SWords shellCommands[] = { 63: {"alter database <db_name> <alter_db_options> <anyword> <alter_db_options> <anyword> <alter_db_options> <anyword> " 64: "<alter_db_options> <anyword> <alter_db_options> <anyword> ;",	
	污染路径: 无	
233	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 34
	步骤描述: 数组 opt3 定义的没有显式的边界限定	
	代码片段: 32: }; 33: 34: static const struct argp_option opt3[] = 35: { 36: { "opt3", '3', "NUMBER", 0, "Option 3" },	
	污染路径: 无	
234	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 363
	步骤描述: 数组 tsSysNetFile 定义的没有显式的边界限定	
	代码片段: 361: 362: static pid_t tsProclId; 363: static char tsSysNetFile[] = "/proc/net/dev"; 364: static char tsSysCpuFile[] = "/proc/stat"; 365: static char tsCpuPeriodFile[] = "/sys/fs/cgroup/cpu/cpu.cfs_period_us";	
	污染路径: 无	
235	缺陷发生位置: /source/libs/metrics/src/metrics.c	行号: 121
	步骤描述:	

	数组 dnode_labels 定义的没有显式的边界限定	
	代码片段: 119: 120: // Initialize global dnode counters 121: const char *dnode_labels[] = {"metric_type", "cluster_id", "dnode_id", "dnode_ep"}; 122: dnode_rpc_queue_memory_allowed = taos_collector_registry_must_register_metric(123: taos_gauge_new(DNODE_RPC_QUEUE_MEMORY_ALLOWED, "RPC queue memory allowed", 4, dnode_labels));	
	污染路径: 无	
236	缺陷发生位置:	行号:
	/source/util/test/stringTest.cpp	10
	步骤描述: 数组 t1 定义的没有显式的边界限定	
	代码片段: 8: 9: TEST(testCase, string_dequote_test) { 10: char t1[] = "abc"; 11: int32_t len = strdequote(t1); 12:	
237	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/monitor/src/monFramework.c	110
	步骤描述: 数组 dnodes_sample_labels 定义的没有显式的边界限定	
	代码片段: 108: 109: int32_t dnodes_label_count = 3; 110: const char *dnodes_sample_labels[] = {"cluster_id", "dnode_id", "dnode_ep"}; 111: char *dnodes_gauges[] = {UPTIME, CPU_ENGINE, CPU_SYSTEM, CPU_CORE, MEM_ENGINE, MEM_FREE, 112: MEM_TOTAL, MEM_CACHE_BUFFER, DISK_ENGINE, DISK_USED, DISK_TOTAL, NET_IN,	
	污染路径: 无	

238	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 879
	步骤描述: 数组 pScopeStr 定义的没有显式的边界限定	
	代码片段: 877: 878: TEST(TaosSetSlowLogScopeTest, QueryScopeInput) { 879: char pScopeStr[] = "query"; 880: int32_t scope = 0; 881: int32_t result = taosSetSlowLogScope2(pScopeStr, &scope);	
	污染路径: 无	
239	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 18
	步骤描述: 数组 fullColList 定义的没有显式的边界限定	
	代码片段: 16: 17: int32_t shortColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_INT}; 18: int32_t fullColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_BOOL, TSDB_DATA_TYPE_TINYINT, 19: TSDB_DATA_TYPE_UTINYINT, TSDB_DATA_TYPE_SMALLINT, TSDB_DATA_TYPE_USMALLINT, 20: TSDB_DATA_TYPE_INT, TSDB_DATA_TYPE_UINT, TSDB_DATA_TYPE_BIGINT,	
	污染路径: 无	
240	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 3186
	步骤描述: 数组 result 定义的没有显式的边界限定	
	代码片段: 3184: int32_t type; 3185: int32_t otype = TSDB_DATA_TYPE_DOUBLE; 3186: double result[] = {0.42016703682664092, 0.99060735569487035, 0.65028784015711683}; 3187: 3188: // TINYINT	

	污染路径: 无	
241	缺陷发生位置: /tools/shell/test/shellTest.cpp	行号: 102
	步骤描述: 数组 s1 定义的没有显式的边界限定	
	代码片段: 100: 101: // str true 102: const char *s1[] = { 103: "create table st(ts timestamp ", 104: "create table st(ts timestamp, age int ",	
	污染路径: 无	
242	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 698
	步骤描述: 数组 units 定义的没有显式的边界限定	
	代码片段: 696: 697: int8_t qptGetRandTimestampUnit() { 698: static int8_t units[] = {TIME_UNIT_NANOSECOND, TIME_UNIT_MICROSECOND, TIME_UNIT_MILLISECOND, TIME_UNIT_SECOND, 699: TIME_UNIT_MINUTE, TIME_UNIT_HOUR, TIME_UNIT_DAY, TIME_UNIT_WEEK, TIME_UNIT_MONTH, TIME_UNIT_YEAR}; 700:	
	污染路径: 无	
243	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 559
	步骤描述: 数组 sql 定义的没有显式的边界限定	
	代码片段: 557: ASSERT_NE(info, nullptr); 558: 559: const char *sql[] = { 560: "twudyr 1626006833641 \"abcd`~!@#\$\$%^&*()_- {[]];;<.>?lfja\\\" id=twudyr_17102_17825 t0=t t1=127i8 t2=32767i16 t3=2147483647i32	

	194: } 195: 196: const char *object_name_arr[] = {object_name}; 197: code = azDeleteObjects(object_name_arr, 1); 198: GTEST_ASSERT_EQ(code, 0);	
	污染路径: 无	
247	缺陷发生位置: /source/libs/index/src/indexFstCommon.c	行号: 276
	步骤描述: 数组 COMMON_INPUTS_INV 定义的没有显式的边界限定	
	代码片段: 274: }; 275: 276: const char COMMON_INPUTS_INV[] = { 277: 't', 'e', '/', 'o', 'a', 's', 'r', 'i', 'p', 'c', 'n', 'w', '.', 'h', 278: 'l', 'm', '-', 'd', 'u', '0', '1', '2', 'g', '=', ':', 'b', 'f', '3',	
	污染路径: 无	
248	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 31
	步骤描述: 数组 locations 定义的没有显式的边界限定	
	代码片段: 29: "abcdefghijklmnopqrstuvwxyABCDEFGHIJKLMNOPQRSTUVWXYZ YZ1234567890"; 30: 31: const char* locations[] = { 32: "California.SanFrancisco", "California.LosAngles", 33: "California.SanDiego", "California.SanJose",	
	污染路径: 无	
249	缺陷发生位置: /utls/test/c/tmq_taosx_ci.c	行号: 1012
	步骤描述: 数组 result 定义的没有显式的边界限定	
	代码片段:	

	<pre> 1010: } 1011: } else { 1012: char* result[] = { 1013: {"type\":"create\","tableType\":"normal\","tableName\":"tb 1\","columns\":[{"name\":"ts\"," 1014: {"type\:9,"isPrimarykey\:false,"encode\":"delta- i\","compress\":"lz4\","level\":"medium\","}{" </pre>	
	污染路径: 无	
250	缺陷发生位置: /source/common/src/ttime.c	行号: 1488
	步骤描述: 数组 fullMonths 定义的没有显式的边界限定	
	代码片段: <pre> 1486: "Thursday", "Friday", "Saturday", "NULL"}; 1487: static const char* const shortWeekDays[] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "NULL"}; 1488: static const char* const fullMonths[] = {"January", "February", "March", "April", "May", "June", "July", 1489: "August", "September", "October", "November", "December", NULL}; 1490: static const char* const months[] = {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", </pre>	
	污染路径: 无	
251	缺陷发生位置: /source/libs/azure/test/azTest.cpp	行号: 235
	步骤描述: 数组 object_name 定义的没有显式的边界限定	
	代码片段: <pre> 233: } 234: 235: const char object_name[] = "azut.bin"; 236: char path[PATH_MAX] = {0}; 237: char path_download[PATH_MAX] = {0}; </pre>	
	污染路径: 无	

252	缺陷发生位置: /utils/test/c/sml_test.c	行号: 129
	步骤描述: 数组 sql2 定义的没有显式的边界限定	
	代码片段: 127: taos_free_result(pRes); 128: 129: const char *sql2[] = {"sysif.bytes.out 1479496700 1.6E0 host=web01 interface=eth0"}; 130: pRes = taos_schemaless_insert(taos, (char **)sql2, sizeof(sql2) / sizeof(sql2[0]), TSDB_SML_TELNET_PROTOCOL, 131: TSDB_SML_TIMESTAMP_NANO_SECONDS);	
	污染路径: 无	
253	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 2435
	步骤描述: 数组 ratio 定义的没有显式的边界限定	
	代码片段: 2433: pInfo->pHisto->numOfElems, pInfo->pHisto-> numOfEntries, pInfo->pHisto, pInfo->pHisto->elems); 2434: 2435: double ratio[] = {pInfo->percent}; 2436: double* res = NULL; 2437: int32_t code = tHistogramUniform(pInfo->pHisto, ratio, 1, &res);	
	污染路径: 无	
254	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 921
	步骤描述: 数组 g_aggreFunc 定义的没有显式的边界限定	
	代码片段: 919: /* ***** Global variables ***** */ 920: extern char * g_aggreFuncDemo[]; 921: extern char * g_aggreFunc[]; 922: extern SArguments * g_arguments; 923: extern SQueryMetaInfo g_queryInfo;	
	污染路径:	

	无	
255	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 2067
	步骤描述: 数组 crc32_table 定义的没有显式的边界限定	
	代码片段: 2065: 2066: /* pre-calculated table for 32-bit CRC */ 2067: static uint32_t crc32_table[] = { 2068: 0x00000000, 0x77073096, 0xEE0E612C, 0x990951BA, 0x076DC419, 0x706AF48F, 0xE963A535, 0x9E6495A3, 0x0EDB8832, 2069: 0x79DCB8A4, 0xE0D5E91E, 0x97D2D988, 0x09B64C2B, 0x7EB17CBD, 0xE7B82D07, 0x90BF1D91, 0x1DB71064, 0x6AB020F2,	
	污染路径: 无	
256	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 29
	步骤描述: 数组 influxHttpOk 定义的没有显式的边界限定	
	代码片段: 27: char resHttp[] = "HTTP/1.1 "; 28: char resHttpOk[] = "HTTP/1.1 200 OK"; 29: char influxHttpOk[] = "HTTP/1.1 204"; 30: char opentsdbHttpOk[] = "HTTP/1.1 400"; 31:	
	污染路径: 无	
257	缺陷发生位置: /source/util/src/tcompression.c	行号: 490
	步骤描述: 数组 selector_to_elems 定义的没有显式的边界限定	
	代码片段: 488: // Selector value: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 489: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 490: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 491:	

	492: const char *ip = input + 1;	
	污染路径: 无	
258	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1327
	步骤描述: 数组 ops 定义的没有显式的边界限定	
	代码片段: 1325: int32_t code = TSDB_CODE_SUCCESS; 1326: 1327: SMetaTableOp ops[] = { 1328: {META_ENTRY_TABLE, META_TABLE_OP_INSERT}, // 1329: {META_UID_IDX, META_TABLE_OP_INSERT}, //	
	污染路径: 无	
259	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 519
	步骤描述: 数组 sql 定义的没有显式的边界限定	
	代码片段: 517: ASSERT_NE(info, nullptr); 518: 519: const char *sql[] = { 520: "sys.procs.running 14794961040 42 host=web01", 521: "sys.procs.running 14791040 42 host=web01",	
	污染路径: 无	
260	缺陷发生位置: /source/libs/monitor/src/monFramework.c	行号: 131
	步骤描述: 数组 dnodes_data_gauges 定义的没有显式的边界限定	
	代码片段: 129: int32_t dnodes_data_label_count = 5; 130: const char *dnodes_data_sample_labels[] = { "cluster_id", "dnode_id", "dnode_ep", "data_dir_name", "data_dir_level"}; 131: char *dnodes_data_gauges[] = {DNODE_DATA_AVAIL, DNODE_DATA_USED, DNODE_DATA_TOTAL}; 132: for(int32_t i = 0; i < 3; i++){ 133: gauge= taos_gauge_new(dnodes_data_gauges[i], "",	

	dnodes_data_label_count, dnodes_data_sample_labels);	
	污染路径: 无	
261	缺陷发生位置: /source/dnode/mgmt/exe/dmMain.c	行号: 1057
	步骤描述: 数组 indent 定义的没有显式的边界限定	
	代码片段: 1055: 1056: static void dmPrintHelp() { 1057: char indent[] = " "; 1058: printf("Usage: %sd [OPTION...] \n\n", CUS_PROMPT); 1059: printf("%s%s%s%s\n", indent, "-a,", indent, DM_APOLLO_URL);	
	污染路径: 无	
262	缺陷发生位置: /source/common/src/tglobal.c	行号: 3320
	步骤描述: 数组 debugOptions 定义的没有显式的边界限定	
	代码片段: 3318: 3319: { // 'bool/int32_t/int64_t/float/double' variables with general modification function 3320: static OptionNameAndVar debugOptions[] = { 3321: {"cDebugFlag", &cDebugFlag}, {"dDebugFlag", &dDebugFlag}, {"fsDebugFlag", &fsDebugFlag}, 3322: {"idxDebugFlag", &idxDebugFlag}, {"jniDebugFlag", &jniDebugFlag}, {"qDebugFlag", &qDebugFlag},	
	污染路径: 无	
263	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 920
	步骤描述: 数组 test_mk 定义的没有显式的边界限定	
	代码片段: 918: } 919: 920: struct {	

	921: const char * env; 922: time_t expected;	
	污染路径: 无	
264	缺陷发生位置: /tools/shell/test/shellTest.cpp	行号: 47
	步骤描述: 数组 s1 定义的没有显式的边界限定	
	代码片段: 45: 46: // str true 47: const char *s1[] = { 48: "create table st(ts timestamp ", 49: "create table st(ts timestamp, age int ",	
	污染路径: 无	
265	缺陷发生位置: /source/util/test/cacheTest.cpp	行号: 45
	步骤描述: 数组 data5 定义的没有显式的边界限定	
	代码片段: 43: 44: char key5[] = "test5"; 45: char data5[] = "data5kkkkk"; 46: cachedObj2 = (char*)taosCachePut(tscMetaCache, key5, strlen(key5), data5, strlen(data5) + 1, 20); 47:	
	污染路径: 无	
266	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 314
	步骤描述: 数组 long_options 定义的没有显式的边界限定	
	代码片段: 312: int option_index, option_value; 313: option_index = 0; 314: static struct option long_options[] = { 315: {"help", no_argument, NULL, 'h'}, 316: {"voter", no_argument, NULL, 'v'},	

	污染路径: 无	
267	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 3286
	步骤描述: 数组 result 定义的没有显式的边界限定	
	代码片段: 3284: int32_t type; 3285: int32_t otype = TSDB_DATA_TYPE_DOUBLE; 3286: double result[] = {0.90744678145019619, 0.13673721820783361, -0.75968791285882131}; 3287: 3288: // TINYINT	
	污染路径: 无	
268	缺陷发生位置: /contrib/test/cos/main.c	行号: 30
	步骤描述: 数组 TEST_UIN 定义的没有显式的边界限定	
	代码片段: 28: static char TEST_BUCKET_NAME[] = "test0711- 1309024725"; 29: // 对象拥有者, 比如用户 UIN: 100000000001 30: static char TEST_UIN[] = "<Uin>"; // your uin 31: // 地域信息, 枚举值可参见 https://cloud.tencent.com/document/product/436/6224 32: // 文档, 例如: ap-beijing、ap-hongkong、eu-frankfurt 等	
	污染路径: 无	
269	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 3631
	步骤描述: 数组 dec32table 定义的没有显式的边界限定	
	代码片段: 3629: if (sequence.offset < 8) { 3630: /* close range match, overlap */ 3631: static const U32 dec32table[] = { 0, 1, 2, 1, 4, 4, 4, 4 }; /* added */ 3632: static const int dec64table[] = { 8, 8, 8, 7, 8, 9,10,11 }; /* subtracted */ 3633: int const sub2 = dec64table[sequence.offset];	

	污染路径: 无	
270	缺陷发生位置: /source/client/src/clientSml.c	行号: 112
	步骤描述: 数组 smlFactorS 定义的没有显式的边界限定	
	代码片段: 110: int64_t smlToMilli[] = {3600000LL, 60000LL, 1000LL}; 111: int64_t smlFactorNS[] = {NANOSECOND_PER_MSEC, NANOSECOND_PER_USEC, 1}; 112: int64_t smlFactorS[] = {1000LL, 1000000LL, 1000000000LL}; 113: 114: static int32_t smlCheckAuth(SSmlHandle *info, SRequestConnInfo *conn, const char *pTabName, EPrivType type, EPrivObjType objType) {	
	污染路径: 无	
271	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 582
	步骤描述: 数组 tz_test 定义的没有显式的边界限定	
	代码片段: 580: } 581: 582: char *tz_test[] = { 583: "2023-09-16 17:00:00+", 584: "2023-09-16 17:00:00+a",	
	污染路径: 无	
272	缺陷发生位置: /utils/test/c/tmq_td37436.c	行号: 27
	步骤描述: 数组 result2 定义的没有显式的边界限定	
	代码片段: 25: 26: char* result1 = {"type\":"create\","tableType\":"normal\","tableName\":"st ream_query\","columns\ [{"name\":"ts\","type\":"9\","isPrimarykey\":"false\","encode\":"	

	<pre>delta-i\","compress\":"lz4\","level\":"medium\"}, {\\\"name\\\":\\\"id\\\",\\\"type\\\":4,\\\"isPrimarykey\\\":false,\\\"encode\\\":\\\"simple8b\\\",\\\"compress\\\":\\\"lz4\\\",\\\"level\\\":\\\"medium\\\"}],\\\"tags\\\": []}\"; 27: char* result2 = {\\\"type\\\":\\\"create\\\",\\\"tableType\\\":\\\"normal\\\",\\\"tableName\\\":\\\"stream_out\\\",\\\"columns\\\": [\\\"name\\\":\\\"_twstart\\\",\\\"type\\\":9,\\\"isPrimarykey\\\":false,\\\"encode\\\":\\\"delta-i\\\",\\\"compress\\\":\\\"lz4\\\",\\\"level\\\":\\\"medium\\\"}, {\\\"name\\\":\\\"avg(id)\\\",\\\"type\\\":7,\\\"isPrimarykey\\\":false,\\\"encode\\\":\\\"bss\\\",\\\"compress\\\":\\\"lz4\\\",\\\"level\\\":\\\"medium\\\"}],\\\"tags\\\": []}\"; 28: 29: static void msg_process(TAOS_RES* msg) {</pre>	
	污染路径: 无	
273	缺陷发生位置: /source/libs/monitor/src/monFramework.c	行号: 130
	步骤描述: 数组 dnodes_data_sample_labels 定义的没有显式的边界限定	
	代码片段: 128: 129: int32_t dnodes_data_label_count = 5; 130: const char *dnodes_data_sample_labels[] = {"cluster_id", "dnode_id", "dnode_ep", "data_dir_name", "data_dir_level"}; 131: char *dnodes_data_gauges[] = {DNODE_DATA_AVAIL, DNODE_DATA_USED, DNODE_DATA_TOTAL}; 132: for(int32_t i = 0; i < 3; i++){	
	污染路径: 无	
274	缺陷发生位置: /test/new_test_framework/script/api/stmt-insert-dupkeys.c	行号: 196
	步骤描述: 数组 ts1 定义的没有显式的边界限定	
	代码片段: 194: 195: // insert with interlace mode, send non-duplicate ts, expect insert 3 overlapped rows 196: int64_t ts1[] = {1648432611234, 1648432611345, 1648432611456};	

	197: int voltage1[] = {219, 220, 221}; 198: insertDataInterlace(taos, ts1, current0, voltage1, phase0);	
	污染路径: 无	
275	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 61
	步骤描述: 数组 joinFullType 定义的没有显式的边界限定	
	代码片段: 59: 60: char* getFullJoinTypeString(EJoinType type, EJoinSubType stype) { 61: static char* joinFullType[][8] = { 62: {"INNER", "INNER", "INNER", "INNER", "INNER", "INNER", "INNER", "ANY", "INNER", "INNER"}, 63: {"LEFT", "LEFT", "LEFT OUTER", "LEFT SEMI", "LEFT ANTI", "LEFT ANY", "LEFT ASOF", "LEFT WINDOW"},	
	污染路径: 无	
276	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 574
	步骤描述: 数组 ERR_strings 定义的没有显式的边界限定	
	代码片段: 572: #define ERROR_CONVERTTOSTRING(String) #String, 573: #define ERROR_GENERATE_STRING(EXPR) ERROR_CONVERTTOSTRING(EXPR) 574: static const char* ERR_strings[] = { ERROR_LIST(ERROR_GENERATE_STRING) }; 575: 576: ERR_STATIC unsigned ERR_isError(size_t code) { return (code > ERROR(maxCode)); }	
	污染路径: 无	
277	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 27
	步骤描述: 数组 resHttp 定义的没有显式的边界限定	
	代码片段:	

	25: char resEncodingChunk[] = "Encoding: chunked"; 26: char succMessage[] = "succ"; 27: char resHttp[] = "HTTP/1.1 "; 28: char resHttpOk[] = "HTTP/1.1 200 OK"; 29: char influxHttpOk[] = "HTTP/1.1 204";	
	污染路径: 无	
278	缺陷发生位置: /source/libs/azure/test/azTest.cpp	行号: 108
	步骤描述: 数组 object_name 定义的没有显式的边界限定	
	代码片段: 106: } 107: 108: const char object_name[] = "azut.bin"; 109: char path[PATH_MAX] = {0}; 110: char path_download[PATH_MAX] = {0};	
	污染路径: 无	
279	缺陷发生位置: /source/client/test/stmt2Test.cpp	行号: 39
	步骤描述: 数组 STMT_STATUS_NAMES 定义的没有显式的边界限定	
	代码片段: 37: #include "taos.h" 38: 39: const char* STMT_STATUS_NAMES[] = { 40: "INVALID", "STMT_INIT", "STMT_PREPARE", "STMT_SETTBNAME", "STMT_SETTAGS", "STMT_FETCH_FIELDS", 41: "STMT_BIND", "STMT_BIND_COL", "STMT_ADD_BATCH", "STMT_EXECUTE", "STMT_MAX",	
	污染路径: 无	
280	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 142
	步骤描述: 数组 arr3 定义的没有显式的边界限定	
	代码片段: 140: } 141:	

	142: uint64_t arr3[] = {3, 5, 10, 11}; 143: f = (SArray *)taosArrayGetP(src, 2); 144: for (int i = 0; i < sizeof(arr3) / sizeof(arr3[0]); i++) {	
	污染路径: 无	
281	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 49
	步骤描述: 数组 options 定义的没有显式的边界限定	
	代码片段: 47: 48: #ifndef WINDOWS 49: static struct argp_option options[] = { 50: {"tables", 't', "NUMBER", 0, "Number of child tables, default is 10000."}, 51: {"records", 'n', "NUMBER", 0,	
	污染路径: 无	
282	缺陷发生位置: /source/libs/planner/test/planTestMain.cpp	行号: 82
	步骤描述: 数组 long_options 定义的没有显式的边界限定	
	代码片段: 80: const char* optstring = ""; 81: // clang-format off 82: static struct option long_options[] = { 83: {"dump", optional_argument, NULL, 'd'}, 84: {"skipSql", required_argument, NULL, 's'},	
	污染路径: 无	
283	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 17
	步骤描述: 数组 shortColList 定义的没有显式的边界限定	
	代码片段: 15: #define TIME_PRECISION_NANO 2 16: 17: int32_t shortColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_INT}; 18: int32_t fullColList[] = {TSDB_DATA_TYPE_TIMESTAMP,	

	TSDB_DATA_TYPE_BOOL, TSDB_DATA_TYPE_TINYINT, 19: TSDB_DATA_TYPE_UTINYINT, TSDB_DATA_TYPE_SMALLINT, TSDB_DATA_TYPE_USMALLINT,	
	污染路径: 无	
284	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 3217
	步骤描述: 数组 dec64table 定义的没有显式的边界限定	
	代码片段: 3215: { 3216: static const int dec32table[] = { 0, 1, 2, 1, 4, 4, 4, 4 }; /* added */ 3217: static const int dec64table[] = { 8, 8, 8, 7, 8, 9,10,11 }; /* subtracted */ 3218: BYTE* const oLitEnd = op + sequence.litLength; 3219: const size_t sequenceLength = sequence.litLength + sequence.matchLength;	
	污染路径: 无	
285	缺陷发生位置: /contrib/test/cos/main.c	行号: 15
	步骤描述: 数组 TEST_CI_ENDPOINT 定义的没有显式的边界限定	
	代码片段: 13: static char TEST_COS_ENDPOINT[] = "http://cos.ap- beijing.myqcloud.com"; 14: // 数据万象的访问域名，详情请参见 https://cloud.tencent.com/document/product/460/31066 文档 15: static char TEST_CI_ENDPOINT[] = "https://ci.ap- guangzhou.myqcloud.com"; 16: // 开发者拥有的项目身份 ID/ 密钥，可在 https://console.cloud.tencent.com/cam/capi 页面获取 17: static char *TEST_ACCESS_KEY_ID; // your secret_id	
	污染路径: 无	
286	缺陷发生位置: /source/os/src/osSystem.c	行号: 227
	步骤描述: 数组 dangerousChars 定义的没有显式的边界限定	

	代码片段: 225: // 移除危险字符, 用于防止命令注入 226: static bool sanitizeCommand(const char* cmd) { 227: const char* dangerousChars = "; &`\$()<>{ }[]!*?~"; 228: for (const char* p = dangerousChars; *p; p++) { 229: if (strchr(cmd, *p) != NULL) {	
	污染路径: 无	
287	缺陷发生位置: /source/os/test/osStringTests.cpp	行号: 790
	步骤描述: 数组 tz 定义的没有显式的边界限定	
	代码片段: 788: 789: TEST(osStringTests, truncateTimezoneStringRemovesAllLeadingSlashes) { 790: char tz[] = "///Asia/Shanghai CST"; 791: 792: truncateTimezoneString(tz);	
	污染路径: 无	
288	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 28
	步骤描述: 数组 charset 定义的没有显式的边界限定	
	代码片段: 26: 27: 28: const char charset[] = 29: "abcdefghijklmnopqrstuvwxyABCDEFGHIJKLMNOPQRSTUVWXYZ YZ1234567890"; 30:	
	污染路径: 无	
289	缺陷发生位置: /tools/shell/src/shellArguments.c	行号: 61
	步骤描述: 数组 indent 定义的没有显式的边界限定	
	代码片段:	

	59: 60: void shellPrintHelp() { 61: char indent[] = " "; 62: (void)printf("Usage: %s [OPTION...] \r\n\r\n", CUS_PROMPT); 63: (void)printf("%s%s%s%s\r\n", indent, "-a,", indent, SHELL_AUTH);	
	污染路径: 无	
290	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 25
	步骤描述: 数组 taosRefHandle 定义的没有显式的边界限定	
	代码片段: 23: void (*taosCloseHandle[])(void* arg) = {transCloseServer, transCloseClient}; 24: 25: void (*taosRefHandle[])(void* handle) = {transRefSrvHandle, transRefCliHandle}; 26: void (*taosUnRefHandle[])(void* handle) = {transUnrefSrvHandle, NULL}; 27:	
	污染路径: 无	
291	缺陷发生位置: /source/client/src/clientMain.c	行号: 831
	步骤描述: 数组 weekdays 定义的没有显式的边界限定	
	代码片段: 829: } SFetchDateTimeWhiteListInfo; 830: 831: static const char *weekdays[] = {"SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT"}; 832: int32_t fetchDateTimeWhiteListCallbackFn(void *param, SDataBuf *pMsg, int32_t code) { 833: int32_t lino = 0;	
	污染路径: 无	
292	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 187

	步骤描述: 数组 SYSTABLE_SPECIAL_COL 定义的没有显式的边界限定	
	代码片段: 185: int64_t* pRows); 186: 187: static char* SYSTABLE_SPECIAL_COL[] = {"db_name", "vgroup_id"}; 188: 189: static int32_t buildSysDbTableInfo(const SSysTableScanInfo* pInfo, int32_t capacity);	
	污染路径: 无	
293	缺陷发生位置:	行号:
	/source/util/test/stringTest.cpp	22
	步骤描述: 数组 t21 定义的没有显式的边界限定	
	代码片段: 20: EXPECT_STRCASEEQ(t1, "abc"); 21: 22: char t21[] = " abc "; 23: int32_t lx = strstr(t21); 24:	
294	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/tmqtt/mgmt/src/tmqttMgmt.c	133
	步骤描述: 数组 envMqtttd 定义的没有显式的边界限定	
295	代码片段: 131: } 132: 133: char *envMqtttd[] = {dnodeIdEnvItem, thrdPoolSizeEnvItem, taosFqdnEnvItem, NULL}; 134: 135: char **envMqtttdWithPEnv = NULL;	
	污染路径: 无	
	缺陷发生位置:	行号:
295	/source/util/test/stringTest.cpp	134
	步骤描述:	

	数组 a2 定义的没有显式的边界限定	
	代码片段: 132: EXPECT_TRUE(strnchr(a1, '.', strlen(a1), true) != NULL); 133: 134: char a2[] = "abc."; 135: EXPECT_TRUE(strnchr(a2, '.', strlen(a2), true) != NULL); 136:	
	污染路径: 无	
296	缺陷发生位置:	行号:
	/utils/test/c/tmq_ts7402.c	26
	步骤描述: 数组 result1 定义的没有显式的边界限定	
	代码片段: 24: #include "types.h" 25: 26: char* result1[] = { 27: "{\"type\":\"create\", \"tableType\":\"super\", \"tableName\":\"st\", \", \"columns\": [{\"name\":\"ts\", \"type\":9, \"isPrimarykey\":false, \"encode\":\"delta-i\", \"compress\":\"lz4\", \"level\":\"medium\"}, {\"name\":\"id\", \"type\":4, \"isPrimarykey\":false, \"encode\":\"simple8b\", \"compress\":\"lz4\", \"level\":\"medium\"}], \"tags\": [{\"name\":\"t1\", \"type\":4}]}", 28: "{\"type\":\"create\", \"tableType\":\"child\", \"tableName\":\"t1\", \", \"using\":\"st\", \"tagNum\":1, \"tags\": [{\"name\":\"t1\", \"type\":4, \"value\":1}], \"createList\":[]}",	
	污染路径: 无	
297	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/batchprepare.c	19
	步骤描述: 数组 bindColTypeList 定义的没有显式的边界限定	
	代码片段: 17: int32_t shortColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_INT}; 18: int32_t fullColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_BOOL, TSDB_DATA_TYPE_TINYINT, TSDB_DATA_TYPE_UTINYINT, TSDB_DATA_TYPE_SMALLINT,	

	TSDB_DATA_TYPE_USMALLINT, TSDB_DATA_TYPE_INT, TSDB_DATA_TYPE_UINT, TSDB_DATA_TYPE_BIGINT, TSDB_DATA_TYPE_UBIGINT, TSDB_DATA_TYPE_FLOAT, TSDB_DATA_TYPE_DOUBLE, TSDB_DATA_TYPE_BINARY, TSDB_DATA_TYPE_NCHAR}; 19: int32_t bindColTypeList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_NCHAR}; 20: int32_t optrIdxList[] = {5, 11}; 21:	
	污染路径: 无	
298	缺陷发生位置: /test/new_test_framework/script/api/stmt-insert-dupkeys.c	行号: 191
	步骤描述: 数组 voltage0 定义的没有显式的边界限定	
	代码片段: 189: int64_t ts0[] = {1648432611234, 1648432611345, 1648432611456}; 190: float current0[] = {10.1f, 10.2f, 10.3f}; 191: int voltage0[] = {216, 217, 218}; 192: float phase0[] = {0.31f, 0.32f, 0.33f}; 193: insertData(taos, ts0, current0, voltage0, phase0);	
	污染路径: 无	
299	缺陷发生位置: /source/libs/tss/src/tss.c	行号: 177
	步骤描述: 数组 tsSsAccessString 定义的没有显式的边界限定	
	代码片段: 175: 176: int32_t tssCreateDefaultInstance() { 177: extern char tsSsAccessString[]; 178: 179: if (strlen(tsSsAccessString) == 0) {	
	污染路径: 无	
300	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 587
	步骤描述: 数组 hexMap 定义的没有显式的边界限定	

	代码片段: 585: 586: static void shellHexEncode(char *dst, const char *val, int32_t length) { 587: static const char hexMap[] = "0123456789ABCDEF"; 588: 589: for (int32_t i = 0; i < length; i++) {	
	污染路径: 无	
301	缺陷发生位置: /source/libs/tcs/test/tcsTest.cpp	行号: 228
	步骤描述: 数组 object_name_arr 定义的没有显式的边界限定	
	代码片段: 226: } 227: 228: const char *object_name_arr[] = {object_name}; 229: code = tcsDeleteObjects(object_name_arr, 1); 230: GTEST_ASSERT_EQ(code, 0);	
	污染路径: 无	
302	缺陷发生位置: /source/util/test/regexTest.cpp	行号: 103
	步骤描述: 数组 s1 定义的没有显式的边界限定	
	代码片段: 101: TEST(testCase, regexCacheTest2) { 102: int times = 100000; 103: char s1[] = "abc%*"; 104: auto start = std::chrono::high_resolution_clock::now(); 105:	
	污染路径: 无	
303	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 25
	步骤描述: 数组 resEncodingChunk 定义的没有显式的边界限定	
	代码片段: 23: #endif 24:	

	25: char resEncodingChunk[] = "Encoding: chunked"; 26: char succMessage[] = "succ"; 27: char resHttp[] = "HTTP/1.1 ";	
	污染路径: 无	
304	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 134
	步骤描述: 数组 argp_default_options 定义的没有显式的边界限定	
	代码片段: 132: #define OPT_HANG -4 133: 134: static const struct argp_option argp_default_options[] = 135: { 136: {"help", '?', 0, 0, N_("Give this help list"), -1},	
	污染路径: 无	
305	缺陷发生位置: /source/libs/metrics/src/metrics.c	行号: 205
	步骤描述: 数组 label_values 定义的没有显式的边界限定	
	代码片段: 203: snprintf(clusterIdStr, sizeof(clusterIdStr), "%" PRId64, clusterId); 204: snprintf(dnodeIdStr, sizeof(dnodeIdStr), "%d", dnodeId); 205: const char *label_values[] = {DNODE_METRIC, clusterIdStr, dnodeIdStr, dnodeEp ? dnodeEp : ""}; 206: 207: // Update counters using monitorfw	
	污染路径: 无	
306	缺陷发生位置: /source/libs/executor/test/queryPlanTests.cpp	行号: 316
	步骤描述: 数组 qptSink 定义的没有显式的边界限定	
	代码片段: 314: bool qptErrorRerun = false; 315: bool qptInRerun = false; 316: int32_t qptSink[] =	

	{QUERY_NODE_PHYSICAL_PLAN_DISPATCH, QUERY_NODE_PHYSICAL_PLAN_INSERT, QUERY_NODE_PHYSICAL_PLAN_QUERY_INSERT, QUERY_NODE_PHYSICAL_PLAN_DELETE}; 317: 318:	
	污染路径: 无	
307	缺陷发生位置: /utls/test/c/sml_test.c	行号: 121
	步骤描述: 数组 sql1 定义的没有显式的边界限定	
	代码片段: 119: taos_free_result(pRes); 120: 121: const char *sql1[] = {"sysif.bytes.out 1479496100 1.3E0 host=web01 interface=eth0"}; 122: pRes = taos_schemaless_insert(taos, (char **)sql1, sizeof(sql1) / sizeof(sql1[0]), TSDB_SML_TELNET_PROTOCOL, 123: TSDB_SML_TIMESTAMP_NANO_SECONDS);	
	污染路径: 无	
308	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 121
	步骤描述: 数组 GVnodeAsynccs 定义的没有显式的边界限定	
	代码片段: 119: }; 120: 121: struct { 122: const char *label; 123: SVAsync *async;	
	污染路径: 无	
309	缺陷发生位置: /docs/examples/c/multi_bind_example.c	行号: 76
	步骤描述: 数组 ts 定义的没有显式的边界限定	
	代码片段:	

	74: TAOS_MULTI_BIND params[4]; 75: // values to bind 76: int64_t ts[] = {1648432611249, 1648432611749}; 77: float current[] = {10.3, 12.6}; 78: int voltage[] = {219, 218};	
	污染路径: 无	
310	缺陷发生位置: /source/util/test/decompressTest.cpp	行号: 172
	步骤描述: 数组 alg 定义的没有显式的边界限定	
	代码片段: 170: } 171: 172: const char* alg[] = {"disabled", "lz4", "zlib", "zstd", "tsz", "xz"}; 173: const char* end[] = {"disabled", "simppe8b", "delta", "test", "test"}; 174: void compressImplTestByAlg(void* pVal, int8_t type, int32_t num, uint32_t cmprAlg) {	
	污染路径: 无	
311	缺陷发生位置: /docs/examples/c/sml_insert_demo.c	行号: 73
	步骤描述: 数组 lines 定义的没有显式的边界限定	
	代码片段: 71: 72: // influxdb line protocol 73: char *lines[] = {line_demo}; 74: result = taos_schemaless_insert(taos, lines, 1, TSDB_SML_LINE_PROTOCOL, TSDB_SML_TIMESTAMP_MILLI_SECONDS); 75: code = taos_errno(result);	
	污染路径: 无	
312	缺陷发生位置: /source/util/src/tcompression.c	行号: 359
	步骤描述: 数组 selector_to_elems 定义的没有显式的边界限定	

	步骤描述: 数组 options 定义的没有显式的边界限定	
	代码片段: 3325: }; 3326: 3327: static OptionNameAndVar options[] = {{"asyncLog", &tsAsyncLog}, 3328: {"compressMsgSize", &tsCompressMsgSize}, 3329: {"countAlwaysReturnValue", &tsCountAlwaysReturnValue},	
	污染路径: 无	
316	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 285
	步骤描述: 数组 functions 定义的没有显式的边界限定	
	代码片段: 283: }; 284: 285: char* functions[] = { 286: "count(", "sum(", 287: "avg(", "last(",	
	污染路径: 无	
317	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 18
	步骤描述: 数组 fullColList 定义的没有显式的边界限定	
	代码片段: 16: 17: int32_t shortColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_INT}; 18: int32_t fullColList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_BOOL, TSDB_DATA_TYPE_TINYINT, TSDB_DATA_TYPE_UTINYINT, TSDB_DATA_TYPE_SMALLINT, TSDB_DATA_TYPE_USMALLINT, TSDB_DATA_TYPE_INT, TSDB_DATA_TYPE_UINT, TSDB_DATA_TYPE_BIGINT, TSDB_DATA_TYPE_UBIGINT, TSDB_DATA_TYPE_FLOAT, TSDB_DATA_TYPE_DOUBLE, TSDB_DATA_TYPE_BINARY, TSDB_DATA_TYPE_NCHAR};	

	19: int32_t bindColTypeList[] = {TSDB_DATA_TYPE_TIMESTAMP, TSDB_DATA_TYPE_NCHAR}; 20: int32_t optrIdxList[] = {5, 11};	
	污染路径: 无	
318	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 33
	步骤描述: 数组 g_tfile_info 定义的没有显式的边界限定	
	代码片段: 31: static int32_t stt_from_json(const cJSON *json, STFile *file); 32: 33: static const struct { 34: const char *suffix; 35: int32_t (*to_json)(const STFile *file, cJSON *json);	
	污染路径: 无	
319	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 2871
	步骤描述: 数组 dec32table 定义的没有显式的边界限定	
	代码片段: 2869: const BYTE* const base, const BYTE* const vBase, const BYTE* const dictEnd) 2870: { 2871: static const int dec32table[] = { 0, 1, 2, 1, 4, 4, 4, 4 }; /* added */ 2872: static const int dec64table[] = { 8, 8, 8, 7, 8, 9,10,11 }; /* subtracted */ 2873: BYTE* const oLitEnd = op + sequence.litLength;	
	污染路径: 无	
320	缺陷发生位置: /source/dnode/vnode/src/bse/bseUtil.c	行号: 45
	步骤描述: 数组 bseCompressFuncSet 定义的没有显式的边界限定	
	代码片段: 43: } SCompressFuncSet; 44: 45: static SCompressFuncSet bseCompressFuncSet[] = {	

	46: {"plain", plainCompress, plainDecompress}, {"lz4", lz4Compress, lz4Decompress}, 47: {"zlib", zlibCompress, zlibDecompress}, {"zstd", zstdCompress, zstdDecompress},	
	污染路径: 无	
321	缺陷发生位置: /test/new_test_framework/script/api/stmt-insert-dupkeys.c	行号: 190
	步骤描述: 数组 current0 定义的没有显式的边界限定	
	代码片段: 188: // initial insert, expect insert 3 rows 189: int64_t ts0[] = {1648432611234, 1648432611345, 1648432611456}; 190: float current0[] = {10.1f, 10.2f, 10.3f}; 191: int voltage0[] = {216, 217, 218}; 192: float phase0[] = {0.31f, 0.32f, 0.33f};	
	污染路径: 无	
322	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 80
	步骤描述: 数组 codes 定义的没有显式的边界限定	
	代码片段: 78: 79: // single code 80: int32_t codes[] = {0x0000ffff, TSDB_CODE_VND_STOPPED}; 81: for(int32_t i = 0; i < sizeof(codes)/sizeof(int32_t); i++) { 82: if (code == codes[i]) {	
	污染路径: 无	
323	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 155
	步骤描述: 数组 arr1 定义的没有显式的边界限定	
	代码片段: 153: clearFinalArray(rsIt); 154: 155: uint64_t arr1[] = {13, 14, 15};	

	156: SArray * f = (SArray *)taosArrayGetP(src, 0); 157: for (int i = 0; i < sizeof(arr1) / sizeof(arr1[0]); i++) {	
	污染路径: 无	
324	缺陷发生位置: /source/client/src/clientMonitor.c	行号: 355
	步骤描述: 数组 result_state 定义的没有显式的边界限定	
	代码片段: 353: 354: const char* monitorResultStr(SQL_RESULT_CODE code) { 355: static const char* result_state[] = {"Success", "Failed", "Cancel"}; 356: return result_state[code]; 357: }	
	污染路径: 无	
325	缺陷发生位置: /docs/examples/c/telnet_line_example.c	行号: 30
	步骤描述: 数组 lines 定义的没有显式的边界限定	
	代码片段: 28: executeSQL(taos, "CREATE DATABASE test"); 29: executeSQL(taos, "USE test"); 30: char *lines[] = { 31: "meters.current 1648432611249 10.3 location=California.SanFrancisco groupid=2", 32: "meters.current 1648432611250 12.6 location=California.SanFrancisco groupid=2",	
	污染路径: 无	
326	缺陷发生位置: /source/libs/tfs/test/tfsTest.cpp	行号: 145
	步骤描述: 数组 p3 定义的没有显式的边界限定	
	代码片段: 143: EXPECT_EQ(taosDirExist(ap2), 1); 144: 145: char p3[] = "p3/p2/p1/p0"; 146: char ap3[128] = {0};	

	147: snprintf(ap3, 128, "%s%s%s", root, TD_DIRSEP, p3);	
	污染路径: 无	
327	缺陷发生位置: /source/client/src/clientMonitorSql.c	行号: 23
	步骤描述: 数组 selectMonitorLabels 定义的没有显式的边界限定	
	代码片段: 21: const char* selectMonitorHelp = "count for select sql"; 22: const int selectMonitorLabelCount = 4; 23: const char* selectMonitorLabels[] = {"cluster_id", "sql_type", "username", "result"}; 24: 25: void monitorClientSQLReqInit(int64_t clusterId) {	
	污染路径: 无	
328	缺陷发生位置: /source/common/src/tglobal.c	行号: 2946
	步骤描述: 数组 options 定义的没有显式的边界限定	
	代码片段: 2944: }; 2945: 2946: static OptionNameAndVar options[] = {"audit", &tsEnableAudit}, 2947: {"asynclog", &tsAsyncLog}, 2948: {"countAlwaysReturnValue", &tsCountAlwaysReturnValue},	
	污染路径: 无	
329	缺陷发生位置: /source/libs/monitor/src/monFramework.c	行号: 111
	步骤描述: 数组 dnodes_gauges 定义的没有显式的边界限定	
	代码片段: 109: int32_t dnodes_label_count = 3; 110: const char *dnodes_sample_labels[] = {"cluster_id", "dnode_id", "dnode_ep"}; 111: char *dnodes_gauges[] = {UPTIME, CPU_ENGINE, CPU_SYSTEM, CPU_CORE, MEM_ENGINE, MEM_FREE,	

	112: MEM_TOTAL, MEM_CACHE_BUFFER, DISK_ENGINE, DISK_USED, DISK_TOTAL, NET_IN, 113: NET_OUT, IO_READ, IO_WRITE, IO_READ_DISK, IO_WRITE_DISK, /*ERRORS,*/	
	污染路径: 无	
330	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 15500
	步骤描述: 数组 validConfigs 定义的没有显式的边界限定	
	代码片段: 15498: int32_t code = 0; 15499: 15500: const char* validConfigs[] = { 15501: "encrypt_key", 15502: };	
	污染路径: 无	
331	缺陷发生位置: /utls/test/c/tmq_taosx_ci.c	行号: 990
	步骤描述: 数组 result 定义的没有显式的边界限定	
	代码片段: 988: } else { 989: if (g_conf.subTable) { 990: char* result[] = { 991: "{"type\":"create\","tableType\":"super\","tableName\":"me ters_summary\","columns\":[{"name\":"_ " 992: "wstart\","type\":"9","isPrimarykey\":"false","encode\":"delta- i\","compress\":"lz4\","level\":"	
	污染路径: 无	
332	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 870
	步骤描述: 数组 pScopeStr 定义的没有显式的边界限定	
	代码片段: 868:	

	869: TEST(TaosSetSlowLogScopeTest, AllScopeInput) { 870: char pScopeStr[] = "all"; 871: int32_t scope = 0; 872: int32_t result = taosSetSlowLogScope2(pScopeStr, &scope);	
	污染路径: 无	
333	缺陷发生位置: /source/common/src/ttime.c	行号: 1381
	步骤描述: 数组 formatKeyWords 定义的没有显式的边界限定	
	代码片段: 1379: 1380: // clang-format off 1381: static const TSFormatKeyword formatKeyWords[] = { 1382: //{"AD", 2, TSFKW_AD, false}, 1383: //{"A.D.", 4, TSFKW_A_D},	
	污染路径: 无	
334	缺陷发生位置: /source/client/inc/clientStmt2.h	行号: 245
	步骤描述: 数组 gStmt2StatusStr 定义的没有显式的边界限定	
	代码片段: 243: #define STMT2_TLOG_E(param) qTrace("stmt2:%p, " param, pStmt) 244: 245: extern char *gStmt2StatusStr[]; 246: 247: #define STMT2_LOG_SEQ(n) \	
	污染路径: 无	
335	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 249
	步骤描述: 数组 cliDealFunc 定义的没有显式的边界限定	
	代码片段: 247: static void cliDoReq(queue* h, SCliThrd* pThrd); 248: static void cliDoBatchReq(queue* h, SCliThrd* pThrd);	

	249: static void (*cliDealFunc[])(queue* h, SCLiThrd* pThrd) = {cliDoReq, cliDoBatchReq}; 250: 251: static void (*cliAsyncHandle[])(SCLiThrd* pThrd, SCLiReq* pReq) = {		
	污染路径: 无		
336	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 11	
	步骤描述: 数组 DBBackupPath 定义的没有显式的边界限定		
	代码片段: 9: // const char DBPath[] = "/tmp/rocksdb_c_simple_example"; 10: const char DBPath[] = "rocksdb_c_simple_example"; 11: const char DBBackupPath[] = "/tmp/rocksdb_c_simple_example_backup"; 12: 13: static const int32_t endian_test_var = 1;		
	污染路径: 无		
337	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 4568	
	步骤描述: 数组 MNDXNODE_DEFAULT_SECRET 定义的没有显式的边界限定		
	代码片段: 4566: } agentTokenField; 4567: 4568: const unsigned char MNDXNODE_DEFAULT_SECRET[] = {126, 222, 130, 137, 43, 122, 41, 173, 144, 146, 116, 4569: 138, 153, 244, 251, 99, 50, 55, 140, 238, 218, 232, 4570: 15, 161, 226, 54, 130, 40, 211, 234, 111, 171};		
	污染路径: 无		
338	缺陷发生位置: /source/dnode/mnode/impl/inc/mndStream.h	行号: 52	
	步骤描述: 数组 gMndStreamEvent 定义的没有显式的边界限定		
	代码片段:		

	<pre>50: } SStmLastEvent; 51: 52: static const char* gMndStreamEvent[] = {"ACTIVE_BEGIN", "NORMAL_BEGIN", "CREATE_STREAM", "DROP_STREAM", "STOP_STREAM", "START_STREAM", 53: "CREATE_SNODE", "DROP_SNODE", "LOOP_SDB", "LOOP_MAP", "LOOP_SNODE", "STREAM_TERR", "MAX_VALUE"}; 54:</pre>
	污染路径: 无

9 C/C++缺陷类型-不匹配的类型转换

9.1 分项汇总表

序号	C/C++缺陷类型-不匹配的类型转换	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4064	主机字节顺序与网络字节顺序的错误转换	3	严重
2	WK4749	长整数变量赋值给短整数变量必须强制转换	184	严重
3	WK4753	禁止使用无实质作用的类型转换	140	严重
4	WK4750	double 型变量赋值给 float 型变量必须强制转换	4	严重
5	WK4794	指针或引用类	48	严重

		型的转换中禁止移除 const 或 volatile 属性		
6	WK4793	禁止将不相关指针类型强制转换为对象指针类型	6	严重

9.2 缺陷/安全漏洞详解

9.2.1 严重类

9.2.1.1 WK4064 主机字节顺序与网络字节顺序的错误转换

缺陷数量：3 个		
缺陷描述： 来源于主机边界的多字节的值在发送或者写入到环境之前，未将主机字节顺序转换为网路字节顺序或从外界获取或者读取一个多字节到主机边界时，没有把网络字节顺序转换为主机字节顺序。		
修复意见： 应使用 htons 函数把 unsigned short 类型从主机序转换到网络序；应使用 htonl 函数把 unsigned ong 类型从主机序转换到网络序；应使用 ntohs 函数把 unsigned Short 类型从网络序转换到主机序；应使用 ntohl 函数把 unsigned long 类型从网络序转换到主机序。		
1	缺陷发生位置：	行号：
	/tools/taos-tools/src/benchUtil.c	1560
	步骤描述： 主机字节顺序与网络字节顺序的错误转换	
	代码片段： <pre> 1558: sent = 0; 1559: do { 1560: bytes = send(sockfd, request_buf + sent, 1561: req_str_len - sent, 0); 1562: if (bytes < 0) { </pre>	
2	污染路径： 无	
	缺陷发生位置：	行号：
	/tools/taos-tools/src/benchUtil.c	1032

	步骤描述: 主机字节顺序与网络字节顺序的错误转换	
	代码片段: <pre> 1030: int recvbuflen = LARGE_BUFF_LEN; 1031: char recvbuf[LARGE_BUFF_LEN]; 1032: result = recv(sockfd, recvbuf, recvbuflen, 0); 1033: if (result > 0) { 1034: debugPrint("Socket bytes received: %d\n", result); </pre>	
	污染路径: 无	
3	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1585
	步骤描述: 主机字节顺序与网络字节顺序的错误转换	
	代码片段: <pre> 1583: } 1584: do { 1585: bytes = recv(sockfd, responseBuf + received, 1586: resp_len - received, 0); 1587: responseBuf[resp_len] = 0; </pre>	
	污染路径: 无	

9.2.1.2 WK4749 长整数变量赋值给短整数变量必须强制转换

缺陷数量：184 个		
缺陷描述: 长整数变量赋值给短整数变量必须强制转换。		
修复意见: 无		
1	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1242
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: <pre> 1240: { 1241: count = (short)(bitStream & (2*threshold-1)); </pre>	

	1242: if (count >= threshold) count -= max; 1243: bitCount += nbBits; 1244: }	
	污染路径: 无	
2	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1976
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1974: U32 nextRankVal = 0; 1975: U32 w, consumed; 1976: const int rescale = (memLog-tableLog) - 1; /* tableLog <= memLog */ 1977: U32* rankVal0 = rankVal[0]; 1978: for (w=1; w<=maxW; w++)	
	污染路径: 无	
3	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 2307
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 2305: if (targetLog-nbBits >= minBits) { /* enough room for a second symbol */ 2306: U32 sortedRank; 2307: int minWeight = nbBits + scaleLog; 2308: if (minWeight < 1) minWeight = 1; 2309: sortedRank = rankStart[minWeight];	
	污染路径: 无	
4	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1183
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1181: if (hbSize < 4) return ERROR(srcSize_wrong); 1182: bitStream = MEM_readLE32(ip); 1183: nbBits = (bitStream & 0xF) + FSE_MIN_TABLELOG; /* extract tableLog */	

	1184: if (nbBits > FSE_TABLELOG_ABSOLUTE_MAX) return ERROR(tableLog_tooLarge); 1185: bitStream >>= 4;	
	污染路径: 无	
5	缺陷发生位置: /tools/src/pub.c	行号: 28
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 26: } 27: for (size_t i = 0; i < len; i++) { 28: result[i] = tolower((unsigned char)str[i]); 29: } 30: result[len] = '\0';	
	污染路径: 无	
6	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 1813
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1811: if (bits > 0) { 1812: int v = (buffer << (5 - bits)) & 0x1F; 1813: out[outLen++] = (v >= 26) ? (v - 26 + '2') : (v + 'A'); 1814: } 1815:	
	污染路径: 无	
7	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1184
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1182: if (hbSize < 4) return ERROR(srcSize_wrong); 1183: bitStream = MEM_readLE32(ip); 1184: nbBits = (bitStream & 0xF) + FSE_MIN_TABLELOG; /* extract tableLog */ 1185: if (nbBits > FSE_TABLELOG_ABSOLUTE_MAX) return ERROR(tableLog_tooLarge);	

	1186: bitStream >>= 4;	
	污染路径: 无	
8	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 237
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 235: STireNode* c; 236: char word[MAX_WORD_LEN]; 237: int len = strlen(prefix); 238: for (int i = 0; i < CHAR_CNT; i++) { 239: c = nodes[i];	
	污染路径: 无	
9	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 2155
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 2153: { 2154: U32 rankVal[HUFv05_ABSOLUTEMAX_TABLELOG + 1]; 2155: const int scaleLog = nbBitsBaseline - targetLog; /* note : targetLog >= srcLog, hence scaleLog <= 1 */ 2156: const U32 minBits = nbBitsBaseline - maxWeight; 2157: U32 s;	
	污染路径: 无	
10	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 2098
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 2096: { 2097: U32 rankVal[HUFv07_TABLELOG_ABSOLUTEMAX + 1]; 2098: const int scaleLog = nbBitsBaseline - targetLog; /* note : targetLog >= srcLog, hence scaleLog <= 1 */ 2099: const U32 minBits = nbBitsBaseline - maxWeight; 2100: U32 s;	

	污染路径: 无	
11	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 996
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 994: p = taosDecodeVariantI64(p, &pHandle->range.sseq); 995: p = taosDecodeVariantI64(p, &pHandle->range.eseq); 996: return p - buf; 997: } 998:	
	污染路径: 无	
12	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 324
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 322: 323: const char *paren = strchr(type_str, '('); 324: int type_len = paren ? (paren - type_str) : strlen(type_str); 325: 326: #define CHECK_TYPE(name, type_const) \	
	污染路径: 无	
13	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 615
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 613: resiBits = 0; 614: if (resiBitsLength != 0) { 615: int kMod8 = k % 8; 616: int rightMovSteps = getRightMovingSteps(kMod8, resiBitsLength); 617: if (rightMovSteps > 0) {	
	污染路径: 无	

14	缺陷发生位置: /contrib/TSZ/sz/src/sz_float.c	行号: 105
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 103: int reqLength; // need save bits length for one float . value ragne 9~32 104: float medianValue = medianValue_f; 105: short rangeExpo = getExponent_float(valueRangeSize/2); 106: 107: computeReqLength_float(realPrecision, rangeExpo, &reqLength, &medianValue);	
	污染路径: 无	
15	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 2186
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 2184: /* Build rankVal */ 2185: { U32* const rankVal0 = rankVal[0]; 2186: { int const rescale = (maxTableLog-tableLog) - 1; /* tableLog <= maxTableLog */ 2187: U32 nextRankVal = 0; 2188: U32 w;	
	污染路径: 无	
16	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 140
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 138: if (srcInfo == NULL removeStr == NULL) return; 139: 140: int len = strlen(srcInfo); 141: int new_len = len; 142:	
	污染路径: 无	

17	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 2291
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 2289: { 2290: U32 rankVal[HUFv06_ABSOLUTEMAX_TABLELOG + 1]; 2291: const int scaleLog = nbBitsBaseline - targetLog; /* note : targetLog >= srcLog, hence scaleLog <= 1 */ 2292: const U32 minBits = nbBitsBaseline - maxWeight; 2293: U32 s;	
	污染路径: 无	
18	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 1874
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1872: c -= 'A'; 1873: } else if (c >= '2' && c <= '7') { 1874: c = c - '2' + 26; 1875: } else if (c == '=') { 1876: break; // padding character	
	污染路径: 无	
19	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 1121
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1119: 1120: // column buffer 1121: int buf_size = stb->lenOfCols + stb->cols->size; 1122: char* buf = (char*)benchCalloc(1, buf_size, true); 1123: if (!buf) {	
	污染路径: 无	
20	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 221
	步骤描述:	

	从'int'到类型'char'的缩小转换	
	代码片段: 219: v.bin[i][j] = (char)(i + '0'); 220: } 221: v.blob[i][0] = 'a' + i; 222: v.blob2[i][0] = 'A' + i; 223:	
	污染路径: 无	
21	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 443
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 441: first = a, limit = -1; 442: } else { 443: last = a, depth += 1, limit = ss_ilg(a - first); 444: } 445: }	
	污染路径: 无	
22	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 2099
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 2097: tools_cJSON_GetObjectItem(sqlObj, "sql"); 2098: if (tools_cJSON_IsString(sqlStr)) { 2099: int strLen = strlen(sqlStr->valuelstring) + 1; 2100: sql->command = benchCalloc(1, strLen, true); 2101: TOOLS_STRNCPY(sql->command, sqlStr->valuelstring, strLen);	
	污染路径: 无	
23	缺陷发生位置: /source/common/src/tmisc.c	行号: 67
	步骤描述:	

	从'long'到类型'int'的缩小转换	
	代码片段: <pre> 65: if (!end) return TSDB_CODE_INVALID_PARA; 66: 67: int ipLen = end - ep - 1; 68: if (ipLen >= TSDB_FQDN_LEN) ipLen = TSDB_FQDN_LEN - 1; 69: </pre>	
	污染路径: 无	
24	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 143
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: <pre> 141: int new_len = len; 142: 143: int remove_len = strlen(removeStr); 144: 145: while (new_len > 0) { </pre>	
	污染路径: 无	
25	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 407
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 405: c -= 'A'; 406: } else if (c >= '2' && c <= '7') { 407: c = c - '2' + 26; 408: } else if (c == '=') { 409: break; // padding character </pre>	
	污染路径: 无	
26	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 2092
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 2090: for (int32_t i = 0; i < 5; ++i) { </pre>	

	2091: rightv[i][2] = rightv[i][3] = '0'; 2092: rightv[i][4] = '0' + i; 2093: varDataSetLen(rightv[i], 3); 2094: }	
	污染路径: 无	
27	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1689
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1687: 1688: // show 1689: int count = strlen(part); 1690: shellInsertStr(cmd, part, count); 1691: cntDel = count; // next press tab delete current append count	
	污染路径: 无	
28	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1243
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 1241: { 1242: count = (short)(bitStream & (2*threshold-1)); 1243: if (count >= threshold) count -= max; 1244: bitCount += nbBits; 1245: }	
	污染路径: 无	
29	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 408
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 406: int v, x = 0; 407: 408: for(ssize = 0, limit = ss_ilg(last - first);;) { 409:	

	410: if((last - first) <= SS_INSERTIONSORT_THRESHOLD) {	
	污染路径: 无	
30	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 479
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 477: resiBits = 0; 478: if (resiBitsLength != 0) { 479: int kMod8 = k % 8; 480: int rightMovSteps = getRightMovingSteps(kMod8, resiBitsLength); 481: if (rightMovSteps > 0) {	
	污染路径: 无	
31	缺陷发生位置: /source/util/src/tbase64.c	行号: 139
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 137: static void tbase64_build_decoding_table() { 138: for (int i = 0; i < 64; i++) { 139: tbase64_decoding_table[(unsigned char)tbase64_encoding_table[i]] = i; 140: } 141: }	
	污染路径: 无	
32	缺陷发生位置: /source/libs/sm4/src/sm4.c	行号: 495
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 493: } 494: Key_Expansion_init(pKey,rk); 495: loop = inDataLen / 16; 496: pIVTemp = pIV; 497: for (i = 0; i < loop; i++)	

	无	
36	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v01.c	行号： 475
	步骤描述： 从'int'到类型'short'的缩小转换	
	代码片段： 473: static short FSE_abs(short a) 474: { 475: return a<0? -a : a; 476: } 477:	
	污染路径： 无	
37	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v03.c	行号： 1889
	步骤描述： 从'unsigned int'到类型'int'的缩小转换	
	代码片段： 1887: { 1888: U32 sortedRank; 1889: int minWeight = nbBits + scaleLog; 1890: if (minWeight < 1) minWeight = 1; 1891: sortedRank = rankStart[minWeight];	
	污染路径： 无	
38	缺陷发生位置： /contrib/TSZ/sz/src/dataCompression.c	行号： 525
	步骤描述： 从'int'到类型'short'的缩小转换	
	代码片段： 523: 524: computeRangeSize_double(oriData, nbEle, &valueRangeSize, medianValue); 525: short radExpo = getExponent_double(valueRangeSize/2); 526: 527: int reqLength;	
	污染路径： 无	
39	缺陷发生位置：	行号：

	/tools/taos-tools/src/taosdump.c	549
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 547: 548: // malloc new buff put new sql with new name 549: int newLen = strlen(cmd) + (strlen(newName) - oldLen) + 1; 550: char* newCmd = (char *)malloc(newLen); 551: if (newCmd == NULL) {	
40	污染路径: 无	
	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 136
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
41	代码片段: 134: bitCount += nbBits-1; 135: } else { 136: count = bitStream & (2*threshold-1); 137: if (count >= threshold) count -= max; 138: bitCount += nbBits;	
	污染路径: 无	
	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 184
42	步骤描述: 从'unsigned int'到类型'char'的缩小转换	
	代码片段: 182: } else if (unic <= 0x000007FF) { 183: *(p + 1) = (unic & 0x3F) 0x80; 184: *p = ((unic >> 6) & 0x1F) 0xC0; 185: ret = 2; 186: } else if (unic <= 0x0000FFFF) {	
	污染路径: 无	
42	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttCtx.c	行号: 840
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	

	代码片段: 838: const bool retain = false; 839: int qos = 1; 840: int data_len = strlen(data); 841: tmq_topic_info* tinfo_found; 842:	
	污染路径: 无	
43	缺陷发生位置: /contrib/TSZ/sz/src/transcode.c	行号: 135
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 133: } 134: 135: int md = intervals / 2; 136: int code2type[67]; 137: for (int i = 0; i < 67; i++)	
	污染路径: 无	
44	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttCtx.c	行号: 799
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 797: const bool retain = false; 798: int qos = 1; 799: int data_len = strlen(data); 800: tmq_topic_info* tinfo_found; 801:	
	污染路径: 无	
45	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 670
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 668: 669: /* Initialize global data */ 670: g_displayLevel = parameters.zParams.notificationLevel;	

	671: /* Checks */ 672: if (!COVER_checkParameters(parameters, dictBufferCapacity)) {	
	污染路径: 无	
46	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 353
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 351: int t; 352: 353: t = last - first; 354: middle = first + t / 2; 355:	
	污染路径: 无	
47	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1882
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 1880: } 1881: 1882: return p1 - p; 1883: } 1884:	
	污染路径: 无	
48	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 156
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 154: INLINE int getLeftMovingSteps(size_t k, unsigned char resiBitLength) 155: { 156: return 8 - k%8 - resiBitLength; 157: } 158:	

	污染路径: 无	
49	缺陷发生位置: /source/util/src/tenv.c	行号: 79
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 77: } 78: } 79: return strlen(cfgStr); 80: }	
	污染路径: 无	
50	缺陷发生位置: /source/dnode/mnode/impl/src/mndUser.c	行号: 4698
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 4696: if (bits > 0) { 4697: int v = (buffer << (5 - bits)) & 0x1F; 4698: out[outLen++] = (v >= 26) ? (v - 26 + '2') : (v + 'A'); 4699: } 4700:	
	污染路径: 无	
51	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1269
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 1267: } else { 1268: count = (short)(bitStream & (2*threshold-1)); 1269: if (count >= threshold) count -= max; 1270: bitCount += nbBits; 1271: }	
	污染路径: 无	
52	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 403

	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 401: } 402: 403: ret = fwrite(buf, 1, length, fp); 404: if (ret != length) { 405: errorPrint("Failed to write create db sql: %s. expected written %d but %d.\n",	
	污染路径: 无	
53	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 557
	步骤描述: 从'const int'到类型'char'的缩小转换	
	代码片段: 555: if (oshort (o) && ! find_char (o->key, hol->short_options, so)) 556: /* O has a valid short option which hasn't already been used.*/ 557: *so++ = o->key; 558: o++; 559: }	
	污染路径: 无	
54	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1261
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 1259: * FSE NCount encoding-decoding 1260: ***** */ 1261: static short FSEv06_abs(short a) { return a<0 ? -a : a; } 1262: 1263: size_t FSEv06_readNCount (short* normalizedCounter, unsigned* maxSVPtr, unsigned* tableLogPtr,	
	污染路径:	

	无	
55	缺陷发生位置： /test/new_test_framework/script/api/stmt2-nchar.c	行号： 170
	步骤描述： 从'int'到类型'char'的缩小转换	
	代码片段： 168: v.ts[i] = ts++; 169: 170: v.blob[i][0] = 'a' + i; 171: v.blob2[i][0] = 'f' + i; 172: v.blob3[i][0] = 't' + i;	
	污染路径： 无	
56	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v02.c	行号： 1874
	步骤描述： 从'unsigned int'到类型'int'的缩小转换	
	代码片段： 1872: { 1873: U32 rankVal[HUF_ABSOLUTEMAX_TABLELOG + 1]; 1874: const int scaleLog = nbBitsBaseline - targetLog; /* note : targetLog >= srcLog, hence scaleLog <= 1 */ 1875: const U32 minBits = nbBitsBaseline - maxWeight; 1876: U32 s;	
	污染路径： 无	
57	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v04.c	行号： 2053
	步骤描述： 从'unsigned int'到类型'int'的缩小转换	
	代码片段： 2051: { 2052: U32 sortedRank; 2053: int minWeight = nbBits + scaleLog; 2054: if (minWeight < 1) minWeight = 1; 2055: sortedRank = rankStart[minWeight];	
	污染路径： 无	
58	缺陷发生位置： /contrib/TSZ/zstd/compress/fse_compress.c	行号： 489

	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: <pre> 487: if (FSE_isError(errorCode)) return errorCode; 488: } 489: else normalizedCounter[largest] += (short)stillToDistribute; 490: } 491: </pre>	
	污染路径: 无	
59	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1174
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: <pre> 1172: if (hbSize < 4) return ERROR(srcSize_wrong); 1173: bitStream = MEM_readLE32(ip); 1174: nbBits = (bitStream & 0xF) + FSE_MIN_TABLELOG; /* extract tableLog */ 1175: if (nbBits > FSE_TABLELOG_ABSOLUTE_MAX) return ERROR(tableLog_tooLarge); 1176: bitStream >>= 4; </pre>	
	污染路径: 无	
60	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 856
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 854: int randomInt(int modulus) { return dis_(gen_) % modulus; } 855: char generateSign(float positive_ratio) { return possible(positive_ratio) ? '+' : '-'; } 856: char generateDigit() { return randomInt(10) + '0'; } 857: bool currentShouldGenerateCornerCase(float corner_case_ratio) { return possible(corner_case_ratio); } 858: string generateCornerCase(const DecimalStringRandomGeneratorConfig& config) { </pre>	
	污染路径:	

	无	
61	缺陷发生位置: /source/libs/monitorfw/src/taos_metric.c	行号: 40
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 38: memset(self, 0, sizeof(taos_metric_t)); 39: self->type = metric_type; 40: int len = strlen(name) + 1; 41: self->name = taos_malloc(len); 42: snprintf(self->name, len, "%s", name);	
	污染路径: 无	
62	缺陷发生位置: /source/os/src/osSystem.c	行号: 279
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 277: } 278: 279: return strlen(buf); 280: } 281:	
	污染路径: 无	
63	缺陷发生位置: /contrib/TSZ/sz/src/szd_float.c	行号: 167
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 165: resiBits = 0; 166: if (resiBitsLength != 0) { 167: int kMod8 = k % 8; 168: int rightMovSteps = getRightMovingSteps(kMod8, resiBitsLength); 169: if (rightMovSteps > 0) {	
	污染路径: 无	
64	缺陷发生位置: /source/libs/sm4/src/sm4.c	行号: 535

	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 533: } 534: Key_Expansion_init(pKey,rk); 535: loop = inDataLen / 16; 536: pIVTemp = pIV; 537: for (i = 0; i < loop; i ++)	
	污染路径: 无	
65	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchDataGeometry.c	153
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 151: int coordNum = info->minCoordNum; 152: if (maxCoordNum > info->minCoordNum) { 153: coordNum = info->minCoordNum + taosRandom() % (maxCoordNum - info->minCoordNum); 154: } 155:	
66	缺陷发生位置:	行号:
	/tools/taos-tools/src/taosdump.c	439
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 437: if (arg == NULL) return ; 438: // malloc new 439: int len = strlen(arg); 440: if(len <= 2) { 441: return ;	
67	缺陷发生位置:	行号:
	/tools/taos-tools/src/toolsString.c	68
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段:	

	66: 67: bool toolsIsStringNumber(char *input) { 68: int len = strlen(input); 69: if (0 == len) { 70: return false;	
	污染路径: 无	
68	缺陷发生位置: /contrib/TSZ/sz/src/dictionary.c	行号: 93
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 91: int i ; 92: 93: len = strlen(key); 94: for (hash=0, i=0 ; i<len ; i++) { 95: hash += (unsigned)key[i] ;	
	污染路径: 无	
69	缺陷发生位置: /contrib/TSZ/sz/src/TightDataPointStorageD.c	行号: 217
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 215: if ((*this)->ifAdtFse == 0) { 216: // huffman 217: int stateNum = 2 * intervals; 218: HuffmanTree* huffmanTree = createHuffmanTree(stateNum); 219: encode_withTree(huffmanTree, type, dataSeriesLength, &(*this)->typeArray, &(*this)- >typeArray_size);	
	污染路径: 无	
70	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1871
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1869: {	

	1870: U32 rankVal[HUF_ABSOLUTEMAX_TABLELOG + 1]; 1871: const int scaleLog = nbBitsBaseline - targetLog; /* note : targetLog >= srcLog, hence scaleLog <= 1 */ 1872: const U32 minBits = nbBitsBaseline - maxWeight; 1873: U32 s;	
	污染路径: 无	
71	缺陷发生位置: /contrib/TSZ/sz/src/CompressElement.c	行号: 32
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 30: INLINE short computeGroupNum_double(double value) 31: { 32: short expo = getExponent_double(value); 33: if(expo < 0) 34: expo = -1;	
	污染路径: 无	
72	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1295
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1293: if (hbSize < 4) return ERROR(srcSize_wrong); 1294: bitStream = MEM_readLE32(ip); 1295: nbBits = (bitStream & 0xF) + FSEv05_MIN_TABLELOG; /* extract tableLog */ 1296: if (nbBits > FSEv05_TABLELOG_ABSOLUTE_MAX) return ERROR(tableLog_tooLarge); 1297: bitStream >>= 4;	
	污染路径: 无	
73	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1276
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 1274: * FSEv05 NCount encoding-decoding 1275:	

	<pre> ***** */ 1276: static short FSEv05_abs(short a) { return a<0 ? -a : a; } 1277: 1278: </pre>	
	污染路径: 无	
74	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 261
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: <pre> 259: while (l->name != NULL) 260: if (name != NULL && strcmp (l->name, name) == 0) 261: return l - long_options; 262: else 263: l++; </pre>	
	污染路径: 无	
75	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 2315
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 2313: leftv[i][2] = 'a' + i; 2314: leftv[i][3] = 'b' + i; 2315: leftv[i][4] = '0' + i; 2316: varDataSetLen(leftv[i], 3); 2317: } </pre>	
	污染路径: 无	
76	缺陷发生位置: /source/libs/tss/test/tssTest.cpp	行号: 76
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: <pre> 74: GTEST_ASSERT_TRUE(data != NULL); 75: for (uint64_t i = 0; i < largeFileSizeInMB; i++) { 76: memset(data + i * 1024 * 1024, 'a' + i % 26, 1024 * </pre>	

	31: *reqLength = 9 + rangeExpo - realPrecExpo + 1; //radExpo-reqExpo == reqMantiLength 32: if(*reqLength<9)	
	污染路径: 无	
80	缺陷发生位置: /source/client/src/clientEnv.c	行号: 1214
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1212: #ifndef WINDOWS 1213: char newstr[PATH_MAX]; 1214: int len = strlen(str); 1215: if (len > 1 && str[0] != '"' && str[0] != '\') { 1216: if (len + 2 >= PATH_MAX) {	
	污染路径: 无	
81	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 981
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 979: 980: /* Construct a parser for these arguments. */ 981: err = parser_init (&parser, argp, argc, argv, flags, input); 982: 983: if (! err)	
	污染路径: 无	
82	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 34
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 32: INLINE void computeReqLength_double(double realPrecision, short radExpo, int* reqLength, double* medianValue) 33: {	

	34: short reqExpo = getPrecisionReqLength_double(realPrecision); 35: *reqLength = 12+radExpo - reqExpo; //radExpo- reqExpo == reqMantiLength 36: if(*reqLength<12)	
	污染路径: 无	
83	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2148
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 2146: tools_cJSON_GetArrayItem(2147: pThreadInfo->sml_json_tags, 2148: (int)tableSeq - 2149: pThreadInfo- >start_table_from), 2150: true);	
	污染路径: 无	
84	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 98
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 96: for (int j = 0; j < ROW_NUMS; j++) { 97: ts[i][j] = 1591060628000 + r * 100000 + j; 98: b[i][j] = 'a' + j; 99: } 100: }	
	污染路径: 无	
85	缺陷发生位置: /contrib/lemon/lemon.c	行号: 2014
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 2012: } 2013: ep = 0; 2014: for(i=0; i<LISTSIZE; i++) if(set[i]) ep =	

	merge(set[i],ep,cmp,offset); 2015: return ep; 2016: }	
	污染路径: 无	
86	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 194
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 192: for (int j = 0; j < ROW_NUMS; j++) { 193: ts[i][j] = 1591060628000 + j; 194: b[i][j] = 'a' + j; 195: } 196: }	
	污染路径: 无	
87	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 103
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 101: int reqLength; 102: double medianValue = medianValue_d; 103: short radExpo = getExponent_double(valueRangeSize/2); 104: 105: computeReqLength_double(realPrecision, radExpo, &reqLength, &medianValue);	
	污染路径: 无	
88	缺陷发生位置: /contrib/TSZ/sz/src/CompressElement.c	行号: 88
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 86: *curGroupID = groupNum+2; //+[-1,0,1,2,3,.....,16] is mapped to [1,2,.....,18] 87: else 88: *curGroupID = -(groupNum+2); //-[-	

	1,0,1,2,3,.....,16] is mapped to [-1,-2,.....,-18] 89: } 90:	
	污染路径: 无	
89	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 188
	步骤描述: 从'unsigned int'到类型'char'的缩小转换	
	代码片段: 186: } else if (unic <= 0x0000FFFF) { 187: *(p + 2) = (unic & 0x3F) 0x80; 188: *(p + 1) = ((unic >> 6) & 0x3F) 0x80; 189: *p = ((unic >> 12) & 0x0F) 0xE0; 190: ret = 3;	
	污染路径: 无	
90	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1896
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 1894: // create stream <stream_name> as select 1895: if (strncasecmp(p, "create stream ", 14) == 0) { 1896: return as_pos_end - p; 1897: ; 1898: }	
	污染路径: 无	
91	缺陷发生位置: /source/client/test/stmt2Test.cpp	行号: 196
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 194: for (int j = 0; j < ROW_NUMS; j++) { 195: ts[i][j] = 1591060628000 + r * 100000 + j; 196: b[i][j] = 'a' + j; 197: } 198: }	

	污染路径: 无	
92	缺陷发生位置: /source/libs/tss/test/tssTest.cpp	行号: 253
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 251: GTEST_ASSERT_TRUE(data != NULL); 252: for (uint64_t i = 0; i < largeFileSizeInMB; i++) { 253: memset(data + i * 1024 * 1024, 'a' + i % 26, 1024 * 1024); 254: } 255:	
	污染路径: 无	
93	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 201
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 199: // insert a new word 200: bool deleteWord(STire* tire, char* word) { 201: int len = strlen(word); 202: if (len >= MAX_WORD_LEN) { 203: return false; 204: } 205: }	
	污染路径: 无	
94	缺陷发生位置: /contrib/TSZ/sz/src/szd_double.c	行号: 158
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 156: resiBits = 0; 157: if (resiBitsLength != 0) { 158: int kMod8 = k % 8; 159: int rightMovSteps = getRightMovingSteps(kMod8, resiBitsLength); 160: if (rightMovSteps > 0) {	
	污染路径: 无	

95	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 2049
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 2047: for (int32_t i = 0; i < 5; ++i) { 2048: rightv[i][2] = rightv[i][3] = '0'; 2049: rightv[i][4] = '0' + i; 2050: varDataSetLen(rightv[i], 3); 2051: }	
	污染路径: 无	
96	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2385
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 2383: int64_t startTimestamp = stbInfo->startTimestamp; 2384: int disorderRange = stbInfo->disorderRange; 2385: int rand_num = taosRandom() % 100; 2386: if (rand_num < stbInfo->disorderRatio) { 2387: disorderRange--;	
	污染路径: 无	
97	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 330
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 328: 329: byteIndex = i>>3; //i/8 330: r = i%8; 331: if(((s[byteIndex] >> (7-r)) & 0x01) == 0) 332: n = n->left;	
	污染路径: 无	
98	缺陷发生位置: /source/common/src/ttime.c	行号: 1816
	步骤描述: 从'long'到类型'int'的缩小转换	

	代码片段: 1814: const char *l, *r; 1815: for (l = fmt[0], r = s;; l++, r++) { 1816: if (*l == '\0') return fmt - arr; 1817: if (*r == '\0' tolower(*l) != tolower(*r)) break; 1818: }	
	污染路径: 无	
99	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1892
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1890: { 1891: U32 sortedRank; 1892: int minWeight = nbBits + scaleLog; 1893: if (minWeight < 1) minWeight = 1; 1894: sortedRank = rankStart[minWeight];	
	污染路径: 无	
100	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 535
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 533: } 534: 535: int oldLen = e - s; 536: if(oldLen + 1 > TSDB_DB_NAME_LEN) { 537: return NULL;	
	污染路径: 无	
101	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 264
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 262: void matchPrefixFromList(STire* tire, char* prefix, SMatch* match) { 263: StrName* item = tire->head;	

	264: int len = strlen(prefix); 265: while (item) { 266: if (strncmp(item->name, prefix, len) == 0) {	
	污染路径: 无	
102	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 781
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 779: 780: int compare(const void *a, const void *b) { 781: return *(int64_t *)a - *(int64_t *)b; 782: } 783:	
	污染路径: 无	
103	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 180
	步骤描述: 从'unsigned int'到类型'char'的缩小转换	
	代码片段: 178: int ret = 0; 179: if (unic <= 0x0000007F) { 180: *p = (unic & 0x7F); 181: ret = 1; 182: } else if (unic <= 0x000007FF) {	
	污染路径: 无	
104	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 799
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 797: while (*p != 0 && *p++ != c) { 798: } 799: return p - a; 800: } 801: static int idxCacheJsonTermCompareImpl(char* a, char* b) {	

	污染路径: 无	
105	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2137
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 2135: tools_cJSON_GetArrayItem(2136: pThreadInfo->sml_json_tags, 2137: (int)tableSeq - 2138: pThreadInfo->start_table_from), 2139: true);	
	污染路径: 无	
106	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1223
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1221: if (hbSize < 4) return ERROR(srcSize_wrong); 1222: bitStream = MEM_readLE32(ip); 1223: nbBits = (bitStream & 0xF) + FSEv07_MIN_TABLELOG; /* extract tableLog */ 1224: if (nbBits > FSEv07_TABLELOG_ABSOLUTE_MAX) return ERROR(tableLog_tooLarge); 1225: bitStream >>= 4;	
	污染路径: 无	
107	缺陷发生位置: /source/util/src/tenv.c	行号: 50
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 48: 49: *p = '\0'; 50: return strlen(cfgNameStr); 51: } 52:	

	污染路径: 无	
108	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 361
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 359: const char *comma = strchr(paren, ','); 360: if (comma != NULL) { 361: int precision_len = comma - paren - 1; 362: char precision_str[16] = {0}; 363: if (precision_len > sizeof(precision_str) - 1) {	
	污染路径: 无	
109	缺陷发生位置: /source/client/src/clientSml.c	行号: 1631
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1629: if (lines) { 1630: *tmp = lines[i]; 1631: *len = strlen(*tmp); 1632: } else if (*rawLine) { 1633: *tmp = *rawLine;	
	污染路径: 无	
110	缺陷发生位置: /contrib/lemon/lemon.c	行号: 2008
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 2006: NEXT(ep) = 0; 2007: for(i=0; i<LISTSIZE-1 && set[i]!=0; i++){ 2008: ep = merge(ep,set[i],cmp,offset); 2009: set[i] = 0; 2010: }	
	污染路径: 无	
111	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 1097

	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 1095: p = taosDecodeVariantl64(p, &pMeta->range.sseq); 1096: p = taosDecodeVariantl64(p, &pMeta->range.eseq); 1097: return p - buf; 1098: } 1099: int32_t metaBlockAdd(SBlock *p, SMetaBlock *pBlk) {	
	污染路径: 无	
112	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmt2.c	222
	步骤描述: 从'int'到类型'char'的缩小转换	
113	代码片段: 220: } 221: v.blob[i][0] = 'a' + i; 222: v.blob2[i][0] = 'A' + i; 223: 224: // v.blob2[i] = malloc(strlen("一二三四五六七十九八"));	
	污染路径: 无	
	缺陷发生位置:	行号:
114	/source/common/test/cosCpTest.cpp	81
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 79: const int64_t MULTIPART_CHUNK_SIZE = 64 << 20; // multipart is 768M 80: uint64_t chunk_size = MULTIPART_CHUNK_SIZE >> 3; 81: int totalSeq = (contentLength + chunk_size - 1) / chunk_size; 82: const int max_part_num = 10000; 83: if (totalSeq > max_part_num) {	
114	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/monitorfw/src/taos_metric_formatter.c	168
	步骤描述:	
	从'long'到类型'int'的缩小转换	

	代码片段: 166: memcpy(vgroupid, start, len); 167: vgroupid[len] = '\0'; 168: return strtol(vgroupid, NULL, 10); 169: } 170: }	
	污染路径: 无	
115	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 67
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 65: 66: // Generate the complete SQL statement 67: int sql_len = strlen("select tbname,last(*) from test.meters where tbname in ") + strlen(table_list) + 68: strlen(" partition by tbname;") + 100; 69:	
	污染路径: 无	
116	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 953
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 951: (1 + (kMaxD - kMinD) / 2) * (1 + (kMaxK - kMinK) / kStepSize); 952: /* Local variables */ 953: const int displayLevel = parameters-> zParams.notificationLevel; 954: unsigned iteration = 1; 955: unsigned d;	
	污染路径: 无	
117	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 387
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段:	

	multipart is 768M 125: uint64_t chunk_size = MULTIPART_CHUNK_SIZE >> 3; 126: int totalSeq = (contentLength + chunk_size - 1) / chunk_size; 127: const int max_part_num = 10000; 128: if (totalSeq > max_part_num) {	
	污染路径: 无	
121	缺陷发生位置: /contrib/lemon/lemon.c	行号: 3040
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 3038: z[j] = 0; 3039: exclude = eval_preprocessor_boolean(&z[iBool], lineno); 3040: z[j] = k; 3041: if(!isNot) exclude = !exclude; 3042: if(exclude){	
122	污染路径: 无	
	缺陷发生位置: /source/util/src/tenv.c	行号: 20
	步骤描述: 从'int'到类型'char'的缩小转换	
123	代码片段: 18: #include "tenv.h" 19: 20: static char toLowChar(char c) { return (c > 'Z' c < 'A' ? c : (c - 'A' + 'a')); } 21: 22: int32_t taosEnvNameToCfgName(const char *envNameStr, char *cfgNameStr, int32_t cfgNameMaxLen) {	
	污染路径: 无	
	缺陷发生位置: /contrib/TSZ/sz/src/transcode.c	行号: 42
123	步骤描述: 从'unsigned int'到类型'int'的缩小转换	

	代码片段: 40: 41: // transcoding 42: int md = intervals / 2; 43: uint8_t* type2code = (uint8_t *)malloc(intervals * sizeof(uint8_t)); 44: uint32_t* diff = (uint32_t *)malloc(intervals * sizeof(uint32_t));	
	污染路径: 无	
124	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 1805
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1803: while (bits >= 5) { 1804: int v = (buffer >> (bits - 5)) & 0x1F; 1805: out[outLen++] = (v >= 26) ? (v - 26 + '2') : (v + 'A'); 1806: bits -= 5; 1807: }	
	污染路径: 无	
125	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 854
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 852: they are. */ 853: { 854: char first1 = short1 ? short1 : long1 ? *long1 : 0; 855: char first2 = short2 ? short2 : long2 ? *long2 : 0; 856: #ifdef _tolower	
	污染路径: 无	
126	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 183
	步骤描述: 从'unsigned int'到类型'char'的缩小转换	

	代码片段: <pre> 181: ret = 1; 182: } else if (unic <= 0x000007FF) { 183: *(p + 1) = (unic & 0x3F) 0x80; 184: *p = ((unic >> 6) & 0x1F) 0xC0; 185: ret = 2; </pre>	
	污染路径: 无	
127	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 434
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: <pre> 432: if(1 < (a - first)) { 433: STACK_PUSH(a, last, depth, -1); 434: last = a, depth += 1, limit = ss_ilg(a - first); 435: } else { 436: first = a, limit = -1; </pre>	
	污染路径: 无	
128	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 133
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: <pre> 131: 132: if ((bitStream & (threshold-1)) < (U32)max) { 133: count = bitStream & (threshold-1); 134: bitCount += nbBits-1; 135: } else { </pre>	
	污染路径: 无	
129	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 130
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: <pre> 128: // insert a new word 129: bool insertWord(STire* tire, char* word) { 130: int len = strlen(word); </pre>	

	131: if (len >= MAX_WORD_LEN) { 132: return false;	
	污染路径: 无	
130	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 194
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 192: for (int j = 0; j < ROW_NUMS; j++) { 193: ts[i][j] = 1591060628000 + j; 194: b[i][j] = 'a' + j; 195: } 196: }	
	污染路径: 无	
131	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1233
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 1231: { 1232: count = (short)(bitStream & (2*threshold-1)); 1233: if (count >= threshold) count -= max; 1234: bitCount += nbBits; 1235: }	
	污染路径: 无	
132	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 2313
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 2311: char rightv[3][5] = {0}; 2312: for (int32_t i = 0; i < 5; ++i) { 2313: leftv[i][2] = 'a' + i; 2314: leftv[i][3] = 'b' + i; 2315: leftv[i][4] = '0' + i;	
	污染路径:	

	无	
133	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 498
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: <pre> 496: if (hbSize < 4) return (size_t)- FSE_ERROR_srcSize_wrong; 497: bitStream = FSE_readLE32(ip); 498: nbBits = (bitStream & 0xF) + FSE_MIN_TABLELOG; /* extract tableLog */ 499: if (nbBits > FSE_TABLELOG_ABSOLUTE_MAX) return (size_t)-FSE_ERROR_tableLog_tooLarge; 500: bitStream >>= 4; </pre>	
	污染路径: 无	
134	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 557
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: <pre> 555: { 556: count = (short)(bitStream & (2*threshold-1)); 557: if (count >= threshold) count -= max; 558: bitCount += nbBits; 559: } </pre>	
	污染路径: 无	
135	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 280
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: <pre> 278: int m = 0; 279: STireNode* c = 0; 280: int len = strlen(prefix); 281: if (len >= MAX_WORD_LEN) { 282: return; </pre>	
	污染路径: 无	

136	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 265
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 263: l++; 264: if (name == NULL) 265: return l - long_options; 266: else 267: return -1;	
	污染路径: 无	
137	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 2171
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 2169: if (targetLog-nbBits >= minBits) { /* enough room for a second symbol */ 2170: U32 sortedRank; 2171: int minWeight = nbBits + scaleLog; 2172: if (minWeight < 1) minWeight = 1; 2173: sortedRank = rankStart[minWeight];	
	污染路径: 无	
138	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 600
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 598: sprintf(secondLine, " * The %s CLI supports tab completion for a variety of items, ", shell.info.cusName); 599: printf("%s", secondLine); 600: int secondLineLen = strlen(secondLine); 601: while (89 - (secondLineLen++) > 0) { 602: printf(" ");	
	污染路径: 无	
139	缺陷发生位置:	行号:

	/test/new_test_framework/script/api/stmt2-blob-performance.c		97
	步骤描述: 从'int'到类型'char'的缩小转换		
	代码片段: 95: for (int j = 0; j < ROW_NUMS; j++) { 96: ts[i][j] = 1591060628000 + r * 100000 + j; 97: b[i][j] = 'a' + j; 98: } 99: }		
	污染路径: 无		
140	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c		行号: 229
	步骤描述: 从'unsigned int'到类型'int'的缩小转换		
	代码片段: 227: remaining = tableSize+1; /* +1 for extra accuracy */ 228: threshold = tableSize; 229: nbBits = tableLog+1; 230: 231: while (remaining>1) { /* stops at 1 */		
	污染路径: 无		
141	缺陷发生位置: /tools/taos-tools/src/benchData.c		行号: 187
	步骤描述: 从'unsigned int'到类型'char'的缩小转换		
	代码片段: 185: ret = 2; 186: } else if (unic <= 0x0000FFFF) { 187: *(p + 2) = (unic & 0x3F) 0x80; 188: *(p + 1) = ((unic >> 6) & 0x3F) 0x80; 189: *p = ((unic >> 12) & 0x0F) 0xE0;		
	污染路径: 无		
142	缺陷发生位置: /source/util/src/tcompare.c		行号: 1183
	步骤描述: 从'int'到类型'char'的缩小转换		
	代码片段:		

	1181: } 1182: 1183: char rejectList[2] = {toupper(c), tolower(c)}; 1184: const char *remainStr = str + nMatchChar; 1185: int32_t remainSize = ssize - nMatchChar;	
	污染路径: 无	
143	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 171
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 169: 170: v.blob[i][0] = 'a' + i; 171: v.blob2[i][0] = 'f' + i; 172: v.blob3[i][0] = 't' + i; 173: v.blob4[i][0] = 'A' + i;	
	污染路径: 无	
144	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 898
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 896: int smallBatchCount = 0; 897: int index= pThreadInfo->start_table_from; 898: int tableSum = pThreadInfo->end_table_to - pThreadInfo->start_table_from + 1; 899: if (stbInfo->useTagTableName) { 900: pThreadInfo->childNames = benchCalloc(tableSum, sizeof(char *), false);	
	污染路径: 无	
145	缺陷发生位置: /source/common/test/cosCpTest.cpp	行号: 41
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 39: const int64_t MULTIPART_CHUNK_SIZE = 64 << 20; // multipart is 768M	

	40: uint64_t chunk_size = MULTIPART_CHUNK_SIZE >> 3; 41: int totalSeq = (contentLength + chunk_size - 1) / chunk_size; 42: const int max_part_num = 10000; 43: if (totalSeq > max_part_num) {	
	污染路径: 无	
146	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 98
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 96: quantization_intervals = exe_params->intvCapacity; 97: //updateQuantizationInfo(quantization_intervals); 98: int intvRadius = quantization_intervals/2; 99: 100: size_t i;	
	污染路径: 无	
147	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 395
	步骤描述: 从'const int'到类型'char'的缩小转换	
	代码片段: 393: /* OPT can be used as a short option. */ 394: { 395: *cvt->short_end++ = opt->key; 396: if (real->arg) 397: {	
	污染路径: 无	
148	缺陷发生位置: /contrib/TSZ/sz/src/CompressElement.c	行号: 106
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 104: 105: if(oriValue>=0)	

	106: *curGroupID = groupNum+2; //+[-1,0,1,2,3,.....,16] is mapped to [1,2,.....,18] 107: else 108: *curGroupID = -(groupNum+2); //[-1,0,1,2,3,.....,16] is mapped to [-1,-2,.....,-18]	
	污染路径: 无	
149	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1588
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1586: int64_t startTimestamp = stbInfo->startTimestamp; 1587: if (stbInfo->disorderRatio > 0) { 1588: int rand_num = taosRandom() % 100; 1589: if (rand_num < stbInfo->disorderRatio) { 1590: (*disorderRange)--;	
	污染路径: 无	
150	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 172
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 170: v.blob[i][0] = 'a' + i; 171: v.blob2[i][0] = 'f' + i; 172: v.blob3[i][0] = 't' + i; 173: v.blob4[i][0] = 'A' + i; 174: v.blob5[i][0] = 'G' + i;	
	污染路径: 无	
151	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 577
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 575: } 576: 577: ret = fwrite(buf, 1, length, fp); 578: if (ret != length) {	

	579: errorPrint("Failed to write create stb sql: %s. expected written %d but %d.\n",	
	污染路径: 无	
152	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 2114
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 2112: if (targetLog-nbBits >= minBits) { /* enough room for a second symbol */ 2113: U32 sortedRank; 2114: int minWeight = nbBits + scaleLog; 2115: if (minWeight < 1) minWeight = 1; 2116: sortedRank = rankStart[minWeight];	
	污染路径: 无	
153	缺陷发生位置: /source/common/test/cosCpTest.cpp	行号: 45
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 43: if (totalSeq > max_part_num) { 44: chunk_size = (contentLength + max_part_num - contentLength % max_part_num) / max_part_num; 45: totalSeq = (contentLength + chunk_size - 1) / chunk_size; 46: } 47: SCheckpoint cp;	
	污染路径: 无	
154	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1279
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 1277: if (hbSize < 4) return ERROR(srcSize_wrong); 1278: bitStream = MEM_readLE32(ip); 1279: nbBits = (bitStream & 0xF) + FSEv06_MIN_TABLELOG; /* extract tableLog */	

	1280: if (nbBits > FSEv06_TABLELOG_ABSOLUTE_MAX) return ERROR(tableLog_tooLarge); 1281: bitStream >>= 4;	
	污染路径: 无	
155	缺陷发生位置: /source/libs/tdb/src/db/tdbDb.c	行号: 29
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 27: *ppDb = NULL; 28: 29: dsize = strlen(dbname); 30: zsize = sizeof(*pDb) + dsize * 2 + strlen(TDB_JOURNAL_NAME) + 3; 31:	
	污染路径: 无	
156	缺陷发生位置: /contrib/TSZ/sz/src/CompressElement.c	行号: 24
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 22: INLINE short computeGroupNum_float(float value) 23: { 24: short expo = getExponent_float(value); 25: if(expo < 0) 26: expo = -1;	
	污染路径: 无	
157	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1716
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1714: 1715: // show 1716: int count = strlen(part); 1717: shellInsertStr(cmd, part, count); 1718: cntDel = count; // next press tab delete current	

	append count	
	污染路径: 无	
158	缺陷发生位置: /test/new_test_framework/script/api/insertBlob.c	行号: 89
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 87: 88: for (i = 0; i < BLOB_LEN - 1000; ++i) { 89: buf[i] = 'a' + (i % 26); // fill with 'a' to 'z' 90: } 91:	
	污染路径: 无	
159	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 855
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 853: { 854: char first1 = short1 ? short1 : long1 ? *long1 : 0; 855: char first2 = short2 ? short2 : long2 ? *long2 : 0; 856: #ifdef _tolower 857: int lower_cmp = _tolower (first1) - _tolower (first2);	
	污染路径: 无	
160	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1576
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 1574: } 1575: 1576: resp_len = response_length - 1; 1577: received = 0; 1578:	
	污染路径:	

	无	
161	缺陷发生位置: /contrib/TSZ/sz/src/TightDataPointStorageF.c	行号: 206
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 204: if ((*this)->ifAdtFse == 0) { 205: // huffman 206: int stateNum = 2 * intervals; 207: HuffmanTree* huffmanTree = createHuffmanTree(stateNum); 208: encode_withTree(huffmanTree, type, dataSeriesLength, &(*this)->typeArray, &(*this)- >typeArray_size);	
	污染路径: 无	
162	缺陷发生位置: /source/libs/index/src/index.c	行号: 865
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 863: SERIALIZE_STR_MEM_TO_BUF(buf, key, colName, key->nColName); 864: } 865: return buf - p; 866: }	
	污染路径: 无	
163	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1342
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 1340: } else { 1341: count = (short)(bitStream & (2*threshold-1)); 1342: if (count >= threshold) count -= max; 1343: bitCount += nbBits; 1344: }	
	污染路径: 无	

164	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 799
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 797: for (int i = 0; i < 7; ++i) { 798: // sprintf(&key[count - 2], "%c", i); 799: key[count - 2] = '0' + i; 800: 801: ret = tdbTbInsert(pDb, key, count, NULL, 0, txn);	
	污染路径: 无	
165	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 77
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 75: { 76: if (__fs->p < __fs->end _argp_fmtstream_ensure (__fs, 1)) 77: return *__fs->p++ = __ch; 78: else 79: return EOF;	
	污染路径: 无	
166	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 2314
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 2312: for (int32_t i = 0; i < 5; ++i) { 2313: leftv[i][2] = 'a' + i; 2314: leftv[i][3] = 'b' + i; 2315: leftv[i][4] = '0' + i; 2316: varDataSetLen(leftv[i], 3);	
	污染路径: 无	
167	缺陷发生位置: /source/libs/new-stream/test/dataSinkTest.cpp	行号: 925

	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 923: 924: // Assign to different queues according to groupId 925: int queueIdx = groupId % consumerCount; 926: { 927: std::lock_guard<std::mutex> lock(queueMutexes[queueIdx]);	
	污染路径: 无	
168	缺陷发生位置: /source/common/test/cosCpTest.cpp	行号: 85
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 83: if (totalSeq > max_part_num) { 84: chunk_size = (contentLength + max_part_num - contentLength % max_part_num) / max_part_num; 85: totalSeq = (contentLength + chunk_size - 1) / chunk_size; 86: } 87: SCheckpoint cp;	
	污染路径: 无	
169	缺陷发生位置: /contrib/lemon/lemon.c	行号: 2157
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 2155: case OPT_INT: 2156: case OPT_FINT: 2157: lv = strtol(cp,&end,0); 2158: if(*end){ 2159: if(err){	
	污染路径: 无	
170	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1155
	步骤描述:	

	从'int'到类型'short'的缩小转换	
	代码片段: 1153: static short FSE_abs(short a) 1154: { 1155: return a<0 ? -a : a; 1156: } 1157:	
	污染路径: 无	
171	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 555
	步骤描述: 从'const int'到类型'char'的缩小转换	
	代码片段: 553: { 554: entry->num++; 555: if (oshort (o) && ! find_char (o->key, hol->short_options, so)) 556: /* O has a valid short option which hasn't already been used.*/ 557: *so++ = o->key;	
	污染路径: 无	
172	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 424
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 422: however)... */ 423: cvt->long_end->val = 424: ((opt->key real->key) & USER_MASK) 425: + (((group - cvt->parser->groups) + 1) << USER_BITS); 426:	
	污染路径: 无	
173	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 97

	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 95: for (int j = 0; j < ROW_NUMS; j++) { 96: ts[i][j] = 1591060628000 + r * 100000 + j; 97: b[i][j] = 'a' + j; 98: } 99: }</pre>	
	污染路径: 无	
174	缺陷发生位置:	行号:
	/source/client/src/clientFirewall.c	147
	步骤描述: 从'int'到类型'char'的缩小转换	
175	代码片段: <pre> 145: 146: // Copy other characters (convert to lowercase) 147: *out = tolower(*p); 148: out++; 149: outLen++;</pre>	
	污染路径: 无	
	缺陷发生位置:	行号:
176	/contrib/TSZ/sz/src/sz_float.c	97
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段:	

	<pre> 95: quantization_intervals = exe_params- >intvCapacity; 96: //updateQuantizationInfo(quantization_intervals); 97: int half_interval = quantization_intervals/2; 98: 99: // </pre>	
	污染路径: 无	
177	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 675
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 673: hol_entry_first_short (const struct hol_entry *entry) 674: { 675: return hol_entry_short_iterate (entry, until_short, 676: entry->argp->argp_domain, 0); 677: } </pre>	
	污染路径: 无	
178	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 91
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: <pre> 89: memset(normalizedCounter, 0, (*maxSVPtr+1) * sizeof(normalizedCounter[0])); /* all symbols not present in NCount have a frequency of 0 */ 90: bitStream = MEM_readLE32(ip); 91: nbBits = (bitStream & 0xF) + FSE_MIN_TABLELOG; /* extract tableLog */ 92: if (nbBits > FSE_TABLELOG_ABSOLUTE_MAX) return ERROR(tableLog_tooLarge); 93: bitStream >>= 4; </pre>	
	污染路径: 无	
179	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 174
	步骤描述:	

	从'int'到类型'char'的缩小转换	
	代码片段: 172: v.blob3[i][0] = 't' + i; 173: v.blob4[i][0] = 'A' + i; 174: v.blob5[i][0] = 'G' + i; 175: 176: // v.blob2[i] = malloc(strlen("一二三四五六七十九八"));	
	污染路径: 无	
180	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v06.c	1325
	步骤描述: 从'int'到类型'short'的缩小转换	
	代码片段: 1323: } else { 1324: count = (short)(bitStream & (2*threshold-1)); 1325: if (count >= threshold) count -= max; 1326: bitCount += nbBits; 1327: }	
181	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/test/cos/main.c	1299
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1297: str = (char *)cos_palloc(p, len + 1); 1298: for (; i < len; i++) { 1299: str[i] = 'a' + rand() % 32; 1300: } 1301: str[len] = '\0';	
182	污染路径: 无	
	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmt2-nchar.c	173
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 171: v.blob2[i][0] = 'f' + i; 172: v.blob3[i][0] = 't' + i;	

	173: v.blob4[i][0] = 'A' + i; 174: v.blob5[i][0] = 'G' + i; 175:	
	污染路径: 无	
183	缺陷发生位置:	行号:
	/source/libs/index/src/indexTfile.c	421
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 419: int ret = 0; 420: char* p = idxPackJsonData(tem); 421: int sz = strlen(p); 422: 423: int64_t st = taosGetTimestampUs();	
184	污染路径: 无	
	缺陷发生位置:	行号:
	/source/util/src/tcompression.c	414
	步骤描述: 从'int'到类型'char'的缩小转换	
184	代码片段: 412: elems + 1 <= selector_to_elems[(int32_t) (bit_to_selector[(int32_t)tmp_bit])) { 413: // If can hold another one. 414: selector = selector > bit_to_selector[(int32_t)tmp_bit] ? selector : bit_to_selector[(int32_t)tmp_bit]; 415: elems++; 416: bit = bit_per_integer[(int32_t)selector];	
	污染路径: 无	

9.2.1.3 WK4753 禁止使用无实质作用的类型转换

缺陷数量: 140 个
缺陷描述:

禁止使用无实质作用的类型转换。		
修复意见： 无		
1	缺陷发生位置： /source/util/src/tlog.c	行号： 1192
	步骤描述： 强制转换为相同类型.	
	代码片段： 1190: static void *taosAsyncOutputLog(void *param) { 1191: SLogBuff *pLogBuf = (SLogBuff *)tsLogObj.logHandle; 1192: SLogBuff *pSlowBuf = (SLogBuff *)tsLogObj.slowHandle; 1193: 1194: setThreadName("log");	
	污染路径： 无	
2	缺陷发生位置： /utils/test/c/varbinary_test.c	行号： 464
	步骤描述： 强制转换为相同类型.	
	代码片段： 462: params[j].is_null = ((i == j) ? &is_null : 0); 463: } 464: v.b = (int8_t)i % 2; 465: 466: for (int j = 0; j < vbinLen; ++j) {	
	污染路径： 无	
3	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号： 181
	步骤描述： 强制转换为相同类型.	
	代码片段： 179: fprintf (stderr, "Timed out: the child process was %s\ n", 180: strsignal (WSTOPSIG (status))); 181: else if (WIFSIGNALED (status)) 182: fprintf (stderr, "Timed out: the child process got signal %s\ n", 183: strsignal (WTERMSIG (status)));	

	污染路径: expanded from macro 'WIFSIGNALED' expanded from macro '___WIFSIGNALED'	
4	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 3589
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 3587: char *binTemp = calloc(1, 1+len); 3588: TOOLS_ASSERT(binTemp); 3589: strncpy(binTemp, (char*)value, len); 3590: avro_value_set_string(&branch, binTemp); 3591: free(binTemp); </pre>	
	污染路径: 无	
5	缺陷发生位置: /source/util/src/tcompression.c	行号: 442
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 440: break; 441: case TSDB_DATA_TYPE_BIGINT: 442: curr_value = (int64_t)((int64_t *)input + i)); 443: break; 444: } </pre>	
	污染路径: 无	
6	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 126
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 124: 125: // Check for number (including floats like 25.5) 126: if (isdigit(*p) (*p == '-' && isdigit(*(p + 1)))) { 127: if (!inNumber) { 128: // Start of number </pre>	
	污染路径: expanded from macro 'isdigit'	

	expanded from macro '__isctype'	
7	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 941
	步骤描述: 强制转换为相同类型.	
	代码片段: 939: (void)setConsoleEcho(true); 940: } else { 941: strcpy(arguments->password, (char *)(argv[i] + 2)); 942: strcpy(argv[i], "-p"); 943: }	
	污染路径: 无	
8	缺陷发生位置: /source/util/src/tunit.c	行号: 80
	步骤描述: 强制转换为相同类型.	
	代码片段: 78: int64Val *= factor; 79: } 80: while (isspace((unsigned char)*endPtr)) endPtr++; 81: if (*endPtr) { 82: return terrno = TSDB_CODE_INVALID_CFG_VALUE;	
	污染路径: 无	
9	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 246
	步骤描述: 强制转换为相同类型.	
	代码片段: 244: while (*var) 245: { 246: SKIPWS (var); 247: 248: if (isalpha (*var))	
	污染路径: expanded from macro 'SKIPWS' expanded from macro 'isspace'	

	expanded from macro '__isctype'	
10	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 528
	步骤描述: 强制转换为相同类型.	
	代码片段: 526: 527: unsigned long long endtime = getCurrentTime(); 528: unsigned long long totalRows = (uint32_t)(totalRowsPerTbl * tableNum); 529: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL, (endtime-starttime)/totalRows); 530:	
	污染路径: 无	
11	缺陷发生位置: /test/new_test_framework/script/api/stmt-crash.c	行号: 209
	步骤描述: 强制转换为相同类型.	
	代码片段: 207: code = taos_stmt_set_tbname_tags(stmt, "m2", tags); 208: v.ts = 1591060628000 + 1; 209: v.b = (int8_t)1 % 2; 210: v.v1 = (int8_t)1; 211: v.v2 = (int16_t)(1 * 2);	
	污染路径: 无	
12	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 912
	步骤描述: 强制转换为相同类型.	
	代码片段: 910: static tools_cJSON_bool print_string(const tools_cJSON * const item, printbuffer * const p) 911: { 912: return print_string_ptr((unsigned char*)item->valuelstring, p); 913: } 914:	

	污染路径: 无	
13	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 1694
	步骤描述: 强制转换为相同类型.	
	代码片段: 1692: assert(T[s - 1] <= T[s]); 1693: c0 = T[--s]; 1694: *j = ~((int)c0); 1695: if((0 < s) && (T[s - 1] > c0)) { s = ~s; } 1696: if(c0 != c2) {	
	污染路径: 无	
14	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 990
	步骤描述: 强制转换为相同类型.	
	代码片段: 988: + strlen("--databases=")); 989: } else if (0 == strncmp(argv[i], "-D", strlen("-D")))) { 990: arguments->databasesSeq = (char *)(argv[i] + strlen("-D")); 991: } else if (strlen("--databases") == strlen(argv[i])) { 992: if (argc == i+1) {	
	污染路径: 无	
15	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 375
	步骤描述: 强制转换为相同类型.	
	代码片段: 373: struct timeval systemTime; 374: taosGetTimeOfDay(&systemTime); 375: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 376: } 377:	

	污染路径: 无	
16	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 221
	步骤描述: 强制转换为相同类型.	
	代码片段: 219: 220: unsigned long long endtime = getCurrentTime(); 221: unsigned long long totalRows = (uint32_t)(totalRowsPerTbl * tableNum); 222: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL, (endtime-starttime)/totalRows); 223:	
	污染路径: 无	
17	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1198
	步骤描述: 强制转换为相同类型.	
	代码片段: 1196: " table(s), actual %" PRId64 " table(s) pre created, %" PRId64 1197: " table(s) will be auto created\n", 1198: (float)(end - start) / 1000.0, 1199: g_arguments->totalChildTables, 1200: g_arguments->table_threads,	
	污染路径: expanded from macro 'succPrint'	
18	缺陷发生位置: /source/os/src/osSocket.c	行号: 197
	步骤描述: 强制转换为相同类型.	
	代码片段: 195: serverAdd.sin_addr.s_addr = ip; 196: #endif 197: serverAdd.sin_port = (uint16_t)htons(port); 198:	

	199: fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);	
	污染路径: 无	
19	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 3285
	步骤描述: 强制转换为相同类型.	
	代码片段: 3283: if (srcSize < MIN_CBLOCK_SIZE) return ERROR(corruption_detected); 3284: 3285: switch((litBlockType_t)(istart[0]>> 6)) 3286: { 3287: case lbt_huffman:	
	污染路径: 无	
20	缺陷发生位置: /utils/test/c/tmq_td37265.c	行号: 78
	步骤描述: 强制转换为相同类型.	
	代码片段: 76: float current = (float)0.0001f * j; 77: int voltage = j; 78: float phase = (float)0.0001f * j; 79: params[0].buffer = &ts; 80: params[1].buffer = ¤t;	
	污染路径: 无	
21	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 1188
	步骤描述: 强制转换为相同类型.	
	代码片段: 1186: 1187: int32_t numOfCols = taosArrayGetSize(pBlock->pDataBlock); 1188: char* isNull = (char*)buf; 1189: char* pStart = (char*)buf + sizeof(int8_t) * numOfCols; 1190: for (int32_t i = 0; i < numOfCols; ++i) {	

	污染路径: 无	
22	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 1273
	步骤描述: 强制转换为相同类型.	
	代码片段: 1271: size_t numOfCols = taosArrayGetSize(pBlock->pDataBlock); 1272: 1273: char* isNull = (char*)buf; 1274: char* pStart = (char*)buf + sizeof(int8_t) * numOfCols; 1275: for (int32_t i = 0; i < numOfCols; ++i) {	
	污染路径: 无	
23	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 259
	步骤描述: 强制转换为相同类型.	
	代码片段: 257: var_len = arg - var; 258: 259: SKIPWS (arg); 260: 261: if (*arg == '\0' *arg == ',')	
	污染路径: expanded from macro 'SKIPWS' expanded from macro 'isspace' expanded from macro ' __isctype'	
24	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 576
	步骤描述: 强制转换为相同类型.	
	代码片段: 574: if(cmpSysEndianType!=(unsigned char)sysEndianType) 575: { 576: unsigned char* p = (unsigned char*)	

	(bytes+1+2*nodeCount*sizeof(unsigned char)); 577: size_t i = 0, size = nodeCount*sizeof(unsigned int); 578: while(1)	
	污染路径: 无	
25	缺陷发生位置: /source/client/src/clientTmq.c	行号: 711
	步骤描述: 强制转换为相同类型.	
	代码片段: 709: } 710: SMqCommitCbParam* pParam = (SMqCommitCbParam*)param; 711: SMqCommitCbParamSet* pParamSet = (SMqCommitCbParamSet*)pParam->params; 712: 713: return commitRspCountDown(pParamSet, pParam-> >consumerId, pParam->topicName, pParam->vgId);	
	污染路径: 无	
26	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 989
	步骤描述: 强制转换为相同类型.	
	代码片段: 987: 988: int retConn = connect(989: sockfd, (struct sockaddr *)&(g_arguments-> >serv_addr), 990: sizeof(struct sockaddr)); 991: infoPrint("createSockFd call connect serv_addr=%p retConn=%d\n", &g_arguments->serv_addr, retConn);	
	污染路径: 无	
27	缺陷发生位置: /source/os/src/osSocket.c	行号: 709
	步骤描述: 强制转换为相同类型.	
	代码片段:	

	<pre> 707: #else // Linux like systems 708: uint32_t conn_timeout_ms = timeout; 709: if (-1 == setsockopt(fd, IPPROTO_TCP, TCP_USER_TIMEOUT, (char *)&conn_timeout_ms, sizeof(conn_timeout_ms))) { 710: terrno = TAOS_SYSTEM_ERROR(ERRNO); 711: TAOS_SKIP_ERROR(taosCloseSocketNoCheck1(fd)); </pre>	
	污染路径: 无	
28	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 301
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 299: params[i].sql = g_sql_statements[i]; 300: } else { 301: params[i].sql = (const char*)sql; 302: } 303: params[i].queries_per_thread = queries_per_thread; </pre>	
	污染路径: 无	
29	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 289
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 287: 288: p = buf + (r + 1 - fs->point_col); 289: while (p >= buf && !isblank (*p)) 290: --p; 291: nextline = p + 1; /* This will begin the next line. */ </pre>	
	污染路径: expanded from macro 'isblank' expanded from macro '___isctype'	
30	缺陷发生位置: /source/util/src/tunit.c	行号: 40
	步骤描述: 强制转换为相同类型.	
	代码片段:	

	38: return terrno = TSDB_CODE_INVALID_CFG_VALUE; 39: } 40: while (isspace((unsigned char)*endPtr)) endPtr++; 41: uint64_t factor = 1; 42: if (*endPtr != '\0') {	
	污染路径: 无	
31	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 49
	步骤描述: 强制转换为相同类型.	
	代码片段: 47: } 48: 49: return (uint64_t)tv.tv_sec * 1000000ULL + (uint64_t)tv.tv_usec; 50: } 51:	
	污染路径: 无	
32	缺陷发生位置: /test/new_test_framework/script/api/insert_stb.c	行号: 74
	步骤描述: 强制转换为相同类型.	
	代码片段: 72: for (int tblIdx = 0; tblIdx < 10; ++tblIdx) { 73: int len = 0; 74: len += sprintf(qstr+len, "insert into ct%d using st tags(%d, %f, '%s')", tblIdx, tblIdx, (float)tblIdx, "childtable"); 75: int batchSize = len; 76: for (int batchIdx = 0; batchIdx < 10000; ++batchIdx) {	
	污染路径: 无	
33	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2148
	步骤描述: 强制转换为相同类型.	
	代码片段: 2146: tools_cJSON_GetArrayItem(2147: pThreadInfo->sml_json_tags,	

	2148: (int)tableSeq - 2149: pThreadInfo- >start_table_from), 2150: true);	
	污染路径: 无	
34	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1599
	步骤描述: 强制转换为相同类型.	
	代码片段: 1597: 1598: /* print key */ 1599: if (!print_string_ptr((unsigned char*)current_item->string, output_buffer)) 1600: { 1601: return false;	
	污染路径: 无	
35	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 666
	步骤描述: 强制转换为相同类型.	
	代码片段: 664: if(cmpSysEndianType!=(unsigned char)sysEndianType) 665: { 666: unsigned char* p = (unsigned char*)(bytes+1); 667: size_t i = 0, size = 3*nodeCount*sizeof(unsigned int); 668: while(1)	
	污染路径: 无	
36	缺陷发生位置: /contrib/lemon/lemon.c	行号: 2150
	步骤描述: 强制转换为相同类型.	
	代码片段: 2148: fprintf(err, 2149: "%sillegal character in floating-point	

	<pre> argument.\n",errmsg); 2150: errline(i,(int)((char*)end-(char*)g_argv[i]),err); 2151: } 2152: errcnt++; </pre>	
	污染路径: 无	
37	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 381
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 379: struct timeval systemTime; 380: taosGetTimeOfDay(&systemTime); 381: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 382: } 383: </pre>	
	污染路径: 无	
38	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 4179
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 4177: &item_value, NULL); 4178: avro_value_get_int(&item_value, &n32tmp); 4179: array_u8 += (int8_t)n32tmp; 4180: } 4181: </pre>	
	污染路径: 无	
39	缺陷发生位置: /test/new_test_framework/script/api/stmt-crash.c	行号: 194
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 192: code = taos_stmt_set_tbname_tags(stmt, "m1", tags); 193: v.ts = 1591060628000; 194: v.b = (int8_t)0 % 2; </pre>	

	195: v.v1 = (int8_t)0; 196: v.v2 = (int16_t)(0 * 2);	
	污染路径: 无	
40	缺陷发生位置: /source/util/src/tlog.c	行号: 1191
	步骤描述: 强制转换为相同类型.	
	代码片段: 1189: 1190: static void *taosAsyncOutputLog(void *param) { 1191: SLogBuff *pLogBuf = (SLogBuff *)tsLogObj.logHandle; 1192: SLogBuff *pSlowBuf = (SLogBuff *)tsLogObj.slowHandle; 1193:	
	污染路径: 无	
41	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 914
	步骤描述: 强制转换为相同类型.	
	代码片段: 912: 913: static int tfileStrCompare(const void* a, const void* b) { 914: int ret = strcmp((char*)a, (char*)b); 915: if (ret == 0) { 916: return ret;	
	污染路径: 无	
42	缺陷发生位置: /source/util/src/tcompression.c	行号: 197
	步骤描述: 强制转换为相同类型.	
	代码片段: 195: const char type, int8_t lvl) { 196: uLongf dstLen = outputSize - 1; 197: int32_t ret = compress2((Bytef*)(output + 1), (uLongf *)&dstLen, (Bytef *)input, (uLong)inputSize, lvl); 198: if (ret == Z_OK) { 199: output[0] = 1;	

	污染路径: 无	
43	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 86
	步骤描述: 强制转换为相同类型.	
	代码片段: 84: last = l + strlen(l); 85: while (last > l) { 86: if (!isspace((int)*(last-1))) 87: break ; 88: last -- ;	
	污染路径: expanded from macro 'isspace' expanded from macro '___isctype'	
44	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1809
	步骤描述: 强制转换为相同类型.	
	代码片段: 1807: assert(strcmp(lem.symbols[i-1]->name,"{default}")==0); 1808: lem.nsymbol = i - 1; 1809: for(i=1; ISUPPER(lem.symbols[i]->name[0]); i++); 1810: lem.terminal = i; 1811:	
	污染路径: expanded from macro 'ISUPPER' expanded from macro 'isupper' expanded from macro '___isctype'	
45	缺陷发生位置: /source/os/src/osRand.c	行号: 66
	步骤描述: 强制转换为相同类型.	
	代码片段: 64: int64_t len = taosReadFile(pFile, &seed, sizeof(seed)); 65: if (len < 0) { 66: seed = (int)taosGetTimestampSec(); 67: } 68: TAOS_SKIP_ERROR(taosCloseFile(&pFile));	

	污染路径: 无	
46	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 960
	步骤描述: 强制转换为相同类型.	
	代码片段: 958: 959: buffer.content = (const unsigned char*)value; 960: buffer.length = strlen((const char*)value) + sizeof(""); 961: buffer.offset = 0; 962: buffer.hooks = global_hooks;	
	污染路径: 无	
47	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 266
	步骤描述: 强制转换为相同类型.	
	代码片段: 264: { 265: arg++; 266: SKIPWS (arg); 267: } 268:	
	污染路径: expanded from macro 'SKIPWS' expanded from macro 'isspace' expanded from macro '___isctype'	
48	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.c	行号: 90
	步骤描述: 强制转换为相同类型.	
	代码片段: 88: TSDB_CHECK_CODE(code = TSDB_CODE_NOT_FOUND, lino, _error); 89: } 90: SCacheItem *pItem = (SCacheItem *)*ppItem; 91: 92: pItem->pNode = tdListPopNode(pCache->lruList, pItem-	

	>pNode);	
	污染路径: 无	
49	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 402
	步骤描述: 强制转换为相同类型.	
	代码片段: 400: struct timeval systemTime; 401: taosGetTimeOfDay(&systemTime); 402: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 403: } 404:	
	污染路径: 无	
50	缺陷发生位置: /contrib/lemon/lemon.c	行号: 183
	步骤描述: 强制转换为相同类型.	
	代码片段: 181: if(ISDIGIT(c) (c=='-' && ISDIGIT(zFormat[i+1]))){ 182: if(c=='-') i++; 183: while(ISDIGIT(zFormat[i])) iWidth = iWidth*10 + zFormat[i++] - '0'; 184: if(c=='-') iWidth = -iWidth; 185: c = zFormat[i];	
	污染路径: expanded from macro 'ISDIGIT' expanded from macro 'isdigit' expanded from macro ' __isctype'	
51	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 489
	步骤描述: 强制转换为相同类型.	
	代码片段: 487: if (FSE_isError(errorCode)) return errorCode; 488: } 489: else normalizedCounter[largest] += (short)stillToDistribute;	

	490: } 491:	
	污染路径: 无	
52	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 255
	步骤描述: 强制转换为相同类型.	
	代码片段: 253: const char *arg = var; 254: 255: while (isalnum (*arg) *arg == '-' *arg == '_') 256: arg++; 257: var_len = arg - var;	
	污染路径: expanded from macro 'isalnum' expanded from macro '___isctype'	
53	缺陷发生位置: /test/new_test_framework/script/api/insert_stb.c	行号: 122
	步骤描述: 强制转换为相同类型.	
	代码片段: 120: for (int rowIdx = 0; rowIdx < 100; ++rowIdx) { 121: int i = rowIdx + batchIdx * 100 + tblIdx*10000*100; 122: len += sprintf(qstr+len, " ('ct%d', %d, %f, '%s', %" PRId64 " , %d, %d, %d, %d, %f, %lf, '%s')", tblIdx, tblIdx, (float)tblIdx, "childtable", 123: (uint64_t)(1546300800000 + i), (int8_t)i, (int16_t)i, i, i, i*1.0, i*2.0, "hello"); 124: } 	
	污染路径: 无	
54	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 310
	步骤描述: 强制转换为相同类型.	
	代码片段: 308: do	

	<pre> 309: ++p; 310: while (p < nl && !isblank (*p)); 311: if (p == nl) 312: { </pre>	
	污染路径: expanded from macro 'isblank' expanded from macro ' __isctype'	
55	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 440
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 438: double maxDelay = *(int64_t *) (benchArrayGet(delay_list, (int32_t)(delay_list->size - 1)))/1E6; 439: double p90 = *(int64_t *) (benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.90)))/1E6; 440: double p95 = *(int64_t *) (benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.95)))/1E6; 441: double p99 = *(int64_t *) (benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.99)))/1E6; 442: </pre>	
	污染路径: 无	
56	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1450
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 1448: tools_cJSON_GetObjectItem(stbInfo, "disorder_fill_interval"); 1449: if (tools_cJSON_IsNumber(disFill)) { 1450: superTable->fillIntervalDis = (int)disFill- >valueint; 1451: } 1452: </pre>	
	污染路径: 无	
57	缺陷发生位置: /docs/examples/c/stmt2_insert_demo.c	行号: 176
	步骤描述:	

	强制转换为相同类型.	
	代码片段: 174: 175: for (int j = 0; j < NUM_OF_ROWS; j++) { 176: ts[j] = 1609459200000LL + (int64_t)i * NUM_OF_ROWS * 1000LL + (int64_t)j * 1000LL; 177: current[j] = 10.0f + (float)j * 0.5f; 178: voltage[j] = 100 + j * 10;	
	污染路径: 无	
58	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 730
	步骤描述: 强制转换为相同类型.	
	代码片段: 728: 729: unsigned long long endtime = getCurrentTime(); 730: unsigned long long totalRows = (uint32_t) (totalRowsPerTbl * tableNum); 731: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL, (endtime-starttime)/totalRows); 732:	
	污染路径: 无	
59	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 81
	步骤描述: 强制转换为相同类型.	
	代码片段: 79: if (s==NULL) return NULL ; 80: 81: while (isspace((int)*s) && *s) s++; 82: memset(l, 0, ASCIILINESZ+1); 83: strncpy(l, s, ASCIILINESZ);	
	污染路径: expanded from macro 'isspace' expanded from macro '___isctype'	
60	缺陷发生位置:	行号:

	/contrib/lemon/lemon.c		2001
	步骤描述: 强制转换为相同类型.		
	代码片段: 1999: char *set[LISTSIZE]; 2000: int i; 2001: offset = (unsigned long)((char*)next - (char*)list); 2002: for(i=0; i<LISTSIZE; i++) set[i] = 0; 2003: while(list){		
	污染路径: 无		
61	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c		行号: 1481
	步骤描述: 强制转换为相同类型.		
	代码片段: 1479: tools_cJSON_GetObjectItem(stbInfo, "generate_row_rule"); 1480: if (tools_cJSON_IsNumber(rowRule)) { 1481: superTable->genRowRule = (int8_t)rowRule->valueint; 1482: } 1483:		
	污染路径: 无		
62	缺陷发生位置: /contrib/test/rocksdb/main.c		行号: 100
	步骤描述: 强制转换为相同类型.		
	代码片段: 98: } 99: int kvDeserial(KV *kv, char *buf) { 100: char *p1 = (char *)buf; 101: // p1 += 1; 102: p1 = taosDecodeFixedU64(p1, &kv->k1);		
	污染路径: 无		
63	缺陷发生位置: /tools/taos-tools/src/benchQuery.c		行号: 441
	步骤描述:		

	强制转换为相同类型.	
	代码片段: 439: double p90 = *(int64_t*)(benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.90)))/1E6; 440: double p95 = *(int64_t*)(benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.95)))/1E6; 441: double p99 = *(int64_t*)(benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.99)))/1E6; 442: 443: // show delay min max	
	污染路径: 无	
64	缺陷发生位置:	行号:
	/source/util/src/tcompression.c	391
	步骤描述: 强制转换为相同类型.	
	代码片段: 389: break; 390: case TSDB_DATA_TYPE_BIGINT: 391: curr_value = (int64_t)((int64_t*)input + j)); 392: break; 393: }	
65	污染路径: 无	
	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchUtil.c	1429
	步骤描述: 强制转换为相同类型.	
	代码片段: 1427: // Check if the remaining characters in str1 are all whitespace 1428: while (*str1 != '\0') { 1429: if (!isblank((unsigned char)*str1)) { 1430: return -1; 1431: }	
66	污染路径: expanded from macro 'isblank' expanded from macro '__isctype'	
	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v01.c	1372

	步骤描述: 强制转换为相同类型.	
	代码片段: 1370: { 1371: const BYTE* p = (const BYTE*)memPtr; 1372: return (U16)((U16)p[0] + ((U16)p[1]<<8)); 1373: } 1374: }	
	污染路径: 无	
67	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 768
	步骤描述: 强制转换为相同类型.	
	代码片段: 766: is_null[i] = 0; 767: 768: v.b[i] = (int8_t)i % 2; 769: v.v1[i] = (int8_t)i; 770: v.v2[i] = (int16_t)(i * 2);	
	污染路径: 无	
68	缺陷发生位置: /utils/test/c/sml_test.c	行号: 1271
	步骤描述: 强制转换为相同类型.	
	代码片段: 1269: ASSERT(tmp != NULL); 1270: (void)memcpy(tmp, sql[0], strlen(sql[0])); 1271: *(char*)(tmp + 44) = 0; 1272: int32_t totalRows = 0; 1273: pRes = taos_schemaless_insert_raw(taos, tmp, strlen(sql[0]), &totalRows, TSDB_SML_LINE_PROTOCOL,	
	污染路径: 无	
69	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 344
	步骤描述: 强制转换为相同类型.	
	代码片段:	

	342: } 343: 344: memcpy((char *)((*async) + 1), label, strlen(label) + 1); 345: (*async)->label = (const char *)((*async) + 1); 346:	
	污染路径: 无	
70	缺陷发生位置: /source/os/src/osSocket.c	行号: 190
	步骤描述: 强制转换为相同类型.	
	代码片段: 188: // printf("open tcp server socket:0x%x:%hu", ip, port); 189: 190: bzero((char *)&serverAdd, sizeof(serverAdd)); 191: serverAdd.sin_family = AF_INET; 192: #ifdef WINDOWS	
	污染路径: 无	
71	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 310
	步骤描述: 强制转换为相同类型.	
	代码片段: 308: params[j].is_null = ((i == j) ? &is_null : 0); 309: } 310: v.b = (int8_t)i % 2; 311: v.v1 = (int8_t)i; 312: v.v2 = (int16_t)(i * 2);	
	污染路径: 无	
72	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 439
	步骤描述: 强制转换为相同类型.	
	代码片段: 437: double minDelay = *(int64_t *) (benchArrayGet(delay_list, 0))/1E6; 438: double maxDelay = *(int64_t *) (benchArrayGet(delay_list, (int32_t)(delay_list->size - 1)))/1E6;	

	439: double p90 = *(int64_t*)(benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.90)))/1E6; 440: double p95 = *(int64_t*)(benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.95)))/1E6; 441: double p99 = *(int64_t*)(benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.99)))/1E6;	
	污染路径: 无	
73	缺陷发生位置: /tools/taos-tools/src/toolsString.c	行号: 74
	步骤描述: 强制转换为相同类型.	
	代码片段: 72: 73: for (int i = 0; i < len; i++) { 74: if (!isdigit(input[i])) 75: return false; 76: }	
	污染路径: expanded from macro 'isdigit' expanded from macro ' __isctype'	
74	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2161
	步骤描述: 强制转换为相同类型.	
	代码片段: 2159: "%s %s %" PRIu64, 2160: pthreadInfo 2161: ->sml_tags[(int)tableSeq - 2162: pthreadInfo->start_table_from], 2163: sampleDataBuf + pos * stbInfo->lenOfCols, 	
	污染路径: 无	
75	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 176
	步骤描述: 强制转换为相同类型.	

	代码片段: 174: #endif 175: 176: if (killed == 0 (WIFSIGNALED (status) && WTERMSIG (status) == SIGKILL)) 177: fputs ("Timed out: killed the child process\n", stderr); 178: else if (WIFSTOPPED (status))	
	污染路径: expanded from macro 'WIFSIGNALED' expanded from macro ' __WIFSIGNALED'	
76	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1474
	步骤描述: 强制转换为相同类型.	
	代码片段: 1472: if (delRatio->valueint > 100) delRatio->valueint = 100; 1473: if (delRatio->valueint < 0) delRatio->valueint = 0; 1474: superTable->delRatio = (int8_t)delRatio->valueint; 1475: } 1476:	
	污染路径: 无	
77	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 119
	步骤描述: 强制转换为相同类型.	
	代码片段: 117: struct timeval systemTime; 118: rtGetTimeOfDay(&systemTime); 119: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 120: } 121:	
	污染路径: 无	
78	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1486

	步骤描述: 强制转换为相同类型.	
	代码片段: 1484: for (int n = 0, m = 0; n < userpass_buf_len;) { 1485: uint32_t oct_a = 1486: n < userpass_buf_len ? (unsigned char)userpass_buf[n++] : 0; 1487: uint32_t oct_b = 1488: n < userpass_buf_len ? (unsigned char)userpass_buf[n++] : 0;	
	污染路径: 无	
79	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmt2-test.c	396
	步骤描述: 强制转换为相同类型.	
	代码片段: 394: struct timeval systemTime; 395: taosGetTimeOfDay(&systemTime); 396: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec / 1000; 397: } 398:	
	污染路径: 无	
80	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/dictBuilder/divsufsort.c	1778
	步骤描述: 强制转换为相同类型.	
	代码片段: 1776: 1777: c0 = T[--s]; 1778: *j = ~((int)c0); 1779: if((0 < s) && (T[s - 1] > c0)) { s = ~s; } 1780: if(c0 != c2) {	
	污染路径: 无	
81	缺陷发生位置:	行号:
	/tools/taos-tools/deps/toolscJson/src/toolscJson.c	461
	步骤描述:	

	强制转换为相同类型.	
	代码片段: 459: 460: /* Check whether the original double can be recovered */ 461: if ((sscanf((char*)number_buffer, "%lg", &test) != 1) ((double)test != d)) 462: { 463: /* If not, print with 17 decimal places of precision */	
	污染路径: 无	
82	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 189
	步骤描述: 强制转换为相同类型.	
	代码片段: 187: gzipStream.avail_in = (uLong)srcLen; 188: gzipStream.next_out = (Bytef*)pDest; 189: gzipStream.avail_out = (uLong)(destLen); 190: 191: while (gzipStream.avail_in != 0 && gzipStream.total_out < (uLong)(destLen)) {	
	污染路径: 无	
83	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 242
	步骤描述: 强制转换为相同类型.	
	代码片段: 240: v.ts[i] = ts++; 241: // v.b[i] = (int8_t)i % 2; 242: v.b[i] = (int8_t)i % 10; 243: v.v1[i] = (int8_t)i; 244: v.v2[i] = (int16_t)(i * 2);	
	污染路径: 无	
84	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 438
	步骤描述:	

	强制转换为相同类型.	
	代码片段: 436: double avgDelay = (double)total_delays/delay_list->size/1E6; 437: double minDelay = *(int64_t *) (benchArrayGet(delay_list, 0))/1E6; 438: double maxDelay = *(int64_t *) (benchArrayGet(delay_list, (int32_t)(delay_list->size - 1)))/1E6; 439: double p90 = *(int64_t*)(benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.90)))/1E6; 440: double p95 = *(int64_t*)(benchArrayGet(delay_list, (int32_t)(delay_list->size * 0.95)))/1E6;	
	污染路径: 无	
85	缺陷发生位置: /source/os/src/osFile.c	行号: 1650
	步骤描述: 强制转换为相同类型.	
	代码片段: 1648: } 1649: if (readLen == 0) break; 1650: if (gzwrite(dstFp, data, (int32_t)readLen) == 0) { 1651: terrno = TAOS_SYSTEM_ERROR(ERRNO); 1652: ret = terrno;	
	污染路径: 无	
86	缺陷发生位置: /source/util/test/timerTest.cpp	行号: 145
	步骤描述: 强制转换为相同类型.	
	代码片段: 143: if (gettimeofday(&tv, NULL) != 0) return false; 144: 145: int64_t us = (int64_t)tv.tv_sec * 1000000LL + tv.tv_usec + offsetMs * 1000; 146: tv.tv_sec = us / 1000000; 147: tv.tv_usec = us % 1000000;	
	污染路径: 无	
87	缺陷发生位置:	行号:

	/tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	299
	步骤描述: 强制转换为相同类型.	
	代码片段: 297: do 298: --p; 299: while (p >= buf && isblank (*p)); 300: nl = p + 1; /* The newline will replace the first blank. */ 301: }	
	污染路径: expanded from macro 'isblank' expanded from macro '___isctype'	
88	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 212
	步骤描述: 强制转换为相同类型.	
	代码片段: 210: v.c5 = (double)0.000000001; 211: for (int j = 0; j < sizeof(v.c6); j++) { 212: v.c6[j] = (char)('a'); 213: } 214: v.c7 = 32767;	
	污染路径: 无	
89	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 177
	步骤描述: 强制转换为相同类型.	
	代码片段: 175: initEnv(taos); 176: // cols 177: int64_t** ts = (int64_t*)malloc(CTB_NUMS * sizeof(int64_t)); 178: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 179: // ctbnames	
	污染路径: 无	
90	缺陷发生位置:	行号:

	/source/util/src/tutil.c		499
	步骤描述: 强制转换为相同类型.		
	代码片段: 497: return terrno = TSDB_CODE_INVALID_CFG_VALUE; 498: } 499: while (isspace((unsigned char)*endPtr)) endPtr++; 500: if (*endPtr != '\0') { 501: return terrno = TSDB_CODE_INVALID_CFG_VALUE;		
	污染路径: 无		
91	缺陷发生位置: /source/libs/transport/src/thttp.c		行号: 187
	步骤描述: 强制转换为相同类型.		
	代码片段: 185: 186: gzipStream.next_in = (Bytef*)pSrc; 187: gzipStream.avail_in = (uLong)srcLen; 188: gzipStream.next_out = (Bytef*)pDest; 189: gzipStream.avail_out = (uLong)(destLen);		
	污染路径: 无		
92	缺陷发生位置: /tools/taos-tools/src/benchInsert.c		行号: 1158
	步骤描述: 强制转换为相同类型.		
	代码片段: 1156: g_arguments->totalChildTables, g_arguments->table_threads); 1157: } 1158: int64_t start = (int64_t)toolsGetTimestampMs(); 1159: 1160: for (int i = 0; i < g_arguments->databases->size		
	污染路径: 无		
93	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c		行号: 3187
	步骤描述: 强制转换为相同类型.		

	代码片段: 3185: matchLength = MEM_readLE32(dumps) & 0xFFFFFFFF; /* no pb : dumps is always followed by seq tables > 1 byte */ 3186: if (matchLength&1) matchLength>>=1, dumps += 3; 3187: else matchLength = (U16)(matchLength)>>1, dumps += 2; 3188: } 3189: if (dumps > de) { matchLength = MaxML+255; } /* late correction, to avoid using uninitialized memory */	
	污染路径: 无	
94	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.c	行号: 115
	步骤描述: 强制转换为相同类型.	
	代码片段: 113: SCacheItem **ppltem = taosHashGet(pCache->pCache, key, keyLen); 114: if (ppltem != NULL && *ppltem != NULL) { 115: SCacheItem *pltem = (SCacheItem *)*ppltem; 116: if ((tdListPopNode(pCache->lruList, pltem->pNode)) == NULL) { 117: bseWarn("node not exist in lru list");	
	污染路径: 无	
95	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 210
	步骤描述: 强制转换为相同类型.	
	代码片段: 208: v.c3 = (int64_t)2147483648; 209: v.c4 = (float)0.1; 210: v.c5 = (double)0.000000001; 211: for (int j = 0; j < sizeof(v.c6); j++) { 212: v.c6[j] = (char)('a');	
	污染路径: 无	

96	缺陷发生位置: /utils/test/c/tmq_td37265.c	行号: 76
	步骤描述: 强制转换为相同类型.	
	代码片段: 74: for (int j = 0; j < ROW_NUMS; j++) { 75: int64_t ts = 1591060628000 + j + k * 100000; 76: float current = (float)0.0001f * j; 77: int voltage = j; 78: float phase = (float)0.0001f * j;	
	污染路径: 无	
97	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1828
	步骤描述: 强制转换为相同类型.	
	代码片段: 1826: 1827: /* call tools_cJSON_AddItemToObjectCS for code reuse */ 1828: tools_cJSON_AddItemToObjectCS(object, (char*)tools_cJSON_strdup((const unsigned char*)string, &global_hooks), item); 1829: /* remove tools_cJSON_StringIsConst flag */ 1830: item->type &= ~tools_cJSON_StringIsConst;	
	污染路径: 无	
98	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 614
	步骤描述: 强制转换为相同类型.	
	代码片段: 612: if(cmpSysEndianType!=(unsigned char)sysEndianType) 613: { 614: unsigned char* p = (unsigned char*)(bytes+1); 615: size_t i = 0, size = 2*nodeCount*sizeof(unsigned short); 616:	
	污染路径:	

	无	
99	缺陷发生位置: /source/util/src/tutil.c	行号: 394
	步骤描述: 强制转换为相同类型.	
	代码片段: 392: // char *by; 393: 394: len = (int32_t)strlen((char *)hexstr) / 2; 395: 396: for (i = 0; i < len; i++) {	
	污染路径: 无	
100	缺陷发生位置: /docs/examples/c/prepare.c	行号: 179
	步骤描述: 强制转换为相同类型.	
	代码片段: 177: params[j].is_null = ((i == j) ? &is_null : 0); 178: } 179: v.b = (int8_t)i % 2; 180: v.v1 = (int8_t)i; 181: v.v2 = (int16_t)(i * 2);	
	污染路径: 无	
101	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 113
	步骤描述: 强制转换为相同类型.	
	代码片段: 111: struct timeval systemTime; 112: rtGetTimeOfDay(&systemTime); 113: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 114: } 115:	
	污染路径: 无	
102	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1005

	步骤描述: 强制转换为相同类型.	
	代码片段: 1003: data->smallData[i] = (int16_t)i; 1004: data->usmallData[i] = (uint16_t)(i + 1); 1005: data->intData[i] = (int32_t)i; 1006: data->uintData[i] = (uint32_t)(i + 1); 1007: data->bigData[i] = (int64_t)i;	
	污染路径: 无	
103	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/apitest.c	509
	步骤描述: 强制转换为相同类型.	
	代码片段: 507: is_null[i] = 0; 508: 509: v.b[i] = (int8_t)i % 2; 510: v.v1[i] = (int8_t)i; 511: v.v2[i] = (int16_t)(i * 2);	
	污染路径: 无	
104	缺陷发生位置:	行号:
	/source/client/src/clientSmlLine.c	560
	步骤描述: 强制转换为相同类型.	
	代码片段: 558: elements->timestamp = sql; 559: while (sql < sqlEnd) { 560: if (unlikely(isspace(*sql))) { 561: break; 562: }	
	污染路径: expanded from macro 'isspace' expanded from macro '___isctype'	
105	缺陷发生位置:	行号:
	/tools/src/pub.c	28
	步骤描述: 强制转换为相同类型.	
	代码片段:	

	<pre> 26: } 27: for (size_t i = 0; i < len; i++) { 28: result[i] = tolower((unsigned char)str[i]); 29: } 30: result[len] = '\0'; </pre>	
	污染路径: 无	
106	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 406
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 404: 405: unsigned long long endtime = getCurrentTime(); 406: unsigned long long totalRows = (uint32_t) (totalRowsPerTbl * tableNum); 407: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL, (endtime-starttime)/totalRows); 408: </pre>	
	污染路径: 无	
107	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 436
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 434: double time_cost = spend / 1E6; 435: double qps = (time_cost > 0) ? (totalQueried / time_cost) : 0.0; 436: double avgDelay = (double)total_delays/delay_list- >size/1E6; 437: double minDelay = *(int64_t *) (benchArrayGet(delay_list, 0))/1E6; 438: double maxDelay = *(int64_t *) (benchArrayGet(delay_list, (int32_t)(delay_list->size - 1)))/1E6; </pre>	
	污染路径: 无	
108	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchData.c	740
	步骤描述: 强制转换为相同类型.	
	代码片段: 738: + floatTmp / 1000000000); 739: } else if (g_arguments->demo_mode && i == 2) { 740: floatTmp = (float)((105 + taosRandom() % 10 741: + floatTmp / 1000000000) / 360); 742: }	
	污染路径: 无	
109	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 323
	步骤描述: 强制转换为相同类型.	
	代码片段: 321: do 322: ++p; 323: while (isblank (*p)); 324: /* The next line will start here. */ 325: nextline = p;	
	污染路径: expanded from macro 'isblank' expanded from macro '__isctype'	
110	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1420
	步骤描述: 强制转换为相同类型.	
	代码片段: 1418: // Compare characters case-insensitively 1419: while (*str2 != '\0') { 1420: if (tolower((unsigned char)*str1) != tolower((unsigned char)*str2)) { 1421: return -1; 1422: }	
	污染路径: 无	
111	缺陷发生位置: /source/client/test/stmtTest.cpp	行号: 183

	步骤描述: 强制转换为相同类型.	
	代码片段: 181: (&tv, NULL); 182: int64_t ts = 1591060628000 + j + k * 100000; 183: float current = (float)0.0001f * j; 184: int voltage = j; 185: float phase = (float)0.0001f * j;	
	污染路径: 无	
112	缺陷发生位置:	行号:
	/contrib/libmqtt/ttq/src/ttqMisc.c	83
	步骤描述: 强制转换为相同类型.	
	代码片段: 81: if (str == NULL) return NULL; 82: 83: while (isspace(str[0])) { 84: str++; 85: }	
	污染路径: expanded from macro 'isspace' expanded from macro '___isctype'	
113	缺陷发生位置:	行号:
	/source/client/test/stmtTest.cpp	185
	步骤描述: 强制转换为相同类型.	
	代码片段: 183: float current = (float)0.0001f * j; 184: int voltage = j; 185: float phase = (float)0.0001f * j; 186: params[0].buffer = &ts; 187: params[1].buffer = ¤t;	
	污染路径: 无	
114	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/batchprepare.c	988
	步骤描述: 强制转换为相同类型.	
	代码片段:	

	986: data->smallData[i] = (int16_t)i; 987: data->usmallData[i] = (uint16_t)(i+1); 988: data->intData[i] = (int32_t)i; 989: data->uintData[i] = (uint32_t)(i+1); 990: data->bigData[i] = (int64_t)i;	
	污染路径: 无	
115	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 5120
	步骤描述: 强制转换为相同类型.	
	代码片段: 5118: } else if (IS_FLOAT_TYPE(type)) { 5119: double *out = (double *)pOutputData->pData; 5120: *(double *)out = *out / (double)count; 5121: } else if (IS_DECIMAL_TYPE(type)) { 5122: Decimal128 *out = (Decimal128 *)pOutputData->pData;	
	污染路径: 无	
116	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 169
	步骤描述: 强制转换为相同类型.	
	代码片段: 167: { 168: char buf[10]; 169: if (isprint (key)) 170: sprintf (buf, "%c", key); 171: else	
	污染路径: expanded from macro 'isprint' expanded from macro '__isctype'	
117	缺陷发生位置: /source/client/src/clientTmq.c	行号: 1529
	步骤描述: 强制转换为相同类型.	
	代码片段: 1527: code = taosWriteQitem(tmq->mqueue, pWrapper);	

	1528: if (code != 0) { 1529: tmqFreeRspWrapper((SMqRspWrapper*)pWrapper); 1530: taosFreeQitem(pWrapper); 1531: tqErrorC("consumer:0x%" PRIx64 " put ep res into mqqueue failed, code:%d", tmq->consumerId, code);	
	污染路径: 无	
118	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 211
	步骤描述: 强制转换为相同类型.	
	代码片段: 209: 210: v.ts[i] = ts++; 211: v.b[i] = (int8_t)i % 2; 212: v.v1[i] = (int8_t)i; 213: v.v2[i] = (int16_t)(i * 2);	
	污染路径: 无	
119	缺陷发生位置: /contrib/lemon/lemon.c	行号: 181
	步骤描述: 强制转换为相同类型.	
	代码片段: 179: lemon_addtext(str, &nUsed, &zFormat[j], i-j, 0); 180: c = zFormat[++i]; 181: if(ISDIGIT(c) (c=='-' && ISDIGIT(zFormat[i+1]))){ 182: if(c=='-') i++; 183: while(ISDIGIT(zFormat[i])) iWidth = iWidth*10 + zFormat[i++] - '0';	
	污染路径: expanded from macro 'ISDIGIT' expanded from macro 'isdigit' expanded from macro ' __isctype'	
120	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqMisc.c	行号: 87
	步骤描述: 强制转换为相同类型.	
	代码片段: 85: }	

	86: endptr = &str[strlen(str) - 1]; 87: while (endptr > str && isspace(endptr[0])) { 88: endptr[0] = '\0'; 89: endptr--;	
	污染路径: expanded from macro 'isspace' expanded from macro '__isctype'	
121	缺陷发生位置: /examples/c/prepare.c	行号: 179
	步骤描述: 强制转换为相同类型.	
	代码片段: 177: params[j].is_null = ((i == j) ? &is_null : 0); 178: } 179: v.b = (int8_t)i % 2; 180: v.v1 = (int8_t)i; 181: v.v2 = (int16_t)(i * 2);	
	污染路径: 无	
122	缺陷发生位置: /test/new_test_framework/script/api/insert_stb.c	行号: 123
	步骤描述: 强制转换为相同类型.	
	代码片段: 121: int i = rowIdx + batchIdx * 100 + tblIdx*10000*100; 122: len += sprintf(qstr+len, " ('ct%d', %d, %f, '%s', %" PRId64 " , %d, %d, %d, %d, %f, %lf, '%s')", tblIdx, tblIdx, (float)tblIdx, "childtable", 123: (uint64_t)(1546300800000 + i), (int8_t)i, (int16_t)i, i, i, i*1.0, i*2.0, "hello"); 124: } 125: TAOS_RES *result1 = taos_query(taos, qstr);	
	污染路径: 无	
123	缺陷发生位置: /test/new_test_framework/script/api/insert_stb.c	行号: 82
	步骤描述: 强制转换为相同类型.	
	代码片段: 80: for (int rowIdx = 0; rowIdx < 100; ++ rowIdx) {	

	81: int i = rowIdx + batchIdx * 100 + tblIdx*10000*100; 82: len += sprintf(qstr+len, " (%" PRIu64 " ", %d, %d, %d, %d, %f, %lf, '%s')", (uint64_t)(1546300800000 + i), (int8_t)i, (int16_t)i, i, i, i*1.0, i*2.0, "hello"); 83: } 84: TAOS_RES *result1 = taos_query(taos, qstr);	
	污染路径: 无	
124	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 987
	步骤描述: 强制转换为相同类型.	
	代码片段: 985: } else if (0 == strncmp(argv[i], "--databases=", 986: strlen("--databases="))) { 987: arguments->databasesSeq = (char *) (argv[i] 988: + strlen("--databases=")); 989: } else if (0 == strncmp(argv[i], "-D", strlen("- D"))) {	
	污染路径: 无	
125	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1414
	步骤描述: 强制转换为相同类型.	
	代码片段: 1412: int trimCaseCmp(char *str1, char *str2) { 1413: // Skip leading whitespace in str1 1414: while (isblank((unsigned char)*str1)) { 1415: str1++; 1416: }	
	污染路径: expanded from macro 'isblank' expanded from macro '__isctype'	
126	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 2416
	步骤描述: 强制转换为相同类型.	
	代码片段: 2414: return code;	

	2415: } 2416: char* isNull = (char*)keyBuf; 2417: char* pStart = (char*)keyBuf + sizeof(int8_t) * LIST_LENGTH(pGroupNode); 2418:	
	污染路径: 无	
127	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 3153
	步骤描述: 强制转换为相同类型.	
	代码片段: 3151: litLength = MEM_readLE32(dumps) & 0xFFFFF; /* no risk : dumps is always followed by seq tables > 1 byte */ 3152: if (litLength&1) litLength>>=1, dumps += 3; 3153: else litLength = (U16)(litLength)>>1, dumps += 2; 3154: } 3155: if (dumps > de) { litLength = MaxLL+255; } /* late correction, to avoid using uninitialized memory */	
	污染路径: 无	
128	缺陷发生位置: /source/os/src/osSocket.c	行号: 716
	步骤描述: 强制转换为相同类型.	
	代码片段: 714: #endif 715: 716: return (int)fd; 717: } 718:	
	污染路径: 无	
129	缺陷发生位置: /source/os/src/osSocket.c	行号: 746
	步骤描述: 强制转换为相同类型.	
	代码片段: 744: #else	

	"update_fill_interval"); 1464: if (tools_cJSON_IsNumber(updFill)) { 1465: superTable->fillIntervalUpd = (uint64_t)updFill->valueint; 1466: } 1467:	
	污染路径: 无	
133	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.c	行号: 173
	步骤描述: 强制转换为相同类型.	
	代码片段: 171: TSDB_CHECK_CODE(code = TSDB_CODE_NOT_FOUND, lino, _error); 172: } 173: SCacheItem *pItem = (SCacheItem *)*ppItem; 174: 175: code = taosHashRemove(pCache->pCache, key, keyLen);	
	污染路径: 无	
134	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2137
	步骤描述: 强制转换为相同类型.	
	代码片段: 2135: tools_cJSON_GetArrayItem(2136: pThreadInfo->sml_json_tags, 2137: (int)tableSeq - 2138: pThreadInfo->start_table_from), 2139: true);	
	污染路径: 无	
135	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 164
	步骤描述: 强制转换为相同类型.	
	代码片段:	

	162: "values(%" PRIu64 ", %f, %" PRIu64", " 163: "'%c%d', '%s%c%d', '%c%d')", 164: i, 1650000000000+j+1, (float)j, j, 165: 'a'+(int)j%25, rand(), 166: "数据", 'z' - (int)j%25, rand(),
	污染路径: 无
136	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c
	行号: 1460
	步骤描述: 强制转换为相同类型.
	代码片段: 1458: if (updRatio->valueint > 100) updRatio->valueint = 100; 1459: if (updRatio->valueint < 0) updRatio->valueint = 0; 1460: superTable->updRatio = (int8_t)updRatio->valueint; 1461: } 1462: tools_cJSON *updFill =
	污染路径: 无
137	缺陷发生位置: /source/os/src/osRand.c
	行号: 62
	步骤描述: 强制转换为相同类型.
	代码片段: 60: pFile = taosOpenFile("/dev/urandom", TD_FILE_READ); 61: if (pFile == NULL) { 62: seed = (int)taosGetTimestampSec(); 63: } else { 64: int64_t len = taosReadFile(pFile, &seed, sizeof(seed));
	污染路径: 无
138	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c
	行号: 177
	步骤描述: 强制转换为相同类型.
	代码片段: 175: initEnv(taos);

	176: // cols 177: int64_t** ts = (int64_t*)malloc(CTB_NUMS * sizeof(int64_t)); 178: char** b = (char**)malloc(CTB_NUMS * sizeof(char*)); 179: // ctbnames	
	污染路径: 无	
139	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 332
	步骤描述: 强制转换为相同类型.	
	代码片段: 330: is_null[i] = 0; 331: 332: v.b[i] = (int8_t)i % 2; 333: v.v1[i] = (int8_t)i; 334: v.v2[i] = (int16_t)(i * 2);	
	污染路径: 无	
140	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 248
	步骤描述: 强制转换为相同类型.	
	代码片段: 246: SKIPWS (var); 247: 248: if (isalpha (*var)) 249: { 250: size_t var_len;	
	污染路径: expanded from macro 'isalpha' expanded from macro '__isctype'	

9.2.1.4 WK4750 double 型变量赋值给 float 型变量必须强制

转换

缺陷数量： 4 个		
缺陷描述： double 型变量赋值给 float 型变量必须强制转换。		
修复意见： 无		
1	缺陷发生位置： /tools/taos-tools/src/benchJsonOpt.c	行号： 173
	步骤描述： double 型变量 atof 赋值给 float 型变量必须强制转换	
	代码片段： 171: key5 = strstr(key3, "-"); 172: if(key5) 173: *base = (float)atof(key5+1) * -1; 174: } 175: }	
	污染路径： 无	
2	缺陷发生位置： /tools/taos-tools/src/benchJsonOpt.c	行号： 150
	步骤描述： double 型变量 atof 赋值给 float 型变量必须强制转换	
	代码片段： 148: key3 = strstr(key2, "-"); 149: if(key3) { 150: *addend = (float)atof(key3 + 1) * -1; 151: key3 += 1; 152: }	
	污染路径： 无	
3	缺陷发生位置： /contrib/TSZ/sz/src/sz_float.c	行号： 409
	步骤描述： double 型变量 fabs 赋值给 float 型变量必须强制转换	
	代码片段：	

	407: totalSampleSize++; 408: pred_value = data_pos[-1]; 409: pred_err = fabs(pred_value - *data_pos); 410: double dbri = (pred_err/realPrecision+1)/2; 411: if(dbri >= confparams_cpr->maxRangeRadius) {	
	污染路径: 无	
4	缺陷发生位置: /contrib/TSZ/sz/src/sz_float.c	行号: 55
	步骤描述: double 型变量 fabs 赋值给 float 型变量必须强制转换	
	代码片段: 53: //pred_value = 2*oriData[i-1] - oriData[i-2]; 54: pred_value = oriData[i-1]; 55: pred_err = fabs(pred_value - oriData[i]); 56: double dbri = (pred_err/realPrecision+1)/2; 57: if(dbri >= confparams_cpr->maxRangeRadius) {	
	污染路径: 无	

9.2.1.5 WK4794 指针或引用类型的转换中禁止移除 **const** 或

volatile 属性

缺陷数量: 48 个		
缺陷描述: 指针或引用类型的转换中禁止移除 const 或 volatile 属性。		
修复意见: 无		
1	缺陷发生位置: /source/util/src/tarray.c	行号: 587
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 585: }	

	586: } 587: return (void*)buf; 588: } 589:	
	污染路径: 无	
2	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 3642
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 3640: } 3641: 3642: char* outName = (char*)tbName; 3643: char tableName[TSDB_TABLE_NAME_LEN + 1]; 3644: if(g_args.dotReplace && replaceCopy(tableName, (char*)tbName)) {	
	污染路径: 无	
3	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 3082
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 3080: char **jsonSchema, bool onlyColumn) { 3081: 3082: char* outName = (char*)tbName; 3083: char tableName[TSDB_TABLE_NAME_LEN + 1]; 3084: if(g_args.dotReplace && replaceCopy(tableName, (char*)tbName)) {	
	污染路径: 无	
4	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 878
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段:	

	876: SWord* word = (SWord*)taosMemoryMalloc(sizeof(SWord)); 877: memset(word, 0, sizeof(SWord)); 878: word->word = (char*)p; 879: word->len = len; 880:	
	污染路径: 无	
5	缺陷发生位置: /source/dnode/mnode/impl/src/mndDef.c	行号: 103
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 101: } 102: buf = taosDecodeSEpSet(buf, &pVgEp->epSet); 103: return (void *)buf; 104: } 105:	
	污染路径: 无	
6	缺陷发生位置: /source/common/src/ttime.c	行号: 53
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 51: 52: bool checkTzPresent(const char* str, int32_t len) { 53: char* seg = forwardToTimeStringEnd((char*)str); 54: int32_t seg_len = len - (int32_t)(seg - str); 55:	
	污染路径: 无	
7	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReaderWriter.c	行号: 47
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段:	

	45: 46: int32_t vid = 0; 47: const char *object_name = taosDirEntryBaseName((char *)path); 48: (void)sscanf(object_name, "v%df%dver%" PRIu64, &vid, &pFD->fid, &pFD->cid); 49:	
	污染路径: 无	
8	缺陷发生位置: /source/libs/sync/test/syncRaftCfgTest.cpp	行号: 65
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 63: void test3() { 64: SSyncCfg* pCfg = createSyncCfg(); 65: char* s = (char*)"./test3_raft_cfg.json"; 66: 67: if (taosCheckExistFile(s)) {	
	污染路径: 无	
9	缺陷发生位置: /source/os/src/osString.c	行号: 755
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 753: if (toupper(*str) == toupper(*pattern)) { 754: for (i = 1;; i++) { 755: if (!pattern[i]) return (char *)str; 756: if (toupper(str[i]) != toupper(pattern[i])) break; 757: }	
	污染路径: 无	
10	缺陷发生位置: /tools/taos-tools/test/taosdumpTest.cpp	行号: 23
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	

	代码片段: 21: TEST(taosdump, toolsSys) { 22: // errorPrintReqArg3 23: errorPrintReqArg3((char *)"taosdump", (char *)"test parameters"); 24: printf("ut function errorPrintReqArg3 [Passed]\n"); 25:	
	污染路径: 无	
11	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 1497
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1495: } 1496: if (fdoc && fdoc != tdoc) 1497: free ((char *)fdoc); /* Free user's modified doc string. */ 1498: 1499: if (child)	
	污染路径: 无	
12	缺陷发生位置: /contrib/lemon/lemon.c	行号: 4029
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 4027: 4028: /* This const cast is wrong but harmless, if we're careful. */ 4029: for(cp=(char *)rp->code; *cp; cp++){ 4030: if(cp==zSkip){ 4031: append_str(zOvwrt,0,0,0);	
	污染路径: 无	
13	缺陷发生位置:	行号:

	/source/common/src/tdatablock.c		2638
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 2636: } 2637: 2638: return (void*)buf; 2639: _error: 2640: for (int32_t i = 0; i < sz; ++i) {		
	污染路径: 无		
14	缺陷发生位置: /tools/taos-tools/src/taosdump.c		行号: 6779
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 6777: if (stable) { 6778: strcat(dirToCreate, STBNAME_FILE); 6779: if (writeFile(dirToCreate, (char *)stable)) { 6780: warnPrint("create file stbname failed. stb:%s file:%s\n", stable, dirToCreate); 6781: }		
	污染路径: 无		
15	缺陷发生位置: /source/dnode/mnode/impl/src/mndDef.c		行号: 432
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 430: } 431: 432: return (void *)buf; 433: } 434:		
	污染路径: 无		
16	缺陷发生位置:		行号:

	/source/dnode/vnode/src/vnd/vnodeStream.c		1354
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 1352: pData = tTagValToData((const STagVal*)p, false); 1353: } else { 1354: pData = (char*)p; 1355: } 1356:		
	污染路径: 无		
17	缺陷发生位置: /contrib/test/rocksdb/main.c		行号: 76
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 74: 75: char *p1 = (char *)aBuf; 76: char *p2 = (char *)bBuf; 77: // p1 += 1; 78: // p2 += 1;		
	污染路径: 无		
18	缺陷发生位置: /source/libs/sync/test/syncTest.cpp		行号: 37
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 35: void test2() { 36: sDebugFlag = DEBUG_DEBUG + DEBUG_SCREEN + DEBUG_FILE; 37: logTest((char*)__FUNCTION__); 38: } 39:		
	污染路径: 无		
19	缺陷发生位置:		行号:

	/source/os/src/osString.c	749
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 747: return NULL; 748: } 749: if (!pattern !*pattern) return (char *)str; 750: 751: size_t i;	
	污染路径: 无	
20	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 133
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 131: char* sRet; 132: if((sRet = strchr(s, c_in)) == NULL) 133: return (char*) (s + strlen(s)); 134: else 135: return sRet;	
	污染路径: 无	
21	缺陷发生位置: /source/libs/sync/test/syncTest.cpp	行号: 42
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 40: void test3() { 41: sDebugFlag = DEBUG_INFO + DEBUG_SCREEN + DEBUG_FILE; 42: logTest((char*)__FUNCTION__); 43: } 44:	
	污染路径: 无	

22	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 140
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 138: pBt->info.nLevel = 1; 139: pBt->info.nData = 0; 140: pBt->tname = (char *)tname; 141: 142: ret = tdbTbInsert(pPager->pEnv->pMainDb, tname, strlen(tname) + 1, &pBt->info, sizeof(pBt->info), txn);	
	污染路径: 无	
23	缺陷发生位置: /source/dnode/mnode/impl/src/mndDef.c	行号: 412
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 410: } 411: } 412: return (void *)buf; 413: } 414:	
	污染路径: 无	
24	缺陷发生位置: /source/os/test/osEnvTests.cpp	行号: 135
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 133: char *argv[] = {(char *)"/usr/local/bin/taosd", NULL}; 134: #else 135: char *argv[] = {(char *)"/usr/bin/taosd", NULL}; 136: #endif 137: osSetProcPath(1, argv);	
	污染路径: 无	

25	缺陷发生位置： /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号： 1851
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 1849: global_hooks.deallocate(item->string); 1850: } 1851: item->string = (char*)string; 1852: item->type = tools_cJSON_StringIsConst; 1853: tools_cJSON_AddItemToArray(object, item);	
	污染路径： 无	
26	缺陷发生位置： /tools/shell/src/shellEngine.c	行号： 219
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 217: } 218: 219: p = (char *)pstr + len - 3; 220: if (strcmp(p, "\\G;") == 0) { 221: return p;	
	污染路径： 无	
27	缺陷发生位置： /test/new_test_framework/script/api/batchprepare.c	行号： 369
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 367: if (ptr == NULL) return; 368: 369: return free((void*)ptr); 370: } 371:	
	污染路径： 无	
28	缺陷发生位置：	行号：

	/source/libs/parser/src/parUtil.c		684
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 682: 683: if (needFree) { 684: taosMemoryFree((char*)valueStr); 685: } 686:		
	污染路径: 无		
29	缺陷发生位置: /source/libs/parser/test/parAlterToBalanceTest.cpp		行号: 682
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 680: clearAlterTbReq(); 681: 682: setAlterTableOptions("t1", -1, (char*)"test"); 683: run("ALTER TABLE t1 COMMENT 'test'"); 684: clearAlterTbReq();		
	污染路径: 无		
30	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c		行号: 1297
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 1295: } 1296: if (fstr && fstr != tstr) 1297: free ((char *) fstr); 1298: } 1299: }		
	污染路径: 无		
31	缺陷发生位置:		行号:

	/source/dnode/mnode/impl/src/mndDef.c		594
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 592: buf = tDecodeOffRows(buf, &pSub->offsetRows, sver); 593: } 594: return (void *)buf; 595: } 596:		
	污染路径: 无		
32	缺陷发生位置: /contrib/test/rocksdb/main.c		行号: 220
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 218: const char *value = rocksdb_iter_value(iter, &vlen); 219: KV kv; 220: kvDeserial(&kv, (char *)key); 221: printf("kv1: %d\t kv2: %d, len:%d, value = %s\n", (int)(kv.k1), (int)(kv.k2), (int)(klen), value); 222: }		
	污染路径: 无		
33	缺陷发生位置: /source/libs/sync/test/syncTest.cpp		行号: 47
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段: 45: void test4() { 46: sDebugFlag = DEBUG_ERROR + DEBUG_SCREEN + DEBUG_FILE; 47: logTest((char*)__FUNCTION__); 48: } 49:		
	污染路径:		

	无	
34	缺陷发生位置: /source/util/src/tutil.c	行号: 150
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 148: 149: if (haystack[i] == needle) { 150: return (char *)&haystack[i]; 151: } 152: }	
	污染路径: 无	
35	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 7231
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 7229: 7230: avro_value_set_branch(&value, 1, &branch); 7231: char* outSName = (char*)stable; 7232: char stableName[TSDB_TABLE_NAME_LEN + 1]; 7233: if(g_args.dotReplace && replaceCopy(stableName, (char*)stable)) {	
	污染路径: 无	
36	缺陷发生位置: /source/dnode/mnode/impl/src/mndConfig.c	行号: 796
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 794: tstrncpy(configName, cfgReq.config, sizeof(configName)); 795: const char *p = strstr(configName, " "); 796: if (p) *(char *)p = 0; 797: EPrivType privType = cfgGetPrivType(tsCfg, configName, 0); 798: if ((code = mndCheckSysObjPrivilege(pMnode,	

	pOperUser, RPC_MSG_TOKEN(pReq), privType, 0, 0, NULL, NULL))) {	
	污染路径: 无	
37	缺陷发生位置: /source/common/src/ttime.c	行号: 232
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 230: 231: int64_t fraction = 0; 232: str = forwardToTimeStringEnd((char*)timestr); 233: 234: if ((str[0] == 'Z' str[0] == 'z') && str[1] == '\0') {	
	污染路径: 无	
38	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号: 231
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 229: text, params->foonly_default); 230: else 231: new_text = (char *)text; 232: 233: return new_text;	
	污染路径: 无	
39	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 1256
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1254: } 1255: if (fstr && fstr != tstr)	

	1256: free ((char *) fstr); 1257: 1258: /* Reset the left margin. */	
	污染路径: 无	
40	缺陷发生位置: /source/dnode/mnode/impl/src/mndDef.c	行号: 349
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 347: } 348: 349: return (void *)buf; 350: } 351:	
	污染路径: 无	
41	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 6142
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 6140: const char *namespace = avro_schema_namespace((const avro_schema_t)schema); 6141: if(g_args.renameHead) { 6142: char* newDbName = findNewName((char *)namespace); 6143: if(newDbName) { 6144: infoPrint(" ----- rename DB Name %s to %s -----\n", namespace, newDbName);	
	污染路径: 无	
42	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 390
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段:	

	388: if (ptr == NULL) return; 389: 390: return free((void *)ptr); 391: } 392:	
	污染路径: 无	
43	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 214
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 212: } 213: 214: char *p = (char *)pstr + len - 2; 215: if (strcmp(p, "\\G") == 0) { 216: return p;	
	污染路径: 无	
44	缺陷发生位置: /source/libs/sync/test/syncTest.cpp	行号: 32
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 30: void test1() { 31: sDebugFlag = DEBUG_TRACE + DEBUG_SCREEN + DEBUG_FILE; 32: logTest((char*)__FUNCTION__); 33: } 34:	
	污染路径: 无	
45	缺陷发生位置: /source/common/src/msg/tmsg.c	行号: 254
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段:	

	252: buf = taosDecodeStringTo(buf, pEp->eps[i].fqdn); 253: } 254: return (void *)buf; 255: } 256:	
	污染路径: 无	
46	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 75
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 73: memset(&w2, 0, sizeof(w2)); 74: 75: char *p1 = (char *)aBuf; 76: char *p2 = (char *)bBuf; 77: // p1 += 1;	
	污染路径: 无	
47	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 1103
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1101: 1102: if (fstr != tstr) 1103: free ((char *) fstr); 1104: } 1105:	
	污染路径: 无	
48	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 118
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	

	代码片段: 116: is preceded by a blank line)."); 117: else 118: return (char *)text; 119: } 120:
	污染路径: 无

9.2.1.6 WK4793 禁止将不相关指针类型强制转换为对象指针类

型

缺陷数量：6 个		
缺陷描述： 禁止将不相关指针类型强制转换为对象指针类型。		
修复意见： 无		
1	缺陷发生位置： /source/libs/decimal/src/detail/intx/intx.hpp	行号： 379
	步骤描述： 不要使用 reinterpret_cast	
	代码片段： 377: inline uint8_t* as_bytes(uint<N>& x) noexcept 378: { 379: return reinterpret_cast<uint8_t*>(as_words(x)); 380: } 381:	
	污染路径： 无	
2	缺陷发生位置： /source/os/test/osMathTests.cpp	行号： 40
	步骤描述： 不要使用 reinterpret_cast	
	代码片段： 38: int cmpFunc(const void* a, const void* b) { 39: const TestStruct* pa = reinterpret_cast<const	

	TestStruct*>(a); 40: const TestStruct* pb = reinterpret_cast<const TestStruct*>(b); 41: if (pa->a < pb->a) { 42: return -1;	
	污染路径: 无	
3	缺陷发生位置: /source/util/test/talgoTest.cpp	行号: 12
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 10: // Define a custom comparison function for testing 11: int cmpFunc(const void* a, const void* b) { 12: const TestStruct* pa = reinterpret_cast<const TestStruct*>(a); 13: const TestStruct* pb = reinterpret_cast<const TestStruct*>(b); 14: if (pa->a < pb->a) {	
	污染路径: 无	
4	缺陷发生位置: /source/util/test/talgoTest.cpp	行号: 13
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 11: int cmpFunc(const void* a, const void* b) { 12: const TestStruct* pa = reinterpret_cast<const TestStruct*>(a); 13: const TestStruct* pb = reinterpret_cast<const TestStruct*>(b); 14: if (pa->a < pb->a) { 15: return -1;	
	污染路径: 无	
5	缺陷发生位置: /source/os/test/osMathTests.cpp	行号: 39
	步骤描述: 不要使用 reinterpret_cast	
	代码片段:	

	<pre>37: // Define a custom comparison function for testing 38: int cmpFunc(const void* a, const void* b) { 39: const TestStruct* pa = reinterpret_cast<const TestStruct*>(a); 40: const TestStruct* pb = reinterpret_cast<const TestStruct*>(b); 41: if (pa->a < pb->a) {</pre>	
	污染路径: 无	
6	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 385
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 383: inline const uint8_t* as_bytes(const uint<N>& x) noexcept 384: { 385: return reinterpret_cast<const uint8_t*>(as_words(x)); 386: } 387:	
	污染路径: 无	

10 C/C++缺陷类型-比较的问题

10.1 分项汇总表

序号	C/C++缺陷类型-比较的问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4759	禁止对指针进行大于或小于的逻辑比较	121	严重
2	WK4761	禁止对无符号	1	严重

		数进行大于等于零或小于零的比较		
3	WK4762	禁止无符号数和有符号数之间的直接比较	61	严重

10.2 缺陷/安全漏洞详解

10.2.1 严重类

10.2.1.1 WK4759 禁止对指针进行大于或小于的逻辑比较

缺陷数量：121 个		
缺陷描述： 禁止对指针进行大于或小于的逻辑比较。		
修复意见： 无		
1	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号： 55
	步骤描述： 禁止对指针 remainder 进行大于或小于的逻辑比较	
	代码片段： 53: } 54: 55: if (remainder > 0) { 56: p = taosMemoryCalloc(remainder, sizeof(STableBlockScanInfo)); 57: TSDB_CHECK_NULL(p, code, lino, _end, terno);	
	污染路径： 无	
2	缺陷发生位置： /source/libs/executor/src/sysscanoperator.c	行号： 2582
	步骤描述： 禁止对指针 tempRes 进行大于或小于的逻辑比较	
	代码片段： 2580:	

	2581: int32_t tempRes = tSerializeSTableInfoReq(buf, contLen, &infoReq); 2582: if (tempRes < 0) { 2583: code = terrno; 2584: QUERY_CHECK_CODE(code, lino, _return);	
	污染路径: 无	
3	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 515
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 513: { 514: n0+=24; 515: if (ip < iend-5) 516: { 517: ip+=2;	
	污染路径: 无	
4	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 1261
	步骤描述: 禁止对指针 idx 进行大于或小于的逻辑比较	
	代码片段: 1259: SExtWinTimeWindow* pWin = NULL; 1260: int32_t idx = taosArraySearchIdx(pExtW->pWins, tsCol, extWinTsWinCompare, TD_EQ); 1261: if (idx >= 0) { 1262: for (int i = idx - 1; i >= 0; --i) { 1263: pWin = TARRAY_GET_ELEM(pExtW->pWins, i);	
	污染路径: 无	
5	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 974
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: 972: { U32 nbBytes = bitD->bitsConsumed >> 3; 973: BITv06_DStream_status result = BITv06_DStream_unfinished;	

	<pre> 974: if (bitD->ptr - nbBytes < bitD->start) { 975: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ 976: result = BITv06_DStream_endOfBuffer; </pre>	
	污染路径: 无	
6	缺陷发生位置: /source/client/src/clientSml.c	行号: 1634
	步骤描述: 禁止对指针 rawLine 进行大于或小于的逻辑比较	
	代码片段: <pre> 1632: } else if (*rawLine) { 1633: *tmp = *rawLine; 1634: while (*rawLine < rawLineEnd) { 1635: if ((*rawLine)++) == '\n') { 1636: break; </pre>	
	污染路径: 无	
7	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 404
	步骤描述: 禁止对指针 op 进行大于或小于的逻辑比较	
	代码片段: <pre> 402: do 403: COPY8(op, ip) 404: while (op < oend); 405: } 406: </pre>	
	污染路径: 无	
8	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 739
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: <pre> 737: return BIT_DStream_overflow; 738: 739: if (bitD->ptr >= bitD->start + sizeof(bitD- >bitContainer)) 740: { </pre>	

	741: bitD->ptr -= bitD->bitsConsumed >> 3;	
	污染路径: 无	
9	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 263
	步骤描述: 禁止对指针 lp 进行大于或小于的逻辑比较	
	代码片段: 261: int result = COVER_cmp(g_ctx, lp, rp); 262: if (result == 0) { 263: result = lp < rp ? -1 : 1; 264: } 265: return result;	
	污染路径: 无	
10	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 548
	步骤描述: 禁止对指针 op 进行大于或小于的逻辑比较	
	代码片段: 546: do 547: COPY8(op, ip) 548: while (op < oend); 549: } 550:	
	污染路径: 无	
11	缺陷发生位置: /source/client/src/clientSmiTelnet.c	行号: 105
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 103: const char *value = sql; 104: size_t valueLen = 0; 105: while (sql < sqlEnd) { 106: // parse value 107: if (unlikely(*sql == SPACE)) {	
	污染路径: 无	

12	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 76
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 74: 75: const char *sql = data; 76: while (sql < sqlEnd) { 77: JUMP_SPACE(sql, sqlEnd) 78: if (unlikely(*sql == '\0' *sql == '\n')) break;	
	污染路径: 无	
13	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1952
	步骤描述: 禁止对指针 i 进行大于或小于的逻辑比较	
	代码片段: 1950: int64_t *output = (int64_t *)pOutputCol->pData; 1951: int32_t numOfRows = TMAX(pLeft->numOfRows, pRight->numOfRows); 1952: for (; i < numOfRows && i >= 0; i += step, output += 1) { 1953: int32_t leftidx = pLeft->numOfRows == 1 ? 0 : i; 1954: int32_t rightidx = pRight->numOfRows == 1 ? 0 : i;	
	污染路径: 无	
14	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 739
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: 737: return FSE_DStream_tooFar; 738: 739: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) 740: { 741: bitD->ptr -= bitD->bitsConsumed >> 3;	
	污染路径: 无	
15	缺陷发生位置:	行号:

	/contrib/TSZ/zstd/common/bitstream.h		435
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较		
	代码片段: 433: { U32 nbBytes = bitD->bitsConsumed >> 3; 434: BIT_DStream_status result = BIT_DStream_unfinished; 435: if (bitD->ptr - nbBytes < bitD->start) { 436: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ 437: result = BIT_DStream_endOfBuffer;		
	污染路径: 无		
16	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c		行号: 475
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较		
	代码片段: 473: U32 nbBytes = bitD->bitsConsumed >> 3; 474: BIT_DStream_status result = BIT_DStream_unfinished; 475: if (bitD->ptr - nbBytes < bitD->start) 476: { 477: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */		
	污染路径: 无		
17	缺陷发生位置: /source/common/src/tvariant.c		行号: 204
	步骤描述: 禁止对指针 s 进行大于或小于的逻辑比较		
	代码片段: 202: const char *end = z + n; 203: bool pure = true; 204: while (s < end) { 205: if (*s < '0' *s > '9') { 206: pure = false;		
	污染路径: 无		
18	缺陷发生位置:		行号:

	/tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	201
	步骤描述: 禁止对指针 fs 进行大于或小于的逻辑比较	
	代码片段: <pre> 199: /* We are starting a new line. Print spaces to the left margin. */ 200: const size_t pad = fs->lmargin; 201: if (fs->p + pad < fs->end) 202: { 203: /* We can fit in them in the buffer by moving the </pre>	
	污染路径: 无	
19	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	行号: 193
	步骤描述: 禁止对指针 buf 进行大于或小于的逻辑比较	
	代码片段: <pre> 191: /* Scan the buffer for newlines. */ 192: buf = fs->buf + fs->point_offs; 193: while (buf < fs->p) 194: { 195: size_t r; </pre>	
	污染路径: 无	
20	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1201
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: <pre> 1199: { 1200: n0+=24; 1201: if (ip < iend-5) 1202: { 1203: ip+=2; </pre>	
	污染路径: 无	
21	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1911

	步骤描述: 禁止对指针 i 进行大于或小于的逻辑比较	
	代码片段: 1909: int64_t *output = (int64_t *)pOutputCol->pData; 1910: int32_t numRows = TMAX(pLeft->numOfRows, pRight->numOfRows); 1911: for (; i < numRows && i >= 0; i += step, output += 1) { 1912: int32_t leftidx = pLeft->numOfRows == 1 ? 0 : i; 1913: int32_t rightidx = pRight->numOfRows == 1 ? 0 : i;	
	污染路径: 无	
22	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 707
	步骤描述: 禁止对指针 group 进行大于或小于的逻辑比较	
	代码片段: 705: 706: /* Since we didn't exit, give each parser an error indication. */ 707: for (group = parser->groups; group < parser->egroup; group++) 708: group_parse (group, &parser->state, ARGV_KEY_ERROR, 0); 709: }	
	污染路径: 无	
23	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 644
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: 642: { U32 nbBytes = bitD->bitsConsumed >> 3; 643: BITv07_DStream_status result = BITv07_DStream_unfinished; 644: if (bitD->ptr - nbBytes < bitD->start) { 645: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ 646: result = BITv07_DStream_endOfBuffer;	

	污染路径: 无	
24	缺陷发生位置: /source/libs/scalar/src/scvector.c	行号: 1786
	步骤描述: 禁止对指针 i 进行大于或小于的逻辑比较	
	代码片段: 1784: 1785: int32_t numOfRows = TMAX(pLeft->numOfRows, pRight->numOfRows); 1786: for (; i < numOfRows && i >= 0; i += step, output += 1) { 1787: int32_t leftidx = pLeft->numOfRows == 1 ? 0 : i; 1788: int32_t rightidx = pRight->numOfRows == 1 ? 0 : i;	
	污染路径: 无	
25	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 460
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: 458: return BIT_DStream_overflow; 459: 460: if (bitD->ptr >= bitD->start + sizeof(bitD-> bitContainer)) 461: { 462: bitD->ptr -= bitD->bitsConsumed >> 3;	
	污染路径: 无	
26	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 595
	步骤描述: 禁止对指针 group 进行大于或小于的逻辑比较	
	代码片段: 593: parser->groups->input = input; 594: for (group = parser->groups; 595: group < parser->egroup && (!err err == EBADKEY); 596: group++) 597: {	

	污染路径: 无	
27	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 150
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 148: } 149: 150: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 151: ip += bitCount>>3; 152: bitCount &= 7;	
	污染路径: 无	
28	缺陷发生位置: /contrib/TSZ/zstd/common/zstd_internal.h	行号: 165
	步骤描述: 禁止对指针 op 进行大于或小于的逻辑比较	
	代码片段: 163: do 164: COPY8(op, ip) 165: while (op < oend); 166: } 167:	
	污染路径: 无	
29	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 1397
	步骤描述: 禁止对指针 snao_end 进行大于或小于的逻辑比较	
	代码片段: 1395: hol_entry_short_iterate (entry, add_argless_short_opt, 1396: entry->argp->argp_domain, &snao_end); 1397: if (snao_end > short_no_arg_opts) 1398: { 1399: *snao_end++ = 0;	
	污染路径:	

	无	
30	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 76
	步骤描述: 禁止对指针 __fs 进行大于或小于的逻辑比较	
	代码片段: 74: argp_fmtstream_putc (argp_fmtstream_t __fs, int __ch) 75: { 76: if (__fs->p < __fs->end _argp_fmtstream_ensure (__fs, 1)) 77: return *__fs->p++ = __ch; 78: else	
	污染路径: 无	
31	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 1494
	步骤描述: 禁止对指针 start 进行大于或小于的逻辑比较	
	代码片段: 1492: 1493: char *start = setstr; 1494: for (int32_t j = 0; start < setstr + setLen; j++) { 1495: char *end = start; 1496:	
	污染路径: 无	
32	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1710
	步骤描述: 禁止对指针 length 进行大于或小于的逻辑比较	
	代码片段: 1708: 1709: int32_t length = taosUcs4ToMbs((TdUcs4*)buf, bufSize, str, NULL); 1710: if (length <= 0) { 1711: return TSDB_CODE_TSC_INVALID_VALUE; 1712: }	
	污染路径: 无	

33	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 1745
	步骤描述: 禁止对指针 start 进行大于或小于的逻辑比较	
	代码片段: 1743: 1744: char *start = setstr; 1745: for (int32_t j = 0; start < setstr + setLen; j++) { 1746: char *end = start; 1747:	
	污染路径: 无	
34	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 247
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 245: bool valueEscaped = false; 246: size_t valueLenEscaped = 0; 247: while (*sql < sqlEnd) { 248: // parse value 249: if (unlikely(IS_SPACE(*sql,escapeChar) IS_COMMA(*sql,escapeChar))) {	
	污染路径: 无	
35	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	行号: 50
	步骤描述: 禁止对指针 __fs 进行大于或小于的逻辑比较	
	代码片段: 48: size_t __len) 49: { 50: if (__fs->p + __len <= __fs->end __argp_fmtstream_ensure (__fs, __len)) 51: { 52: memcpy (__fs->p, __str, __len);	
	污染路径: 无	
36	缺陷发生位置:	行号:

	/utils/tsim/src/simExec.c	413
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 411: *(end + interval) = 0; 412: 413: for (char *p = end; p >= begin; p--) { 414: *(p + interval) = *p; 415: }	
	污染路径: 无	
37	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 860
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: 858: return BITv05_DStream_overflow; 859: 860: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) { 861: bitD->ptr -= bitD->bitsConsumed >> 3; 862: bitD->bitsConsumed &= 7;	
	污染路径: 无	
38	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 4049
	步骤描述: 禁止对指针 contLen 进行大于或小于的逻辑比较	
	代码片段: 4047: mInfo("vgld:%d, build compact vnode config req", pVgroup->vgld); 4048: int32_t contLen = tSerializeSCompactVnodeReq(NULL, 0, &compactReq); 4049: if (contLen < 0) { 4050: terrno = TSDB_CODE_OUT_OF_MEMORY; 4051: return NULL;	
	污染路径: 无	
39	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1784

	步骤描述: 禁止对指针 p1 进行大于或小于的逻辑比较	
	代码片段: 1782: 1783: // like 'select count(*),sum(age) fr' need return true 1784: if (plast && plast > p3 && p2 > p1 && plast > p2 && p1 > p3) { 1785: return true; 1786: }	
	污染路径: 无	
40	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1253
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1251: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 1252: while (charnum < n0) normalizedCounter[charnum++] = 0; 1253: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 1254: ip += bitCount>>3; 1255: bitCount &= 7;	
	污染路径: 无	
41	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 3403
	步骤描述: 禁止对指针 code 进行大于或小于的逻辑比较	
	代码片段: 3401: int32_t lino = 0; 3402: int32_t code = pMetaReaderFn->getTableEntryByUid(pMReader, uid); 3403: if (code < 0) { 3404: qError("failed to get table meta, uid:%" PRIu64 " ", code:%s, %s", uid, strerror(errno), idStr); 3405: return code;	
	污染路径: 无	

42	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 530
	步骤描述: 禁止对指针 e 进行大于或小于的逻辑比较	
	代码片段: 528: e = e1; 529: } else if(e && e1) { 530: if (e > e1) { 531: e = e1; 532: }	
	污染路径: 无	
43	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1754
	步骤描述: 禁止对指针 p1 进行大于或小于的逻辑比较	
	代码片段: 1752: // like select count(' ' 1753: return false; 1754: } else if (p1 && p2 && p1 > p2) { 1755: // like select sum(age), count(' ' 1756: return false;	
	污染路径: 无	
44	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 240
	步骤描述: 禁止对指针 U1 进行大于或小于的逻辑比较	
	代码片段: 238: } 239: 240: return U1 < U1n ? 241: (U2 < U2n ? *U1 - *U2 : 1) : 242: (U2 < U2n ? -1 : 0);	
	污染路径: 无	
45	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1282
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	

	代码片段: 1280: } 1281: 1282: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 1283: ip += bitCount>>3; 1284: bitCount &= 7;	
	污染路径: 无	
46	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 85
	步骤描述: 禁止对指针 last 进行大于或小于的逻辑比较	
	代码片段: 83: strncpy(l, s, ASCIILINESZ); 84: last = l + strlen(l); 85: while (last > l) { 86: if (!isspace((int)*(last-1))) 87: break ;	
	污染路径: 无	
47	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 340
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 338: static int32_t smlParseColLine(SSmlHandle *info, char **sql, char *sqlEnd, SSmlLineInfo *currElement) { 339: int cnt = 0; 340: while (*sql < sqlEnd) { 341: if (unlikely(IS_SPACE(*sql,NULL))) { 342: break;	
	污染路径: 无	
48	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 236
	步骤描述: 禁止对指针 U1 进行大于或小于的逻辑比较	
	代码片段: 234: U1n = T + *(p1 + 1) + 2,	

	235: U2n = T + *(p2 + 1) + 2; 236: (U1 < U1n) && (U2 < U2n) && (*U1 == *U2); 237: ++U1, ++U2) { 238: }	
	污染路径: 无	
49	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 473
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: 471: U32 nbBytes = bitD->bitsConsumed >> 3; 472: BIT_DStream_status result = BIT_DStream_unfinished; 473: if (bitD->ptr - nbBytes < bitD->start) 474: { 475: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */	
	污染路径: 无	
50	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 273
	步骤描述: 禁止对指针 bitC 进行大于或小于的逻辑比较	
	代码片段: 271: BIT_addBitsFast(bitC, 1, 1); /* endMark */ 272: BIT_flushBits(bitC); 273: if (bitC->ptr >= bitC->endPtr) return 0; /* overflow detected */ 274: return (bitC->ptr - bitC->startPtr) + (bitC->bitPos > 0); 275: }	
	污染路径: 无	
51	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1222
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1220: if (n0 > *maxSVPtr) return	

	ERROR(maxSymbolValue_tooSmall); 1221: while (charnum < n0) normalizedCounter[charnum++] = 0; 1222: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) 1223: { 1224: ip += bitCount>>3;	
	污染路径: 无	
52	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1784
	步骤描述: 禁止对指针 plast 进行大于或小于的逻辑比较	
	代码片段: 1782: 1783: // like 'select count(*),sum(age) fr' need return true 1784: if (plast && plast > p3 && p2 > p1 && plast > p2 && p1 > p3) { 1785: return true; 1786: }	
	污染路径: 无	
53	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 103
	步骤描述: 禁止对指针 remainder 进行大于或小于的逻辑比较	
	代码片段: 101: 102: remainder = pBuf->numOfTables % pBuf->numPerBucket; 103: if (remainder > 0) { 104: TSDB_CHECK_CONDITION(taosArrayGetSize(pBuf->pData) > 0, code, lino, _end, TSDB_CODE_INVALID_PARA); 105: px = taosArrayPop(pBuf->pData);	
	污染路径: 无	
54	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 121
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	

	代码片段: <pre> 119: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 120: while (charnum < n0) normalizedCounter[charnum++] = 0; 121: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend- 4)) { 122: assert((bitCount >> 3) <= 3); /* For first condition to work */ 123: ip += bitCount>>3; </pre>	
	污染路径: 无	
55	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1784
	步骤描述: 禁止对指针 p2 进行大于或小于的逻辑比较	
	代码片段: <pre> 1782: 1783: // like 'select count(*),sum(age) fr' need return true 1784: if (plast && plast > p3 && p2 > p1 && plast > p2 && p1 > p3) { 1785: return true; 1786: } </pre>	
	污染路径: 无	
56	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 1808
	步骤描述: 禁止对指针 idx 进行大于或小于的逻辑比较	
	代码片段: <pre> 1806: static int32_t extWinGetMultiTbWinFromTs(SOperatorInfo* pOperator, SArray* pWins, int64_t* tsCol, int64_t rowNum, int32_t* startPos) { 1807: int32_t idx = taosArraySearchIdx(pWins, tsCol, extWinTsWinCompare, TD_EQ); 1808: if (idx >= 0) { 1809: *startPos = 0; 1810: return idx; </pre>	
	污染路径: 	

	无	
57	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1237
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: <pre> 1235: while ((bitStream & 0xFFFF) == 0xFFFF) { 1236: n0+=24; 1237: if (ip < iend-5) { 1238: ip+=2; 1239: bitStream = MEM_readLE32(ip) >> bitCount;</pre>	
	污染路径: 无	
58	缺陷发生位置: /contrib/lemon/lempar.c	行号: 905
	步骤描述: 禁止对指针 yypParser 进行大于或小于的逻辑比较	
	代码片段: <pre> 903: 904: while(1){ /* Exit by "break" */ 905: assert(yypParser->yptos>=yypParser->yystack); 906: assert(yyact==yypParser->yptos->stateno); 907: yyact = yy_find_shift_action((YYCODETYPE)yymajor,yyact);</pre>	
	污染路径: 无	
59	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 223
	步骤描述: 禁止对指针 op 进行大于或小于的逻辑比较	
	代码片段: <pre> 221: 222: /* 4 symbols per loop */ 223: for (; (BIT_reloadDStream(&bitD)==BIT_DStream_unfinished) & (op<olimit) ; op+=4) { 224: op[0] = FSE_GETSYMBOL(&state1); 225:</pre>	
	污染路径:	

	无	
60	缺陷发生位置: /source/util/test/memPoolTest.cpp	行号: 424
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 422: char* pEnd = (char*)pStart + size - 1; 423: char* p = (char*)pStart; 424: while (p <= pEnd) { 425: *p = 'a' + taosRand() % 26; 426: p += 4096;	
	污染路径: 无	
61	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 667
	步骤描述: 禁止对指针 group 进行大于或小于的逻辑比较	
	代码片段: 665: err = group_parse (group, &parser->state, ARGP_KEY_NO_ARGS, 0); 666: for (group = parser->egroup - 1; 667: group >= parser->groups && (!err err==EBADKEY); 668: group--) 669: err = group_parse (group, &parser->state, ARGP_KEY_END, 0);	
	污染路径: 无	
62	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1212
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1210: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 1211: while (charnum < n0) normalizedCounter[charnum++] = 0; 1212: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) 1213: {	

	1214: ip += bitCount>>3;	
	污染路径: 无	
63	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 59
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 57: 58: static void smlParseTelnetElement(char **sql, char *sqlEnd, char **data, int32_t *len) { 59: while (*sql < sqlEnd) { 60: if (unlikely((*sql != SPACE && !(*data)))) { 61: *data = *sql;	
	污染路径: 无	
64	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 983
	步骤描述: 禁止对指针 idx 进行大于或小于的逻辑比较	
	代码片段: 981: static int32_t extWinGetMultiTbWinFromTs(SOperatorInfo* pOperator, SExternalWindowOperator* pExtW, int64_t* tsCol, int64_t rowNum, int32_t* startPos) { 982: int32_t idx = taosArraySearchIdx(pExtW->pWins, tsCol, extWinTsWinCompare, TD_EQ); 983: if (idx >= 0) { 984: *startPos = 0; 985: return idx;	
	污染路径: 无	
65	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 112
	步骤描述: 禁止对指针 key1 进行大于或小于的逻辑比较	
	代码片段: 110: if(p1 && key1 > p1) 111: found = false; 112: if(p2 && key1 > p2)	

	113: found = false; 114:	
	污染路径: 无	
66	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 129
	步骤描述: 禁止对指针 remainder 进行大于或小于的逻辑比较	
	代码片段: 127: } 128: 129: if (remainder > 0) { 130: p = taosMemoryCalloc(remainder, sizeof(STableBlockScanInfo)); 131: TSDB_CHECK_NULL(p, code, lino, _end, terrno);	
	污染路径: 无	
67	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1200
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1198: { 1199: n0+=24; 1200: if (ip < iend-5) 1201: { 1202: ip+=2;	
	污染路径: 无	
68	缺陷发生位置: /contrib/TSZ/zstd/common/zstd_internal.h	行号: 175
	步骤描述: 禁止对指针 op 进行大于或小于的逻辑比较	
	代码片段: 173: do 174: COPY8(op, ip) 175: while (op < oend); 176: } 177:	

	污染路径: 无	
69	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 261
	步骤描述: 禁止对指针 bitC 进行大于或小于的逻辑比较	
	代码片段: 259: MEM_writeLEST(bitC->ptr, bitC->bitContainer); 260: bitC->ptr += nbBytes; 261: if (bitC->ptr > bitC->endPtr) bitC->ptr = bitC->endPtr; 262: bitC->bitPos &= 7; 263: bitC->bitContainer >>= nbBytes*8;	
	污染路径: 无	
70	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 105
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 103: while ((bitStream & 0xFFFF) == 0xFFFF) { 104: n0 += 24; 105: if (ip < iend-5) { 106: ip += 2; 107: bitStream = MEM_readLE32(ip) >> bitCount;	
	污染路径: 无	
71	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 754
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: 752: U32 nbBytes = bitD->bitsConsumed >> 3; 753: U32 result = FSE_DStream_unfinished; 754: if (bitD->ptr - nbBytes < bitD->start) 755: { 756: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */	
	污染路径: 无	

72	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 84
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 82: 83: // parse key 84: while (sql < sqlEnd) { 85: if (unlikely(*sql == SPACE)) { 86: smlBuildInvalidDataMsg(&info->msgBuf, "invalid data", sql);	
	污染路径: 无	
73	缺陷发生位置: /contrib/lemon/lempar.c	行号: 1064
	步骤描述: 禁止对指针 i 进行大于或小于的逻辑比较	
	代码片段: 1062: char cDiv = '['; 1063: fprintf(yyTraceFILE,"%sReturn. Stack=",yyTracePrompt); 1064: for(i=&yypParser->yystack[1]; i<=yypParser->yynos; i++){ 1065: fprintf(yyTraceFILE,"%c%s", cDiv, yyTokenName[i- >major]); 1066: cDiv = ' ';	
	污染路径: 无	
74	缺陷发生位置: /source/common/src/tvariant.c	行号: 212
	步骤描述: 禁止对指针 endPtr 进行大于或小于的逻辑比较	
	代码片段: 210: } 211: if (pure) { 212: if (endPtr + 1 < end && endPtr[1] > '4') { 213: const char *p = z; 214: while (*p == ' ') {	
	污染路径: 无	

75	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 572
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 570: 571: { 572: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) 573: { 574: ip += bitCount>>3;	
	污染路径: 无	
76	缺陷发生位置: /contrib/libmqt/ttq/src/ttqMisc.c	行号: 87
	步骤描述: 禁止对指针 endptr 进行大于或小于的逻辑比较	
	代码片段: 85: } 86: endptr = &str[strlen(str) - 1]; 87: while (endptr > str && isspace(endptr[0])) { 88: endptr[0] = '\0'; 89: endptr--;	
	污染路径: 无	
77	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 458
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: 456: return BIT_DStream_overflow; 457: 458: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) 459: { 460: bitD->ptr -= bitD->bitsConsumed >> 3;	
	污染路径: 无	
78	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 873

	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: <pre> 871: U32 nbBytes = bitD->bitsConsumed >> 3; 872: BITv05_DStream_status result = BITv05_DStream_unfinished; 873: if (bitD->ptr - nbBytes < bitD->start) { 874: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ 875: result = BITv05_DStream_endOfBuffer; </pre>	
	污染路径: 无	
79	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1309
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: <pre> 1307: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 1308: while (charnum < n0) normalizedCounter[charnum++] = 0; 1309: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 1310: ip += bitCount>>3; 1311: bitCount &= 7; </pre>	
	污染路径: 无	
80	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 962
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: <pre> 960: return BITv06_DStream_overflow; 961: 962: if (bitD->ptr >= bitD->start + sizeof(bitD- >bitContainer)) { 963: bitD->ptr -= bitD->bitsConsumed >> 3; 964: bitD->bitsConsumed &= 7; </pre>	
	污染路径: 无	

81	缺陷发生位置： /source/libs/executor/src/externalwindowoperator.c	行号： 2092
	步骤描述： 禁止对指针 idx 进行大于或小于的逻辑比较	
	代码片段： 2090: SExtWinTimeWindow* pWin = NULL; 2091: int32_t idx = taosArraySearchIdx(pWins, tsCol, extWinTsWinCompare, TD_EQ); 2092: if (idx >= 0) { 2093: for (int i = idx - 1; i >= 0; --i) { 2094: pWin = TARRAY_GET_ELEM(pWins, i);	
	污染路径： 无	
82	缺陷发生位置： /source/libs/parser/src/parser.c	行号： 268
	步骤描述： 禁止对指针 pSql 进行大于或小于的逻辑比较	
	代码片段： 266: 267: // where clause 268: if (pSql < pEnd) { 269: bool inWhereClause = true; 270: int32_t bracketLevel = 0;	
	污染路径： 无	
83	缺陷发生位置： /contrib/TSZ/zstd/common/bitstream.h	行号： 422
	步骤描述： 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段： 420: return BIT_DStream_overflow; 421: 422: if (bitD->ptr >= bitD->limitPtr) { 423: bitD->ptr -= bitD->bitsConsumed >> 3; 424: bitD->bitsConsumed &= 7;	
	污染路径： 无	
84	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v06.c	行号： 1293
	步骤描述：	

	禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: <pre> 1291: while ((bitStream & 0xFFFF) == 0xFFFF) { 1292: n0+=24; 1293: if (ip < iend-5) { 1294: ip+=2; 1295: bitStream = MEM_readLE32(ip) >> bitCount;</pre>	
	污染路径: 无	
85	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 592
	步骤描述: 禁止对指针 parser 进行大于或小于的逻辑比较	
	代码片段: <pre> 590: /* Call each parser for the first time, giving it a chance to propagate 591: values to child parsers. */ 592: if (parser->groups < parser->egroup) 593: parser->groups->input = input; 594: for (group = parser->groups;</pre>	
	污染路径: 无	
86	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 1659
	步骤描述: 禁止对指针 range 进行大于或小于的逻辑比较	
	代码片段: <pre> 1657: } 1658: /* Job source in multiple segments not supported yet */ 1659: assert(range.start <= mtctx- >jobs[wJobID].src.start); 1660: return range; 1661: }</pre>	
	污染路径: 无	
87	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/	行号: 662

	argp/argp-parse.c	
	步骤描述: 禁止对指针 group 进行大于或小于的逻辑比较	
	代码片段: <pre> 660: { 661: for (group = parser->groups; 662: group < parser->egroup && (!err err==EBADKEY); 663: group++) 664: if (group->args_processed == 0) </pre>	
88	污染路径: 无	
	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 707
	步骤描述: 禁止对指针 input_pointer 进行大于或小于的逻辑比较	
89	代码片段: <pre> 705: output_pointer = output; 706: /* loop through the string literal */ 707: while (input_pointer < input_end) 708: { 709: if (*input_pointer != '\\') </pre>	
	污染路径: 无	
	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 207
90	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: <pre> 205: int cnt = 0; 206: 207: while (*sql < sqlEnd) { 208: if (unlikely(IS_SPACE(*sql,NULL))) { 209: break; </pre>	
	污染路径: 无	
90	缺陷发生位置: /contrib/lemon/lempar.c	行号: 426
	步骤描述: 禁止对指针 pParser 进行大于或小于的逻辑比较	

	代码片段: 424: yyStackEntry *yytos; 425: assert(pParser->yytos!=0); 426: assert(pParser->yytos > pParser->yystack); 427: yytos = pParser->yytos--; 428: #ifndef NDEBUG	
	污染路径: 无	
91	缺陷发生位置: /source/common/src/tvariant.c	行号: 303
	步骤描述: 禁止对指针 s 进行大于或小于的逻辑比较	
	代码片段: 301: const char *end = z + n; 302: bool pure = true; 303: while (s < end) { 304: if (*s < '0' *s > '9') { 305: pure = false;	
	污染路径: 无	
92	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1737
	步骤描述: 禁止对指针 p1 进行大于或小于的逻辑比较	
	代码片段: 1735: 1736: if (p1 && p2) { 1737: return p1 > p2 ? p1 + 1 : p2 + 1; 1738: } else if (p1) { 1739: return p1 + 1;	
	污染路径: 无	
93	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 536
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 534: if (n0 > *maxSVPtr) return (size_t)- FSE_ERROR_maxSymbolValue_tooSmall; 535: while (charnum < n0)	

	<pre>normalizedCounter[charnum++] = 0; 536: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) 537: { 538: ip += bitCount>>3;</pre>	
	污染路径: 无	
94	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 110
	步骤描述: 禁止对指针 key1 进行大于或小于的逻辑比较	
	代码片段: <pre>108: char* p1 = strstr(value+1, "+"); 109: char* p2 = strstr(value+1, "-"); 110: if(p1 && key1 > p1) 111: found = false; 112: if(p2 && key1 > p2)</pre>	
	污染路径: 无	
95	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 632
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: <pre>630: return BITv07_DStream_overflow; 631: 632: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) { 633: bitD->ptr -= bitD->bitsConsumed >> 3; 634: bitD->bitsConsumed &= 7;</pre>	
	污染路径: 无	
96	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 273
	步骤描述: 禁止对指针 lp 进行大于或小于的逻辑比较	
	代码片段: <pre>271: int result = COVER_cmp8(g_ctx, lp, rp); 272: if (result == 0) { 273: result = lp < rp ? -1 : 1;</pre>	

	274: } 275: return result;	
	污染路径: 无	
97	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1325
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1323: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 1324: while (charnum < n0) normalizedCounter[charnum++] = 0; 1325: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 1326: ip += bitCount>>3; 1327: bitCount &= 7;	
	污染路径: 无	
98	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 245
	步骤描述: 禁止对指针 op 进行大于或小于的逻辑比较	
	代码片段: 243: /* note : BIT_reloadDStream(&bitD) >= FSE_DStream_partiallyFilled; Ends at exactly BIT_DStream_completed */ 244: while (1) { 245: if (op>(omax-2)) return ERROR(dstSize_tooSmall); 246: *op++ = FSE_GETSYMBOL(&state1); 247: if (BIT_reloadDStream(&bitD)==BIT_DStream_overflow) {	
	污染路径: 无	
99	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 351
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 349: size_t keyLenEscaped = 0;	

	<pre>350: const char *escapeChar = NULL; 351: while (*sql < sqlEnd) { 352: if (unlikely(IS_SPACE(*sql,escapeChar) IS_COMMA(*sql,escapeChar))) { 353: smlBuildInvalidDataMsg(&info->msgBuf, "SML line invalid data", *sql);</pre>	
	污染路径: 无	
100	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 428
	步骤描述: 禁止对指针 newSize 进行大于或小于的逻辑比较	
	代码片段: 426: taosArrayClear(pldx); 427: int32_t newSize = (int32_t)taosArrayGetSize(pNewIdx); 428: if (newSize > 0) { 429: void* dest = taosArrayReserve(pldx, newSize); 430: TSDB_CHECK_NULL(dest, code, lino, _exit, terrno);	
	污染路径: 无	
101	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 241
	步骤描述: 禁止对指针 U2 进行大于或小于的逻辑比较	
	代码片段: 239: 240: return U1 < U1n ? 241: (U2 < U2n ? *U1 - *U2 : 1) : 242: (U2 < U2n ? -1 : 0); 243: }	
	污染路径: 无	
102	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1257
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1255: 1256: { 1257: if ((ip <= iend-7) (ip + (bitCount>>3) <=	

	iend-4)) 1258: { 1259: ip += bitCount>>3;	
	污染路径: 无	
103	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1258
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1256: 1257: { 1258: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) 1259: { 1260: ip += bitCount>>3;	
	污染路径: 无	
104	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1309
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1307: while ((bitStream & 0xFFFF) == 0xFFFF) { 1308: n0+=24; 1309: if (ip < iend-5) { 1310: ip+=2; 1311: bitStream = MEM_readLE32(ip) >> bitCount;	
	污染路径: 无	
105	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 513
	步骤描述: 禁止对指针 op 进行大于或小于的逻辑比较	
	代码片段: 511: do 512: COPY8(op, ip) 513: while (op < oend); 514: }	

	515:	
	污染路径: 无	
106	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 175
	步骤描述: 禁止对指针 fs 进行大于或小于的逻辑比较	
	代码片段: 173: { 174: _argp_fmtstream_update (fs); 175: if (fs->p > fs->buf) 176: { 177: fwrite (fs->buf, 1, fs->p - fs->buf, fs->stream);	
	污染路径: 无	
107	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 252
	步骤描述: 禁止对指针 op 进行大于或小于的逻辑比较	
	代码片段: 250: } 251: 252: if (op>(omax-2)) return ERROR(dstSize_tooSmall); 253: *op++ = FSE_GETSYMBOL(&state2); 254: if (BIT_reloadDStream(&bitD)==BIT_DStream_overflow) {	
	污染路径: 无	
108	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 2526
	步骤描述: 禁止对指针 vgNum 进行大于或小于的逻辑比较	
	代码片段: 2524: 2525: int32_t vgNum = (int32_t)taosArrayGetSize(dbInfo->vgArray); 2526: if (vgNum <= 0) { 2527: qError("db vgroup cache invalid, db:%s, vgroup number:%d", dbFName, vgNum);	

	2528: code = TSDB_CODE_TSC_DB_NOT_SELECTED;	
	污染路径: 无	
109	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 242
	步骤描述: 禁止对指针 U2 进行大于或小于的逻辑比较	
	代码片段: 240: return U1 < U1n ? 241: (U2 < U2n ? *U1 - *U2 : 1) : 242: (U2 < U2n ? -1 : 0); 243: } 244:	
	污染路径: 无	
110	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1191
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: 1189: { 1190: n0+=24; 1191: if (ip < iend-5) 1192: { 1193: ip+=2;	
	污染路径: 无	
111	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 411
	步骤描述: 禁止对指针 beg 进行大于或小于的逻辑比较	
	代码片段: 409: find_char (char ch, char *beg, char *end) 410: { 411: while (beg < end) 412: if (*beg == ch) 413: return 1;	
	污染路径:	

	无	
112	缺陷发生位置: /source/util/src/talgo.c	行号: 255
	步骤描述: 禁止对指针 run_ptr 进行大于或小于的逻辑比较	
	代码片段: 253: char *run_ptr; 254: 255: for (run_ptr = tmp_ptr + size; run_ptr <= thresh; run_ptr += size) 256: if ((*cmp)((void *)run_ptr, (void *)tmp_ptr, arg) < 0) tmp_ptr = run_ptr; 257:	
	污染路径: 无	
113	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 808
	步骤描述: 禁止对指针 tagStart 进行大于或小于的逻辑比较	
	代码片段: 806: if (stbInfo->useTagName) { 807: char *firstComma = strchr(tagStart, ','); 808: if (firstComma == NULL && tagStart >= firstComma) { 809: errorPrint("Invalid tag data format: %s\n", tagStart); 810: return -1;	
	污染路径: 无	
114	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 236
	步骤描述: 禁止对指针 U2 进行大于或小于的逻辑比较	
	代码片段: 234: U1n = T + *(p1 + 1) + 2, 235: U2n = T + *(p2 + 1) + 2; 236: (U1 < U1n) && (U2 < U2n) && (*U1 == *U2); 237: ++U1, ++U2) { 238: }	
	污染路径:	

	无	
115	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1221
	步骤描述: 禁止对指针 ip 进行大于或小于的逻辑比较	
	代码片段: <pre> 1219: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 1220: while (charnum < n0) normalizedCounter[charnum++] = 0; 1221: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) 1222: { 1223: ip += bitCount>>3; </pre>	
	污染路径: 无	
116	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 754
	步骤描述: 禁止对指针 bitD 进行大于或小于的逻辑比较	
	代码片段: <pre> 752: U32 nbBytes = bitD->bitsConsumed >> 3; 753: BIT_DStream_status result = BIT_DStream_unfinished; 754: if (bitD->ptr - nbBytes < bitD->start) 755: { 756: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ </pre>	
	污染路径: 无	
117	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 219
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: <pre> 217: const char *escapeChar = NULL; 218: 219: while (*sql < sqlEnd) { 220: if (unlikely(IS_SPACE(*sql,escapeChar) IS_COMMA(*sql,escapeChar))) { 221: smlBuildInvalidDataMsg(&info->msgBuf, "invalid </pre>	

	data", *sql);	
	污染路径: 无	
118	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 4002
	步骤描述: 禁止对指针 numCols 进行大于或小于的逻辑比较	
	代码片段: 4000: int32_t numRows = pBlock->info.rows; 4001: int32_t numCols = taosArrayGetSize(pBlock->pDataBlock); 4002: if (numCols < 2) { 4003: qError("%s invalid external-window block: expected at least 2 columns, got %d", __func__, numCols); 4004: TAOS_CHECK_EXIT(TSDB_CODE_INVALID_PARA);	
	污染路径: 无	
119	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 145
	步骤描述: 禁止对指针 start 进行大于或小于的逻辑比较	
	代码片段: 143: } 144: 145: if (start < current) { 146: processSingleToken(start, fields, index, isTag); 147: }	
	污染路径: 无	
120	缺陷发生位置: /contrib/lemon/lempar.c	行号: 447
	步骤描述: 禁止对指针 yytos 进行大于或小于的逻辑比较	
	代码片段: 445: ** element left in the stack */ 446: yyStackEntry *yytos = pParser->yytos; 447: while(yytos>pParser->yystack){ 448: #ifndef NDEBUB 449: if(yyTraceFILE){	

	污染路径: 无	
121	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 2422
	步骤描述: 禁止对指针 tempRes 进行大于或小于的逻辑比较	
	代码片段: 2420: 2421: int32_t tempRes = tSerializeSUseDbReq(buf, contLen, &usedbReq); 2422: if (tempRes < 0) { 2423: code = terrno; 2424: QUERY_CHECK_CODE(code, lino, _return);	
	污染路径: 无	

10.2.1.2 WK4761 禁止对无符号数进行大于等于零或小于零的

比较

缺陷数量：1 个		
缺陷描述: 禁止对无符号数进行大于等于零或小于零的比较。		
修复意见: 无		
1	缺陷发生位置: /contrib/lemon/lemon.c	行号: 91
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 89: static void *lemon_malloc(size_t nByte){ 90: MemChunk *p; 91: if(nByte<0) return 0; 92: p = malloc(nByte + sizeof(MemChunk)); 93: if(p==0){	
	污染路径: 无	

10.2.1.3 WK4762 禁止无符号数和有符号数之间的直接比较

缺陷数量： 61 个		
缺陷描述： 禁止无符号数和有符号数之间的直接比较。		
修复意见： 无		
1	缺陷发生位置： /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号： 1631
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：T,c0	
	代码片段： 1629: *j = ~s; 1630: c0 = T[--s]; 1631: if((0 < s) && (T[s - 1] > c0)) { s = ~s; } 1632: if(c0 != c2) { 1633: if(0 <= c2) { BUCKET_B(c2, c1) = k - SA; } 	
	污染路径： 无	
2	缺陷发生位置： /source/client/test/connectOptionsTest.cpp	行号： 945
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,test_mk	
	代码片段： 943: tm.tm_sec = 15; 944: 945: for (unsigned int i = 0; i < sizeof (test_mk) / sizeof (test_mk[0]); ++i) 946: { 947: timezone_t tzp = tzalloc(test_mk[i].env); 	
	污染路径： 无	
3	缺陷发生位置： /tools/taos-tools/src/benchInsertMix.c	行号： 336
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,info	
	代码片段：	

	334: 335: // other cols data 336: for(uint16_t i = 0; i< info->nBatCols; i++) { 337: Field* fd = benchArrayGet(stb->cols, GET_IDX(i)); 338: size += dataGenByCalcTs(fd, pstr, len + size, ts);	
	污染路径: 无	
4	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 276
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: lackBits,bitSize	
	代码片段: 274: 275: bitSize -= 64; 276: if(lackBits < bitSize) 277: { 278: *p = (*p) (unsigned char) ((huffmanTree->code[state])[0] >> (64 - lackBits));	
	污染路径: 无	
5	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 968
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,test_mk	
	代码片段: 966: tm.tm_sec = 15; 967: 968: for (unsigned int i = 0; i < sizeof (test_mk) / sizeof (test_mk[0]); ++i) 969: { 970: timezone_t tz = tzalloc(test_mk[i].env);	
	污染路径: 无	
6	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 308
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,info	
	代码片段: 306: 307: // other cols data	

	308: for(uint16_t i = 0; i< info->nBatCols; i++) { 309: Field* fd = benchArrayGet(stb->cols, GET_IDX(i)); 310: char* prefix = "";	
	污染路径: 无	
7	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 253
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: lackBits,bitSize	
	代码片段: 251: { 252: *p = (*p) (unsigned char)((huffmanTree->code[state])[0] >> (64 - lackBits)); 253: if(lackBits < bitSize) 254: { 255: p++;	
	污染路径: 无	
8	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1686
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,x	
	代码片段: 1684: pNext = rp->next; 1685: rp->next = 0; 1686: for(i=0; i<sizeof(x)/sizeof(x[0])-1 && x[i]; i++){ 1687: rp = Rule_merge(x[i], rp); 1688: x[i] = 0;	
	污染路径: 无	
9	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1106
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,mod	
	代码片段: 1104: } 1105: pThreadInfo->start_table_from = tableFrom; 1106: pThreadInfo->ntables = i < mod ? div + 1 : div; 1107: pThreadInfo->end_table_to = i < mod ?	

	tableFrom + div : tableFrom + div - 1; 1108: tableFrom = pthreadInfo->end_table_to + 1;	
	污染路径: 无	
10	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 900
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,test_tz	
	代码片段: 898: 899: TEST(timezoneCase, format_timezone_Test) { 900: for (unsigned int i = 0; i < sizeof (test_tz) / sizeof (test_tz[0]); ++i){ 901: timezone_str_test(test_tz[i].name, test_tz[i].t, test_tz[i].timezone); 902: timezone_str_test(test_tz[i].name, test_tz[i].t, test_tz[i].timezone);	
	污染路径: 无	
11	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 519
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: Td,v	
	代码片段: 517: } else { 518: limit += 1; 519: if(Td[PA[*first] - 1] < v) { 520: first = ss_partition(PA, first, last, depth); 521: limit = ss_ilg(last - first);	
	污染路径: 无	
12	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1694
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,x	
	代码片段: 1692: } 1693: rp = 0; 1694: for(i=0; i<sizeof(x)/sizeof(x[0]); i++){ 1695: rp = Rule_merge(x[i], rp);	

	1696: }	
	污染路径: 无	
13	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 132
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,arr1	
	代码片段: 130: uint64_t arr1[] = {2, 3, 4, 5}; 131: SArray * f = (SArray *)taosArrayGetP(src, 0); 132: for (int i = 0; i < sizeof(arr1) / sizeof(arr1[0]); i++) { 133: taosArrayPush(f, &arr1[i]); 134: }	
	污染路径: 无	
14	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1101
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,bindNum	
	代码片段: 1099: } 1100: 1101: for (int32_t i = 0; i < bindNum; ++i) { 1102: data->tsData[i] = bpTs + tblIdx*gCurCase->rowNum + rand()%gCurCase->rowNum; 1103: data->boolData[i] = (bool)(tblIdx*gCurCase->rowNum + rand() % gCurCase->rowNum);	
	污染路径: 无	
15	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 226
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,nbSamples	
	代码片段: 224: size_t sum = 0; 225: size_t i; 226: for (i = 0; i < nbSamples; ++i) { 227: sum += samplesSizes[i]; 228: }	

	污染路径: 无	
16	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1884
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: FSE_reloadDStream,FSE_DStream_completed	
	代码片段: 1882: FSE_initDState(&(seqState.stateML), &(seqState.DStream), DTableML); 1883: 1884: for (; (FSE_reloadDStream(&(seqState.DStream)) <= FSE_DStream_completed) && (nbSeq>0) ;) 1885: { 1886: size_t oneSeqSize;	
	污染路径: 无	
17	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 268
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: rand_num, stb	
	代码片段: 266: if (stb->disorderRatio > 0) { 267: unsigned int rand_num = taosRandom() % 100; 268: if (rand_num < stb->disorderRatio) { 269: disorderRange--; 270: if (0 == disorderRange) {	
	污染路径: 无	
18	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1118
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i, bindNum	
	代码片段: 1116: } 1117: 1118: for (int32_t i = 0; i < bindNum; ++i) { 1119: data->tsData[i] = bpTs + tblIdx * gCurCase->rowNum + rand() % gCurCase->rowNum; 1120: data->boolData[i] = (bool)(tblIdx * gCurCase->rowNum + rand() % gCurCase->rowNum);	

	污染路径: 无	
22	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 2015
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,nSubmitTbData	
	代码片段: 2013: ST_TASK_DLOG("%s nSubmitTbData:%" PRIu64 ", ver: %"PRIu64 " bodyLen:%d", __func__, nSubmitTbData, ver, len); 2014: 2015: for (int32_t i = 0; i < nSubmitTbData; i++) { 2016: uint64_t gid = -1; 2017: int64_t uid = 0;	
	污染路径: 无	
23	缺陷发生位置: /source/client/test/session.cpp	行号: 687
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,params1	
	代码片段: 685: * 3. test timestamp with string format 686: */ 687: for (unsigned int i = 0; i < sizeof(params1) / sizeof(params1[0]); ++i) { 688: do_insert(params1[i]); 689: do_select(params1[i]);	
	污染路径: 无	
24	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 613
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,tz_test	
	代码片段: 611: TAOS* pConn = getConnWithGlobalOption("UTC-8"); 612: 613: for (unsigned int i = 0; i < sizeof (tz_test) / sizeof (tz_test[0]); ++i){ 614: char sql[1024] = {0}; 615: (void)snprintf(sql, sizeof(sql), "insert into db1.ctb1 values('%s', '%s', 1)", tz_test[i], tz_test[i]);	

	污染路径: 无	
25	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 445
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: bufCnt,maxFill	
	代码片段: 443: 444: if (force) { 445: rdFill = bufCnt > maxFill ? maxFill : bufCnt; 446: } else { 447: if (rdFill > bufCnt) {	
	污染路径: 无	
26	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1019
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: b,allRowNum	
	代码片段: 1017: } 1018: 1019: for (int b = 0; b < (allRowNum / gCurCase->bindRowNum); b++) { 1020: for (int c = 0; c < gCurCase->bindColNum; ++c) { 1021: prepareColData(false, BP_BIND_COL, data, b * gCurCase->bindColNum + c, b * gCurCase->bindRowNum, c);	
	污染路径: 无	
27	缺陷发生位置: /source/libs/tss/test/tssTest.cpp	行号: 93
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,dataSize	
	代码片段: 91: GTEST_ASSERT_EQ(code, TSDB_CODE_SUCCESS); 92: GTEST_ASSERT_EQ(dataSize, 1024 * 1024); 93: for(uint64_t i = 0; i < dataSize; i++) { 94: GTEST_ASSERT_EQ(data[i], 'b'); 95: }	
	污染路径:	

	无	
28	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 1649
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: T,c2	
	代码片段: 1647: the sorted order of type B suffixes. */ 1648: k = SA + BUCKET_A(c2 = T[n - 1]); 1649: *k++ = (T[n - 2] < c2) ? ~(n - 1) : (n - 1); 1650: /* Scan the suffix array from left to right. */ 1651: for(i = SA, j = SA + n; i < j; ++i) {	
	污染路径: 无	
29	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 1075
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: iRow,nRow	
	代码片段: 1073: } 1074: 1075: for (int32_t iRow = 0; iRow < nRow; ++iRow) { 1076: SRow *pRow = (SRow *) (pCoder->data + pCoder->pos); 1077: pCoder->pos += pRow->len;	
	污染路径: 无	
30	缺陷发生位置: /source/client/test/session.cpp	行号: 607
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,tz_test	
	代码片段: 605: TAOS* pConn = getConnWithGlobalOption("UTC-8"); 606: 607: for (unsigned int i = 0; i < sizeof(tz_test) / sizeof(tz_test[0]); ++i) { 608: char sql[1024] = {0}; 609: (void)snprintf(sql, sizeof(sql), "insert into db1.ctb1 values('%s', '%s', 1)", tz_test[i], tz_test[i]);	
	污染路径: 无	

31	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1992
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: ds_len,stbInfo	
	代码片段: <pre> 1990: } 1991: // check buffer enough 1992: if (ds_len(pThreadInfo->buffer) 1993: > stbInfo->max_sql_len) { 1994: errorPrint("sql buffer length (%"PRIu64") " </pre>	
	污染路径: 无	
32	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1870
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,nBatchTable	
	代码片段: <pre> 1868: 1869: // loop each table 1870: for (i = 0; i < nBatchTable; i++) { 1871: if (g_arguments->terminate) { 1872: goto free_of_interlace; </pre>	
	污染路径: 无	
33	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1151
	步骤描述: 禁止无符号数和有符号数之间的直接比较 , 两个变量名称分别为 : FSE_reloadDStream,FSE_DStream_completed	
	代码片段: <pre> 1149: bitTail.bitContainer = bitD1.bitContainer; // required in case of FSE_DStream_endOfBuffer 1150: bitTail.start = start1; 1151: for (; (FSE_reloadDStream(&bitTail) < FSE_DStream_completed) && (op<omax) ; op++) 1152: { 1153: HUF_DECODE_SYMBOL_0(0, bitTail); </pre>	
	污染路径: 无	

34	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 991
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,allRowNum	
	代码片段: 989: } 990: 991: for (int32_t i = 0; i < allRowNum; ++i) { 992: if (gCurCase->duplicateValue) { 993: data->tsData[i] = bpTs;	
	污染路径: 无	
35	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 2803
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: b,batch	
	代码片段: 2801: param->length = pThreadInfo->lengths[c]; 2802: 2803: for (int b = 0; b < batch; b++) { 2804: param->length[b] = length; 2805: }	
	污染路径: 无	
36	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 830
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,prec	
	代码片段: 828: } else { 829: uint8_t prec = randomInt(config.prec - config.scale), scale = randomInt(config.scale); 830: for (int i = 0; i < prec; ++i) { 831: ret.push_back(generateDigit()); 832: }	
	污染路径: 无	
37	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 144
	步骤描述:	

	禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,arr3	
	代码片段: <pre> 142: uint64_t arr3[] = {3, 5, 10, 11}; 143: f = (SArray *)taosArrayGetP(src, 2); 144: for (int i = 0; i < sizeof(arr3) / sizeof(arr3[0]); i++) { 145: taosArrayPush(f, &arr3[i]); 146: }</pre>	
	污染路径: 无	
38	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 847
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: FSE_reloadDStream,FSE_DStream_unfinished	
	代码片段: <pre> 845: 846: if (FSE_MAX_TABLELOG*4+7 > sizeof(bitD.bitContainer)*8) /* This test must be static */ 847: { if (FSE_reloadDStream(&bitD) > FSE_DStream_unfinished) { op+=2; break; } } 848: 849: op[2] = FSE_GETSYMBOL(&state1);</pre>	
	污染路径: 无	
39	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1475
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,k	
	代码片段: <pre> 1473: } else { 1474: int len = 0; 1475: for (int i = 0; i < k; i++) { 1476: if (strlen(pThreadInfo->lines[i]) != 0) { 1477: int n;</pre>	
	污染路径: 无	
40	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 163
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,arr2	

	代码片段: 161: uint64_t arr2[] = {8, 10, 12, 13}; 162: f = (SArray *)taosArrayGetP(src, 1); 163: for (int i = 0; i < sizeof(arr2) / sizeof(arr2[0]); i++) { 164: taosArrayPush(f, &arr2[i]); 165: }	
	污染路径: 无	
41	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 486
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: v,Td	
	代码片段: 484: 485: a = first + (b - a), c = last - (d - c); 486: b = (v <= Td[PA[*a] - 1]) ? a : ss_partition(PA, a, c, depth); 487: 488: if((a - first) <= (last - c)) {	
	污染路径: 无	
42	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 138
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,arr2	
	代码片段: 136: uint64_t arr2[] = {1, 2, 3, 5}; 137: f = (SArray *)taosArrayGetP(src, 1); 138: for (int i = 0; i < sizeof(arr2) / sizeof(arr2[0]); i++) { 139: taosArrayPush(f, &arr2[i]); 140: }	
	污染路径: 无	
43	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1061
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,bindNum	
	代码片段: 1059: } 1060:	

	1061: for (int32_t i = 0; i < bindNum; ++i) { 1062: data->tsData[i] = bpTs + tblIdx * gCurCase->rowNum + rand() % gCurCase->rowNum; 1063: data->boolData[i] = (bool)(tblIdx * gCurCase->rowNum + rand() % gCurCase->rowNum);	
	污染路径: 无	
44	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1082
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: b,bindNum	
	代码片段: 1080: } 1081: 1082: for (int b = 0; b < bindNum; b++) { 1083: for (int c = 0; c < gCurCase->bindColNum; ++c) { 1084: prepareColData(true, BP_BIND_COL, data, b * gCurCase->bindColNum + c, b * gCurCase->bindRowNum, c);	
	污染路径: 无	
45	缺陷发生位置: /contrib/TSZ/sz/src/transcode.c	行号: 46
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,intervals	
	代码片段: 44: uint32_t* diff = (uint32_t *)malloc(intervals * sizeof(uint32_t)); 45: 46: for (int i = md; i < intervals; i++) 47: { 48: type2code[i] = (uint8_t)Int2code(i - md);	
	污染路径: 无	
46	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 984
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: value,MEM_32bits	
	代码片段: 982: & (value < ZSTDMT_JOBSIZE_MIN)) 983: value = ZSTDMT_JOBSIZE_MIN;	

	984: if (value > ZSTDMT_JOBSIZE_MAX) 985: value = ZSTDMT_JOBSIZE_MAX; 986: params->jobSize = value;	
	污染路径: 无	
47	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 243
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: index,queryTimes	
	代码片段: 241: } 242: 243: while (index < (int64_t)queryTimes) { 244: // use cancel 245: if(g_arguments->terminate) {	
	污染路径: 无	
48	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1107
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,mod	
	代码片段: 1105: pThreadInfo->start_table_from = tableFrom; 1106: pThreadInfo->ntables = i < mod ? div + 1 : div; 1107: pThreadInfo->end_table_to = i < mod ? tableFrom + div : tableFrom + div - 1; 1108: tableFrom = pThreadInfo->end_table_to + 1; 1109: pThreadInfo->tables_created = 0;	
	污染路径: 无	
49	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 687
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,params1	
	代码片段: 685: * 3. test timestamp with string format 686: */ 687: for (unsigned int i = 0; i < sizeof (params1) / sizeof (params1[0]); ++i){	

	688: do_insert(params1[i]); 689: do_select(params1[i]);	
	污染路径: 无	
50	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 428
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: Td,v	
	代码片段: <pre> 426: } 427: } 428: if(Td[PA[*first] - 1] < v) { 429: first = ss_partition(PA, first, a, depth); 430: } </pre>	
	污染路径: 无	
51	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 157
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,arr1	
	代码片段: <pre> 155: uint64_t arr1[] = {13, 14, 15}; 156: SArray * f = (SArray *)taosArrayGetP(src, 0); 157: for (int i = 0; i < sizeof(arr1) / sizeof(arr1[0]); i++) { 158: taosArrayPush(f, &arr1[i]); 159: } </pre>	
	污染路径: 无	
52	缺陷发生位置: /source/libs/tss/test/tssTest.cpp	行号: 252
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,largeFileSizeInMB	
	代码片段: <pre> 250: data = (char*)taosMemoryMalloc(dataSize); 251: GTEST_ASSERT_TRUE(data != NULL); 252: for (uint64_t i = 0; i < largeFileSizeInMB; i++) { 253: memset(data + i * 1024 * 1024, 'a' + i % 26, 1024 * 1024); 254: } </pre>	

	污染路径: 无	
53	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 840
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,scale	
	代码片段: 838: } 839: ret.push_back('.'); 840: for (int i = 0; i < scale; ++i) { 841: ret.push_back(generateDigit()); 842: }	
	污染路径: 无	
54	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 2859
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: n,num	
	代码片段: 2857: (void)printf("%d - expired stableNum:%d\n", i, num); 2858: if (stable) { 2859: for (int32_t n = 0; n < num; ++n) { 2860: (void)printf("suid:%" PRIu64 " ", dbFName:%s, 2861: stable[n].dbFName, stable[n].stbName, stable[n].sversion, stable[n].tversion);	
	污染路径: 无	
55	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1064
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: b,bindNum	
	代码片段: 1062: } 1063: 1064: for (int b = 0; b < bindNum; b++) { 1065: for (int c = 0; c < gCurCase->bindColNum; ++c) { 1066: prepareColData(true, BP_BIND_COL, data, b*gCurCase->bindColNum+c, b*gCurCase->bindRowNum, c);	
	污染路径:	

	无	
56	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1044
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,bindNum	
	代码片段: 1042: } 1043: 1044: for (int32_t i = 0; i < bindNum; ++i) { 1045: data->tsData[i] = bpTs + tblIdx*gCurCase->rowNum + rand()%gCurCase->rowNum; 1046: data->boolData[i] = (bool)(tblIdx*gCurCase->rowNum + rand() % gCurCase->rowNum);	
	污染路径: 无	
57	缺陷发生位置: /source/libs/tss/test/tssTest.cpp	行号: 270
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,dataSize	
	代码片段: 268: GTEST_ASSERT_EQ(code, TSDB_CODE_SUCCESS); 269: GTEST_ASSERT_EQ(dataSize, 1024 * 1024); 270: for(uint64_t i = 0; i < dataSize; i++) { 271: GTEST_ASSERT_EQ(data[i], 'b'); 272: }	
	污染路径: 无	
58	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 2885
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: j,batch	
	代码片段: 2883: colsTb[i].buffer_type = TSDB_DATA_TYPE_TIMESTAMP; 2884: colsTb[i].length = pThreadInfo->lengths[0]; 2885: for (int32_t j = 0; j < batch; j++) { 2886: colsTb[i].length[j] = sizeof(int64_t); 2887: }	
	污染路径: 无	

59	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1001
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: b,allRowNum	
	代码片段: 999: } 1000: 1001: for (int b = 0; b < (allRowNum/gCurCase->bindRowNum); b++) { 1002: for (int c = 0; c < gCurCase->bindColNum; ++c) { 1003: prepareColData(false, BP_BIND_COL, data, b*gCurCase->bindColNum+c, b*gCurCase->bindRowNum, c);	
	污染路径: 无	
60	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 1704
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: count,total	
	代码片段: 1702: if (count[s] != 0 && norm == 0) 1703: norm = 1; 1704: assert(count[s] < total); 1705: cost += count[s] * kInverseProbabiltyLog256[norm]; 1706: }	
	污染路径: 无	
61	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 861
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: FSE_reloadDStream,FSE_DStream_completed	
	代码片段: 859: while (1) 860: { 861: if ((FSE_reloadDStream(&bitD)>FSE_DStream_completed) (op==omax) (FSE_endOfDStream(&bitD) && (fast FSE_endOfDState(&state1)))) 862: break; 863:	

	污染路径: 无

11 C/C++缺陷类型-字符串问题

11.1 分项汇总表

序号	C/C++缺陷类型-字符串问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK5639	禁止字符串中单独使用“\”，字符串的终止必须使用\0	12	严重
2	WK4778	用于表示字符串的数组必须以\0结束	19	严重

11.2 缺陷/安全漏洞详解

11.2.1 严重类

11.2.1.1 WK5639 禁止字符串中单独使用“\”，字符串的终止必须使用\0

缺陷数量：12 个
缺陷描述： 只用“\”并不能控制字符串的终止，应该用“\0”控制字符串的终止。

修复意见： 无		
1	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp1.c	行号： 58
	步骤描述： 禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段： 56: { "rounds", 'r', "NUMBER", 0, "Number of rounds each thread runs" }, 57: 58: { NULL, 0, NULL, 0, "\ 59: There are a number of different methods how thread creation can be \ 60: synchronized. Synchronization is necessary since the number of concurrently \	
	污染路径： 无	
2	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp1.c	行号： 66
	步骤描述： 禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段： 64: { "sync-join", OPT_SYNC_JOIN, NULL, 0, "Synchronize using pthread_join" }, 65: 66: { NULL, 0, NULL, 0, "\ 67: One parameter for each threads execution is the size of the stack. If this \ 68: parameter is not used the system's default stack size is used. If many \	
	污染路径： 无	
3	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号： 298
	步骤描述： 禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段： 296: dgettext (state == NULL ? NULL	

	43: implies --sync-join" }, 44: 45: { NULL, 0, NULL, 0, "\n 46: Each thread can do one of two things: sleep or do work. The latter is 100% \n 47: CPU bound. The work load is the probability a thread does work. All values \n	
	污染路径: 无	
7	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 1285
	步骤描述: 禁止字符串中单独使用\\,字符串的终止必须使用\\0	
	代码片段: 1283: { 1284: const char *tstr = dgettext (state == NULL ? NULL 1285: : state->root_argp->argp_domain, "\n 1286: Mandatory or optional arguments to long options are also mandatory or \n 1287: optional for any corresponding short options.");	
	污染路径: 无	
8	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/argp-test.c	行号: 115
	步骤描述: 禁止字符串中单独使用\\,字符串的终止必须使用\\0	
	代码片段: 113: { 114: if (key == ARGV_KEY_HELP_EXTRA) 115: return strdup ("This is some extra text from the sub parser (note that it \n 116: is preceded by a blank line)."); 117: else	
	污染路径: 无	
9	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 831
	步骤描述:	

	禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段: 829: // define in getDuration() function 830: printf(831: "\n 832: Timestamp expression Format:\n\ 833: b - nanosecond \n\ 污染路径: 无	
10	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp1.c	行号: 34
	步骤描述: 禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段: 32: static const struct argp_option test_options[] = 33: { 34: { NULL, 0, NULL, 0, "\n 35: This is a test for threads so we allow ther user to selection the number of \ 36: threads which are used at any one time. Independently the total number of \ 污染路径: 无	
11	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 308
	步骤描述: 禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段: 306: argp_failure (state, 0, 0, 307: dgettext (state == NULL ? NULL 308: : state->root_argp->argp_domain, "\n 309: %.*s: Unknown ARGV_HELP_FMT parameter"), 310: (int) var_len, var); 污染路径: 无	
12	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 1708

	步骤描述: 禁止字符串中单独使用\,字符串的终止必须使用\0
	代码片段: 1706: if (flags & ARGP_HELP_SEE) 1707: { 1708: argp_fmtstream_printf (fs, dgettext (argp->argp_domain, "\n"), 1709: Try `'%s --help' or `'%s --usage' for more information.\n"), 1710: name, name);
	污染路径: 无

11.2.1.2 WK4778 用于表示字符串的数组必须以 \0 结束

缺陷数量：19 个		
缺陷描述： 用于表示字符串的数组必须以 \0 结束		
修复意见： 无		
1	缺陷发生位置： /source/util/src/tconfig.c	行号： 1187
	步骤描述： 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段： 1185: 1186: int32_t cfgLoadFromEnvVar(SConfig *pConfig) { 1187: char line[1024], *name, *value, *value2, *value3, *value4; 1188: int32_t olen, vlen, vlen2, vlen3, vlen4; 1189: int32_t code = 0;	
	污染路径： 无	
2	缺陷发生位置： /source/os/src/osSocket.c	行号： 581
	步骤描述： 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段：	

	579: } 580: #endif 581: char hostname[1024]; 582: hostname[1023] = '\0'; 583: int32_t code = taosGetlocalhostname(hostname, 1023);	
	污染路径: 无	
3	缺陷发生位置: /source/util/src/tcompression.c	行号: 489
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 487: 488: // Selector value: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 489: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 490: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 491:	
	污染路径: 无	
4	缺陷发生位置: /source/util/src/tuuid.c	行号: 23
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 21: int32_t taosGetSystemUUIDU32(uint32_t *uuid) { 22: if (uuid == NULL) return TSDB_CODE_APP_ERROR; 23: char uid[37] = {0}; 24: int32_t code = taosGetSystemUIDLimit36(uid, sizeof(uid)); 25: uid[36] = 0;	
	污染路径: 无	
5	缺陷发生位置: /utls/test/c/tmqSim.c	行号: 502
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段:	

	500: } 501: 502: char quotationStr[2]; 503: quotationStr[0] = '\\'; 504: quotationStr[1] = 0;	
	污染路径: 无	
6	缺陷发生位置: /test/new_test_framework/script/api/stmt2-example.c	行号: 108
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 106: int32_t b_len[2], t64_len[2], n_len[2]; 107: char is_null[2] = {0}; 108: char is_null2[2] = {0, 2}; 109: // TAOS_STMT2_OPTION option = {0}; 110: // TAOS_STMT2_OPTION option = {0, true, true, stmtAsyncQueryCb, NULL};	
	污染路径: 无	
7	缺陷发生位置: /source/util/src/tuuid.c	行号: 37
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 35: int32_t taosGetSystemUUIDU64(uint64_t *uuid) { 36: if (uuid == NULL) return TSDB_CODE_APP_ERROR; 37: char uid[37] = {0}; 38: int32_t code = taosGetSystemUIDLimit36(uid, sizeof(uid)); 39: uid[36] = 0;	
	污染路径: 无	
8	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 128
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 126: TAOS_STMT2_BIND params[10]; 127: char is_null2[10] = {0};	

	128: char is_null[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 2}; 129: 130: params[0].buffer_type = TSDB_DATA_TYPE_TIMESTAMP;	
	污染路径: 无	
9	缺陷发生位置: /source/util/src/tcompare.c	行号: 1183
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 1181: } 1182: 1183: char rejectList[2] = {toupper(c), tolower(c)}; 1184: const char *remainStr = str + nMatchChar; 1185: int32_t remainSize = ssize - nMatchChar;	
	污染路径: 无	
10	缺陷发生位置: /source/util/src/tcompression.c	行号: 360
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 358: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 359: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 360: char bit_to_selector[] = {0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 11, 12, 12, 12, 13, 13, 13, 13, 13, 361: 14, 14, 14, 14, 14, 14, 14, 14, 14, 14, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 362: 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15};	
	污染路径: 无	
11	缺陷发生位置: /source/util/src/tconfig.c	行号: 1628
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 1626: }	

	1627: 1628: char line[1024]; 1629: char **pEnv = environ; 1630: line[1023] = 0;	
	污染路径: 无	
12	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-exmaple.c	行号: 30
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 28: int32_t b_len[2], t64_len[2], n_len[2]; 29: char is_null[2] = {0}; 30: char is_null2[2] = {0, 2}; 31: // TAOS_STMT2_OPTION option = {0}; 32: // TAOS_STMT2_OPTION option = {0, true, true, stmtAsyncQueryCb, NULL};	
	污染路径: 无	
13	缺陷发生位置: /contrib/lemon/lemon.c	行号: 3968
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 3966: 3967: if(rp->code==0){ 3968: static char newlinestr[2] = { '\n', '\0' }; 3969: rp->code = newlinestr; 3970: rp->line = rp->ruleline;	
	污染路径: 无	
14	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-exmaple.c	行号: 108
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 106: int32_t b_len[2], t64_len[2], n_len[2]; 107: char is_null[2] = {0}; 108: char is_null2[2] = {0, 2}; 109: // TAOS_STMT2_OPTION option = {0};	

	110: // TAOS_STMT2_OPTION option = {0, true, true, stmtAsyncQueryCb, NULL};	
	污染路径: 无	
15	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1466
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 1464: void encodeAuthBase64() { 1465: char userpass_buf[INPUT_BUF_LEN]; 1466: static char base64[] = { 1467: 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 1468: 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z',	
	污染路径: 无	
16	缺陷发生位置: /test/new_test_framework/script/api/stmt2-example.c	行号: 30
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 28: int32_t b_len[2], t64_len[2], n_len[2]; 29: char is_null[2] = {0}; 30: char is_null2[2] = {0, 2}; 31: // TAOS_STMT2_OPTION option = {0}; 32: // TAOS_STMT2_OPTION option = {0, true, true, stmtAsyncQueryCb, NULL};	
	污染路径: 无	
17	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 374
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 372: } 373: 374: char quotationStr[2] = {"", 0}; 375: int32_t width; 376:	

	污染路径: 无	
18	缺陷发生位置: /source/util/src/tcompression.c	行号: 358
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 356: // Selector value: 0 1 2 3 4 5 6 7 8 9 10 11 357: // 12 13 14 15 358: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 359: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 360: char bit_to_selector[] = {0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 11, 12, 12, 12, 13, 13, 13, 13, 13,	
	污染路径: 无	
19	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 344
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 342: // get all items from tires tree 343: void enumFromTree(STire* tire, SMatch* match) { 344: char pre[2] = {0, 0}; 345: STireNode* c; 346:	
	污染路径: 无	

12 C/C++缺陷类型-返回值问题

12.1 分项汇总表

序号	C/C++缺陷类型-返回值问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4733	具有返回值的函数,其返回值如果不被使用,调用是应有 void 说明	18	严重

12.2 缺陷/安全漏洞详解

12.2.1 严重类

12.2.1.1 WK4733 具有返回值的函数,其返回值如果不被使用,

调用是应有 void 说明

缺陷数量：18 个		
缺陷描述： 具有返回值的函数,其返回值如果不被使用,调用时应有 void 说明。		
修复意见： 无		
1	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v03.c	行号： 1787
	步骤描述： 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明。	
	代码片段：	

	1785: HUF_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1786: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1787: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 1788: HUF_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); 1789:	
	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
2	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSttFileRW.c	行号: 174
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 172: reader->footer->tombBlkPtr->size, 0, pEncryptData); 173: if (code) { 174: taosMemoryFree(data); 175: return code; 176: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
3	缺陷发生位置: /source/libs/executor/src/groupcacheoperator.c	行号: 206
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 204: taosMemoryFree(pGrpCacheOperator->execInfo.pDownstreamBlkNum); 205: 206: taosMemoryFreeClear(param); 207: } 208:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
4	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 2072
	步骤描述:	

	不使用返回值的函数'HUFv05_decodeStreamX2'应使用'void'说明.	
	代码片段: 2070: 2071: /* finish bitStreams one by one */ 2072: HUFv05_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 2073: HUFv05_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2074: HUFv05_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog);	
	污染路径: 函数'HUFv05_decodeStreamX2'的定义点.	
5	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMerge.c	行号: 529
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 527: 528: tsdbTFileSetClear(&merger->fset); 529: taosMemoryFree(arg); 530: return code; 531: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
6	缺陷发生位置: /source/libs/parser/src/parInsertUtil.c	行号: 1299
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 1297: pColSchema->name, tDataTypes[pColSchema->type].name, pColSchema->bytes, tDataTypes[*fields].name, 1298: *(int32_t*)(fields + sizeof(int8_t)); 1299: uError("checkSchema %s", buf); 1300: } 1301: return TSDB_CODE_INVALID_PARA;	
	污染路径: 函数'uError'的定义点.	
7	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 360

	步骤描述: 不使用返回值的函数'tsdbCacherowsReaderClose'应使用'void'说明.	
	代码片段: 358: } 359: if (p != NULL) { 360: tsdbCacherowsReaderClose(p); 361: } 362: return code;	
	污染路径: 函数'tsdbCacherowsReaderClose'的定义点.	
8	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp	行号: 418
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 416: 417: for (int i = 1; i <= nData; i++) { 418: sprintf(key, "key%d", i); 419: // sprintf(val, "value%d", i); 420:	
	污染路径: 无	
9	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReaderWriter.c	行号: 65
	步骤描述: 不使用返回值的函数'tsdbInfo'应使用'void'说明.	
	代码片段: 63: } 64: } else { 65: tsdbInfo("no file: %s", path); 66: TSDB_CHECK_CODE(code = terrno, lino, _exit); 67: }	
	污染路径: 函数'tsdbInfo'的定义点.	
10	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 41
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 39: TAOS_RES* result = taos_query(taos, "drop database if	

	exists test;"); 40: taos_free_result(result); 41: usleep(100000); 42: result = taos_query(taos, "create database test;"); 43:	
	污染路径: 无	
11	缺陷发生位置: /source/libs/tss/src/fs.c	行号: 111
	步骤描述: 不使用返回值的函数'tssError'应使用'void'说明.	
	代码片段: 109: 110: if (ss->baseDir == NULL) { 111: tssError("baseDir is not configured in access string: %s", as); 112: return false; 113: }	
	污染路径: 函数'tssError'的定义点.	
12	缺陷发生位置: /source/util/src/tworker.c	行号: 256
	步骤描述: 不使用返回值的函数'DestoryThreadLocalRegComp'应使用'void'说明.	
	代码片段: 254: taosUpdateItemSize(qinfo.queue, 1); 255: } 256: DestoryThreadLocalRegComp(); 257: closeThreadNotificationConn(); 258:	
	污染路径: 函数'DestoryThreadLocalRegComp'的定义点.	
13	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet.c	行号: 957
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 955: } 956: if (net__socket_nonblock(&sv[1])) { 957: UNUSED(COMPAT_CLOSE(sv[0]));	

	958: return TTQ_ERR_ERRNO; 959: }	
	污染路径: 函数'UNUSED'的定义点.	
14	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 237
	步骤描述: 不使用返回值的函数'sclError'应使用'void'说明.	
	代码片段: 235: *p = NULL; 236: } else { 237: sclError("getVectorBigintValueFn invalid srcType : %d", srcType); 238: return TSDB_CODE_SCALAR_CONVERT_ERROR; 239: }	
	污染路径: 函数'sclError'的定义点.	
15	缺陷发生位置: /examples/c/asyncdemo.c	行号: 72
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 70: taos_free_result(pSql); 71: taos_close(taos); 72: taos_cleanup(); 73: exit(EXIT_FAILURE); 74: }	
	污染路径: 函数'taos_cleanup'的定义点.	
16	缺陷发生位置: /source/os/src/osTimer.c	行号: 85
	步骤描述: 不使用返回值的函数'TAOS_SKIP_ERROR'应使用'void'说明.	
	代码片段: 83: static void taosDeleteTimer(void *tharg) { 84: timer_t *pTimer = tharg; 85: TAOS_SKIP_ERROR(timer_delete(*pTimer)); 86: } 87:	

	污染路径: 函数'TAOS_SKIP_ERROR'的定义点.	
17	缺陷发生位置: /source/os/src/osSocket.c	行号: 227
	步骤描述: 不使用返回值的函数'TAOS_SKIP_ERROR'应使用'void'说明.	
	代码片段: 225: } 226: 227: TAOS_SKIP_ERROR(taosCloseSocket(&pSocket)); 228: 229: return 1;	
	污染路径: 函数'TAOS_SKIP_ERROR'的定义点.	
18	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamErrorInjection.c	行号: 50
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 48: } else if (choseltem == 1) { 49: // 2. repeat send update chkpt msg 50: mError("***repeat send update-checkpoint/consensus/checkpoint trans msg 3times to vnode***"); 51: 52: mError("***repeat 1***");	
	污染路径: 函数'mError'的定义点.	

13 C/C++缺陷类型-浮点数问题

13.1 分项汇总表

序号	C/C++缺陷类型-浮点数问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4748	浮点数变量赋值给整型变量必须强制转换.	6	严重
2	WK4101	浮点数相等或不等比较	3	严重

13.2 缺陷/安全漏洞详解

13.2.1 严重类

13.2.1.1 WK4748 浮点数变量赋值给整型变量必须强制转换.

缺陷数量： 6 个		
缺陷描述： 浮点数变量赋值给整型变量必须强制转换。		
修复意见： 无		
1	缺陷发生位置： /utils/test/c/tmqDemo.c	行号： 170
	步骤描述： 浮点数变量赋值给整型变量必须强制转换	
	代码片段： 168: } 169: 170: g_stConflInfo.totalRowsOfT2 =	

	g_stConflInfo.totalRowsOfPerTbl * g_stConflInfo.ratio; 171: 172: #if 0	
	污染路径: 无	
2	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 345
	步骤描述: 浮点数变量赋值给整型变量必须强制转换	
	代码片段: 343: if(twoStage) 344: { 345: tmpByteData = (unsigned char*)malloc(r1*sizeof(double)*1.2); 346: } 347:	
	污染路径: 无	
3	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 163
	步骤描述: 浮点数变量赋值给整型变量必须强制转换	
	代码片段: 161: if(predAbsErr<checkRadius) 162: { 163: state = (predAbsErr*recip_realPrecision+1)*0.5; 164: if(curData>=pred) 165: {	
	污染路径: 无	
4	缺陷发生位置: /contrib/TSZ/sz/src/utility.c	行号: 45
	步骤描述: 浮点数变量赋值给整型变量必须强制转换	
	代码片段: 43: estimatedCompressedSize = 200; 44: else 45: estimatedCompressedSize = dataLength*1.2; 46: /*compressBytes = (unsigned	

	char*)malloc(estimatedCompressedSize); // comment by tickduan 47: outSize = ZSTD_compress(compressBytes, estimatedCompressedSize, data, dataLength, level); //default setting of level is 3	
	污染路径: 无	
5	缺陷发生位置: /utls/test/c/tmqSim.c	行号: 1413
	步骤描述: 浮点数变量赋值给整型变量必须强制转换	
	代码片段: 1411: queryDbExec(taos, sql, NO_INSERT_TYPE); 1412: 1413: int32_t producerRate = ceil(((double)g_stConflInfo.producerRate) / g_stConflInfo.producers); 1414: 1415: printf("==== create %d produce thread ====\\n", g_stConflInfo.producers);	
	污染路径: 无	
6	缺陷发生位置: /utls/test/c/tmqDemo.c	行号: 663
	步骤描述: 浮点数变量赋值给整型变量必须强制转换	
	代码片段: 661: totalRows = g_stConflInfo.totalRowsOfPerTbl * g_stConflInfo.numOfTables; 662: } else { 663: totalRows = g_stConflInfo.totalRowsOfPerTbl * (1 + g_stConflInfo.ratio); 664: } 665:	
	污染路径: 无	

13.2.1.2 WK4101 浮点数相等或不等比较

缺陷数量： 3 个		
缺陷描述： 两个浮点型或双精度浮点型的值使用 “==”或 “=”运算符进行比较。		
修复意见： 通过计算两个浮点数之间的差的绝对值是否小于一个允许的误差值判断浮点数之间是否相等或不等。		
1	缺陷发生位置： /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号： 2613
	步骤描述： 对浮点数进行相等或不等比较	
	代码片段： 2611: 2612: case tools_cJSON_Number: 2613: if (a->valuedouble == b->valuedouble) 2614: { 2615: return true;	
	污染路径： 无	
2	缺陷发生位置： /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号： 461
	步骤描述： 对浮点数进行相等或不等比较	
	代码片段： 459: 460: /* Check whether the original double can be recovered */ 461: if ((sscanf((char*)number_buffer, "%lg", &test) != 1) ((double)test != d)) 462: { 463: /* If not, print with 17 decimal places of precision */	
	污染路径： 无	
3	缺陷发生位置： /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号： 451
	步骤描述： 对浮点数进行相等或不等比较	
	代码片段：	

	<pre>449: 450: /* This checks for NaN and Infinity */ 451: if ((d * 0) != 0) 452: { 453: length = sprintf((char*)number_buffer, "null");</pre>
	污染路径: 无

14 C/C++缺陷类型-资源泄露

14.1 分项汇总表

序号	C/C++缺陷类型-资源泄露	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4800	禁止可导致非资源性对象数据被外部修改的成员函数返回	3	严重

14.2 缺陷/安全漏洞详解

14.2.1 严重类

14.2.1.1 WK4800 禁止可导致非资源性对象数据被外部修改的

成员函数返回

缺陷数量：3 个		
缺陷描述： 禁止可导致非资源性对象数据被外部修改的成员函数返回。		
修复意见： 无		
1	缺陷发生位置： /source/libs/index/test/fstUT.cc	行号： 187
	步骤描述： 禁止可导致非资源性对象数据 fr 被外部修改的成员函数 Get 返回	
	代码片段： 185: return fw->Put(k, v); 186: } 187: bool Get(const std::string& k, uint64_t* v) { 188: if (fr == NULL) { 189: return false;	
	污染路径： 无	
2	缺陷发生位置： /source/libs/index/test/fstUT.cc	行号： 181
	步骤描述： 禁止可导致非资源性对象数据 fw 被外部修改的成员函数 Put 返回	
	代码片段： 179: fr = NULL; 180: } 181: bool Put(const std::string& k, uint64_t v) { 182: if (fw == NULL) { 183: return false;	

3	污染路径: 无	
	缺陷发生位置: /source/libs/parser/test/mockCatalogService.cpp	行号: 389
	步骤描述: 禁止可导致非资源性对象数据 id_ 被外部修改的成员函数 getNextId 返回	
	代码片段: 387: typedef std::map<string, SdbCfgInfo> DbCfgCache; 388: 389: uint64_t getNextId() { return id_++; } 390: 391: void genEpSet(SEpSet* pEpSet) {	
	污染路径: 无	

15 C/C++缺陷类型-其他内存相关问题

15.1 分项汇总表

序号	C/C++缺陷类型-其他内存相关问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4085	释放未动态分配的内存	33	严重
2	WK4115	为指针分配的内存大小与类型不匹配	2	严重
3	WK5388	释放并不在堆上的内存	74	严重

15.2 缺陷/安全漏洞详解

15.2.1 严重类

15.2.1.1 WK4085 释放未动态分配的内存

缺陷数量：33 个		
缺陷描述： 在指针还未指向动态分配的内存时,释放指针指向的内存。		
修复意见： 无		
1	缺陷发生位置： /contrib/test/traft/join_into_vgroup/node_follower10000.c	行号： 52
	步骤描述： 禁止释放未动态分配的内存'ptr'.	
	代码片段： 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径： 无	
2	缺陷发生位置： /contrib/TSZ/sz/src/Huffman.c	行号： 205
	步骤描述： 禁止释放未动态分配的内存'freq'.	
	代码片段： 203: 204: build_code(huffmanTree, huffmanTree->qq[1], 0, 0, 0); 205: free(freq); 206: } 207:	
	污染路径： 无	
3	缺陷发生位置：	行号：

	/tools/taos-tools/src/taosdump.c		2490
	步骤描述: 禁止释放未动态分配的内存'sql'.		
	代码片段: 2488: 2489: free(jsonSchema); 2490: free(sql); 2491: 2492: return 0;		
	污染路径: 无		
4	缺陷发生位置: /tools/taos-tools/src/taosdump.c		行号: 2514
	步骤描述: 禁止释放未动态分配的内存'sql'.		
	代码片段: 2512: return -1; 2513: } 2514: free(sql); 2515: return 0; 2516: }		
	污染路径: 无		
5	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c		行号: 1051
	步骤描述: 禁止释放未动态分配的内存'dictList'.		
	代码片段: 1049: 1050: /* clean up */ 1051: free(dictList); 1052: return dictSize; 1053: }		
	污染路径: 无		
6	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_leader10002.c		行号: 52
	步骤描述: 禁止释放未动态分配的内存'ptr'.		
	代码片段:		

	50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
7	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1439
	步骤描述: 禁止释放未动态分配的内存'tmpBuf'.	
	代码片段: 1437: 1438: int ret = fprintf(fp, "%s\n", tmpBuf); 1439: free(tmpBuf); 1440: 1441: return ret;	
	污染路径: 无	
8	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 714
	步骤描述: 禁止释放未动态分配的内存'treeBytes'.	
	代码片段: 712: memcpy(*out+4, buffer, 4); 713: memcpy(*out+8, treeBytes, treeByteSize); 714: free(treeBytes); 715: size_t enCodeSize = 0; 716: encode(huffmanTree, s, length, *out+8+treeByteSize, &enCodeSize);	
	污染路径: 无	
9	缺陷发生位置: /contrib/test/traft/cluster/node10000.c	行号: 52
	步骤描述: 禁止释放未动态分配的内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr);	

	53: free(req); 54: }	
	污染路径: 无	
10	缺陷发生位置: /contrib/test/traft/cluster/node10000_restart.c	行号: 52
	步骤描述: 禁止释放未动态分配的内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
11	缺陷发生位置: /source/libs/parser/test/parStreamTest.cpp	行号: 599
	步骤描述: 禁止释放未动态分配的内存'expectCalcPlan'.	
	代码片段: 597: cJSON_Delete(j1); 598: cJSON_Delete(j2); 599: free(expectCalcPlan); 600: free(reqCalcPlan); 601: }	
	污染路径: 无	
12	缺陷发生位置: /source/libs/parser/test/parStreamTest.cpp	行号: 600
	步骤描述: 禁止释放未动态分配的内存'reqCalcPlan'.	
	代码片段: 598: cJSON_Delete(j2); 599: free(expectCalcPlan); 600: free(reqCalcPlan); 601: } 602:	
	污染路径:	

	无	
13	缺陷发生位置: /source/libs/parser/test/parStreamTest.cpp	行号: 570
	步骤描述: 禁止释放未动态分配的内存'reqTriggerScanStr'.	
	代码片段: 568: cJSON_Delete(j2); 569: free(expectTriggerScanStr); 570: free(reqTriggerScanStr); 571: } 572:	
	污染路径: 无	
14	缺陷发生位置: /contrib/test/traft/cluster/node10001.c	行号: 52
	步骤描述: 禁止释放未动态分配的内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
15	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 971
	步骤描述: 禁止释放未动态分配的内存'dictList'.	
	代码片段: 969: /* checks */ 970: if (!dictList) return ERROR(memory_allocation); 971: if (maxDictSize < ZDICT_DICTSIZE_MIN) { free(dictList); return ERROR(dstSize_tooSmall); } /* requested dictionary size is too small */ 972: if (samplesBuffSize < ZDICT_MIN_SAMPLES_SIZE) { free(dictList); return ERROR(dictionaryCreation_failed); } /* not enough source to create dictionary */ 973:	
	污染路径:	

	无	
16	缺陷发生位置： /source/libs/parser/test/parStreamTest.cpp	行号： 569
	步骤描述： 禁止释放未动态分配的内存'expectTriggerScanStr'.	
	代码片段： 567: cJSON_Delete(j1); 568: cJSON_Delete(j2); 569: free(expectTriggerScanStr); 570: free(reqTriggerScanStr); 571: }	
	污染路径： 无	
17	缺陷发生位置： /contrib/test/traft/single_node/singleNode.c	行号： 52
	步骤描述： 禁止释放未动态分配的内存'ptr'.	
	代码片段： 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径： 无	
18	缺陷发生位置： /contrib/TSZ/sz/src/sz_double.c	行号： 415
	步骤描述： 禁止释放未动态分配的内存'intervals'.	
	代码片段： 413: powerOf2 = 32; 414: 415: free(intervals); 416: return powerOf2; 417: }	
	污染路径： 无	
19	缺陷发生位置： /contrib/TSZ/zstd/dictBuilder/zdict.c	行号： 1005
	步骤描述：	

	禁止释放未动态分配的内存'dictList'.	
	代码片段: <pre> 1003: /* create dictionary */ 1004: { U32 dictContentSize = ZDICT_dictSize(dictList); 1005: if (dictContentSize < ZDICT_CONTENTSIZE_MIN) { free(dictList); return ERROR(dictionaryCreation_failed); } /* dictionary content too small */ 1006: if (dictContentSize < targetDictSize/4) { 1007: DISPLAYLEVEL(2, "! warning : selected content significantly smaller than requested (%u < %u) \n", dictContentSize, (U32)maxDictSize); </pre>	
	污染路径: 无	
20	缺陷发生位置: /contrib/test/traft/cluster/node10001_restart.c	行号: 52
	步骤描述: 禁止释放未动态分配的内存'ptr'.	
	代码片段: <pre> 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: } </pre>	
	污染路径: 无	
21	缺陷发生位置: /contrib/TSZ/sz/src/sz_float.c	行号: 83
	步骤描述: 禁止释放未动态分配的内存'intervals'.	
	代码片段: <pre> 81: powerOf2 = 32; 82: 83: free(intervals); 84: //printf("accIntervals=%d, powerOf2=%d\n", accIntervals, powerOf2); 85: return powerOf2; </pre>	
	污染路径: 无	
22	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2511

	步骤描述: 禁止释放未动态分配的内存'sql'.	
	代码片段: 2509: if (len != strlen(sql) + 1) { 2510: errorPrint(" write create sql failed. expect=%ld real=%d\n sql=%s\n", strlen(sql), len, sql); 2511: free(sql); 2512: return -1; 2513: }	
	污染路径: 无	
23	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 180
	步骤描述: 禁止释放未动态分配的内存'fs'.	
	代码片段: 178: } 179: free (fs->buf); 180: free (fs); 181: } 182:	
	污染路径: 无	
24	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10001.c	行号: 52
	步骤描述: 禁止释放未动态分配的内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
25	缺陷发生位置: /contrib/test/traft/cluster/node10002.c	行号: 52
	步骤描述: 禁止释放未动态分配的内存'ptr'.	

	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
26	缺陷发生位置: /contrib/TSZ/sz/src/szd_double.c	行号: 205
	步骤描述: 禁止释放未动态分配的内存'leadNum'.	
	代码片段: 203: } 204: 205: free(leadNum); 206: free(type); 207: return;	
	污染路径: 无	
27	缺陷发生位置: /contrib/TSZ/sz/src/szd_float.c	行号: 214
	步骤描述: 禁止释放未动态分配的内存'leadNum'.	
	代码片段: 212: } 213: 214: free(leadNum); 215: free(types); 216: return;	
	污染路径: 无	
28	缺陷发生位置: /contrib/test/traft/cluster/node10002_restart.c	行号: 52
	步骤描述: 禁止释放未动态分配的内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr);	

	53: free(req); 54: }	
	污染路径: 无	
29	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 972
	步骤描述: 禁止释放未动态分配的内存'dictList'.	
	代码片段: 970: if (!dictList) return ERROR(memory_allocation); 971: if (maxDictSize < ZDICT_DICTSIZE_MIN) { free(dictList); return ERROR(dstSize_tooSmall); } /* requested dictionary size is too small */ 972: if (samplesBuffSize < ZDICT_MIN_SAMPLES_SIZE) { free(dictList); return ERROR(dictionaryCreation_failed); } /* not enough source to create dictionary */ 973: 974: /* init */	
	污染路径: 无	
30	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 84
	步骤描述: 禁止释放未动态分配的内存'intervals'.	
	代码片段: 82: powerOf2 = 32; 83: 84: free(intervals); 85: //printf("accIntervals=%d, powerOf2=%d\n", accIntervals, powerOf2); 86: return powerOf2;	
	污染路径: 无	
31	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 187
	步骤描述: 禁止释放未动态分配的内存'freq'.	
	代码片段: 185: 186: build_code(huffmanTree, huffmanTree->qq[1], 0, 0, 0);	

	187: free(freq); 188: } 189:	
	污染路径: 无	
32	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 3358
	步骤描述: 禁止释放未动态分配的内存'command'.	
	代码片段: 3356: } 3357: 3358: free(command); 3359: return res; 3360: }	
	污染路径: 无	
33	缺陷发生位置: /contrib/TSZ/sz/src/sz_float.c	行号: 438
	步骤描述: 禁止释放未动态分配的内存'intervals'.	
	代码片段: 436: powerOf2 = 32; 437: 438: free(intervals); 439: return powerOf2; 440: }	
	污染路径: 无	

15.2.1.2 WK4115 为指针分配的内存大小与类型不匹配

缺陷数量: 2 个
缺陷描述: 为某个类型的指针分配的内存大小小于该类型的大小。
修复意见: 采用 sizeof 关键字对指针指向的数据类型的大小进行计算并分配内存。

1	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 27
	步骤描述: 为指针分配的内存大小与类型不匹配	
	代码片段: 25: { 26: *outSize = dataLength*sizeof(double); 27: unsigned char* out = (unsigned char*)malloc(dataLength*sizeof(double)); 28: memcpy(out, data, dataLength*sizeof(double)); 29: return out;	
	污染路径: 无	
2	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 109
	步骤描述: 为指针分配的内存大小与类型不匹配	
	代码片段: 107: 108: fname = (char *) malloc (strlen (test_dir) + 1 + strlen (base) 109: + sizeof ("XXXXXX")); 110: if (fname == NULL) 111: {	
	污染路径: 无	

15.2.1.3 WK5388 释放并不在堆上的内存

缺陷数量: 74 个		
缺陷描述: 应用程序在指向未使用关联的堆分配函数（如 malloc（）、calloc（）或 realloc（））分配的内存的指针上调用 free（）。		
修复意见: 在释放指针之前，程序员应确保指针以前已在堆上分配，并且内存属于程序员。释放未分配的指针将导致程序中出现未定义的行为。		
1	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchJsonOpt.c		965
	步骤描述: 禁止释放非堆内存'tsma'.		
	代码片段: 963: if (!tools_cJSON_IsString(tsma_sliding_obj)) { 964: errorPrint("%s", "Invalid tsma sliding format in json\n"); 965: free(tsma); 966: return -1; 967: }		
	污染路径: 无		
2	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c		行号: 714
	步骤描述: 禁止释放非堆内存'treeBytes'.		
	代码片段: 712: memcpy(*out+4, buffer, 4); 713: memcpy(*out+8, treeBytes, treeByteSize); 714: free(treeBytes); 715: size_t enCodeSize = 0; 716: encode(huffmanTree, s, length, *out+8+treeByteSize, &enCodeSize);		
	污染路径: 无		
3	缺陷发生位置: /tools/taos-tools/src/benchData.c		行号: 1080
	步骤描述: 禁止释放非堆内存'tmp'.		
	代码片段: 1078: char *tmp = benchCalloc(1, field->length + 1, false); 1079: if (0 != tmpStr(tmp, stbInfo->iface, field, index)) { 1080: free(tmp); 1081: return -1; 1082: }		
	污染路径: 无		
4	缺陷发生位置:		行号:

	/contrib/TSZ/sz/src/sz_float.c		438
	步骤描述: 禁止释放非堆内存'intervals'.		
	代码片段: 436: powerOf2 = 32; 437: 438: free(intervals); 439: return powerOf2; 440: }		
	污染路径: 无		
5	缺陷发生位置: /tools/taos-tools/src/benchUtil.c		行号: 1853
	步骤描述: 禁止释放非堆内存'responseBuf'.		
	代码片段: 1851: } 1852: free_of_post: 1853: free(responseBuf); 1854: return code; 1855: }		
	污染路径: 无		
6	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c		行号: 187
	步骤描述: 禁止释放非堆内存'freq'.		
	代码片段: 185: 186: build_code(huffmanTree, huffmanTree->qq[1], 0, 0, 0); 187: free(freq); 188: } 189:		
	污染路径: 无		
7	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleConnect.c		行号: 277
	步骤描述: 禁止释放非堆内存'auth_data_out'.		
	代码片段:		

	275: } 276: } 277: free(auth_data_out); 278: auth_data_out = NULL; 279:	
	污染路径: 无	
8	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_leader10002.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
9	缺陷发生位置: /contrib/test/traft/cluster/node10001.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
10	缺陷发生位置: /source/libs/parser/test/parStreamTest.cpp	行号: 569
	步骤描述: 禁止释放非堆内存'expectTriggerScanStr'.	
	代码片段: 567: cJSON_Delete(j1); 568: cJSON_Delete(j2); 569: free(expectTriggerScanStr); 570: free(reqTriggerScanStr);	

	571: }	
	污染路径: 无	
11	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 938
	步骤描述: 禁止释放非堆内存'tsma'.	
	代码片段: 936: if (!tools_cJSON_IsString(tsma_name_obj)) { 937: errorPrint("%s", "Invalid tsma name format in json\n"); 938: free(tsma); 939: return -1; 940: }	
	污染路径: 无	
12	缺陷发生位置: /contrib/test/traft/cluster/node10001_restart.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
13	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1696
	步骤描述: 禁止释放非堆内存'responseBuf'.	
	代码片段: 1694: } 1695: free_of_getversion: 1696: free(responseBuf); 1697: return server_ver; 1698: }	
	污染路径:	

	无	
14	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 972
	步骤描述: 禁止释放非堆内存'dictList'.	
	代码片段: 970: if (!dictList) return ERROR(memory_allocation); 971: if (maxDictSize < ZDICT_DICTSIZE_MIN) { free(dictList); return ERROR(dstSize_tooSmall); } /* requested dictionary size is too small */ 972: if (samplesBuffSize < ZDICT_MIN_SAMPLES_SIZE) { free(dictList); return ERROR(dictionaryCreation_failed); } /* not enough source to create dictionary */ 973: 974: /* init */	
	污染路径: 无	
15	缺陷发生位置: /source/libs/parser/test/parStreamTest.cpp	行号: 600
	步骤描述: 禁止释放非堆内存'reqCalcPlan'.	
	代码片段: 598: cJSON_Delete(j2); 599: free(expectCalcPlan); 600: free(reqCalcPlan); 601: } 602:	
	污染路径: 无	
16	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 84
	步骤描述: 禁止释放非堆内存'intervals'.	
	代码片段: 82: powerOf2 = 32; 83: 84: free(intervals); 85: //printf("accIntervals=%d, powerOf2=%d\n", accIntervals, powerOf2); 86: return powerOf2;	

	污染路径: 无	
17	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleConnect.c	行号: 292
	步骤描述: 禁止释放非堆内存'auth_data_out'.	
	代码片段: 290: return rc; 291: error: 292: free(auth_data_out); 293: mqtt_property_free_all(&connack_props); 294: return rc;	
	污染路径: 无	
18	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1145
	步骤描述: 禁止释放非堆内存'command'.	
	代码片段: 1143: dbName, taos_errstr(res)); 1144: taos_free_result(res); 1145: free(command); 1146: taos_close(taos); 1147: return 0;	
	污染路径: 无	
19	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 459
	步骤描述: 禁止释放非堆内存'command'.	
	代码片段: 457: free(condition); 458: } 459: free(command); 460: return NULL; 461: }	
	污染路径: 无	
20	缺陷发生位置: /source/libs/parser/test/parStreamTest.cpp	行号: 570

	步骤描述: 禁止释放非堆内存'reqTriggerScanStr'.	
	代码片段: 568: cJSON_Delete(j2); 569: free(expectTriggerScanStr); 570: free(reqTriggerScanStr); 571: } 572:	
	污染路径: 无	
21	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 63
	步骤描述: 禁止释放非堆内存'old'.	
	代码片段: 61: SNode *old = next; 62: next = next->next; 63: free(old); 64: } 65: }	
	污染路径: 无	
22	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 1321
	步骤描述: 禁止释放非堆内存'tmp'.	
	代码片段: 1319: char *tmp = benchCalloc(1, field->length + 1, false); 1320: if (0 != tmpStr(tmp, stbInfo->iface, field, k)) { 1321: free(tmp); 1322: return -1; 1323: }	
	污染路径: 无	
23	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 415
	步骤描述: 禁止释放非堆内存'intervals'.	

	代码片段: 413: powerOf2 = 32; 414: 415: free(intervals); 416: return powerOf2; 417: }	
	污染路径: 无	
24	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 440
	步骤描述: 禁止释放非堆内存'command'.	
	代码片段: 438: printErrCmdCodeStr(command, code, res); 439: free(condition); 440: free(command); 441: return NULL; 442: }	
	污染路径: 无	
25	缺陷发生位置: /contrib/TSZ/sz/src/szd_float.c	行号: 214
	步骤描述: 禁止释放非堆内存'leadNum'.	
	代码片段: 212: } 213: 214: free(leadNum); 215: free(types); 216: return;	
	污染路径: 无	
26	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 335
	步骤描述: 禁止释放非堆内存'command'.	
	代码片段: 333: __func__, __LINE__); 334: free(colsBuf); 335: free(command);	

	336: tmfree(tagsBuf); 337: return -1;	
	污染路径: 无	
27	缺陷发生位置: /source/libs/parser/test/parStreamTest.cpp	行号: 599
	步骤描述: 禁止释放非堆内存'expectCalcPlan'.	
	代码片段: 597: cJSON_Delete(j1); 598: cJSON_Delete(j2); 599: free(expectCalcPlan); 600: free(reqCalcPlan); 601: }	
	污染路径: 无	
28	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 971
	步骤描述: 禁止释放非堆内存'dictList'.	
	代码片段: 969: /* checks */ 970: if (!dictList) return ERROR(memory_allocation); 971: if (maxDictSize < ZDICT_DICTSIZE_MIN) { free(dictList); return ERROR(dstSize_tooSmall); } /* requested dictionary size is too small */ 972: if (samplesBuffSize < ZDICT_MIN_SAMPLES_SIZE) { free(dictList); return ERROR(dictionaryCreation_failed); } /* not enough source to create dictionary */ 973:	
	污染路径: 无	
29	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 439
	步骤描述: 禁止释放非堆内存'condition'.	
	代码片段: 437: if (code != 0) { 438: printErrCmdCodeStr(command, code, res); 439: free(condition);	

	440: free(command); 441: return NULL;	
	污染路径: 无	
30	缺陷发生位置: /tools/taos-tools/src/benchTmq.c	行号: 417
	步骤描述: 禁止释放非堆内存'infos'.	
	代码片段: 415: tmq_over: 416: free(pids); 417: free(infos); 418: return ret; 419: }	
	污染路径: 无	
31	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 205
	步骤描述: 禁止释放非堆内存'freq'.	
	代码片段: 203: 204: build_code(huffmanTree, huffmanTree->qq[1], 0, 0, 0); 205: free(freq); 206: } 207:	
	污染路径: 无	
32	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 1103
	步骤描述: 禁止释放非堆内存'tmp'.	
	代码片段: 1101: char *tmp = benchCalloc(1, bufferSize, false); 1102: if (0 != tmpGeometry(tmp, stbInfo->iface, field, i)) { 1103: free(tmp); 1104: return -1; 1105: }	

	污染路径: 无	
33	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1213
	步骤描述: 禁止释放非堆内存'command'.	
	代码片段: 1211: taos_free_result(res); 1212: taos_close(taos); 1213: free(command); 1214: 1215: if (isSet) {	
	污染路径: 无	
34	缺陷发生位置: /contrib/TSZ/sz/src/sz_float.c	行号: 83
	步骤描述: 禁止释放非堆内存'intervals'.	
	代码片段: 81: powerOf2 = 32; 82: 83: free(intervals); 84: //printf("acclIntervals=%d, powerOf2=%d\n", acclIntervals, powerOf2); 85: return powerOf2;	
	污染路径: 无	
35	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1349
	步骤描述: 禁止释放非堆内存'command'.	
	代码片段: 1347: taos_close(taos); 1348: 1349: free(command); 1350: return count; 1351: }	
	污染路径: 无	

36	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 1005
	步骤描述: 禁止释放非堆内存'dictList'.	
	代码片段: 1003: /* create dictionary */ 1004: { U32 dictContentSize = ZDICT_dictSize(dictList); 1005: if (dictContentSize < ZDICT_CONTENTSIZE_MIN) { free(dictList); return ERROR(dictionaryCreation_failed); } /* dictionary content too small */ 1006: if (dictContentSize < targetDictSize/4) { 1007: DISPLAYLEVEL(2, "! warning : selected content significantly smaller than requested (%u < %u) \n", dictContentSize, (U32)maxDictSize);	
	污染路径: 无	
37	缺陷发生位置: /contrib/test/traft/cluster/node10000_restart.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
38	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 315
	步骤描述: 禁止释放非堆内存'dsnc'.	
	代码片段: 313: if (code) { 314: errorPrint("%s dsn=%s\n", error, dsnc); 315: free(dsnc); 316: return NULL; 317: }	
	污染路径: 无	

39	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10001.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
40	缺陷发生位置: /contrib/TSZ/sz/src/szd_double.c	行号: 205
	步骤描述: 禁止释放非堆内存'leadNum'.	
	代码片段: 203: } 204: 205: free(leadNum); 206: free(type); 207: return;	
	污染路径: 无	
41	缺陷发生位置: /test/new_test_framework/script/api/insertBlob.c	行号: 145
	步骤描述: 禁止释放非堆内存'buf'.	
	代码片段: 143: free(temp); 144: } 145: free(buf); 146: 147: taos_free_result(result);	
	污染路径: 无	
42	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 718
	步骤描述: 禁止释放非堆内存'out1'.	

	代码片段: 716: 717: if (out2) { 718: if (out1) free(out1); 719: return out2; 720: }	
	污染路径: 无	
43	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 156
	步骤描述: 禁止释放非堆内存'fs'.	
	代码片段: 154: if (! fs->buf) 155: { 156: free (fs); 157: fs = 0; 158: }	
	污染路径: 无	
44	缺陷发生位置: /contrib/test/traft/cluster/node10002_restart.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
45	缺陷发生位置: /tools/taos-tools/src/benchTmq.c	行号: 141
	步骤描述: 禁止释放非堆内存'buf'.	
	代码片段: 139: totalRows++; 140: }	

	141: free(buf); 142: return totalRows; 143: }	
	污染路径: 无	
46	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 1051
	步骤描述: 禁止释放非堆内存'dictList'.	
	代码片段: 1049: 1050: /* clean up */ 1051: free(dictList); 1052: return dictSize; 1053: }	
	污染路径: 无	
47	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 334
	步骤描述: 禁止释放非堆内存'colsBuf'.	
	代码片段: 332: errorPrint("%s() LN%d snprintf overflow\n", 333: __func__, __LINE__); 334: free(colsBuf); 335: free(command); 336: tmmfree(tagsBuf);	
	污染路径: 无	
48	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 947
	步骤描述: 禁止释放非堆内存'tsma'.	
	代码片段: 945: if (!tools_cJSON_IsString(tsma_func_obj)) { 946: errorPrint("%s", "Invalid tsma function format in json\n"); 947: free(tsma); 948: return -1; 949: }	

	污染路径: 无	
49	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10000.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
50	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 319
	步骤描述: 禁止释放非堆内存'colsBuf'.	
	代码片段: 317: 318: if (stbInfo->tags->size == 0) { 319: free(colsBuf); 320: free(command); 321: return 0;	
	污染路径: 无	
51	缺陷发生位置: /tools/taos-tools/src/benchTmq.c	行号: 416
	步骤描述: 禁止释放非堆内存'pids'.	
	代码片段: 414: 415: tmq_over: 416: free(pids); 417: free(infos); 418: return ret;	
	污染路径: 无	
52	缺陷发生位置:	行号:

	/tools/taos-tools/src/dumpUtil.c		499
	步骤描述: 禁止释放非堆内存'entry'.		
	代码片段: 497: freeStbChange((StbChange *)entry->value); 498: free(entry->key); 499: free(entry); 500: 501: // next		
	污染路径: 无		
53	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c		行号: 277
	步骤描述: 禁止释放非堆内存'buf'.		
	代码片段: 275: if (readLen != size) { 276: errorPrint("read file failed. expect read=%ld real= %ld file=%s error=%s\n", size, readLen, filename, strerror(errno)); 277: free(buf); 278: buf = NULL; 279: }		
	污染路径: 无		
54	缺陷发生位置: /tools/rocks-reader/rreader.cpp		行号: 137
	步骤描述: 禁止释放非堆内存'pLastCol'.		
	代码片段: 135: return 0; 136: } else if (offset > size) { 137: free(pLastCol); 138: 139: return -3;		
	污染路径: 无		
55	缺陷发生位置: /tools/taos-tools/src/benchInsert.c		行号: 466
	步骤描述:		

	禁止释放非堆内存'command'.	
	代码片段: 464: 465: int ret = queryDbExec(database, stbInfo, command); 466: free(command); 467: return ret; 468: }	
	污染路径: 无	
56	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 513
	步骤描述: 禁止释放非堆内存'jsonStr'.	
	代码片段: 511: } 512: 513: free(jsonStr); 514: jsonStr = NULL; 515: }	
	污染路径: 无	
57	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 436
	步骤描述: 禁止释放非堆内存'databuf'.	
	代码片段: 434: if (toFile) { 435: appendResultBufToFile(databuf, filePath); 436: free(databuf); 437: } 438: return rows;	
	污染路径: 无	
58	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 601
	步骤描述: 禁止释放非堆内存'cl'.	
	代码片段: 599: {	

	600: struct hol_cluster *next = cl->next; 601: free (cl); 602: cl = next; 603: }	
	污染路径: 无	
59	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 377
	步骤描述: 禁止释放非堆内存'dsnc'.	
	代码片段: 375: 376: if (dsnc) { 377: free(dsnc); 378: } 379: return taos;	
	污染路径: 无	
60	缺陷发生位置: /contrib/test/traft/cluster/node10000.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }	
	污染路径: 无	
61	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 893
	步骤描述: 禁止释放非堆内存'cfg'.	
	代码片段: 891: } else { 892: errorPrint("Invalid value format for %s\n", cfg->name); 893: free(cfg); 894: return -1;	

	895: }	
	污染路径: 无	
62	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 180
	步骤描述: 禁止释放非堆内存'fs'.	
	代码片段: 178: } 179: free (fs->buf); 180: free (fs); 181: } 182:	
	污染路径: 无	
63	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 1092
	步骤描述: 禁止释放非堆内存'tmp'.	
	代码片段: 1090: char *tmp = benchCalloc(1, field->length + 1, false); 1091: if (0 != tmpStr(tmp, stbInfo->iface, field, index)) { 1092: free(tmp); 1093: return -1; 1094: }	
	污染路径: 无	
64	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 320
	步骤描述: 禁止释放非堆内存'command'.	
	代码片段: 318: if (stbInfo->tags->size == 0) { 319: free(colsBuf); 320: free(command); 321: return 0; 322: }	

	污染路径: 无	
65	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 541
	步骤描述: 禁止释放非堆内存'command'.	
	代码片段: 539: totalT / 1000000); 540: } 541: free(command); 542: return NULL; 543: }	
	污染路径: 无	
66	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1551
	步骤描述: 禁止释放非堆内存'request_buf'.	
	代码片段: 1549: } 1550: if (r >= req_buf_len) { 1551: free(request_buf); 1552: errorPrint("%s", "too long request"); 1553: goto free_of_postImpl;	
	污染路径: 无	
67	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 169
	步骤描述: 禁止释放非堆内存'pLastCol'.	
	代码片段: 167: 168: if (offset > size) { 169: free(pLastCol); 170: 171: return -3;	
	污染路径: 无	
68	缺陷发生位置:	行号:

	/tools/taos-tools/src/taosdump.c		1276
	步骤描述: 禁止释放非堆内存'dupSeq'.		
	代码片段: 1274: } 1275: 1276: free(dupSeq); 1277: } 1278:		
	污染路径: 无		
69	缺陷发生位置: /contrib/lemon/lemon.c		行号: 133
	步骤描述: 禁止释放非堆内存'memChunkList'.		
	代码片段: 131: while(memChunkList){ 132: MemChunk *pNext = memChunkList->pNext; 133: free(memChunkList); 134: memChunkList = pNext; 135: }		
	污染路径: 无		
70	缺陷发生位置: /contrib/test/traft/cluster/node10002.c		行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.		
	代码片段: 50: printf("putValueCb ok \n"); 51: } 52: free(ptr); 53: free(req); 54: }		
	污染路径: 无		
71	缺陷发生位置: /tools/taos-tools/src/benchData.c		行号: 1298
	步骤描述: 禁止释放非堆内存'tmp'.		
	代码片段:		

	1296: char *tmp = benchCalloc(1, field->length + 1, false); 1297: if (0 != tmpStr(tmp, stbInfo->iface, field, k)) { 1298: free(tmp); 1299: return -1; 1300: }	
	污染路径: 无	
72	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 457
	步骤描述: 禁止释放非堆内存'condition'.	
	代码片段: 455: t / 1000000); 456: } 457: free(condition); 458: } 459: free(command);	
	污染路径: 无	
73	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 956
	步骤描述: 禁止释放非堆内存'tsma'.	
	代码片段: 954: if (!tools_cJSON_IsString(tsma_interval_obj)) { 955: errorPrint("%s", "Invalid tsma interval format in json\n"); 956: free(tsma); 957: return -1; 958: }	
	污染路径: 无	
74	缺陷发生位置: /contrib/test/traft/single_node/singleNode.c	行号: 52
	步骤描述: 禁止释放非堆内存'ptr'.	
	代码片段: 50: printf("putValueCb ok \n");	

	51: } 52: free(ptr); 53: free(req); 54: }
	污染路径: 无